



TEAMCENTER

Advanced Classification — Deployment and Administration

Software Version 2606

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: <https://www.sw.siemens.com/en-US/trademarks/>. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center

Log in to Support Center to access software downloads and updates, product and release documentation, Knowledge Base articles, videos, other self-service resources, and to open support cases: <https://support.sw.siemens.com>.

If you do not have a Support Center login, register here: <https://support.sw.siemens.com/register>.

To send feedback on documentation: https://support.sw.siemens.com/doc_feedback_form.

Table of Contents

- Deploying and administering Classification..... 1-1
- Overview of Advanced Classification, ECLASS, and licensing..... 2-1
- What is the difference between basic and advanced classification?..... 3-1
- About the Classification Standard Taxonomy architecture.....4-1
- About the objects that you work with..... 5-1
- ECLASS version support for the current version of Teamcenter.....6-1
- Deploying classification..... 7-1
 - Planning the deployment of advanced classification.....7-1
 - Install classification using Deployment Center..... 7-1
 - Install classification using Teamcenter Environment Manager..... 7-3
 - Configuring advanced classification..... 7-3
 - Enable the Presentation Layer..... 7-3
 - About working with advanced and basic classification simultaneously.....7-4
 - Classify with Advanced and Basic Classification simultaneously.....7-4
 - Extend classes to the Presentation Layer using clsutility.....7-6
 - Synchronize the Presentation Layer with the Classification hierarchy.....7-7
 - Migrating Basic Classification to Advanced Classification Standard Taxonomy.....7-8
 - Configuring search for classification..... 7-11
 - Configure search for advanced classification.....7-11
 - Make properties searchable in the global search.....7-11
 - Searching using localized classification values.....7-11
 - Re-index classification data after upgrading Teamcenter.....7-13
 - Classification business constants for search.....7-13
 - Add classification properties to a table.....7-14
 - Displaying classification attributes in structure columns.....7-16
 - Configuring the classification interface..... 7-21
 - Adding images and icons to a classification class or hierarchy.....7-21
 - Add images and an icon to a node.....7-23
 - Delete an image or an icon from a node.....7-24
 - Changing the default icon of visual navigation cards.....7-24
 - Configuring what information appears in the classification tiles.....7-25
 - Enable the display of DWG and CGM class images in the Classification tab.....7-26
 - Configure a visual indicator for classification on an object.....7-27
 - Setup advanced classification with ECLASS data..... 7-28
 - ECLASS and Teamcenter terminology.....7-28

Importing and exporting classification data.....	7-28
Organizing classes with hierarchical positions in ECLASS.....	7-77
Managing multiple ECLASS releases.....	7-77
Converting between XML and JSON file format.....	7-82
Setting up classification with artificial intelligence.....	7-93
Overview of configuring Classification Artificial Intelligence.....	7-93
Install Classification AI components.....	7-94
Deploying the classificationaiserving Docker image on Linux.....	7-94
When to run Classification AI training.....	7-95
Train the Classification AI engine.....	7-95
Using Classification AI with traditional and Classification Standard Taxonomy data.....	7-96
Classifying objects in batch mode.....	7-97
Troubleshooting Classification AI.....	7-98
Configure access for classification.....	7-99
Give access to users for classifying objects.....	7-99
Controlling access to property blocks or groups.....	7-100
Control property block and group access.....	7-103
Set block access using clsutility.....	7-104
Granting access to non-dba users for Classification Manager tasks.....	7-105
Troubleshooting the installation and configuration.....	7-105
Enforce classification on specific object types.....	7-106

Administering classification.....8-1

Viewing your hierarchy definitions.....	8-1
Authoring hierarchy definitions.....	8-2
Creating classification hierarchy definitions.....	8-2
Creating Key LOVs.....	8-2
Create a property.....	8-18
Create a class.....	8-20
Create a node.....	8-27
Indexing classification data.....	8-27
About indexing classification data.....	8-27
Index classification data.....	8-28
Starting and stopping the indexing synchronization process.....	8-30
Classifying objects using clsutility.....	8-30
Understanding object classification with clsutility.....	8-30
Classify objects using clsutility.....	8-33
Classify objects in multiple classes using clsutility	8-34
Modify a classification using clsutility.....	8-35
Find the UID of a classified object.....	8-36
Can I delete classifications using clsutility?.....	8-38
Updating the status of classification objects.....	8-38
Change the status of Classification objects.....	8-38
Change the status of a class to Released using the clsutility command.....	8-39
Change the status of a class to Released from the user interface.....	8-40
When to revise classification definitions.....	8-40
Updating advanced classification objects without revisioning.....	8-42
Revise a class.....	8-44

Administering classification.....	8-44
Deleting CST structures and data.....	8-44
Enable notifications for the objects you follow that have classification criteria.....	8-45
About cardinality on property blocks.....	8-46
About polymorphism.....	8-48
About aspects.....	8-49
Add classification properties to a table.....	8-52
Creating and managing views for class in Active Workspace.....	8-56
About classification views.....	8-56
Create a view.....	8-56
Assign attributes to a view.....	8-58
Apply additional properties to the attributes in a view.....	8-59
Delete a view.....	8-60
Using classification views to filter what information users can see.....	8-60
What are classification views?.....	8-60
Create base views in bulk using clsutility.....	8-61
Create user, group, or role views using clsutility.....	8-61
Extract existing view definition using clsutility.....	8-64
List the class descriptor for a view using clsutility.....	8-65
Delete view definitions using clsutility.....	8-65
Importing and exporting classification data.....	8-66
Importing hierarchy definitions with the Classification Manager.....	8-66
Import classified data for advanced classification.....	8-68
Importing JSON files using the clsutility command.....	8-69
Preparing JSON files for import (example).....	8-70
Export classification data in JSON format.....	8-113
Export classified data for advanced classification BMEcat XML format.....	8-114
Export classified objects using the clsutility command.....	8-114
Share objects with classification data across sites with Briefcase files.....	8-115
Enable auditing for classification events.....	8-115
Overview of auditing classification events.....	8-115
Make audit logs visible for users.....	8-116
Allow auditing for certain event types.....	8-116
Allow auditing for class attributes using JSON file.....	8-116
Allow auditing for class attributes using the user interface.....	8-118
View audit logs for classification events.....	8-119
Classification preferences and utilities.....	8-120
eclass2json.pl.....	8-120
runClsAITraining.....	8-122
cls_ai_auto_classify.....	8-123
clsutility.....	8-125
clsutility reference tables.....	8-127
Examine the hierarchy using the clsutility -list -hierarchy utility.....	8-130
Setting Classification preferences.....	8-134

1. Deploying and administering Classification

Teamcenter Classification helps to manage and classify product data efficiently.

As an installer, you can install and setup classification for the business users and classification administrator

As a classification administrator, you can create a classification hierarchy containing classes and class attributes that helps you describe the objects that you want to reuse in your organization. The classification administrator sets up such features as the sharing of hierarchy and classified data, adding images to classes, or enabling graphics creation for part family templates.

Alternatively, you can import a standardized ECLASS hierarchy to describe your data.

Where do I go from here?

 Business User		You can use Advanced Classification for reuse and standardization. Alternatively, you can also use the old version of classification using the rich client.
 Installer		
Plan the deployment of classification		Planning the deployment of advanced classification
Learn which components must be installed for various classification scenarios		Install classification
 Classification administrator		
You use classification in the Teamcenter rich client and now want to search for and author classification data in Active Workspace.		Configure full-feature classification for traditional classification data
You are just beginning with classification and want to create hierarchies so that you can easily share your data.		Configure Classification Standard Taxonomy
You want to work with Basic and Advanced Classification simultaneously.		Classify with Advanced and Basic Classification simultaneously
You use the ECLASS standard for classifying data and want to download and use such hierarchies and data.		Configure classification for ECLASS compliant data
How do you create classification hierarchy for basic and Advanced Classification data?		Refer to authoring classification hierarchy objects in Classification Manager

2. Overview of Advanced Classification, ECLASS, and licensing

Advanced classification is based on the *Classification Standard Taxonomy* (CST) framework. This framework supports using standardized classification hierarchies and data so that classification data can easily be shared between customers and suppliers. In particular, the framework supports the ECLASS standard that describes products and services across a wide range of industries through the use of classes and properties with unique identifiers.

For more information about the ECLASS standard, visit their website:

<https://eclass.eu/en/eclass-standard/introduction>

For information about what versions of ECLASS are supported, see [ECLASS version support for the current version of Teamcenter](#).

Teamcenter Classification for ECLASS introduces CST to support ECLASS structures. CST classes are available in the classification hierarchy, and if desired, alongside traditional classification classes (ICS) to classify your data. Using Teamcenter Classification for ECLASS, you can import and export ECLASS-compliant data.

The use of the ECLASS standard requires registration and licensing.

Advanced Classification can also be used as generic classification system where you can build your own hierarchy and classification definitions by using the new features and data constructs. Most of the new features and data constructs are supported in Advanced Classification even without ECLASS.

If you want standards based classification system then ECLASS Standard is supported in Advanced Classification by importing ECLASS taxonomy and classification definitions.

Classification Standard Taxonomy features (CST) are available with a regular classification authoring license. Reading, authoring, and browsing of a custom CST hierarchy is possible.

If, however, you download and use hierarchy data from ECLASS, you require a **cls_eiclass_user** named user license. With this license, you can perform create, update, and delete operations on an ECLASS hierarchy and data.

For complete information about licenses required, contact your Siemens Digital Industries Software representative.

3. What is the difference between basic and advanced classification?

When using the classification feature, there are two types of data that can exist in your classification hierarchy — Basic and Advanced. These are some of the differences between these two types of data:

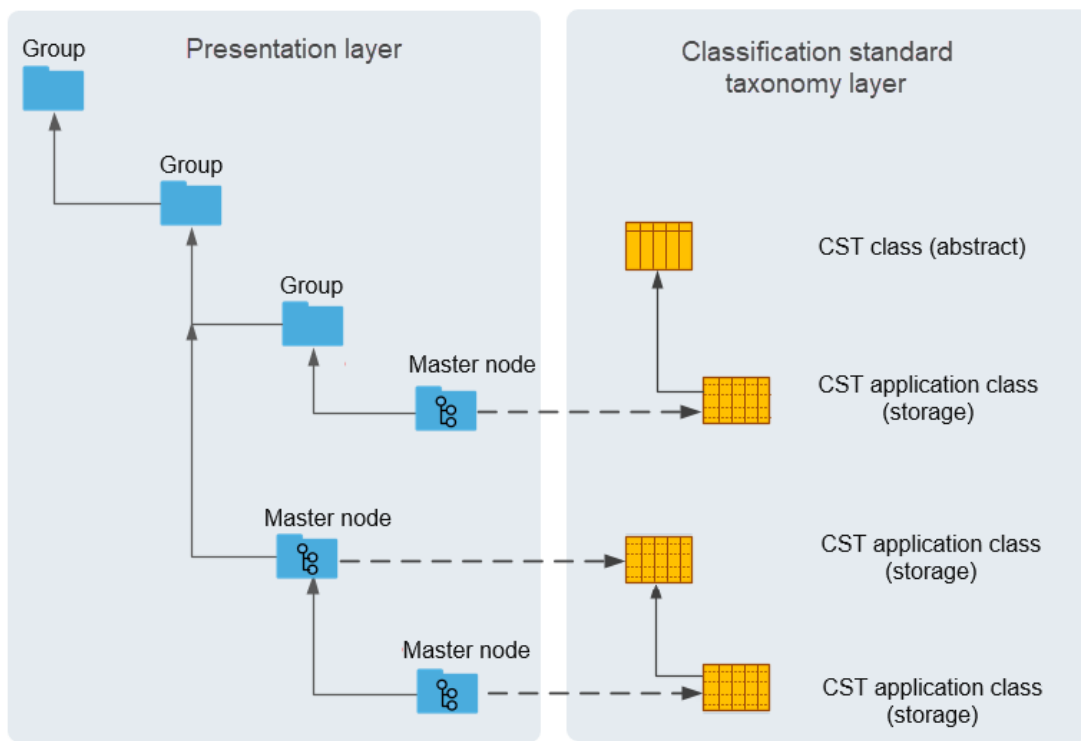
	Basic classification	Advanced classification
Availability	Available on the rich client and Active Workspace.	Available on Active Workspace.
Effectiveness	Effective in defining objects uniquely with properties for reuse.	Effective in capturing overall product information for PIM and MDM systems with support for ECLASS standard class hierarchy and definitions.
Standard Features	Some of the basic classification features: • Classification hierarchical representation • Flat list of properties • List of values • Unit of measure • Support for views • Limited to 200 Properties in a class	Some of the advanced classification features: • Class definition versioning • Flexible data modeling with attribute blocks, aspects, cardinality, and polymorphism. • Complex data type attributes • Native data storage • Support for namespaces • Unlimited number of properties • No Array limitations
Data model standards	Supports DIN-4000 standard and conforms to underlying specifications of ISO/TS-13399 standard.	Supports ECLASS standard and conforms to DIN 4002, ISO 13584-32, ISO 13584-42, IEC 61360 standards.
Hierarchy and class definitions	Define custom hierarchy and class definitions.	Define custom hierarchy and class definitions. Additionally, supports standard based ECLASS hierarchy spanning more than 48 domains.
Data exchange support	Supports PLMXML, TCXML, and Multi-Site.	Supports JSON, OntoML, BMEcat.

There is very little difference between these two types of data in the user interface. These data types are, however, installed and configured differently. If you have questions, contact your classification administrator.

4. About the Classification Standard Taxonomy architecture

The Classification Standard Taxonomy (CST) is displayed through a hierarchy of groups containing master nodes that reference classes in which data is stored. This hierarchy is referred to as the *Presentation Layer*. The Presentation Layer is installed by default.

The Presentation Layer does not actually store the classes. It references classes that are stored in the CST layer. A node in the hierarchy can hold a reference to a class that stores data. Group nodes are used for organizational structure, and master nodes reference application classes in the CST layer. The CST layer also stores properties and key-LOVs. Classification data is stored in the Teamcenter server layer.



The significance of this architecture becomes apparent when [understanding how CST and traditional Classification \(SML\) work together](#).

5. About the objects that you work with

IRDI

The International Registration Data Identifier (IRDI) originates from the [ECLASS](#) standard. This identifier is used by the Classification Standard Taxonomy to uniquely identify all classes, properties, and key-LOVs. Most importantly, using an IRDI supports:

- Unique namespaces
- Revisioning

In CST, the IRDI is displayed using the following format:

<aaaa...a>#<bb>-<cccc>#<nnn>

Example:

XMPL#01-CLS001#001

Where:

IRDI components	Description
<aaaa...a>	Unique alphanumeric namespace Numeric namespaces are reserved as companies and organizations can register them with ISO. For example, the following namespaces are reserved: 0173 is reserved by the ECLASS organization 0175 is reserved by Siemens Choose a namespace that is unique to your organization. This is particularly important if you plan to share data with other organizations.
#	Separator character
<bb>	Data type 01 : class

Class

There are three types of classes in CST:

- Application class

An *application class* is referenced by nodes and can be used to store data. They hold the properties used to define the class. An application class can reference properties and aspect classes.

- Property block class

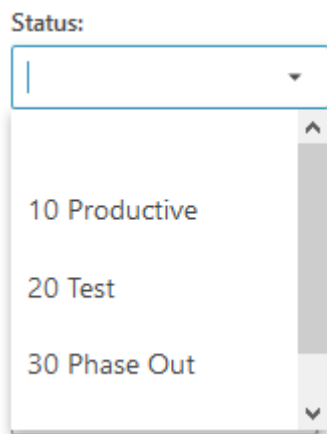
A *property block class* groups sets of properties together. You can group properties that are frequently used, avoiding the need to repeatedly assign each property to a class. For example, a milling tool generally comprises a holder, an adapter, and an insert. Each of these components is described by many properties. The insert, for example, can be described by properties such as thickness, cutting length, grade, and material. Because these properties apply to every insert, you can group them into one property block class called **Insert** and reuse this property block like a simple property in every class that contains an insert.

- Aspect class

An *aspect class* groups properties that pertain to overarching aspects of many objects and can be reused across multiple object types. They often group such parameters as commercial information, for example, supplier contact information, that have nothing to do with product definition. Aspect classes can only be referenced directly by application classes.

Key list of values

A *key list of values (key-LOV)* holds selection lists.



These lists can be nested.

Key-LOVs are referenced by properties.

Property

A *property* is used to describe attributes of a class. It can have multiple data types:

- String
- Integer
- Double

- Boolean
- Reference

Reference property types point to another object such as a key-LOV definition or a property block class.

Property group

A *property group* is the user interface representation of a property block class. It is a group of properties that you can navigate using the **Property Groups** pane.

Classification object

An Internal Classification Object (ICO) is used to classify Teamcenter data.

View

Used to display all or a subset of the properties in a class in the user interface. There are two types of views used in Classification classes:

- Base view
- User, group, or role view

6. ECLASS version support for the current version of Teamcenter

Advanced Classification in Teamcenter version 2606 supports all ECLASS versions from ECLASS 9 to ECLASS11.1.

7. Deploying classification

Planning the deployment of advanced classification

Before deploying classification, plan your deployment considering the following scenarios:

☐	You only want to use the advanced classification functionality.
☐	You want to use both basic and advanced classification functionality.
Additional scenarios are addressed when you have installed the advanced classification functionality:	
☐	You want to use the ECLASS taxonomy.
☐	You want to use AI assisted classification.

Once you have identified the scenarios that are applicable in your case, you can [install Teamcenter Classification](#) and then set up classification by creating the classification hierarchy and performing additional configurations.

Install classification using Deployment Center

Add the Classification application to your existing Teamcenter environment.

Procedure

1. Log on to Deployment Center and select the environment to which you want to add Classification.
2. Go to the **Applications** task. Click **Add or Remove Selected Applications** .
3. In the **Available Applications** panel, use the web browser search to find the following applications depending on your scenario:

Scenario	Applications to select for Active Workspace based solution	Applications to select for Rich Client based solution
Advanced classification Based on new ISO, IEC standard classification system supporting advanced features	Advanced Classification Classification Active Workspace	Not supported

Scenario	Applications to select for Active Workspace based solution	Applications to select for Rich Client based solution
like attribute blocks, cardinality, and polymorphism		
Traditional Basic Classification and Advanced Classification together	Advanced Classification Classification Active Workspace	Not supported
Teamcenter Classification for ECLASS Importing of ECLASS Standard taxonomy into Teamcenter and classifying of Teamcenter data into ECLASS taxonomy	Advanced Classification Classification Active Workspace	Not supported
AI assisted classification	Classification Active Workspace	Not supported
Classification localization support Supports internalization and localization of classification	Classification L10N	Classification L10N

Select the application, and then click **Update Selected Applications**.

Deployment Center automatically selects any additional dependent applications.

- Go to the **Components** task.
- In the **Selected Components** list, note any remaining components whose configuration status is not **100%**. Select each incomplete component, enter required parameters, and save component settings until all components in the environment show a configuration status of **100%**.

When all components are fully configured, the **Deploy** task is enabled.

- Go to the **Deploy** task. Click **Generate Install Scripts** to generate deployment scripts you will use to update affected machines.

When script generation is complete, note any special instructions in the **Deploy Instructions** panel.

- Locate deployment scripts, copy each script to its target machine, and then run each script on its target machine.

For more information about running deployment scripts, see *Run the deployment scripts* in the *Deployment Center — Usage* help.

If you face any issues during the deployment, see *Troubleshooting deployment* in the *Deployment Center — Usage* help.

What to do next

If you want to use classification with Active Workspace, ensure that you [enable the presentation layer](#).

Install classification using Teamcenter Environment Manager

You can install the classification functionality through Teamcenter Environment Manager.

Procedure

1. Launch Teamcenter Environment Manager.
2. Proceed to the **Features** panel and select the features based on your requirements.

Functionality required	Features to select
Install advanced classification	• Extensions > Reuse and Standardization > Advanced Classification
Use traditional basic classification and advanced classification with ECLASS data simultaneously	• Active Workspace > Client > Reuse and Standardization > Classification Client • Active Workspace > Server Extensions > Reuse and Standardization > Classification Server • Active Workspace > Server Extensions > Reuse and Standardization > Presentation Layer - Next Generation Classification Server • Extensions > Reuse and Standardization > Advanced Classification
Install AI based Classification	• Microservices > Classification AI Serving • Active Workspace > Server Extensions > Reuse and Standardization > Classification AI

3. After selecting the features perform the next steps to complete the installation.

Configuring advanced classification

Enable the Presentation Layer

The Presentation Layer provides the ability to deal with traditional Basic Classification features (such as creating a class hierarchy, adding Classification objects to a Classification hierarchy, classifying workspace objects, searching for Classification objects, and modifying and deleting objects from a Classification hierarchy) and advanced CST and Classification library management features simultaneously. Additionally,

the Presentation Layer adds **Classification** to the **home page** and allows you to configure visual navigation cards for searching.

Procedure

Make the node hierarchy (Presentation Layer) visible in the Active Workspace user interface by updating the value of the **CLS_is_presentation_hierarchy_active** user preference to **true**.

The screenshot shows a web-based configuration interface for a user preference. At the top, there are tabs: 'Definition' (selected), 'Instances', 'Category', 'Import', and 'Export'. Below the tabs is a text box with instructions: 'Click on the "Edit" button to modify the definition and update any field in order for the "Save" button to be enabled. Note that the "Description" field must not be empty. Click on the "Save" button to save the definition of the existing preference.' The form fields are as follows: 'Name' is 'CLS_is_presentation_hierarchy_active'; 'Location' is 'Site'; 'Protection Scope' is 'User' (dropdown); 'Category' is 'Active Workspace' (dropdown); 'Environment' is 'Disabled' (dropdown); 'Type' is 'Logical' (dropdown); 'Multiple' is 'Single' (dropdown); 'Description' is 'See presentation layer nodes' (text area); and 'Value' is 'true' (text field).

Being able to turn the visibility of the node hierarchy on and off per user allows users of both traditional Classification and CST to classify objects in the same environment. Depending on the setting, a user can see either the traditional Classification classes or the CST classes.

Note

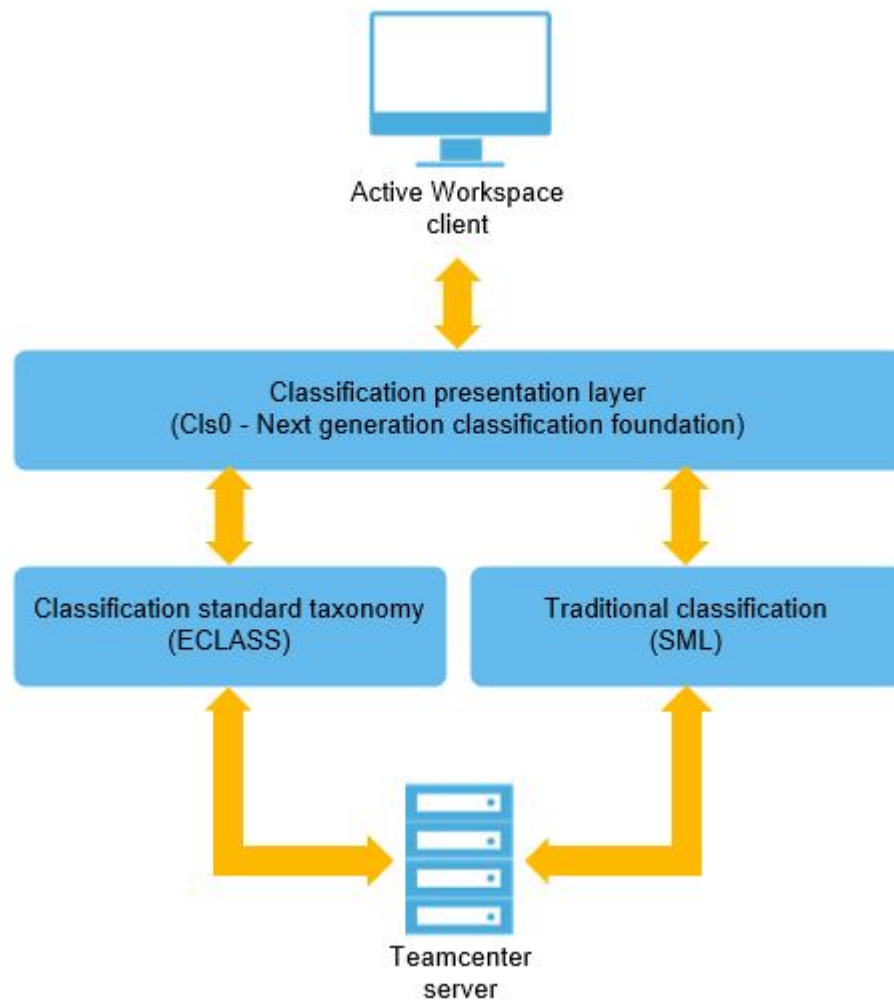
CST classes are not subject to access control. Therefore, if your environment contains both CST and traditional Classification classes, if the presentation hierarchy is not active (**CLS_is_presentation_hierarchy_active=false**), CST classes are displayed in the search results for traditional Classification users.

About working with advanced and basic classification simultaneously

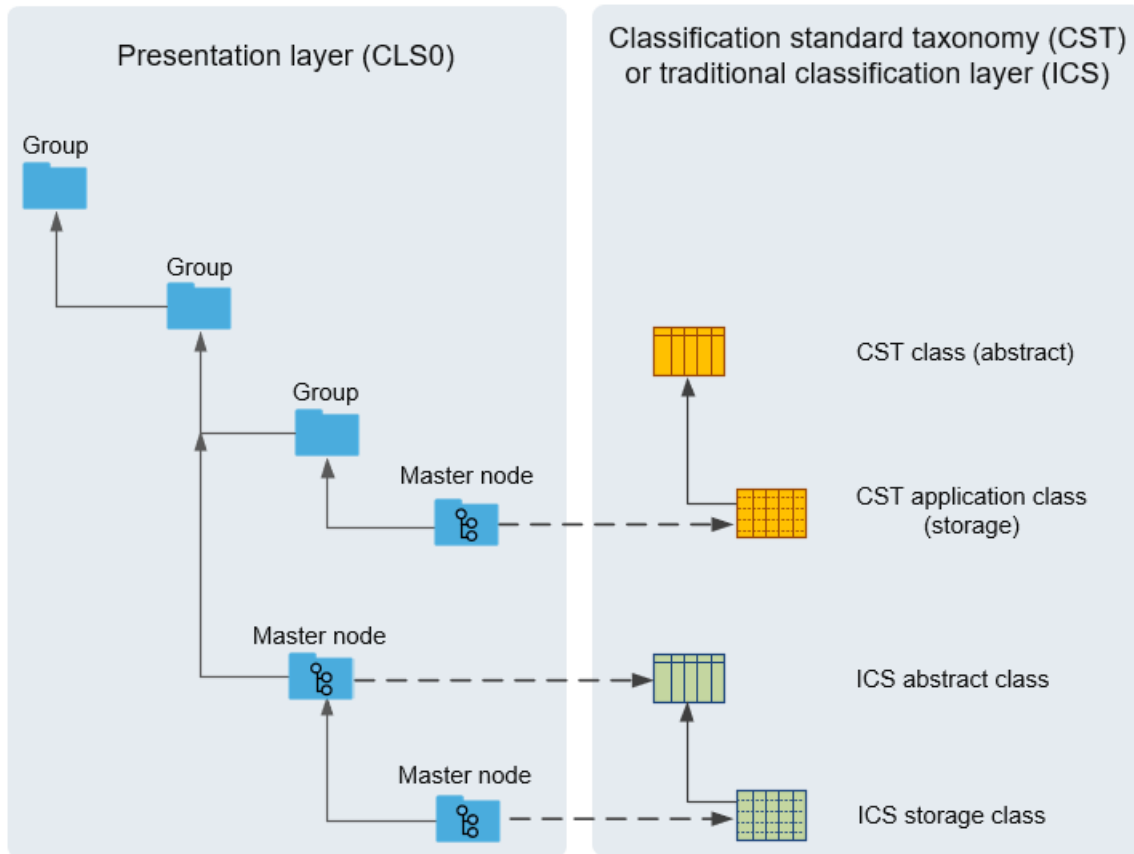
Classify with Advanced and Basic Classification simultaneously

Advanced Classification Standard Taxonomy (CST) offers some features that traditional basic classification does not. Additionally, in the future, CST will be regularly enhanced. For this reason, you may want to consider **migrating your traditional basic classification classes to the advanced CST format**.

Both advanced and basic classification classes can co-exist in a single environment.



In this case, the nodes reference the appropriate class type: For basic classification, a node references ICS classes and for advanced, it references CST classes.



The **CLS_is_presentation_hierarchy_active** user preference determines whether nodes or classes are displayed in the hierarchy. If this preference is set to **true**, only nodes are displayed in the hierarchy. If it is set to **false**, traditional classes are displayed.

Note

The **CLS_is_presentation_hierarchy_active** preference is not delivered by default. You must add it manually.

Basic classification (ICS) classes must be extended to the presentation hierarchy as follows:

1. **Extend classes to the Presentation Layer using clsutility.**
2. **Synchronize the Presentation Layer with the Classification hierarchy.**

Extend classes to the Presentation Layer using clsutility

Procedure

1. Extend Classification data to the Presentation Layer by running the following command line utility:

clsutility -import -hierarchy -cid=<group-or-class-ID>

This command extends the Classification subhierarchy under the specified group or class.

- For Classification groups, running this utility creates Presentation Layer group nodes with the same ID as the associated groups.
- For Classification classes, running this utility creates Presentation Layer master nodes with the **storage_class_type** property corresponding to their associated classes.

Additional optional arguments:

- **-include_instances**
Includes ICOs in the subhierarchy.
- **-exclude_children**
Excludes the subhierarchy. It extends the specified group or class only.

2. (Optional) **Synchronize the Presentation Layer with the Classification hierarchy.**

Extend classes to the Presentation Layer when using only a subset of the Classification hierarchy

When using only a subset of the Classification hierarchy, you must run the **clsutility** on the entire hierarchy. In this case, the argument **-include_instances** is not required since you only need to extend the classes to the nodes that are required.

Example

```
clsutility -import -hierarchy -cid=ICM
```

Synchronize the Presentation Layer with the Classification hierarchy

Use one of the following methods to synchronize the Presentation Layer with the underlying Classification hierarchy:

- **Manually run clsutility.**
- Enable automatic synchronization that shadows specific operations by setting the **CLS_auto_sync_node_hierarchy** preference to **true**.

When at least the top-level node of the storage and presentation hierarchies are **linked**, the synchronization mechanism is triggered by a change to groups, classes, or ICOs in the storage hierarchy and results in the following changes to the presentation hierarchy.

Operation on class hierarchy	Operation required on node hierarchy
Add/delete group	Add/delete group node
Add/delete class	Add/delete master node
Add/delete class ICO	Add/delete Classification element

Operation on class hierarchy	Operation required on node hierarchy
Copy/paste or cut/paste of a group	Copy/paste or cut/paste of a group node
Copy/paste or cut/paste of a class	Copy/paste or cut/paste of a master node

The synchronization mechanism functions as follows:

- If a target is not found in the presentation hierarchy, it is created (applies to node or element).
 - ◊ To create a node in the presentation hierarchy, Teamcenter matches the parent node with the parent class.
 - ◊ To create an element in the presentation node, Teamcenter matches the node with the storage class.
- To find targets, Teamcenter searches for the following:
 - ◊ A group node with the same ID as the Classification group
 - ◊ A master node with the storage class type property pointing to the associated classification class
 - ◊ A Classification element referencing the associated ICO

The objects in the hierarchies are mapped as follows.

Object types from Classification (class hierarchy)	Corresponding object types from the Presentation Layer (node hierarchy)
Group	Group node
Abstract class	Master node
Storage class	Master node
Classification object (ICO)	Classification element (ICO)

Migrating Basic Classification to Advanced Classification Standard Taxonomy

Advanced Classification Standard Taxonomy (CST) offers some features that traditional Basic Classification does not. Additionally, in the future, CST will be regularly enhanced. For this reason, you may want to consider migrating your traditional Basic Classification classes to the advanced CST format. To do this, you can run the following utility:

```
clsutility -migrate -classification2cst -cid=<class-ID> [-recursive | -confirm] -include_icos]
```

Caution

It is strongly recommended that you test the migration of your classes in a staging environment and evaluate the results. Deploy the migration in production only when you are completely satisfied with the test results.

This utility migrates traditional classification classes and all the objects (properties, dictionary attributes, key-LOVs, and associated images) referenced by that class, as well as all the parent classes, to the CST format. Optionally, the utility can also convert all child classes (**-recursive**), and any ICOs already existing in the class (**-include_icos**).

Running the utility creates a class in the database with an automatically generated IRDI. This IRDI is displayed in the utility output. Additionally, node definitions are created for application classes so that the migrated CST classes are visible and can be used for classification.

Migration summary for traditional objects

TYPE	SML ID	CST ID	Status	Note
NodeDefinition	AD	SMPL1#AD#001	UNPROCESSED	
PropertyDefinition	-80001	SMPL1#02-f6f89w#001	UNPROCESSED	
PropertyDefinition	-80002	SMPL1#02-fsf89w#001	UNPROCESSED	
PropertyDefinition	-80005	SMPL1#02-e6f89w#001	UNPROCESSED	
PropertyDefinition	-80015	SMPL1#02-ccf89w#001	UNPROCESSED	
PropertyDefinition	-80018	SMPL1#02-bsf89w#001	UNPROCESSED	
KeyLOVDefinition	-80051	SMPL1#09-Tcf89w#001	UNPROCESSED	
PropertyDefinition	-80020	SMPL1#02-bMf89w#001	UNPROCESSED	
ClassDefinition	ADT-CONNECTION	SMPL1#01-ADT-CONNECTION#001	UNPROCESSED	
NodeDefinition	ADT-CONNECTION	SMPL1#ADT-CONNECTION#001	UNPROCESSED	
PropertyDefinition	-82005	SMPL1#02-q5989w#001	UNPROCESSED	
KeyLOVDefinition	-80040	SMPL1#09-WMf89w#001	UNPROCESSED	
PropertyDefinition	-86123	SMPL1#02-la989w#001	UNPROCESSED	
ClassDefinition	ADT-CONN-BUND	SMPL1#01-ADT-CONN-BUND#001	UNPROCESSED	
NodeDefinition	ADT-CONN-BUND	SMPL1#ADT-CONN-BUND#001	UNPROCESSED	

JSON input for KeyLOVDefinitions: C:\Users\ny6wyo\AppData\Local\Temp\KeyLOVDefinitions.json
 JSON input for PropertyDefinitions: C:\Users\ny6wyo\AppData\Local\Temp\PropertyDefinitions.json
 JSON input for ClassDefinitions: C:\Users\ny6wyo\AppData\Local\Temp\ClassDefinitions.json
 JSON input for NodeDefinitions: C:\Users\ny6wyo\AppData\Local\Temp\NodeDefinitions.json

Operation completed successfully.

The **-confirm** argument creates the objects in the database. It is recommended that you run the utility without the **-confirm** argument and review the resulting JSON files. These are created in your *temp* directory. The exact path is included in the utility output. You can modify the JSON files if necessary and import them manually or, if you are satisfied with them, rerun the migration utility using the **-confirm** argument.

You must set the following two preferences to run the migration:

- **CST_default_namespace**

This preference specifies the namespace of all CST objects created by the migration utility.

- **CST_default_migration_status**

This preference specifies the status of the CST objects created by the migration utility. Valid values are **Develop**, **Released**, or **Retired**. Objects set to **Released** can no longer be modified.

The output is displayed in tabular form with a column for status. This status can be one of:

- **UNPROCESSED**

The migration utility was run in dryrun mode (default). You can review the JSON files created for the new object but it does not yet exist in the database.

- **SKIPPED**

The reason that the object was skipped is displayed in the **Note** column.

- **SUCCESS**

The migration utility was run with the **-confirm** argument. The object was successfully migrated to Advanced Classification.

- **FAILURE**

The object was not successfully migrated. See the **Note** column for more information.

Limitations

- Class attribute, class detail, and view attribute options that are not supported in Advanced Classification are migrated as property values only. Your data is not lost, but is not relevant to Advanced Classification. Only the **Mandatory** and **Protected** options are supported in Advanced Classification.
- As Advanced Classification does not support localization for the **UserData1**, **UserData2**, and **Annotation** properties, localized values for these properties are not migrated.
- Default or minimum and maximum values set for properties are stored as individual property values. Advanced Classification does not support inheritance, so if a default or minimum and maximum value is inherited in rich client, the value that would be displayed through that inheritance mechanism is displayed as the property value in the migrated class or view. If one of the supported views contains a value in rich client that differs from the value displayed in the default view, the property is assigned the value displayed in the default view.
- Reference attributes are not supported in CST.
- Interdependent key-LOVs are migrated as individual property values.
- **Only views that are supported in Advanced Classification** (Default, User, Group, and Role views) can be migrated.

Configuring search for classification

Configure search for advanced classification

After installing classification or after adding data (classes, views, properties), you must enable searching for the data by **indexing classification data**.

Any time you create new classification data, such as classifying new objects, you must re-index to be able to search for the new object in the global search or see the property displayed in the list of filters when searching for data. There are two ways to re-index:

- Index all data.

This can be a time-consuming process depending on the amount of data in your database.

- **Index only the workspace objects whose classification data has changed.**

Make properties searchable in the global search

By default, when you add properties to a class, they are displayed as searchable filters. If, additionally, to search for properties in the global search, or when indexing classification data to mark the changed classification objects, you must do the following:

Procedure

1. Run the **bmide_modeltool** utility.

For example, in a Teamcenter command window, change to the `<TC_ROOT>\bin` directory and type:

```
bmide_modeltool -u=username -p=password -g=dba -tool=all -mode=upgrade
-target_dir="<TC_DATA">
```

2. Verify the schema file was correctly updated.
 - a. Verify that the **tc_solr_schema.xml** file in the `TC_DATA\fts\solr_schema_files` folder has the current date and time.
 - b. Open the **tc_solr_schema.xml** file in a text editor and search for **TC_0Y0_CLS_0Y0**. You should find several lines containing this string. Close the text editor without saving.

Searching using localized classification values

Classification values are localized along with other objects using the same localization process.

The following features are fully supported for localization:

- Keyword search
- Property specific search
- Category name in filter panel
- Filter value
- Filtering using localized value

Re-index classification data after upgrading Teamcenter

After upgrading Teamcenter, or after installing any classification features, you must **re-index classification data**.

If the new release to which you are upgrading includes new classification features, it generally also includes schema changes. During this upgrade procedure all classification data is marked for re-indexing and included during your next indexing synchronization. This can cause the synchronization to take a longer than usual.

Classification business constants for search

Business object constant	Object type on which constant can be set	Default setting	Description
Awp0SearchIsIndexed	Item ItemRevision Cls0ClassBase Lbr0LibraryElement	Set to true on the ItemRevision	Enables a business object for indexing. This business object is configured in the AWS2 template owned by the search. Although it is possible to set this constant to true on Item objects, it is recommended to index item revisions only. To enable classification presentation layer classification objects (Cls0ClassBase) and library elements (Lbr0LibraryElement) for indexing, you must install the Cls1ClassificationAW and Lbr0LibraryManagementAW templates which set this constant to true on Cls0ClassBase and Lbr0LibraryElement respectively.
Awp0SearchIsClassifyData Indexed	Specific object type, for example: Item ItemRevision Cls0Object Lbr0LibraryElement	Is set automatically to true when you install the ics1icsaw template.	This business object type constant is specific to classification and is set on a business object to enable its associated classification information for indexing. If in an environment item revisions are classified and are also marked for indexing and if this business object type constant value is set to true, classification information associated to those item revisions are indexed. Administrators needs to install the ics1icsaw template to enable classification indexing which sets this BO constant to true on workspaceObject. All subtypes of WorkspaceObject (Item, ItemRevision, and so on) inherit this configuration.
Awp0SearchClassifySearchEnabled	WorkspaceObject	Installing the ics1icsaw template to enable classification data indexing, sets this business object type constant to true on WorkspaceObject and all its subtypes inherit this configuration.	This business object type constant is set to enable indexed classification properties for search. If this business object type constant is set to false and if the classification information is already indexed, it is not used for searching. Consequently, classification-specific filter categories are not displayed in the Active Workspace client.

Add classification properties to a table

To display specific classification properties in a table such as your search results table, you can add the desired properties to the table columns. This allows you to view the classification information that is most relevant to your needs.

To add classification properties in your search results table, you must:

- Choose the classification properties you want to add to the search results table and find the **Dynamic Compound Property**(DCP) value of the classification property using the **clsutility**.
- Add the DCP value to the table configuration.

Procedure

Find the DCP value of the classification property

2. Run the **clsutility** command:

```
clsutility -list -hierarchy -nodeId=0173-1#AFX043#002 -showType
-showDCP -recursive > C:\output.txt
```

From the command replace the value of the **nodeId** with the **Node ID** of the node.

3. From the output of the **clsutility**, copy the relevant DCP value .

```
143 : [cst00173-1#02-AAM549#001] (String)1 "Supplier item type"
DCP :
GRM(Cls0ClassifiedBy,Cst00173-1#AFR745#002) .REF(cst00173-1#01-ADN228#006,Cst001
73-1#01-ADN228#006) .REF(cst00173-1#02-AAQ376#005,Cst00173-1#01-ADN199#005) .cst0
0173-1#02-AAM549#001
144 : [cst00173-1#02-AAO735#002] (String)1 "Supplier name"
DCP :
GRM(Cls0ClassifiedBy,Cst00173-1#AFR745#002) .REF(cst00173-1#01-ADN228#006,Cst001
73-1#01-ADN228#006) .REF(cst00173-1#02-AAQ376#005,Cst00173-1#01-ADN199#005) .cst0
0173-1#02-AAO735#002
145 : [cst00173-1#02-AAM551#001] (String)1 "Supplier product designation"
DCP :
GRM(Cls0ClassifiedBy,Cst00173-1#AFR745#002) .REF(cst00173-1#01-ADN228#006,Cst001
73-1#01-ADN228#006) .REF(cst00173-1#02-AAQ376#005,Cst00173-1#01-ADN199#005) .cst0
0173-1#02-AAM551#001
```

Note

Select only simple classification properties as complex properties are not supported.

Add the DCP value to the table configuration

5. From the search results page, from the **Table Settings**  menu, choose **Configure Table** .

The screenshot shows the 'Table Configurator' interface. At the top, there are tabs for 'Shape', 'Saved', and 'Advanced'. Below these, there's a search bar with 'Any Category' and a search icon. A classification filter is set to 'eCI@ass ADVANCED 47...' with a 'Clear' button. A table is displayed with columns 'Name', 'Description', and 'Release Status'. The table contains five rows of electronic components. A context menu is open over the table, showing options: 'Arrange', 'Wrap Text', 'Clear all filters', and 'Configure Table'.

Name	Description	Release Status
3RT2018-1FB47	CONTACTOR,AC3:7,5KW 1NO DC24V M. DIODE	
3RH2344-1AF00	CONT.RELAY,4NO+4NC,AC110V 50/60HZ	
3RW5225-1AC05	3RW52 600V 63A 24V screw analog	
3RT2617-1AB03	CONT., AC6B:12.5KVAR 1NO+1NC24VAC50/60HZ	
6ES7144-5KD00-0BA0	ET 200AL, AI 4xU/I/RTD, 4xM12	

The **Table Configurator** opens with the table's column configuration selected.

6. In the **Table Configurator** page, click **Add Property** .
7. In the **Add Property** panel, select **Advanced** and specify the object type and the DCP value.

Add Property

Reset
 Pin Panel
 Close

☐ Basic
 ☒ Advanced

▼ Type

Workspace object

▼ Properties

* Dynamic Compound Property:

GRM(Cls0ClassifiedBy,Cst00173-1#AFR745#002).REF(cst00173-1#01-ADN228#006,Cst00173-1#01-ADN228#006).REF(cst00173-1#02-AAQ376#005,Cst00173-1#01-ADN199#005).cst00173-1#02-AAO735#002

Add

- Click **Add**.
- You can update the behavior for the classification property you added in the table, such as making the property visible by default, by editing the table configuration.

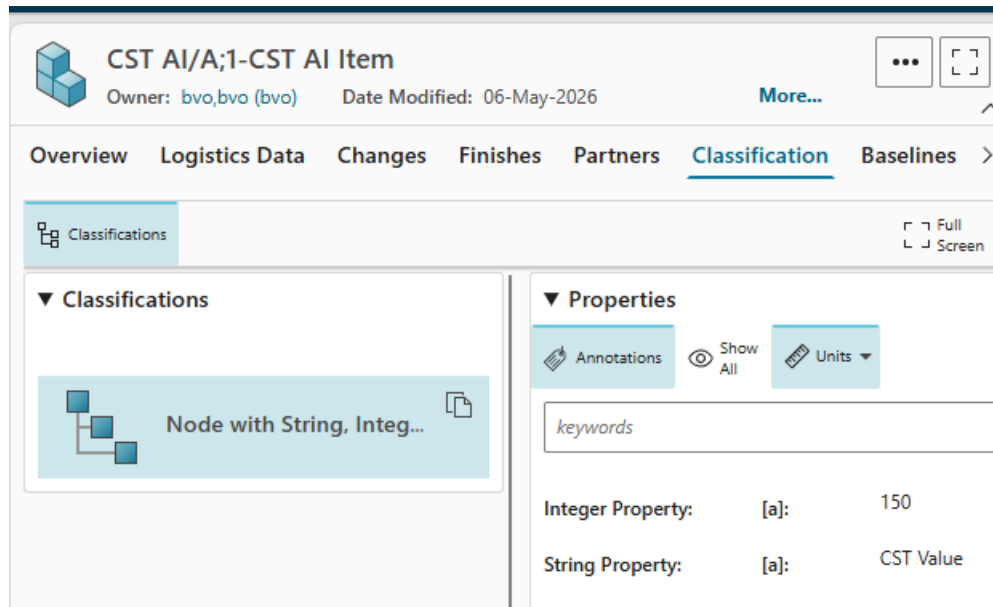
Results

The classification property is now available in the search results table. Users can now arrange the search results table to show or hide the classification properties.

Displaying classification attributes in structure columns

How to add classification attributes to structure columns.

In this scenario, the following object has two classification attributes you want displayed when it is in a structure; **Integer Property** and **String Property** from the **Node with String, Integer, and Double** classification.

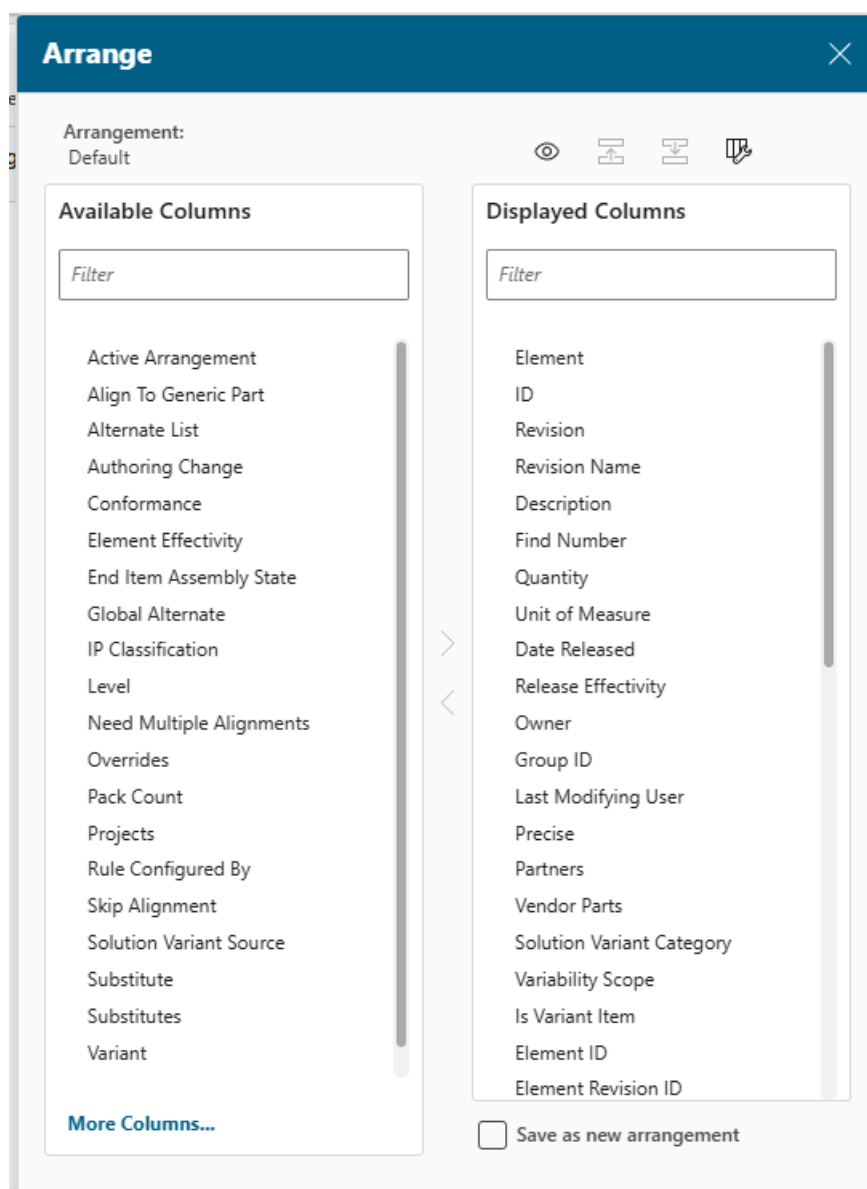


Initial scenario

By default, classification attributes are not displayed in structures.

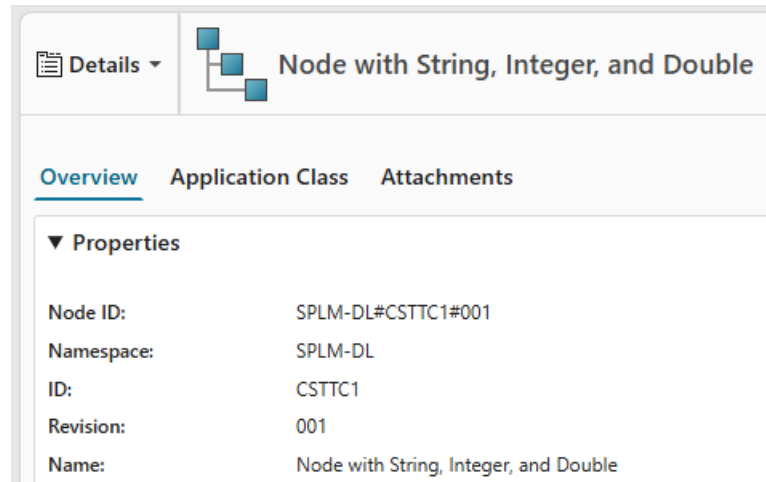
Revision: Global (Latest Working) Date: Today Units: None Variant: No Variant Rule Expansion: No Rule			
+ Add ↗ Edit Structure 🔍 Find 📁 Expand ⚖ Compare Σ Rollup + Duplicate ✓ Selection Mode ✎ Edit ⋮			
Element	ID	Revision	Revision Name
Example Assembly	033610	A	Example Assembly
CST AI Item	CST AI	A	CST AI Item

They are not automatically displayed in the **Arrange** panel for the structure's table either.



Identify classification node ID

Use the **Nodes** tab of the **Classification Manager** to find the **Node ID** of the properties you want to display on the structure columns. The name of the node in this example is **Node with String, Integer, and Double**. In this example, the **Node ID** is **SPLM-DL#CSTTG1#001**.



Export the node's dynamic compound property information

Use the **clsutility** from a Teamcenter command-line environment to retrieve the dynamic compound property information of the classification attributes. This utility has a **-showDCP** argument which writes a GRM string to the output file for each classification attribute. You can use these GRM strings to reference the attributes in Teamcenter.

```
clsutility -list -hierarchy -nodeid=SPLM-DL#CSTTG1#001 -showType -showDCP
-recursive >output.txt
```

Locate the GRM information

Search the *output.txt* file created from the **clsutility** to locate the **SPLM-DL#CSTTG1#001** node. Copy the DCP strings for each the attributes that you want to display in your structure columns. In this example, there are two.

```
"90-01-01-01" [SPLM-DL#CSTTC1#001 ==> SPLM-DL#01-000800#001] "Node with
String, Integer, and Double"
{ DynRTT "Cst0SPLM-DL#000800#001"
  Parent Type : Cls0CstObject
  142 : [cst0SPLM-DL#02-000500#001](Integer)1 "Integer Property"
    DCP : GRM(Cls0ClassifiedBy,Cst0SPLM-DL#000800#001).cst0SPLM-
DL#02-000500#001
  143 : [cst0SPLM-DL#02-000501#001](String)1 "String Property"
    DCP : GRM(Cls0ClassifiedBy,Cst0SPLM-DL#000800#001).cst0SPLM-
DL#02-000501#001
  144 : [cst0SPLM-DL#02-000502#001](Double)1 "Double Property"
    DCP : GRM(Cls0ClassifiedBy,Cst0SPLM-DL#000800#001).cst0SPLM-
DL#02-000502#001
}
```

Add properties to the table

In Active Workspace, use the **Table Configurator** to locate the **Awb0OccurrenceManagement** table. This is the table which controls which properties are displayed when a structure is loaded in Active Workspace. In this example, choose the default site configuration for the table.

Name	Type
Awb0OccurrenceManagement	Table
Site	Site
Default	Table Configuration

Click **Add Property**  and then in the **Add Property** panel:

1. Select **Advanced**.
2. **Type:** Choose **Element**. (**Awb0Element**)
3. **Properties:** Enter **REF(awb0Archetype,ItemRevision)**. and then the DCP string for the classification attribute

For example: `REF(awb0Archetype,ItemRevision).GRM(Cls0ClassifiedBy,Cst0SPLM-DL#000800#001).cst0SPLM-DL#02-000500#001`

4. Click **Add**.

Add Property
Pin Panel
X

☐ Basic
☒ Advanced

▼ Type

Element

▼ Properties

* Dynamic Compound Property:

REF(awb0Archetype,ItemRevision).GRM(Cls0ClassifiedBy,Cst0SPLM-DL#000800#001).cst0SPLM-DL#02-000500#001

Repeat this for each attribute.

Default			
Column Configuration ID: contentColConfig Client Scope URI: Awb0OccurrenceManagement			
▼ Table Contents			
Property Name	Type Name	Internal Property Name	Internal Type Name
Integer Property	Element	REF(awb0Archetype,ItemRevision).GRM(Cls0Classifie...	Awb0Element
String Property	Element	REF(awb0Archetype,ItemRevision).GRM(Cls0Classifie...	Awb0Element

In this example, two classification attributes have been added, **Integer Property** and **String Property**.

Result

Return to the structure and you should see your two new property columns on the table.

(+) Add ▼ Edit Structure ▼ Find Expand ▼ Compare ▼ Rollup ▼ Edit ...				
Element ▾	a... ▾	Finishes ▾	Integer Property	String Property ⚙
▼  Example Assembly				
 CST AI Item			150	CST Value

Configuring the classification interface

Adding images and icons to a classification class or hierarchy

A visual representation of classes helps to quickly navigate the hierarchy or provides additional information about a class. In classification, you can add images and icons to a class.

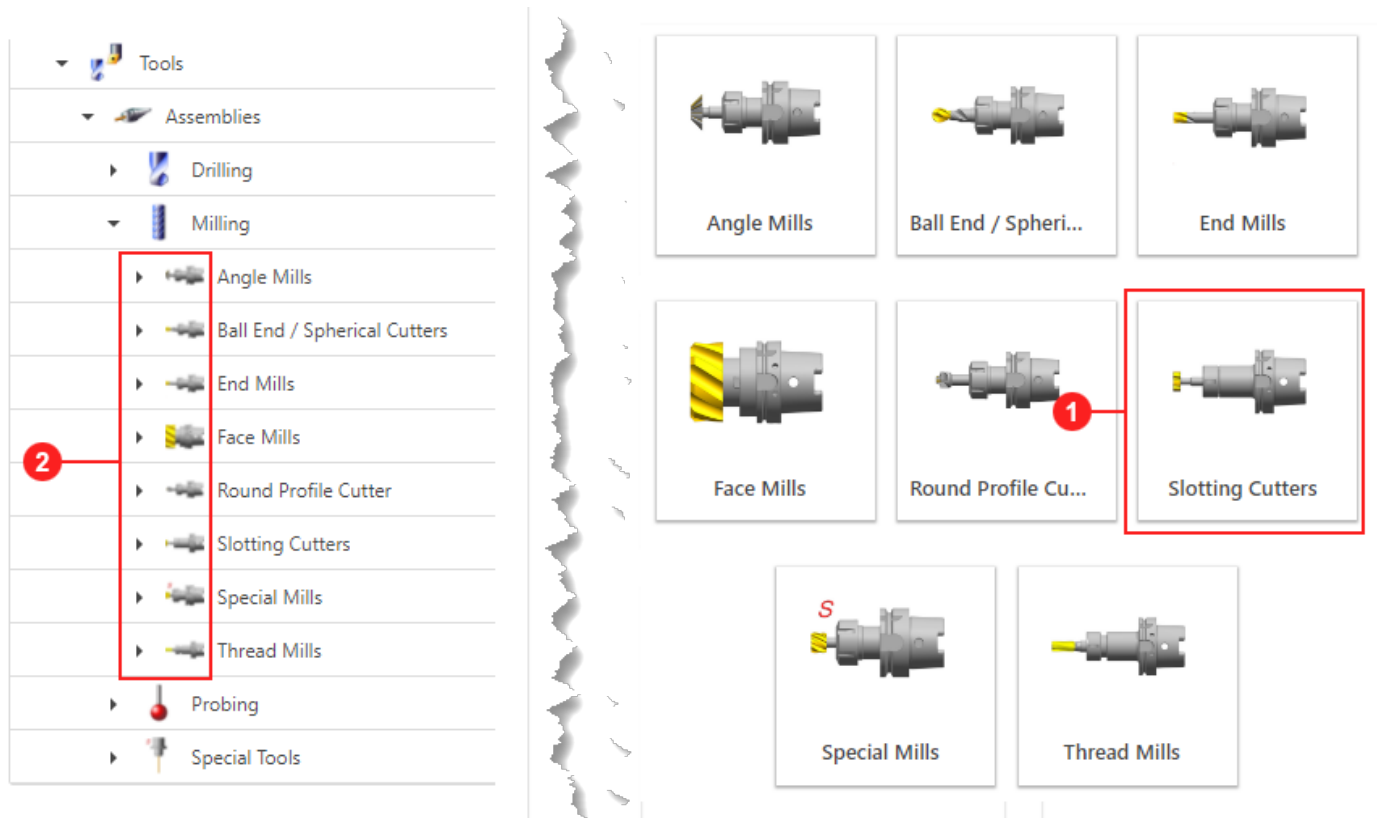
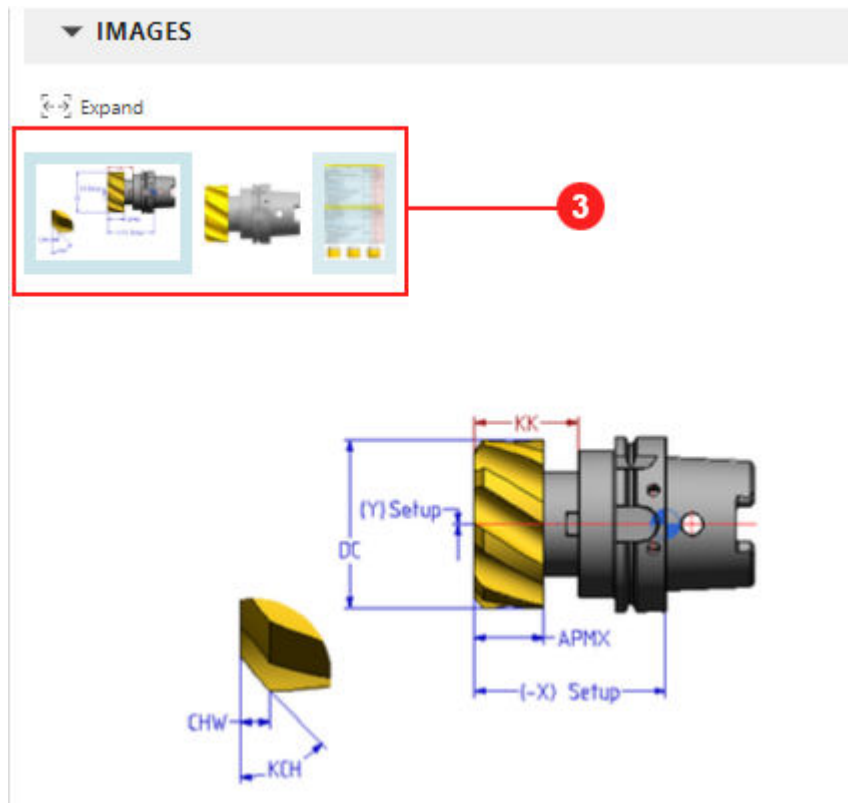


Image ¹ Refers to an image associated with a class. A class can have multiple images associated. These are displayed in the **IMAGES** pane where you can switch between them using the carousel ³.



One image is designated as the primary image. This is the image that is displayed in the visual navigation card.

Icon 2 Refers to the small graphic displayed in the classification hierarchy when you navigate the hierarchy.

The behavior of images and icons varies depending on what type of classification you use.

Import icons and images using the following [clsutility](#) commands:

-import -image -id=<hierarchy-node-ID> -os_path=<path-to-image-file>

-import -icon -id=<hierarchy-node-ID> -os_path=<path-to-icon-file>

Add images and an icon to a node

Adding images or an icon to a node provides visual points of reference within the classification hierarchy. Images help to identify the contents of a node. You can add multiple images to a node which then appear in the **Preview** section within the **Overview**.

Procedure

1. On the **Classification Manager** dashboard, click **Nodes**.
2. Select a node to which you want to add an image or an icon and then select **Attachments**.
3. In the **Files** section, click **Add** ⊕ and click any one of the following:
 - **Add Icon** ⊕ to add an icon in the selected node.
 - **Add Image** ⊕ to add an image in the selected node.
4. In the **Add Icon** or **Add Image** panel, click **Choose File** to select the required file, and click **Add** to add an icon or an image to the node.

When you add an image as an attachment to a node, a preview of the image appears in the **Preview** section of the **Overview**. For a given node, you can include multiple images by adding them one at a time.

When you add an icon as an attachment to a node, it is displayed in the visual navigation card of the node tree. You can add one icon at a time. To replace an icon, you must first delete the previously added icon.

5. Click **List** or **Images** to view all the images and icon added to the node.

Delete an image or an icon from a node

You can delete an image or icon from a node.

Procedure

1. On the **Classification Manager** dashboard, click **Nodes**.
2. Select a node to from which you want to delete the image or icon.
3. Under **Files**, select the image or icon and click **Delete** ✕.
4. In the confirmation message, click **Delete**.

Changing the default icon of visual navigation cards

In the Class Navigator you can view the classification hierarchy by image. These tile images are referred to as *visual navigation cards* (VNCs) and allow the display of larger images while viewing the selected level in the classification node hierarchy.

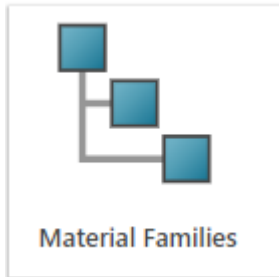
VNCs are visible in the following areas:

- The Classification location when you click on the CLASSIFICATION tile on the home page

- The **Classification** tab where you classify objects

Configuring the image displayed on the VNC

If you have not yet imported node icons, a generic VNC is displayed listing the children of the node.



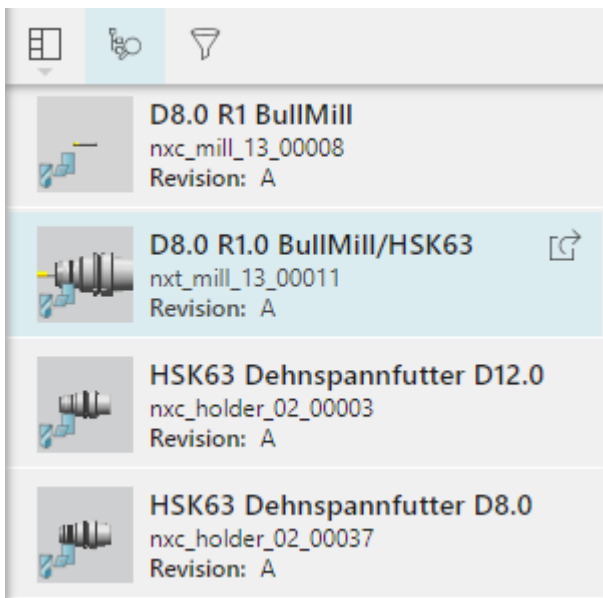
The images are based on the node icons that you import using **clsutility -import -icon -id=<hierarchy-node-ID>**. If a node icon does not exist for a class, there is a fallback mechanism that searches for the first image of a node, if none exists, then the first class image is used.

Tip

To make optimal use of the image space on the VNC, create a square image. Rectangular images are cropped.

Configuring what information appears in the classification tiles

Tiles are displayed in the primary work area, for example, in search results:



Configuration of the object tiles respects the type hierarchy.

Mdl0ModelElement		
	Cls0ClassBase	
		Cls0CstObject
		Cls0Object

Tile properties are configurable on the parent class or on the child classes themselves. Site preferences determine what is displayed on the tiles for **Cls0Object** and **Cls0CstObject**, for example:

✕
🔍

Name ↕	Location ↕	Scope ↕	Values ↕
Cls0ClassBase.CellProperties	Site	User	object_name, cls0object_id
Cls0Context.CellProperties	Site	User	object_name, object_type, last_mod_date
Cls0HierarchyNode.CellProperties	Site	User	object_name, object_type, object_desc
Cls0HierarchyView.CellProperties	Site	User	object_name, object_type, object_desc

Enable the display of DWG and CGM class images in the Classification tab

To use CGM or DWG file formats as class graphics, you must first convert them to PDF using the **prepare** utility delivered with the Lifecycle Visualization installation.

Procedure

1. Run the Lifecycle Visualization installation, and select the **Convert and Print** option.
2. Copy the CGM or DWG file to the **VVCP** directory in the installation location.
3. Double-click the **prepare.exe** utility.
4. Run the utility to convert the **CGM** or **DWG** files to PDF. For example, for a file named **my_drawing.dwg**, enter the following:

```
prepare -pdf my_drawing.dwg
```

This converts the **my_drawing.dwg** file and creates the **my_drawing.pdf** file in the **VVCP** directory. You can add path names to change the input and output directories.

Run `prepare -h` to obtain information about the utility.

- 5. Associate the **PDF** to the required class in Classification Admin (rich client).

Configure a visual indicator for classification on an object

You can set a visual indicator on an object to display that it is classified.

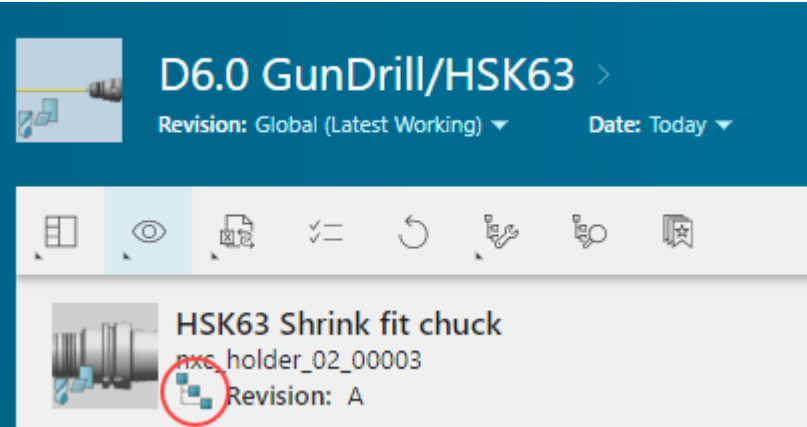
You can use the following runtime properties to define visual indicators for classified objects:

Runtime property on workspace object	Display name	Comments
ics_classified	Classified	Works with basic classification.
CLs0ClassifiedBy	Classified By	Works with basic and advanced classification.

For detailed instructions on how to configure a visual indicator, see Create an indicator help on **Support Center**.

The following icons are available by default which can be used with visual indicator:

Icon	Icon name	Used for
	<i>indicatorClassified16.svg</i>	Object classified
	<i>indicatorPartNotClassified16.svg</i>	Object not classified



Setup advanced classification with ECLASS data

ECLASS and Teamcenter terminology

The Classification Standard Taxonomy objects and concepts have different nomenclature than the ECLASS objects.

The ECLASS website provides in-depth explanations of ECLASS objects:

<https://eclass.eu>

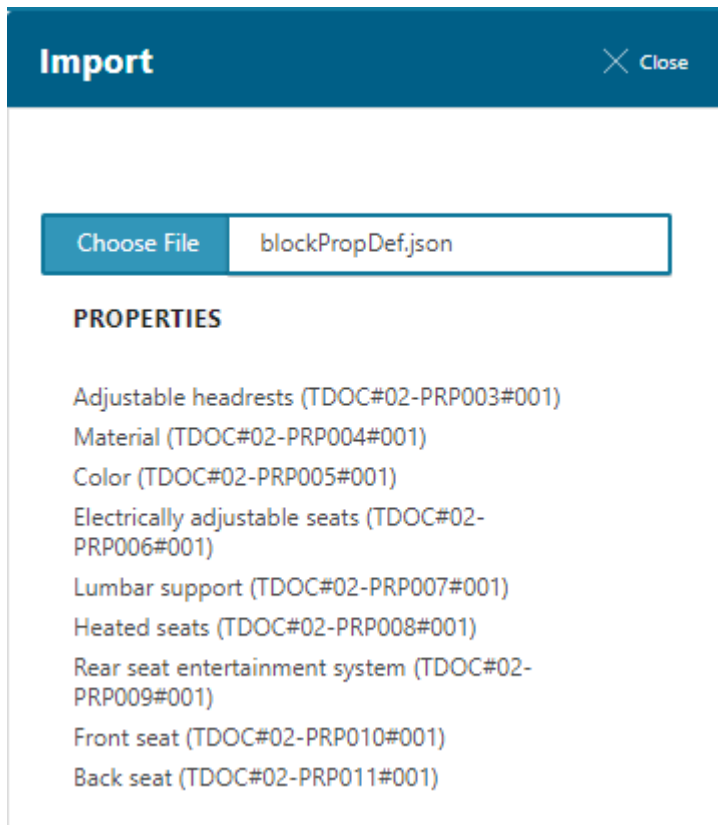
The most commonly used objects are as follows.

ECLASS	CST	Traditional classification (ICS)
Value list	Key-LOV definition	Key-LOV
Property	Property definition	Dictionary attribute
Classification class	Node definition	--
Application class	Class definition	Group/class
IRDI	IRDI	ID
Property block	Property block	--
Application data	Classification object	ICO
Aspect	Aspect	--

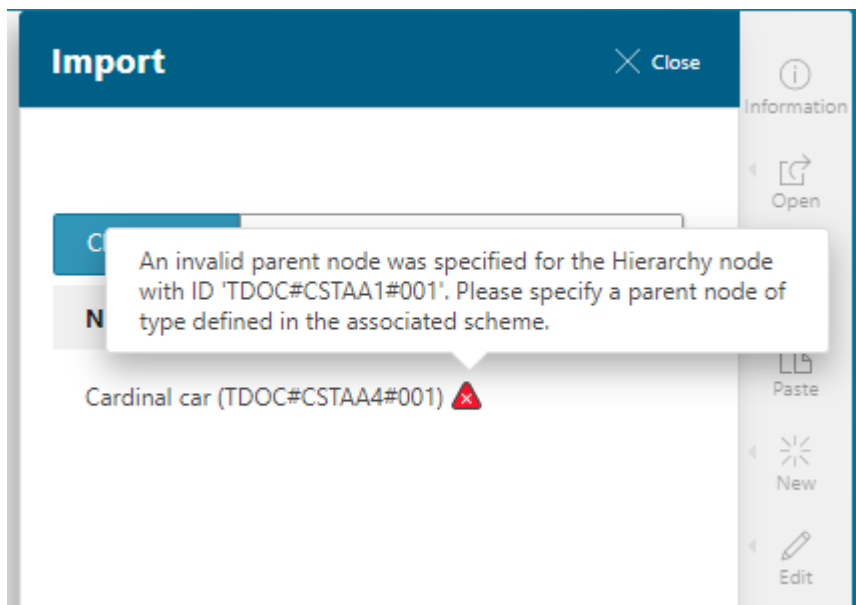
Importing and exporting classification data

Importing hierarchy definitions with the Classification Manager

For definition files based on schemas prior to version 1.5, you can import the definitions directly in the Teamcenter user interface. Use the **Import** button on the **Dashboard** tab or the **Import** command on any of the other tabs in **Classification Manager** to import key-LOV, property, class, and node definitions. They must always be **imported in the correct order**. After you upload the JSON file, you receive an overview of the objects you are about to import.



If there is an error in the files that you import, the error message provides assistance.



Tip

It does not matter on which tab you choose to import, you can import any type of hierarchy definition on any of the tabs.

Import classified data for advanced classification

You can import classified data in BMEcat XML or JSON format in the user interface. The XML files and supporting documents (for example, images) must be available in a local directory that you compress to a ZIP file. The import searches the compressed for the correct input file.

Procedure

1. Compress the directory structure of the data in a ZIP file.
If you download BMEcat files from the Siemens Industry Mall, these are already in the appropriate zipped format.
2. In your **Home** folder, choose **More Commands *** > Import/Export**  **> Import Classification Data**.
3. In the **Choose File** box, select the ZIP file containing the data.
Teamcenter searches inside the compressed file for all eligible import files and presents them to you in a list.
4. In the **Select file to import** box, select the XML file containing the data.
If you import BMEcat data, there may be several versions of the XML file in different languages. These are not localized values. If you import, for example, the English version, and then subsequently import the German version, the German version overwrites the English values. Each of these language files represent master locales.

Results

When the import is complete, you are notified in the **Alert** area where you can view the details of the import. As soon as the data is indexed, it is available for searching.

This feature requires that both the Subscription Manager and Dispatcher Client are running. If the Subscription Manager is not running, no notification is displayed in the **Alert** area, but the operation still occurs in the background.

If the imported data includes classification objects only, at import they are automatically attached to existing objects in the database if they have the same ID. This automatically classifies the existing objects or updates an existing classification on previously classified objects. If there is no existing object with the same ID in the database, a new object is created.

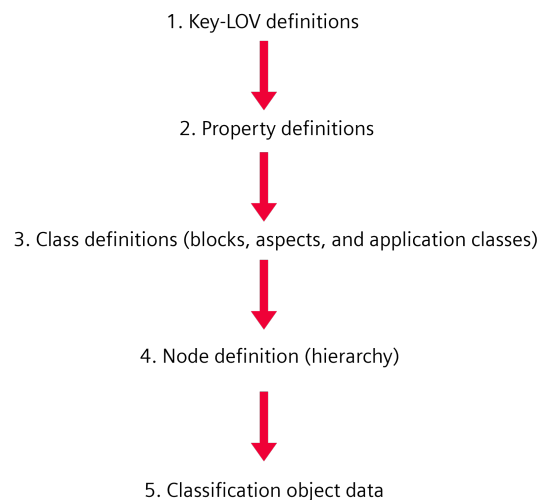
Importing JSON files using the clsutility command

You create a CST hierarchy by importing JSON files containing both hierarchy and data. The schema files defining the JSON format and example files are in the following directory:

```
..\<TC_DATA>\classification\json\schema\advanced
```

The schema used to import classification definitions is evolving over time. Prior to version Teamcenter 14.1 (schema 1.5.0), key-LOVs, properties, class, and nodes had to be defined in separate JSON files and imported in the order listed. Using schema 1.5.0, you can import all types of class definitions in a single JSON file. This schema also supports importing older object definition JSON files with no need to rewrite them. You can define new objects or update existing ones in the same JSON file.

When importing, you can only reference objects that are already in the database. When importing classes that reference other classes (for example, when importing aspects), you must import the referenced classes first and then import the referencing classes. The definition file is read from top to bottom. If you import individual files, the order of import is as follows:



Data is imported from JSON files using the following methods:

- **Use the Classification Manager to import JSON files** in the user interface.
- Use **clsutility** to import a definition file based on any schema up to and including version 1.5.

```
clsutility -u=<user> -p=<password> -g=<group> -save -classification_definitions -request="<path-to-JSON-file>"
```

- Use **clsutility** to import object individual definition files.

The syntax of this utility is:

clsutility-u=<user> -p=<password> -g=<group> -<operation> <input/output file path>

If using schema 1.5, import the definition file using the following syntax:

clsutility -u=<user> -p=<password> -g=<group> -create -keylov_definitions -request="<path-to-JSON-file>"

For more information about the available commands and their exact syntax, type **clsutility -h** for the top-level help and **clsutility -<command-name> -h** for information about each available command.

The following table lists the supported objects and an example of the **clsutility** statements used to import these object types.

Object	Operation	clsutility command
Key-LOV	Import	clsutility -u=<user> -p=<password> -g=<group> -create -keylov_definitions -request="<path-to-JSON-file>"
Property definition	Import	clsutility -u=<user> -p=<password> -g=<group> -create -property_definitions -request="<path-to-JSON-file>"
Class definition	Import	clsutility -u=<user> -p=<password> -g=<group> -create -class_definitions -request="<path-to-JSON-file>"
Nodes	Import	clsutility -u=<user> -p=<password> -g=<group> -create -node_definitions -request="<path-to-JSON-file>"
Application data (ICOs)	Import	clsutility -u=<user> -p=<password> -g=<group> -create -classification_objects -request="<path-to-JSON-file>"

Note

There are pairs of very similar commands in the **clsutility**: **classification_objects** and **classification_object**, or **node_definitions** and **node_definition**. Be sure to use the plural forms of these commands (**objects** and **definitions**) for all CST imports. The singular refers to traditional classification objects.

Preparing JSON files for import (example)

Importing the hierarchy and classes of car seats

This example walks you through creating and importing the JSON files required to describe a class containing attributes that allow you to configure the seats in a car. The example consists of several levels of complexity:

- Classify a car in a class containing simple properties, for example, a key-LOV with the type of seat (front seat or back seat).
- Classify a car in a class containing reusable sets of properties (blocks).
- Specify the number of seats when classifying the car (cardinality).
- Specify the attributes shown depending on the type of seat selected (polymorphism).

- Specify the attributes shown for each seat when specifying multiple seats (cardinality and polymorphism).
- Import a property record into one of the newly created classes.

For each of these examples, the JSON files are explained. Once created, the JSON files can be imported using the Classification Manager or, alternatively, with **clsutility**. The **clsutility** commands required to import the JSON files are also listed [here](#).

To verify that the definitions in the following examples are correct or to create additional definitions, consult the following schema files found in ...\\<TC_DATA>\\classification\\json\\schema\\advanced\\:

- **Classification_Save_KeyLOVDefinitions_Request_advanced.schema.json**
- **Classification_Save_PropertyDefinitions_Request_advanced.schema.json**
- **Classification_Save_ClassDefinitions_Request_advanced.schema.json**
- **Classification_Save_NodeDefinitions_Request_advanced.schema.json**
- **Classification_Save_PropertyRecords_Request_advanced.schema.json**

There are many online JSON validators to assist you in creating JSON files with the proper syntax. We recommend using any of these to verify the JSON files that you create prior to importing them.

Note the following points about these examples:

- The definitions are imported with a release status of **Develop**. This ensures that you can still modify them if required. When you are satisfied with the structure and classes, you must [change the status to Released](#). Objects can be classified only in **Released** classes.
- The **TDOC** name space is used to differentiate the structures imported in this example from other structures that you use. Creating your own name space helps when creating queries. When searching for your newly-imported objects, you can filter the tabs in Classification Manager to only show those with **Namespace = TDOC**.

Import a class with simple properties

The simplest use case is to import a class containing properties. This example shows how to import a **Car** class containing three properties.

- **Type of seat**
A key-LOV with two entries: **Front** and **Back**
- **Manufacturer**
A simple string property
- **Adjustable headrests**
A Boolean property

In the user interface, this class is displayed as follows:

Overview Content **Classification** Finishes Made From Where Used Changes Attachments History

ASSIGNED CLASSIFICATIONS

Automotive Engineering > Assemblies > Cars > Simple Car

AVAILABLE CLASSES

Filter

- Automotive Engineering
 - Assemblies
 - Cars
 - Simple Car**
 - Material Families
 - Miscellaneous

PROPERTIES

keywords

Unit System: * ☒ Metric ☐ Non-Metric

Type of seat:

Manufacturer:

☐ Adjustable headrests

Type of seat:

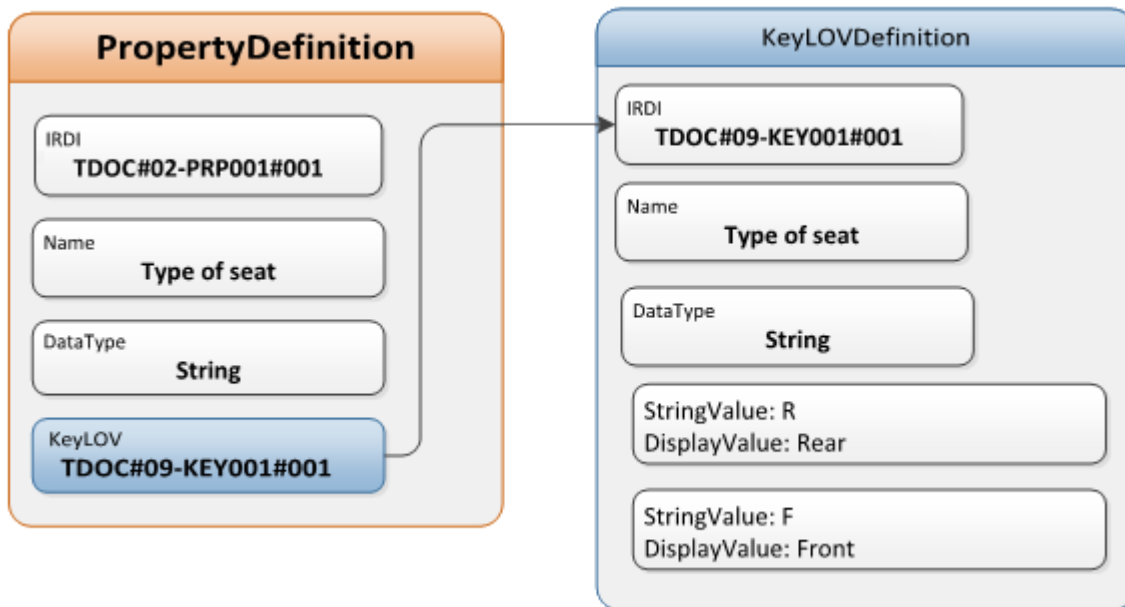
- F Front
- R Rear
- F Front

The order when importing this class is to first import the key-LOV definitions, then the property definitions, then the class definition, and finally the node definition so that the class is visible in the Classification hierarchy.

Procedure

1. Import the key-LOV definition.

In Classification Standard Taxonomy, a key-LOV definition is separate from its property definition.



In this example, there is one key-LOV whose JSON definition is stored in a file titled **simpleKeylovDef.json**:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "KeyLOVDefinitions": [
    {
      "ObjectType": "09",
      "Namespace": "TDOC",
      "ID": "KEY001",
      "Revision": "001",
      "Name": "Type of seat",
      "Status": "Develop",
      "LOVItems": {
        "DataType": "String",
        "LOVStringItems": [
          {
            "StringValue": "R",
            "DisplayValue": "Rear Seat"
          },
          {
            "StringValue": "F",
            "DisplayValue": "Front Seat"
          }
        ]
      }
    }
  ]
}
```

2. Import the property definitions.

PropertyDefinition

IRDI
TDOC#02-PRP001#001

Name
Type of seat

DataType
String

KeyLOV
TDOC#09-KEY001#001

PropertyDefinition

IRDI
TDOC#02-PRP002#001

Name
Manufacturer

DataType
String

PropertyDefinition

IRDI
TDOC#02-PRP003#001

Name
Adjustable headrests

DataType
Boolean

In this example, there are three properties to import, one of which (**Type of seat**) is a key-LOV that references the key-LOV definition imported in the previous step. The JSON definition for the properties is stored in a file titled **simplePropDef.json**:

```
{
  "SchemaVersion": "1.0.0",
```



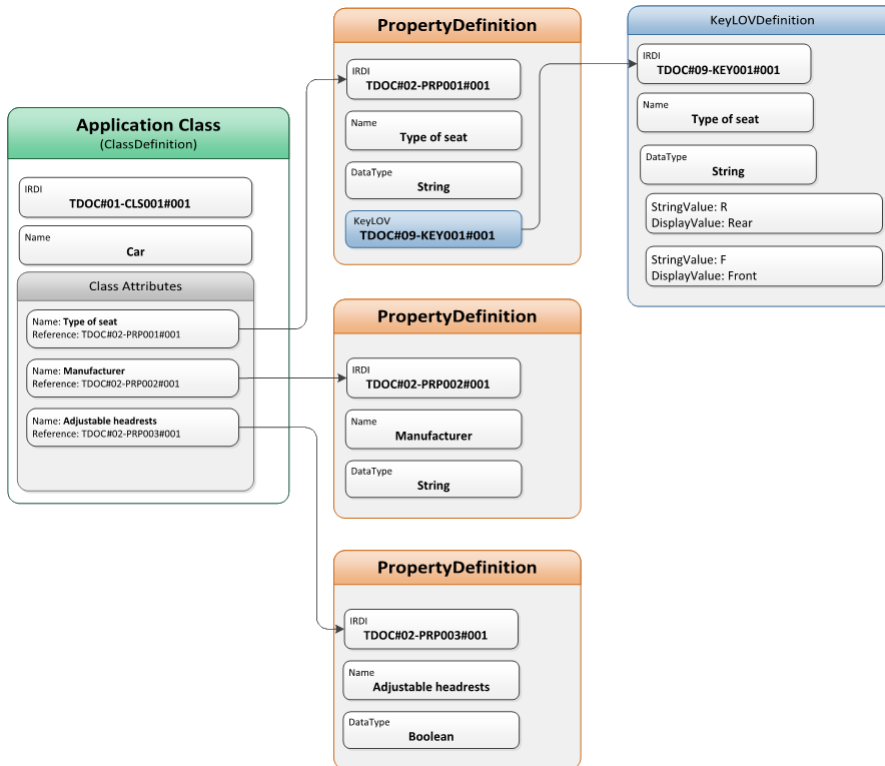
```

"Locale": "en_US",
"PropertyDefinitions": [
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP001",
    "Revision": "001",
    "Status": "Develop",
    "Name": "Type of seat",
    "DataType": {
      "Type": "String",
      "KeyLOV": "TDOC#09-KEY001#001"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP002",
    "Revision": "001",
    "Name": "Manufacturer",
    "Status": "Develop",
    "DataType": {
      "Type": "String"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP003",
    "Revision": "001",
    "Name": "Adjustable headrests",
    "Status": "Develop",
    "DataType": {
      "Type": "Boolean"
    }
  }
]
}

```

3. Import the class definition.

The **Car** class contains only three properties: **Type of seat**, **Manufacturer**, and **Adjustable headrests**:



The JSON file, named **simpleApplicationClassDef.json**, consists of the following:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "ObjectType": "01",
      "Namespace": "TDOC",
      "ID": "CLS001",
      "Revision": "001",
      "Name": "Car",
      "Status": "Develop",
      "IsAbstract": false,
      "UnitSystem": 3,
      "ClassType": "Application Class",
      "ClassAttributes": [
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP001#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP002#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP003#001"
        }
      ]
    }
  ]
}
```

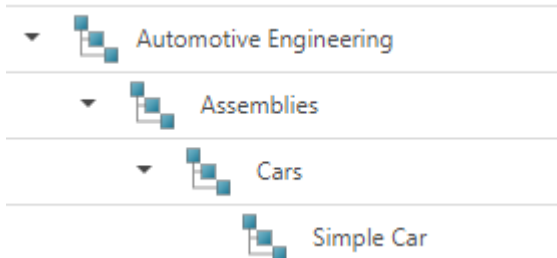
```

    ]
  }
]
}

```

4. Import the node definition that makes the class visible.

A nested hierarchy is required to organize the classes.



To import this hierarchy, create the following JSON file and name it **simpleNodeDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAE0",
      "Revision": "001",
      "Name": "Automotive Engineering"
    },
    {
      "Namespace": "TDOC",
      "ID": "CSTAA0",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAE0",
        "Revision": "001"
      },
      "Name": "Assemblies"
    },
    {
      "Namespace": "TDOC",
      "ID": "CSTAA1",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA0",
        "Revision": "001"
      },
      "Name": "Cars"
    },
    {
      "Namespace": "TDOC",

```

```

    "ID": "CSTAA2",
    "Revision": "001",
    "Parent": {
      "Namespace": "TDOC",
      "ID": "CSTAA1",
      "Revision": "001"
    },
    "Name": "Simple Car",
    "ApplicationClass": {
      "Namespace": "TDOC",
      "ID": "CLS001",
      "Revision": "001"
    }
  }
]
}

```

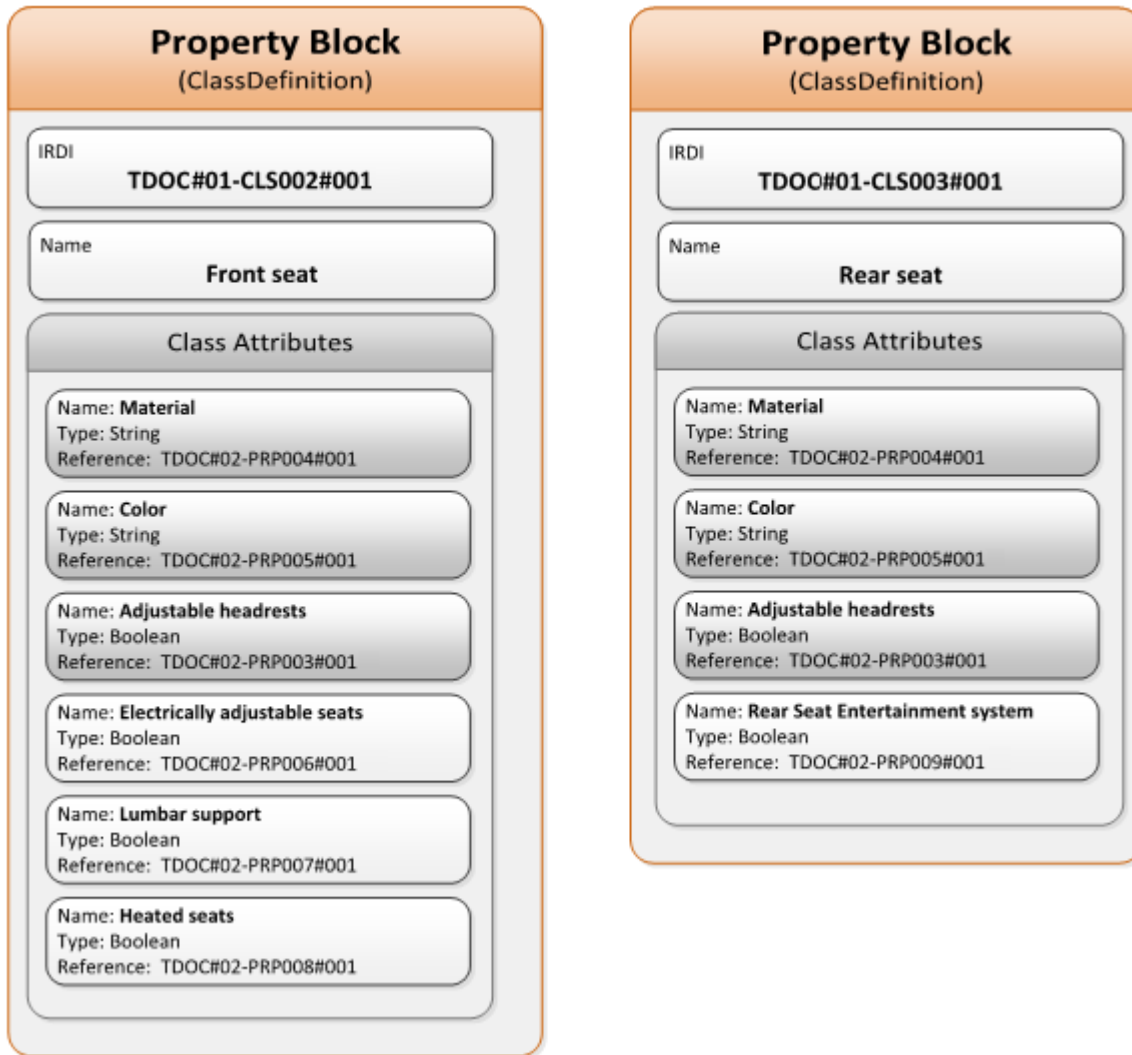
5. Import the definitions into the database using [Classification Manager](#) or with [clsutility commands](#). You must import the JSON files in the following order:
 - a. `simpleKeylovDef.json`
 - b. `simplePropDef.json`
 - c. `simpleApplicationClassDef.json`
 - d. `simpleNodeDef.json`

Import a class with a reusable set of properties (blocks)

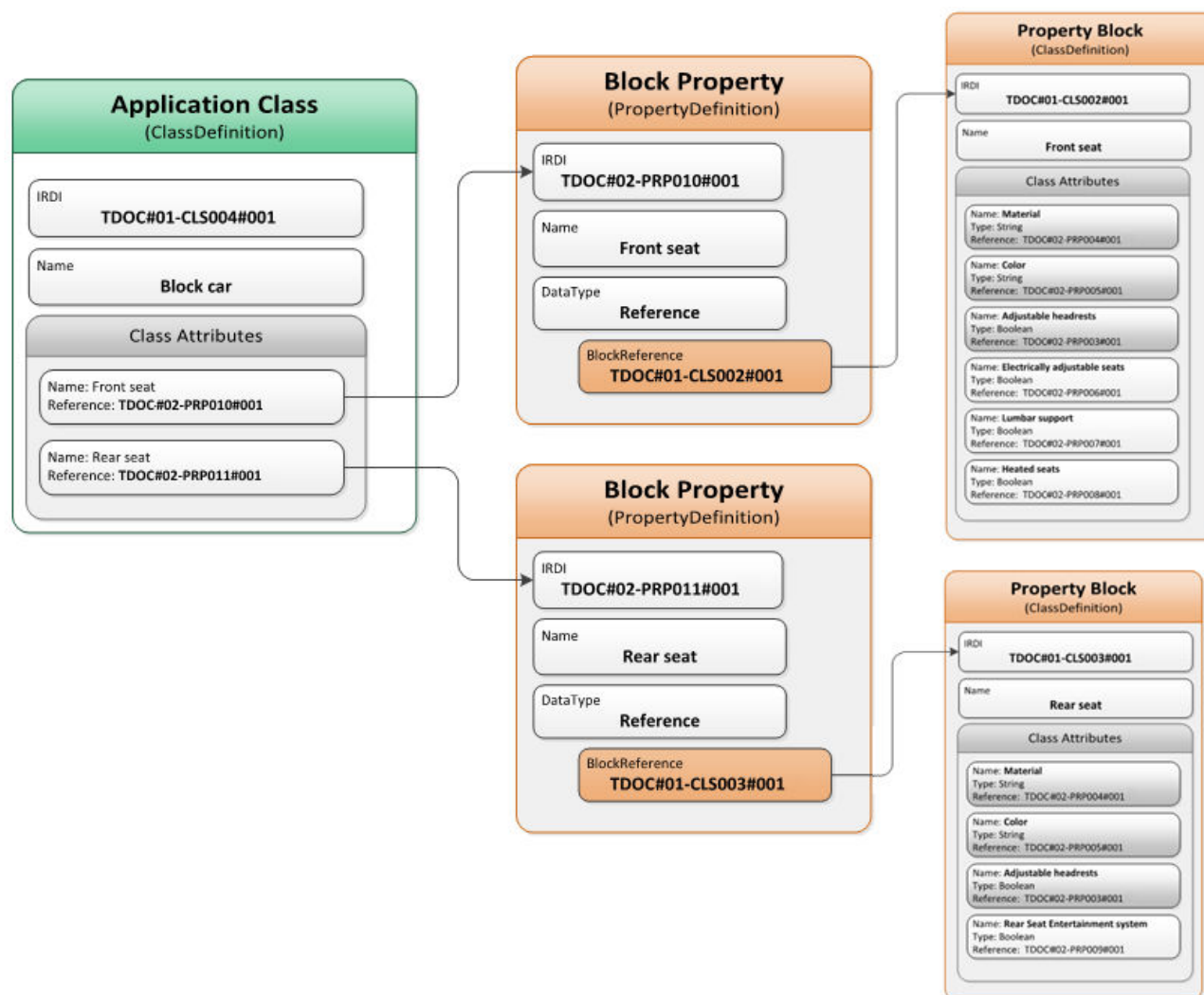
Frequently, there is a group of properties that is always used together. For example, the following set of properties describe a front seat and a rear seat:

Front seat	Rear seat
Material	Material
Color	Color
Adjustable headrests	Adjustable headrests
Electrically adjustable seats	Rear seat entertainment
Lumbar support	
Heated seats	

Each time you create a class, you could manually add each of these properties to the class to describe each of these seat types. However, it is easier to group them and select a single property called **Front seat** that contains all the required properties. This set of properties is called a property block. A property block is a special kind of class.



Property blocks are referenced by block properties which, in turn, are referenced in the application class containing the block properties:



To create the JSON files required to import these classes and properties:

Procedure

1. Create the individual property definitions.

Create the following JSON file and save it in a file named **blockPropDef.json** as follows:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyDefinitions": [
    {
      "ObjectType": "02",
      "Namespace": "TDOC",
      "ID": "PRP004",
      "Revision": "001",
      "Status": "Develop",
```

```

    "Name": "Material",
    "DataType": {
      "Type": "String"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP005",
    "Revision": "001",
    "Name": "Color",
    "Status": "Develop",
    "DataType": {
      "Type": "String"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP003",
    "Revision": "001",
    "Name": "Adjustable headrests",
    "Status": "Develop",
    "DataType": {
      "Type": "Boolean"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP006",
    "Revision": "001",
    "Name": "Electrically adjustable seats",
    "Status": "Develop",
    "DataType": {
      "Type": "Boolean"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP007",
    "Revision": "001",
    "Name": "Lumbar support",
    "Status": "Develop",
    "DataType": {
      "Type": "Boolean"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP008",
    "Revision": "001",
    "Name": "Heated seats",
    "Status": "Develop",

```

```

        "DataType": {
            "Type": "Boolean"
        }
    },
    {
        "ObjectType": "02",
        "Namespace": "TDOC",
        "ID": "PRP009",
        "Revision": "001",
        "Name": "Rear seat entertainment system",
        "Status": "Develop",
        "DataType": {
            "Type": "Boolean"
        }
    },
    {
        "ObjectType": "02",
        "Namespace": "TDOC",
        "ID": "PRP010",
        "Revision": "001",
        "Name": "Front seat",
        "Status": "Develop",
        "DataType": {
            "Type": "Reference",
            "BlockReference": "TDOC#01-CLS002#001"
        }
    },
    {
        "ObjectType": "02",
        "Namespace": "TDOC",
        "ID": "PRP011",
        "Revision": "001",
        "Name": "Back seat",
        "Status": "Develop",
        "DataType": {
            "Type": "Reference",
            "BlockReference": "TDOC#01-CLS003#001"
        }
    }
]
}

```

The last two properties, **PRP010** and **PRP011** reference the two property block classes that hold the sets of properties.

2. Create the class definitions, including the property block classes.

```

{
    "SchemaVersion": "1.0.0",
    "Locale": "en_US",
    "ClassDefinitions": [
        {
            "ObjectType": "01",
            "Namespace": "TDOC",

```



```

    "ID": "CLS002",
    "Revision": "001",
    "Name": "Front seat",
    "Status": "Develop",
    "IsAbstract": true,
    "UnitSystem": 3,
    "ClassType": "Block",
    "ClassAttributes": [
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP004#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP005#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP003#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP006#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP007#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP008#001"
      }
    ]
  },
  {
    "ObjectType": "01",
    "Namespace": "TDOC",
    "ID": "CLS003",
    "Revision": "001",
    "Name": "Back seat",
    "Status": "Develop",
    "IsAbstract": true,
    "UnitSystem": 3,
    "ClassType": "Block",
    "ClassAttributes": [
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP004#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP005#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP003#001"
      }
    ]
  }

```

```

    },
    {
      "Type": "Property",
      "Reference": "TDOC#02-PRP009#001"
    }
  ]
},
{
  "ObjectType": "01",
  "Namespace": "TDOC",
  "ID": "CLS004",
  "Revision": "001",
  "Name": "Car",
  "Status": "Develop",
  "IsAbstract": false,
  "UnitSystem": 3,
  "ClassType": "Application Class",
  "ClassAttributes": [
    {
      "Type": "Property",
      "Reference": "TDOC#02-PRP010#001"
    },
    {
      "Type": "Property",
      "Reference": "TDOC#02-PRP011#001"
    }
  ]
}
]
}

```

The first two classes, **CLS002** and **CLS003**, are the property block classes and the third class, **CLS004**, is the application class that references the two property blocks.

3. Import the node definition that makes the class visible in Active Workspace.

The **Block car** node must be displayed in the **Cars** node that you created in the first example. To do this, import the new node specifying the **Car** node, **CSTAA1**, as the parent. Save the following in a file named **blockNodeDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAA3",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA1",
        "Revision": "001"
      },
      "Name": "Block Car",
    }
  ]
}

```

```

"ApplicationClass": {
  "Namespace": "TDOC",
  "ID": "CLS004",
  "Revision": "001"
}
}
]
}

```

4. Import the definitions into the database using **Classification Manager** or with **clsutility** commands.

Results

The structure is displayed as follows:

ASSIGNED CLASSIFICATIONS

Automotive Engineering > Assemblies > Cars > Block Car

▼ **AVAILABLE CLASSES**

Filter

- Material Families
- Miscellaneous
- Resource Management
- Automotive Engineering
 - Assemblies
 - Cars
 - Simple Car
 - Block Car**

▼ **PROPERTY GROUPS**

keywords

Front seat

Back seat

PROPERTIES

keywords

Unit System: * ☒ Metric ☐ Non-Metric

▼ **Front seat**

Material:

Color:

☐ Adjustable headrests

☐ Electrically adjustable seats

☐ Lumbar support

☐ Heated seats

▼ **Back seat**

Material:

Color:

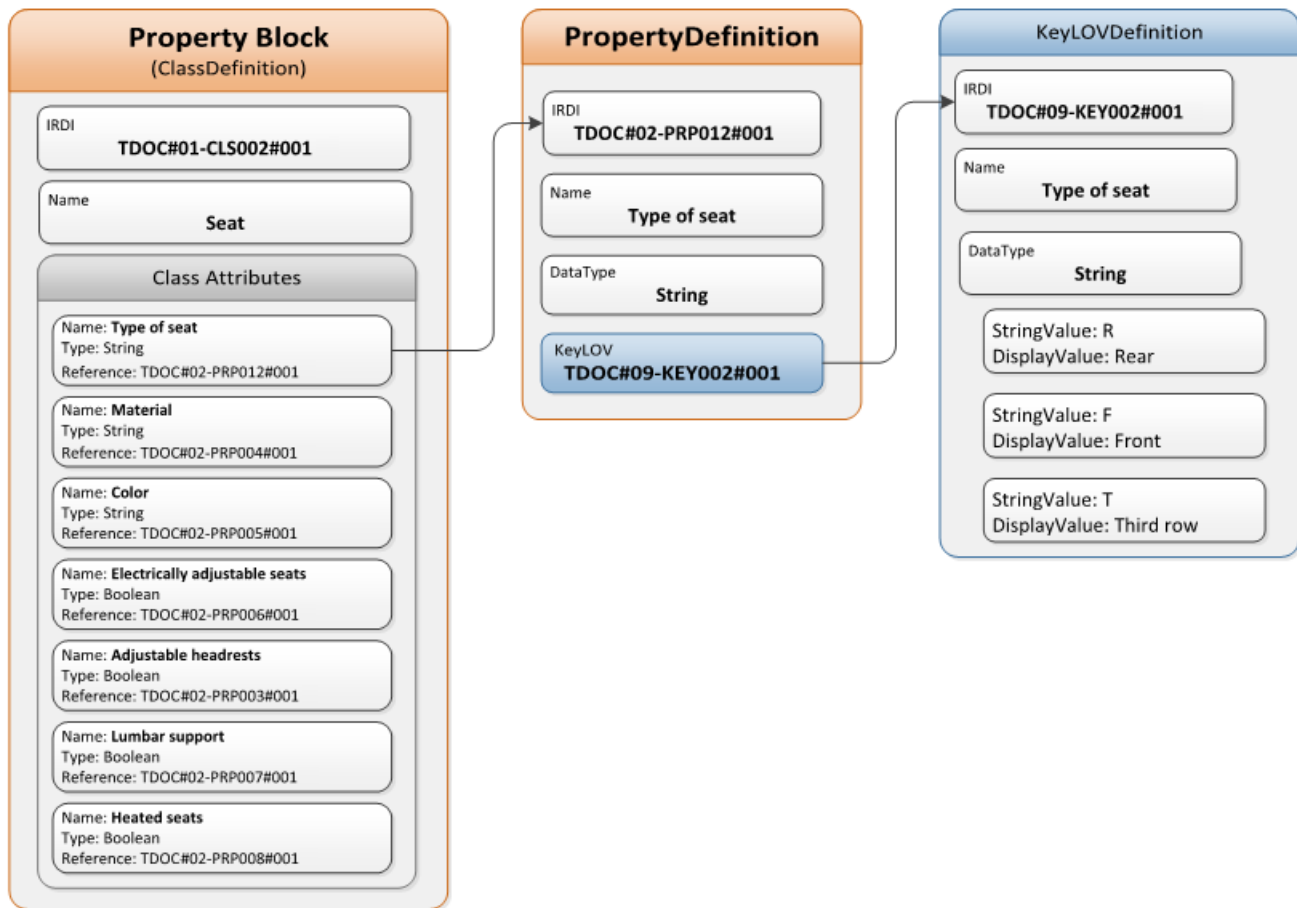
☐ Adjustable headrests

☐ Rear seat entertainment system

Import a class with cardinal properties

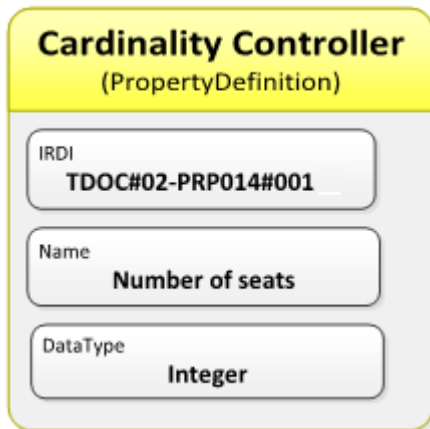
This example shows how to create a class where we can specify the number of seat rows in a car during classification. For example, when classifying a sports car, there is only one row of seats, so only one set of seat properties is necessary. When classifying a family van, there are three rows of seats that each need a set of properties to describe them.

We begin with a set of properties, a property block:

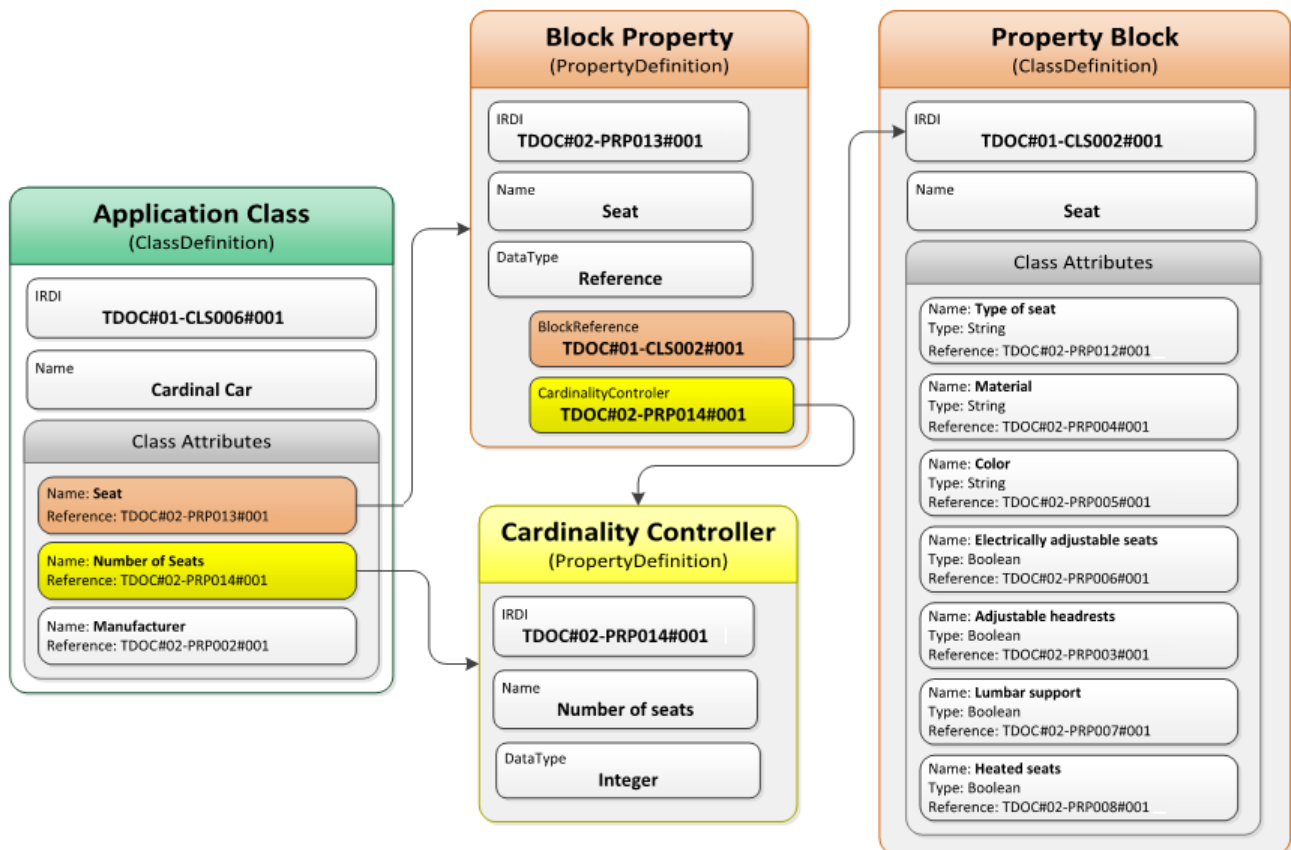


This property block class contains the same attributes as in the previous example, but additionally includes a new key-LOV property, **Type of seat**, where you can specify whether the seats are front, back, or third-row seats.

The number of times that this block of properties is displayed in the user interface is determined by a special property called a *cardinality controller*:



A cardinality controller is referenced by another property and is only of significance if set to **true**. If this is the case, then the user can specify the number of times that the set of properties is displayed.



To create the JSON files required to import these classes and properties:

Procedure

1. Create the key-LOV definitions.

This example introduces a new key-LOV, **Type of seat**, with three values, **Front**, **Rear**, and **Third**. Save the following in a file named **cardinalKeyLOVDef.json**

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "KeyLOVDefinitions": [
    {
      "ObjectType": "09",
      "Namespace": "TDOC",
      "ID": "KEY002",
      "Revision": "001",
      "Name": "Type of seat",
      "Status": "Develop",

      "LOVItems": {
        "DataType": "String",
        "LOVStringItems": [
          {
            "StringValue": "F",
            "DisplayValue": "Front"
          },
          {
            "StringValue": "R",
            "DisplayValue": "Rear"
          },
          {
            "StringValue": "T",
            "DisplayValue": "Third row"
          }
        ]
      }
    }
  ]
}
```

2. Create the individual property definitions.

There are three new properties in this example: **Type of seat** that references the new key-LOV, **Seat**, the property that specifies the property block class that is to be displayed and acts as an indicator that cardinality is involved, and the **Number of seats** that holds the count.

Save the following in a file named **cardinalPropDef.json**:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyDefinitions": [
    {
      "ObjectType": "02",
      "Namespace": "TDOC",
      "ID": "PRP012",
      "Revision": "001",
      "Name": "Type of seat",
```

```

    "Status": "Develop",
    "DataType": {
      "Type": "String",
      "KeyLOV": "TDOC#09-KEY002#001"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP013",
    "Revision": "001",
    "Name": "Seat",
    "Status": "Develop",
    "DataType": {
      "Type": "Reference",
      "BlockReference": "TDOC#01-CLS005#001",
      "CardinalityController": "TDOC#02-PRP014#001"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP014",
    "Revision": "001",
    "Name": "Number of seats",
    "Status": "Develop",
    "DataType": {
      "Type": "Integer"
    }
  }
]
}

```

3. Create the class definitions, including the property block classes.

This example introduces two new classes, **CLS005**, the property block class, and **CLS006**, the application class. Save the following in a file named **cardinalClassDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "ObjectType": "01",
      "Namespace": "TDOC",
      "ID": "CLS005",
      "Revision": "001",
      "Name": "Seat properties",
      "Status": "Develop",
      "IsAbstract": true,
      "UnitSystem": 3,
      "ClassType": "Block",
      "ClassAttributes": [
        {
          "Type": "Property",

```

```

        "Reference": "TDOC#02-PRP012#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP004#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP005#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP006#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP003#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP007#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP008#001"
      }
    ]
  },
  {
    "ObjectType": "01",
    "Namespace": "TDOC",
    "ID": "CLS006",
    "Revision": "001",
    "Name": "Cardinal car",
    "Status": "Develop",
    "IsAbstract": false,
    "UnitSystem": 3,
    "ClassType": "Application Class",
    "ClassAttributes": [
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP013#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP014#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP002#001"
      }
    ]
  }
]
}

```


4. Import the node definition that makes the class visible in.

The **Cardinal car** node must be displayed in the **Cars** node that we created in the first example. To do this, import the new node specifying the **Car** node, **CSTAA1**, as the parent. Save the following in a file named **cardinalNodeDef.json**.

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAA4",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA1",
        "Revision": "001"
      },
      "Name": "Cardinal car",
      "ApplicationClass": {
        "Namespace": "TDOC",
        "ID": "CLS006",
        "Revision": "001"
      }
    }
  ]
}
```

5. Import the definitions into the database using **Classification Manager** or with **clsutility** commands.

Results

The structure is displayed as follows:

▼ AVAILABLE CLASSES

Filter

↕

▶ Material Families

▶ Miscellaneous

▶ Resource Management

▼ Automotive Engineering

▼ Assemblies

▼ Cars

Simple Car

Block Car

Cardinal car

▼ PROPERTY GROUPS

keywords

▶ Seat

PROPERTIES

keywords

Unit System: *

MetricNon-Metric

▼ Seat

Number of seats:

2

1

▼ Seat 1

Type of seat:

F Front

Material:

Color:

☐ Electrically adjustable seats

☐ Adjustable headrests

☐ Lumbar support

☐ Heated seats

▶ Seat 2

Manufacturer:

▼ Seat 2

Type of seat:

F Front

R Rear

T Third row

☐ Electrically adjustable seats

☐ Adjustable headrests

☐ Lumbar support

☐ Heated seats

After specifying the number of seats in 1, that number of **Seat** property blocks is displayed 2. You can assign different values for each seat row. With the new key-LOV 3, you can specify a separate type of seat for each row.

Import a class with polymorphic properties

This example shows how to create a class where we can specify the properties displayed depending on the type of seat row in a car that you select during classification. The following table displays the different properties available for each seat row type:

Front seat	Rear seat
Material	Material
Color	Color
Adjustable headrests	Adjustable headrests
Electrically adjustable seats	Rear seat entertainment
Lumbar support	
Heated seats	

These are the same properties used in the [second example](#). The difference is that in this example, you can specify the seat type during classification and, depending on this selection, display different properties. The concept of displaying different properties depending on the value of an additional property is referred to as *polymorphism*.

ASSIGNED CLASSIFICATIONS

Automotive Engineering > Assemblies > Cars > Polymorphic car

AVAILABLE CLASSES

Filter

- Material Families
- Miscellaneous
- Resource Management
- Automotive Engineering
 - Assemblies
 - Cars
 - Simple Car
 - Block Car
 - Cardinal car
 - Polymorphic car**

PROPERTY GROUPS

keywords

Type of seat

PROPERTIES

keywords

Unit System: * ☒ Metric ☐ Non-Metric

Type of seat

Type of seat: * Front

Material:

Color:

☐ Electrically adjustable seats

☐ Adjustable headrests

☐ Lumbar support

☐ Heated seats

Manufacturer:

Type of seat

Type of seat: * Rear

Material:

Color:

☐ Adjustable headrests

☐ Rear seat entertainment system

This example contains two classes describing the front and rear seats:

Property Block (ClassDefinition)

IRDI

TDOC#01-CLS007#001

Name

Front Seat

Class Attributes

Name: **Type of Seat**
Type: String
Reference: TDOC#02-PRP0016#001

Name: **Material**
Type: String
Reference: TDOC#02-PRP004#001

Name: **Color**
Type: String
Reference: TDOC#02-PRP005#001

Name: **Adjustable headrests**
Type: Boolean
Reference: TDOC#02-PRP003#001

Name: **Electrically adjustable seats**
Type: Boolean
Reference: TDOC#02-PRP006#001

Name: **Lumbar support**
Type: Boolean
Reference: TDOC#02-PRP007#001

Name: **Heated seats**
Type: Boolean
Reference: TDOC#02-PRP008#001

Property Block (ClassDefinition)

IRDI

TDOC#01-CLS008#001

Name

Rear Seat

Class Attributes

Name: **Type of Seat**
Type: String
Reference: TDOC#02-PRP0016#001

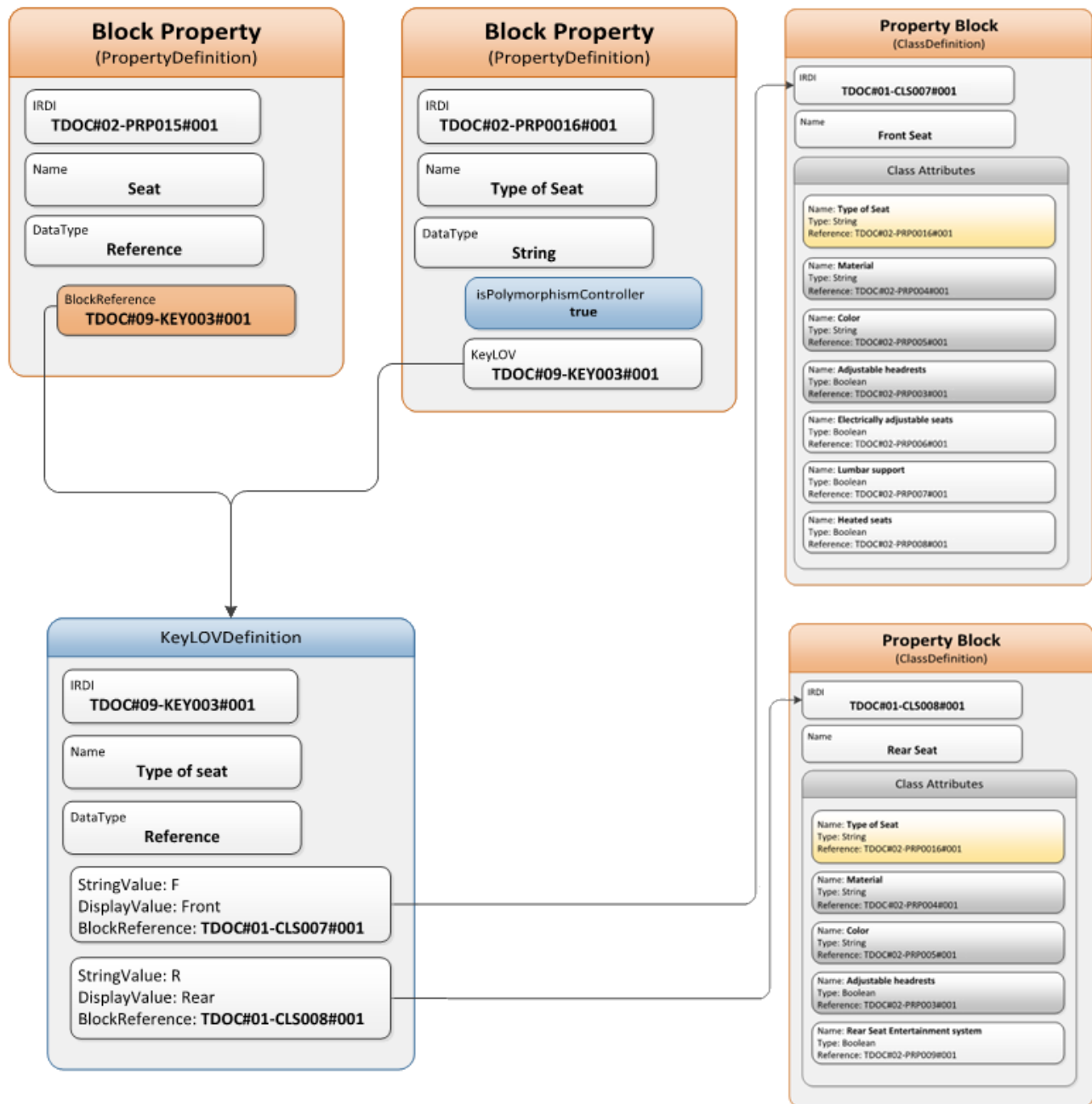
Name: **Material**
Type: String
Reference: TDOC#02-PRP004#001

Name: **Color**
Type: String
Reference: TDOC#02-PRP005#001

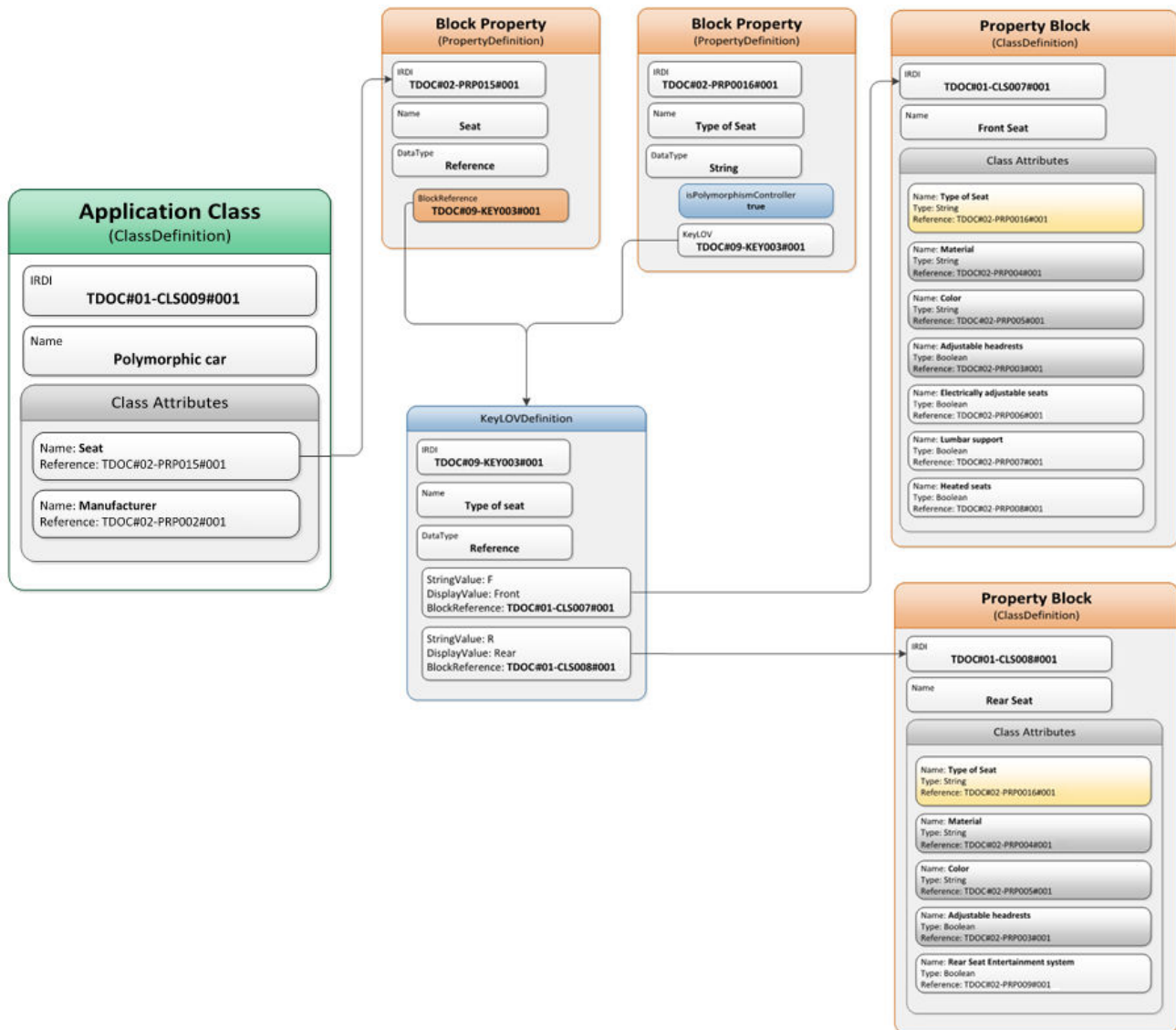
Name: **Adjustable headrests**
Type: Boolean
Reference: TDOC#02-PRP003#001

Name: **Rear Seat Entertainment system**
Type: Boolean
Reference: TDOC#02-PRP009#001

Each of the property block classes contains a special property that specifies the type of seat. This property references a key-LOV with the values **Front** or **Rear**.



The following application class contains the polymorphic **Seat** property that references the key-LOV.



Another property that references the key-LOV, **PRP016**, contains the **isPolymorphismController** attribute. When this property is used in a property block class (**CLS0007** or **CLS0008**), the **isPolymorphismController** attribute indicates that the property is polymorphic and that its value determines the list of class properties displayed in the user interface.

To create the JSON files for this example:

Procedure

1. Create the key-LOV definition.

This example introduces one new key-LOV that is saved in a file called **polymorphicKeylovDef.json**:

```
{
  "SchemaVersion": "1.0.0",
```

```

"Locale": "en_US",
"KeyLOVDefinitions": [
  {
    "ObjectType": "09",
    "Namespace": "TDOC",
    "ID": "KEY003",
    "Revision": "001",
    "Name": "Type of seat",
    "Status": "Develop",
    "LOVItems": {
      "DataType": "Reference",
      "LOVReferenceItems": [
        {
          "StringValue": "F",
          "DisplayValue": "Front",
          "BlockReference": "TDOC#01-CLS007#001"
        },
        {
          "StringValue": "R",
          "DisplayValue": "Rear",
          "BlockReference": "TDOC#01-CLS008#001"
        }
      ]
    }
  }
]
}

```

The values of the key-LOV, **Front** and **Rear**, reference their respective property block classes, **CLS007** and **CLS008**.

2. Create the individual property definitions.

This example uses many of the properties created in the earlier examples, as well as two new properties. Create the following JSON file and save it in a file named **polymorphicPropDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyDefinitions": [
    {
      "ObjectType": "02",
      "Namespace": "TDOC",
      "ID": "PRP015",
      "Revision": "001",
      "Name": "Seat",
      "Status": "Develop",
      "DataType": {
        "Type": "Reference",
        "BlockReference": "TDOC#09-KEY003#001"
      }
    },
    {
      "ObjectType": "02",

```

```

    "Namespace": "TDOC",
    "ID": "PRP016",
    "Revision": "001",
    "Name": "Type of Seat",
    "Status": "Develop",
    "DataType": {
      "Type": "String",
      "KeyLOV": "TDOC#09-KEY003#001",
      "IsPolymorphismController": true
    }
  }
]
}

```

PRP016 contains the **IsPolymorphismController** attribute.

3. Create the class definitions, including the property block classes.

This example introduces two new property block classes very similar to those in the block class example, but additionally containing the polymorphic **PRP016** property. Save this in a file named **polymorphicClassDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "ObjectType": "01",
      "Namespace": "TDOC",
      "ID": "CLS007",
      "Revision": "001",
      "Name": "Front seat",
      "Status": "Develop",
      "IsAbstract": true,
      "UnitSystem": 3,
      "ClassType": "Block",
      "ClassAttributes": [
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP016#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP004#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP005#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP006#001"
        }
      ]
    }
  ]
}

```



```

        "Type": "Property",
        "Reference": "TDOC#02-PRP003#001"
    },
    {
        "Type": "Property",
        "Reference": "TDOC#02-PRP007#001"
    },
    {
        "Type": "Property",
        "Reference": "TDOC#02-PRP008#001"
    }
]
},
{
    "ObjectType": "01",
    "Namespace": "TDOC",
    "ID": "CLS008",
    "Revision": "001",
    "Name": "Back seat",
    "Status": "Develop",
    "IsAbstract": true,
    "UnitSystem": 3,
    "ClassType": "Block",
    "ClassAttributes": [
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP016#001"
        },
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP004#001"
        },
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP005#001"
        },
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP003#001"
        },
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP009#001"
        }
    ]
},
{
    "ObjectType": "01",
    "Namespace": "TDOC",
    "ID": "CLS009",
    "Revision": "001",
    "Name": "Polymorphic car",
    "Status": "Develop",
    "IsAbstract": false,
    "UnitSystem": 3,
    "ClassType": "Application Class",

```

```

    "ClassAttributes": [
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP015#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP002#001"
      }
    ]
  }
]
}

```

The application class **CLS009** references **PRP015**, which is associated with the polymorphic attribute **PRP016** through the key-LOV.

4. Import the node definition that makes the class visible.

The **Polymorphic car** node must be displayed in the **Cars** node that we created in the first example. To do this, import the new node specifying the **Car** node **CSTAA1** as the parent. Save the following in a file named **polymorphicNodeDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAA5",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA1",
        "Revision": "001"
      },
      "Name": "Polymorphic car",
      "ApplicationClass": {
        "Namespace": "TDOC",
        "ID": "CLS009",
        "Revision": "001"
      }
    }
  ]
}

```

5. Import the definitions into the database using **Classification Manager** or with **clsutility commands**.

Import a class with polymorphic and cardinal properties

This example shows how to create a class where you can first specify the number of seat rows a car has during classification. For each seat row, the properties displayed depend on the type of seat row that you select.

ASSIGNED CLASSIFICATIONS  

Automotive Engineering > Assemblies > Cars > Polymorphic cardinal car

▼ **AVAILABLE CLASSES**

Filter 

- Material Families
- Miscellaneous
- Resource Management
- Automotive Engineering
 - Assemblies
 - Cars
 - Simple Car
 - Block Car
 - Cardinal car
 - Polymorphic car
 - Polymorphic cardinal car**

▼ **PROPERTY GROUPS**

keywords

►  **Seat**

PROPERTIES

keywords

Unit System: *  Metric  Non-Metric

▼  **Seat**

Number of seats: 

▼  **Seat 1**

Type of seat: *

Material:

Color:

☐ Electrically adjustable seats

☐ Adjustable headrests

☐ Lumbar support

☐ Heated seats

►  **Seat 2**

Manufacturer:

▼  **Seat 2**

Type of seat: *

Material:

Color:

☐ Adjustable headrests

☐ Rear seat entertainment system

This example uses the same two property block classes as the [previous example](#), describing the front and rear seats:

Property Block (ClassDefinition)

IRDI

TDOC#01-CLS007#001

Name

Front Seat

Class Attributes

Name: **Type of Seat**
Type: String
Reference: TDOC#02-PRP0016#001

Name: **Material**
Type: String
Reference: TDOC#02-PRP004#001

Name: **Color**
Type: String
Reference: TDOC#02-PRP005#001

Name: **Adjustable headrests**
Type: Boolean
Reference: TDOC#02-PRP003#001

Name: **Electrically adjustable seats**
Type: Boolean
Reference: TDOC#02-PRP006#001

Name: **Lumbar support**
Type: Boolean
Reference: TDOC#02-PRP007#001

Name: **Heated seats**
Type: Boolean
Reference: TDOC#02-PRP008#001

Property Block (ClassDefinition)

IRDI

TDOC#01-CLS008#001

Name

Rear Seat

Class Attributes

Name: **Type of Seat**
Type: String
Reference: TDOC#02-PRP0016#001

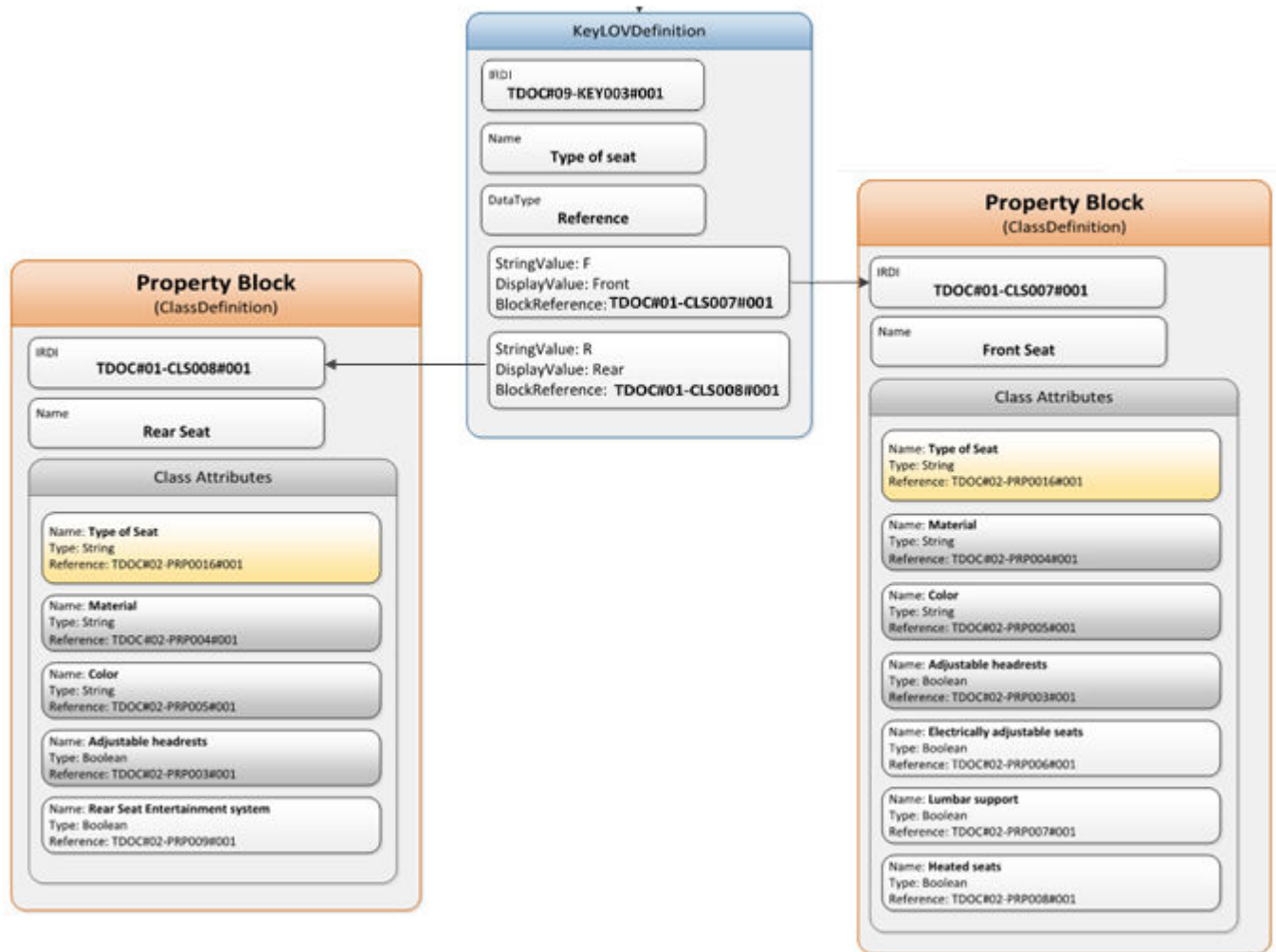
Name: **Material**
Type: String
Reference: TDOC#02-PRP004#001

Name: **Color**
Type: String
Reference: TDOC#02-PRP005#001

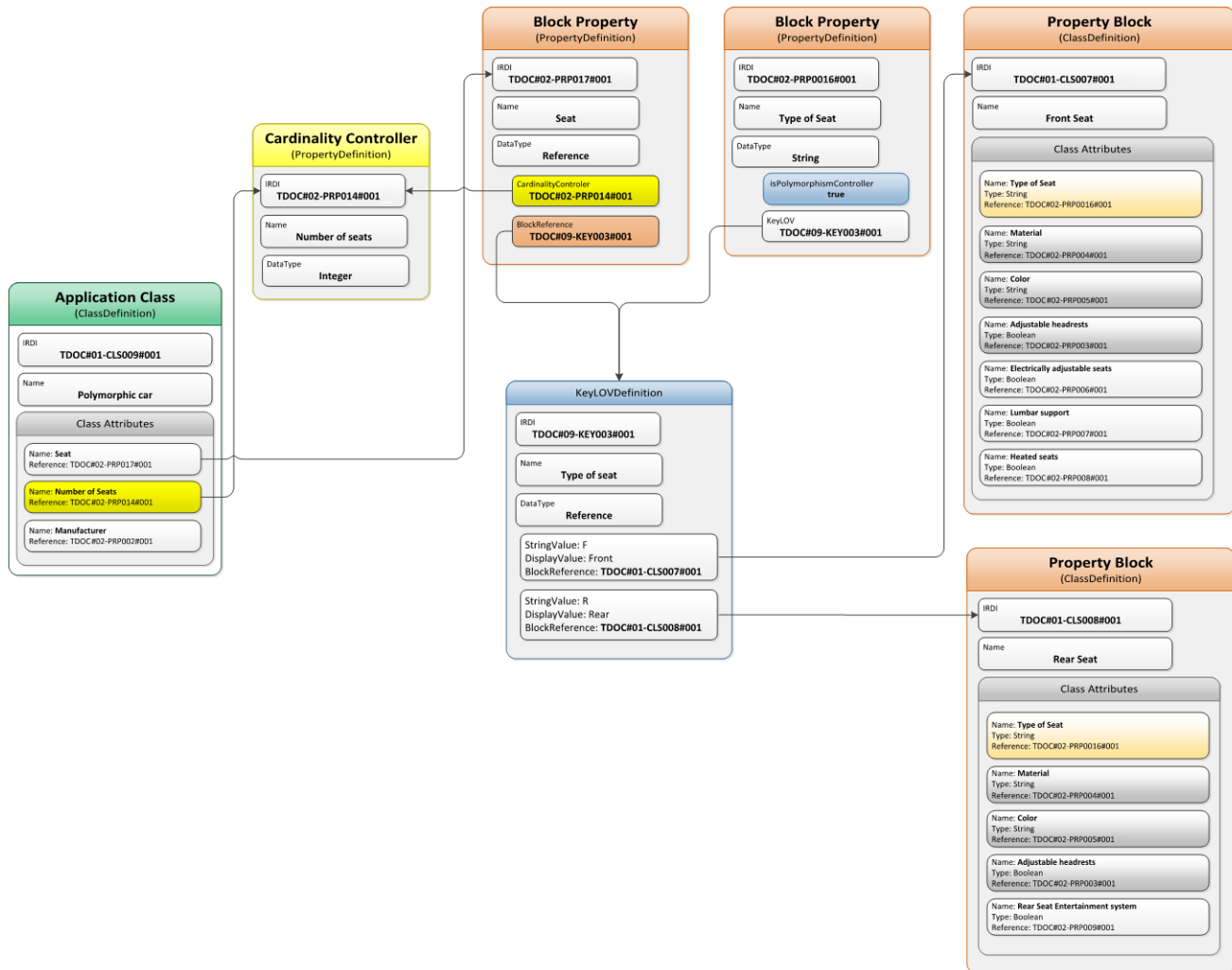
Name: **Adjustable headrests**
Type: Boolean
Reference: TDOC#02-PRP003#001

Name: **Rear Seat Entertainment system**
Type: Boolean
Reference: TDOC#02-PRP009#001

Again, these classes are referenced by the same key-LOV as in the previous example, which determines which of these classes is displayed:



The application class references the properties that determine the polymorphism and cardinality:



After completing the previous examples, there are only a few new objects required to import this example. To create the JSON files for this example:

Procedure

1. Create the individual property definitions.

This example uses **PRP014** created in the cardinality example and **PRP016** created in the polymorphism example. One new property is required that references the key-LOV created in the polymorphism example. Create the following JSON file and save it in a file named **poly_cardPropDef.json**.

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyDefinitions": [
    {
      "ObjectType": "02",
      "Namespace": "TDOC",
```

```

    "ID": "PRP017",
    "Revision": "001",
    "Name": "Seat",
    "Status": "Develop",
    "DataType": {
      "Type": "Reference",
      "BlockReference": "TDOC#09-KEY003#001",
      "CardinalityController": "TDOC#02-PRP014#001"
    }
  }
]
}

```

This property references the key-LOV definition, as well as the cardinality controller property. The polymorphism is achieved by the reference of **PRP016** to the key-LOV.

2. Create the class definitions.

This example introduces only one new application class. Save this in a file named **poly_cardClassDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "ObjectType": "01",
      "Namespace": "TDOC",
      "ID": "CLS010",
      "Revision": "001",
      "Name": "Polymorphic cardinal car",
      "Status": "Develop",
      "IsAbstract": false,
      "UnitSystem": 3,
      "ClassType": "Application Class",
      "ClassAttributes": [
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP014#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP017#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP002#001"
        }
      ]
    }
  ]
}

```

3. Import the node definition that makes the class visible.

The **Polymorphic cardinal car** node must be displayed in the **Cars** node that we created in the first example. To do this, import the new node specifying the **Car** node **CSTAA1** as the parent. Save the following in a file named **poly_cardNodeDef.json**.

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAA6",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA1",
        "Revision": "001"
      },
      "Name": "Polymorphic cardinal car",
      "ApplicationClass": {
        "Namespace": "TDOC",
        "ID": "CLS010",
        "Revision": "001"
      }
    }
  ]
}
```

4. Import the definitions into the database using **Classification Manager** or with **clsutility** commands.

Importing classification objects to CST

Classification objects can be imported using the JSON format. The schema file for this is:

..\TD\classification\json\schema\advanced\Classification_Save_PropertyRecords_Request_advanced.schema.json

When creating the JSON file, you can specify whether only the item for the property record is created, or additionally specify a class into which the item is immediately classified during import. Importing the following example classifies an item into the **Polymorphic cardinal car** class that was created in the **import example**.

Tip

Find the example as text that you can copy [here](#).


```

1  {
2      "SchemaVersion": "1.0.0",
3      "Locale": "en_US",
4      "PropertyRecords": [
5          {
6              "ID": "029159/A",
7              "ObjectType": "PR",
8              "ClassDefinition": "TDOC#01-CLS010#001",
9              "UnitSystem": 1,
10             "ClassifiedObject": {
11                 "ObjectType": "Item",
12                 "ClassifyRevision": true,
13                 "ID": [
14                     {
15                         "PropertyName": "item_id",
16                         "PropertyValue": "029159"
17                     }
18                 ],
19                 "Properties": [
20                     {
21                         "PropertyName": "object_name",
22                         "PropertyValue": "my_car"
23                     }
24                 ]
25             },
26             "Properties": [
27                 {
28                     "ID": "TDOC#02-PRP014#001",
29                     "Name": "Number of seats",
30                     "Value": 2
31                 },
32                 {
33                     "ID": "TDOC#02-PRP017#001",
34                     "Name": "Seat",
35                     "Value": [
36                         {
37                             "Index": 1,
38                             "ClassDefinition": "TDOC#01-CLS008#001",
39                             "Properties": [
40                                 {
41                                     "ID": "TDOC#02-PRP004#001",
42                                     "Name": "Material",
43                                     "Value": "Polyester"
44                                 },
45                                 {
46                                     "ID": "TDOC#02-PRP005#001",
47                                     "Name": "Color",
48                                     "Value": "Black"
49                                 },
50                                 {
51                                     "ID": "TDOC#02-PRP003#001",
52                                     "Name": "Adjustable headrests",
53                                     "Value": true
54                                 },
55                                 {
56                                     "ID": "TDOC#02-PRP009#001",
57                                     "Name": "Rear seat entertainment system",
58                                     "Value": true
59                                 }
60                             ]
61                         }
62                     ]
63                 }
64             ],
65             "ID": "TDOC#02-PRP002#001",
66             "Name": "Manufacturer",
67             "Value": "Fancy Car Company"
68         }
69     ]
70 }

```

In the JSON file:

- Lines 10-25 create the classified item with ID **029159** and name **my_car**.
- Line 30 creates 2 instances (a cardinality of 2) of **Number of seats** and these seats are described in lines 36-99.
- The index shown in line 37 and 73 introduces each of the new seat instances.
- Lines 39 and 75 begin the list of properties for each type of seat.
- The list of properties specific to each seat type is described in lines 40-71 and lines

The **Name** property is not mandatory and is added to this example for purposes of clarity.

To import this property record that is saved in a file called **poly_car_prop_rec.json**, you must use the **clsutility**:

```
clsutility -create -classification_objects -request="poly_car_prop_rec.json"
```

JSON classification object input example

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyRecords": [
    {
      "ID": "029159/A",
      "ObjectType": "PR",
      "ClassDefinition": "TDOC#01-CLS010#001",
      "UnitSystem": 1,
      "ClassifiedObject": {
        "ObjectType": "Item",
        "ClassifyRevision": true,
        "ID": [
          {
            "PropertyName": "item_id",
            "PropertyValue": "029159"
          }
        ],
        "Properties": [
          {
            "PropertyName": "object_name",
            "PropertyValue": "my_car"
          }
        ]
      },
      "Properties": [
        {
          "ID": "TDOC#02-PRP014#001",
          "Name": "Number of seats",
          "Value": 2
        },
        {
          "ID": "TDOC#02-PRP017#001",
          "Name": "Seat",
          "Value": [
            {
              "Index": 1,
              "ClassDefinition": "TDOC#01-CLS007#001",
              "Properties": [
                {
                  "ID": "TDOC#02-PRP004#001",
                  "Name": "Material",
                  "Value": "Leather"
                }
              ],
              {
                "ID": "TDOC#02-PRP005#001",
                "Name": "Color",
                "Value": "Red"
              }
            ],
            {
              "ID": "TDOC#02-PRP006#001",
              "Name": "Electrically adjustable seats",
              "Value": true
            }
          ]
        }
      ]
    }
  ]
}

```

```

    },
    {
      "ID": "TDOC#02-PRP003#001",
      "Name": "Adjustable headrests",
      "Value": true
    },
    {
      "ID": "TDOC#02-PRP007#001",
      "Name": "Lumbar support",
      "Value": true
    },
    {
      "ID": "TDOC#02-PRP008#001",
      "Name": "Heated seats",
      "Value": true
    }
  ]
},
{
  "Index": 2,
  "ClassDefinition": "TDOC#01-CLS008#001",
  "Properties": [
    {
      "ID": "TDOC#02-PRP004#001",
      "Name": "Material",
      "Value": "Polyester"
    },
    {
      "ID": "TDOC#02-PRP005#001",
      "Name": "Color",
      "Value": "Black"
    },
    {
      "ID": "TDOC#02-PRP003#001",
      "Name": "Adjustable headrests",
      "Value": true
    },
    {
      "ID": "TDOC#02-PRP009#001",
      "Name": "Rear seat entertainment",
      "Value": true
    }
  ]
}
]
},
{
  "ID": "TDOC#02-PRP002#001",
  "Name": "Manufacturer",
  "Value": "Fancy Car Company"
}
]
}
system",
}

```

```
} ]
```

clsutility commands required to import example json files

When importing files for [this example](#), if you choose to import using the **clsutility**, run each of the following commands in a Teamcenter command window:

Import a class with simple properties

```
clsutility -u=user -p=password -g=group -create -keylov_definitions
-request="simpleKeylovDef.json"
clsutility -u=user -p=password -g=group -create -property_definitions
-request="simplePropDef.json"
clsutility -u=user -p=password -g=group -create -class_definitions
-request="simpleApplicationClassDef.json"
clsutility -u=user -p=password -g=group -create -node_definitions
-request="simpleNodeDef.json"
```

Import a class with a block properties

```
clsutility -u=user -p=password -g=group -create -property_definitions
-request="blockPropDef.json"
clsutility -u=user -p=password -g=group -create -class_definitions
-request="blockClassDef.json"
clsutility -u=user -p=password -g=group -create -node_definitions
-request="blockNodeDef.json"
```

Import a class with cardinal properties

```
clsutility -u=user -p=password -g=group -create -keylov_definitions
-request="cardinalKeyLOVDef.json"
clsutility -u=user -p=password -g=group -create -property_definitions
-request="cardinalPropDef.json"
clsutility -u=user -p=password -g=group -create -class_definitions
-request="cardinalClassDef.json"
clsutility -u=user -p=password -g=group -create -node_definitions
-request="cardinalNodeDef.json"
```

Import a class with polymorphic properties

```
clsutility -u=user -p=password -g=group -create -keylov_definitions
-request="polymorphicKeylovDef.json"
clsutility -u=user -p=password -g=group -create -property_definitions
-request="polymorphicPropDef.json"
clsutility -u=user -p=password -g=group -create -class_definitions
-request="polymorphicClassDef.json"
clsutility -u=user -p=password -g=group -create -node_definitions
-request="polymorphicNodeDef.json"
```

Import a class with polymorphic and cardinal properties

```
clsutility -u=user -p=password -g=group -create -property_definitions
-request="poly_cardPropDef.json"
clsutility -u=user -p=password -g=group -create -class_definitions
-request="poly_cardClassDef.json"
clsutility -u=user -p=password -g=group -create -node_definitions
-request="poly_cardNodeDef.json"
```

Import a property record

```
clsutility -create -classification_objects -request="poly_car_prop_rec.json"
```

Export classification data in JSON format

You can export classification data in JSON format from the user interface.

Restrictions and limitations

This feature only exports the selected node or class. Any dependent data and groups are not included in the export.

Procedure

1. Open the **Active Admin** workspace.
2. From **Launcher**, search for **Classification Manager**.
If this option is unavailable, search for it. You can pin it from the search results for future access.
3. Select either a node, class, property, or key LOV, and click **Export** .
4. From the **Export** panel, select **Export Dependent Object** to include all dependent objects along with the selected object.

Tip

As a best practice, export Nodes without the dependent data.

5. Click **Export** to start the export.

Results

A notification appears in **Alerts** when the export completes successfully.

The file is available for download in the **Target Object** section of the alert report.

Export classified data for advanced classification BMEcat XML format

You can export classified data in the BMEcat XML from the user interface.

Procedure

1. Open the classified data that you want to export.
2. Choose **More Commands *** > Import/Export**  **> Export Classification Data**.
3. In the **File Name** box, give a name to the file that will be downloaded.
4. Click **Export**.

Results

When the export is complete, you are notified in the **Alert** area where you can view the details of the export.

The file is available in the **Target Object** section of the alert report. Download the file from that section.

This feature requires that Subscription Manager is running. If the Subscription Manager is not running, no notification is displayed in the **Alert** area, but the operation still occurs in the background.

Export classified objects using the clsutility command

To export the classification information (the *ICO*) for an object classified in a CST hierarchy, use the following **clsutility** command:

```
clsutility -u=<user> -p=<password> -g=<group> -find -classification_objects -request=<JSON_file> -output=<path-to-output-file>
```

This command requires a JSON file resembling the following (see the [Classification_FindRequest_advanced.schema.json](#) schema file):

```
{
  "SchemaVersion": "1.0.0",
  "Locale" : "en_US",
  "ObjectIDList": [
    {
      "ID": "026144/A"
    }
  ]
}
```

The input JSON file contains a list of object IDs for which the ICO information is exported to a file of your choice.

Organizing classes with hierarchical positions in ECLASS

The ECLASS standard uses the concept of a *class code* or *coded name* to uniquely identify classes and designate the order in which classes and subclasses are displayed. The coded name consists of a group of four two-digit numbers, separated by a hyphen. Each of these numbers represents a position in the hierarchy.

The ECLASS standard assumes a four-level hierarchy, with the fourth level always consisting of leaf classes. Properties are assigned to the fourth level only. If this is the structure of your classification hierarchy, then you can use the coded name, or the *hierarchical position* to sort the classes in the classification hierarchy. The hierarchical position is then honored by Teamcenter at import of the JSON file. According to the ECLASS standard (see the section on *Blocked Class Codes* in the ECLASS documentation), you can use the number 98 and 99 to embed custom classes into the ECLASS hierarchy.

Caution

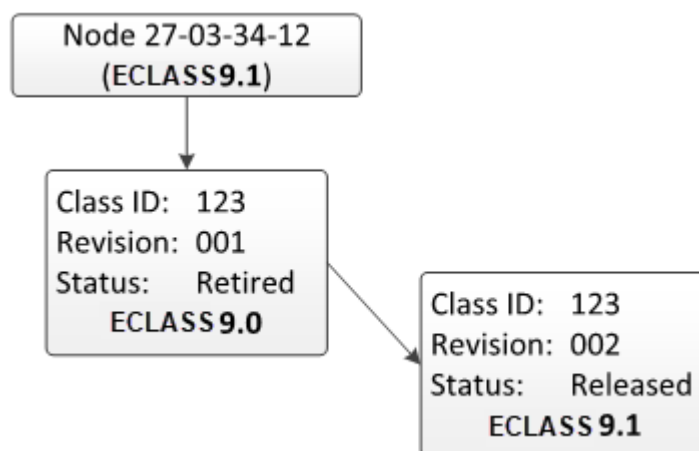
If your hierarchy does not follow this four-level pattern, do not use the **Hierarchical position** option in your data definitions.

Managing multiple ECLASS releases

The ECLASS hierarchy is available in multiple releases. You can import and access in Teamcenter where you can classify objects in different releases of the classification hierarchy simultaneously.

To facilitate having multiple releases of a hierarchy, the concept of a *revision chain* is used. The following scenario takes place during the import of ECLASS releases.

You import the ECLASS 9.0. Later, you decide to update to the ECLASS 9.1 release. Because you cannot have two classes with a **Released** status at the same time, the original class referenced by **Node 27-03-34-12** is set to **Retired** when you [set the new class to Released](#). Even though the new 9.1 **Node 27-03-34-12** still points to **Revision 001** of the class, internally, the node now references **Revision 002** of the class.



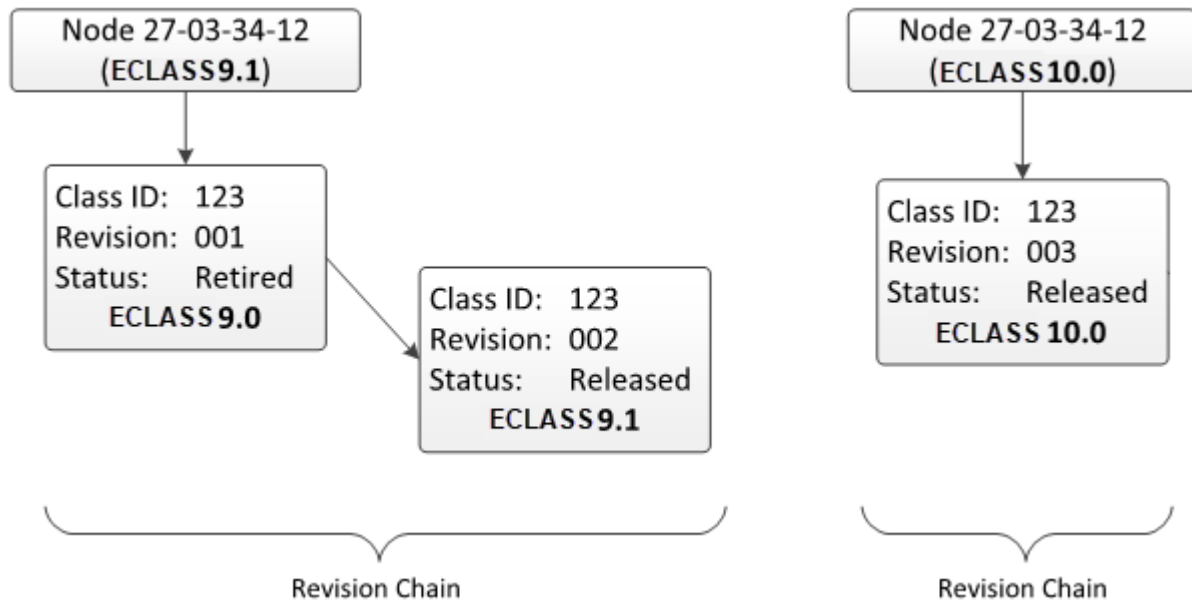
Now your company wants to use the new ECLASS 10.0 release. You still, however, want to classify in (or at least see) the 9.1 hierarchy at the same time. To do this, add the **AddAsNewRelease** option to the JSON files that you import (requires at least schema version 1.2.0):

```
1 {  
2   "SchemaVersion": "1.2.0",  
3   "Locale": "en US",  
4   "AddAsNewRelease": "true",  
5   "NodeDefinitions": [  
    ]  
}
```

Tip

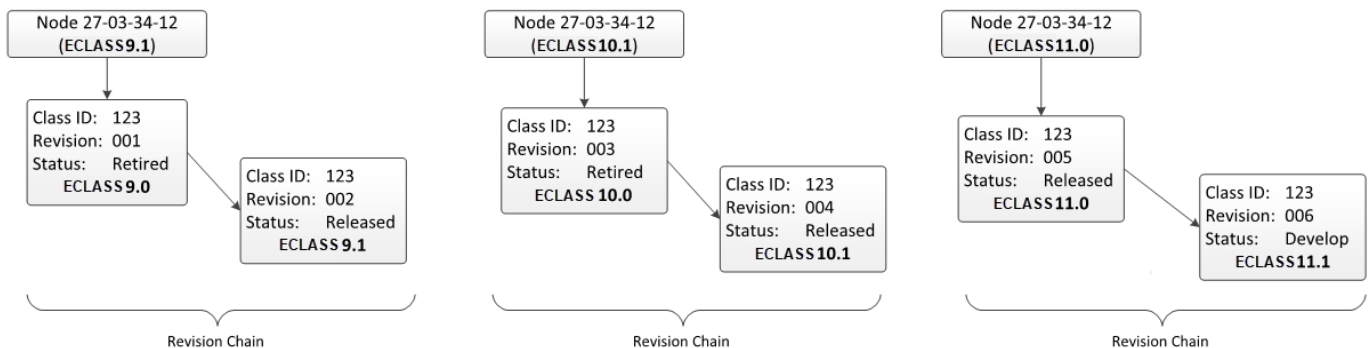
You can add this to a JSON definition when you [convert the hierarchy definitions from ontoML with the eclass2json utility](#).

Importing a JSON definition with the **AddAsNewRelease** option set to **true** creates a new revision chain.

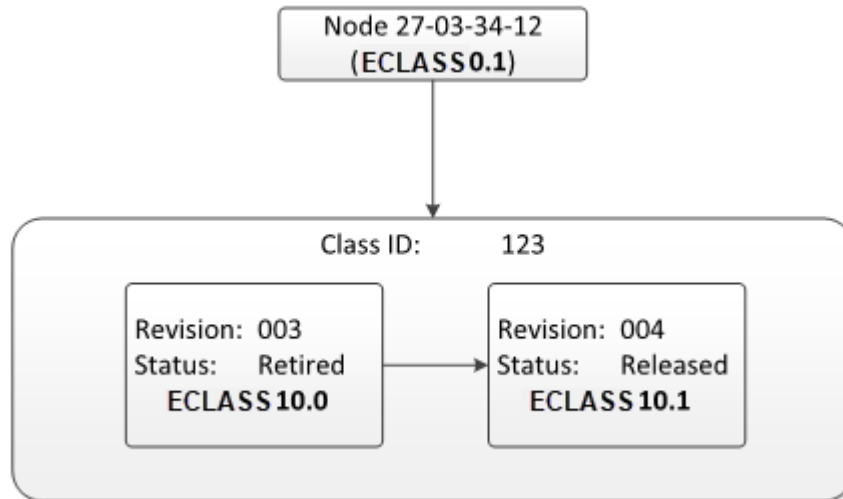


There are now two revisions of a class in the **Released** state but they are in two different revision chains so that users can classify in both 9.1 and 10.0 classes simultaneously.

When you continue upgrading and importing new ECLASS releases, the revision chains may look like this:



Note Revision 006 of the class that was imported with the ECLASS 11.1 release is still in **Develop** state (you haven't yet set it to **Released** with the **clsutility**). Therefore, the node still references **Revision 005** of the class. In other words, a node always references the most recent (in terms of revision number) class in a **Released** state. That class is used for classification.



The fact that a different revision than the one referenced by the node is found when classifying objects is referred to as *imprecise* handling.

To differentiate releases of the hierarchy when working in multiple releases, the schema file (version 1.2.0) contains an optional **SourceStandard** entry. This option in the JSON file is used to specify the hierarchy version for an administrative object. The **SourceStandard** value is mapped to a display name for the release in the **CST_supported_eclass_releases** site preference. The preference values are set in pairs. Examples of possible values are:

```

0173-1#11-ECLASS9#001
ECLASS 9.0
0173-1#11-ECLASS91#001
ECLASS 9.1
0173-1#11-ECLASS10.0.1#001
ECLASS 10.0.1
0173-1#11-ECLASS10.1#001
ECLASS 10.1
0173-1#11-ECLASS11#001
ECLASS 11.0
0173-1#11-ECLASS11.1#001
ECLASS 11.1
  
```

The first entry represents the IRDI that must be added to the **SourceStandard** entry in the JSON file describing the admin objects. The second entry is the display name shown in parentheses in the classification hierarchy.

Classification Manager [Dashboard](#) [Nodes](#) [Classes](#) [Properties](#) [Key LOV](#)

12 results found for "Nodes"

Tree with Summary
 Filters
 Import

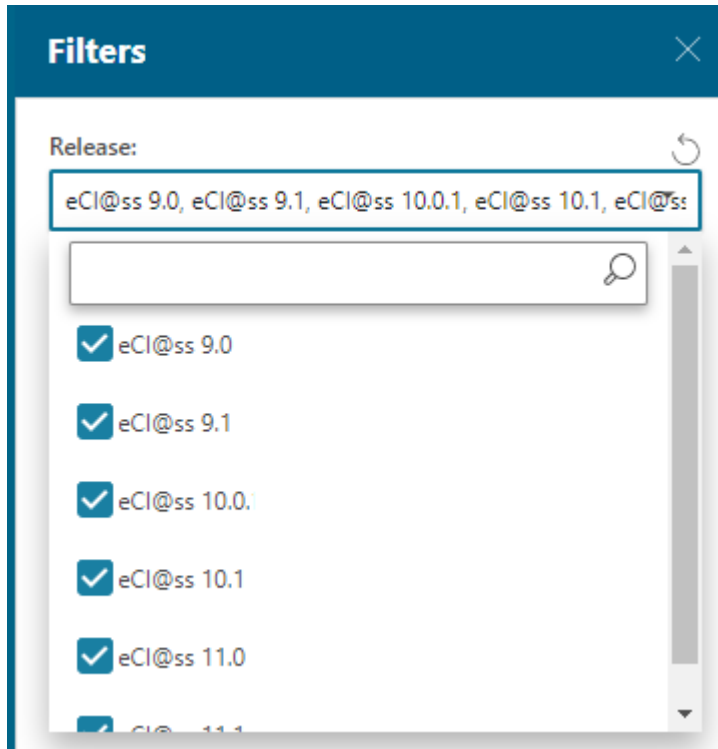
Name
▶ [27-04-02] Current inverter
▶ [27-04-03] Rectifier
▶ [27-04-05] Stand-by unit
▶ [27-04-06] No-break power supply (complete)
▼ [27-04-07] Power supply device
[27-04-07-01] Continuous current supply (eCI@ss 9.0)
[27-04-07-02] Voltage stabilizer (eCI@ss 9.1)
[27-04-07-03] Alternating current supply (eCI@ss 10.0.1)
[27-04-07-04] AC-DC supply (eC@ass 10.1)

[Overview](#) [Application Class](#)

▼ **PROPERTIES**

Node ID:	0173-1#AFX041#002
Namespace:	0173-1
ID:	AFX041
Revision:	001
Minor Revision:	
Is Deprecated:	
Name:	[27-04-07-02] Voltage stabilizer
Short Name:	
Definition:	Device for automatically stabilization tput voltage
Source Standard:	0173-1#11-ECLASS91#001
Note:	
Remark:	
Hierarchical Position:	27-04-07-02

If the **CST_supported_eclass_releases** preference contains any IRDI and display name pairs, a **Release** filter is available showing the display names in the preference. You specify which release is visible in this filter:



By default, all releases are selected. If you select one release only, no release is displayed in the classification hierarchy.

When classifying in a system with multiple hierarchies, filters can be set to specify which releases of the hierarchy to display.

Although this mechanism is intended for use with multiple ECLASS hierarchies, it can be used for any type of hierarchy revisions your company may work with. Specify your own **SourceStandard**/display name pairs for the preference value and add your source standard value to the JSON hierarchy objects.

Converting between XML and JSON file format

Converting files between ECLASS XML formats and JSON

ECLASS data is generally delivered in two formats:

- ontoML XML

Hierarchy (segment) data downloaded from the ECLASS website is delivered in this format.

- BMEcat XML

Classification objects that are downloaded from the Siemens Industry Mall website are delivered in this format.

For more information, see <https://eclass.eu/support>.

There are several scenarios when importing and exporting classification data that involve converting files from XML formats to JSON and from JSON to BMEcat XML.

Scenario 1: You want to import hierarchy definitions into the database:

1. Download ECLASS hierarchy data (ontoML XML) from the ECLASS website.
This data contains hierarchy and class definitions only.
2. Convert this data into JSON using the **eclass2json** utility.
3. Import the JSON files into the database.
4. Create objects and classify them using the new classification classes.

Scenario 2: You want to import classified product data in BMEcat XML format into the database:

1. Convert the BMEcat files to JSON format using the **eclass2json** utility.
2. Import the JSON files into your database.

Scenario 3: You have classified data in your classification hierarchy that you want to share with an external partner who also uses ECLASS:

1. Export the classification object (ICO) using **clsutility**.
2. Convert the ICO from JSON to BMEcat XML using the **eclass2json** utility. The BMEcat files include references to any attachments.

Convert hierarchy definitions to JSON format

To import ECLASS hierarchy definitions into Teamcenter, they must be converted into the JSON file format. Use the **eclass2json** utility to perform these conversions.

The utility functions by outputting temporary files that it then references to create the JSON or BMEcat files.

For this reason, the correct creation of the JSON files requires running the utility to output objects individually in the following sequence:

Sequence	Object	Utility argument	Comment
1	Units	unitsmap	Convert these once per ECLASS release.
2	Hash maps (temporary files)	hashmaps	Convert these once per ECLASS segment.
3	Definitions	definitions	Convert these as required. If necessary, you can convert individual object types using the

Sequence	Object	Utility argument	Comment
			following key words (and in the following order): keylovs properties aspects blocks applicationClasses classificationClasses

The utility requires a specific directory structure that it uses to find its input, as well as to store the completed JSON files. As input, this directory must contain the extracted XML files to be converted. During processing, the utility creates subdirectories for both the temporary hash map files and the newly created JSON files.

Procedure

1. Obtain ontoML XML hierarchy data from the ECLASS website and extract the contents into a directory.
2. Open a command window and navigate to the directory containing the units file, for example, **eClass9_1_UnitsML_EN_DE**.
3. Call the **eclass2json** Perl script to create the unit hashmap file. The utility requires the unit XML file for input, for example:

```
%TC_PERL% %TC_BIN%\eclass2json\eclass2json.pl
-inputFile:eClass9_1_UnitsML_EN_DE.xml -objectType:unitsmap
```

The utility creates the units hashmap text file in a subdirectory of the **e2j_output** directory.

4. Navigate to the directory containing the dictionary objects, for example, **eClass9_1_Dictionary_ADVANCED_XML_EN**.
5. Call the **eclass2json** Perl script to create the hashmap file. The utility requires the dictionary XML file for input, for example:

```
%TC_PERL% %TC_BIN%\eclass2json\eclass2json.pl
-inputFile:eClass9_1_ADVANCED_EN_SG_13.xml -objectType:hashmaps
```

The utility creates the hashmap text files in the **e2j_output** directory.

- In the same directory, using the same input file, convert the definitions:

```
%TC_PERL% %TC_BIN%\eclass2json\eclass2json.pl
-inputFile:eClass9_1_ADVANCED_EN_SG_13.xml -objectType:definitions
```

The **e2j_output** directory now contains a subdirectory with the converted JSON files.

- Import the newly created JSON files into the database as described in [Importing JSON files using the clsutility command](#).

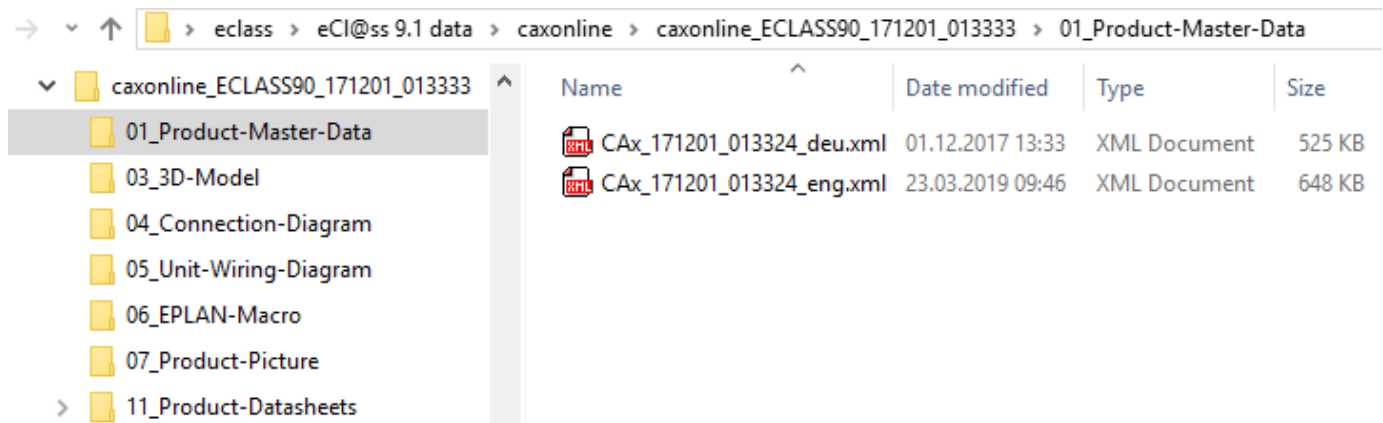
Results

The class hierarchy is now imported and ready for you to classify objects or [import classified objects](#).

Convert BMEcat format to JSON

Many companies use the ECLASS standard to classify their product data. This data is delivered s CAx data in the BMEcat XML file format. The files of interest for the **eclass2json** conversion utility are found in the **01_Product-Master-Data** directory of the download package.

The following describes manually converting BMEcat XML files after which you can import them using the [clsutility](#). Alternatively, you can [import these files automatically in the user interface](#).



During conversion, the **eclass2json** utility creates a classification object for each **PRODUCT** in the XML file **1**. The **FEATURES** are used to create properties. The **MANUFACTURER_PID** is used to create the classification object ID **2**.

BMEcat XML file:

Dr:\Workdir\eclass\ecf@ss 9.1\data\caxonline\caxonline_ECLASS90_171201_013333\01_Product-Ma...CAx_171201_013324_eng.xml

XML	
BMECAT	
version	2005
HEADER	
T_NEW_CATALOG	
PRODUCT	1
SUPPLIER_PID	6ES7142-5AF00-0BA0
PRODUCT_DETAILS	
DESCRIPTION_SHORT	ET 200AL, DQ 8X24VDC/2A, 8XM12
INTERNATIONAL_PID	type=gtin
MANUFACTURER_PID	6ES7142-5AF00-0BA0
MANUFACTURER_NAME	Siemens AG
PRODUCT_FEATURES	
REFERENCE_FEATURE_SYSTEM	ECLASS-9.1
REFERENCE_FEATURE_GROUP	27242604
REFERENCE_FEATURE_GROUP_ID2	type=flat
FEATURE	(39)
FEATURE_GROUP	
FEATURE	(8)
FEATURE_GROUP	(2)

JSON file:

D:\Workdir\eclass\eclass_9.1_data\caxonline\caxonline_ECLASS90_171201_013333\json\CAx_171201_013324_data.json *

SchemaVersion	1.0.0																																
Locale	en_US																																
PropertyRecords	<table> <tr> <td>ID</td><td>6ES7142-5AF00-0BA0/A</td></tr> <tr> <td>ObjectType</td><td>PR</td></tr> <tr> <td>HierarchicalPosition</td><td>27-24-26-04</td></tr> <tr> <td>ClassifiedObject</td><td> <table> <tr> <td>ObjectType</td><td>Item</td></tr> <tr> <td>ClassifyRevision</td><td>true</td></tr> <tr> <td>ID (1)</td><td> <table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 item_id</td><td>6ES7142-5AF00-0BA0</td></tr> </table> </td></tr> <tr> <td>Properties (2)</td><td> <table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 object_name</td><td>BMECAT Object 0</td></tr> <tr> <td>2 object_desc</td><td>ET 200AL, DQ 8X24VDC/2A, 8XM12</td></tr> </table> </td></tr> </table> </td></tr> <tr> <td>ClassDefinition</td><td>0173-1#01-ADN579#006</td></tr> <tr> <td>UnitSystem</td><td>1</td></tr> <tr> <td>Properties (43)</td><td></td></tr> </table>	ID	6ES7142-5AF00-0BA0/A	ObjectType	PR	HierarchicalPosition	27-24-26-04	ClassifiedObject	<table> <tr> <td>ObjectType</td><td>Item</td></tr> <tr> <td>ClassifyRevision</td><td>true</td></tr> <tr> <td>ID (1)</td><td> <table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 item_id</td><td>6ES7142-5AF00-0BA0</td></tr> </table> </td></tr> <tr> <td>Properties (2)</td><td> <table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 object_name</td><td>BMECAT Object 0</td></tr> <tr> <td>2 object_desc</td><td>ET 200AL, DQ 8X24VDC/2A, 8XM12</td></tr> </table> </td></tr> </table>	ObjectType	Item	ClassifyRevision	true	ID (1)	<table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 item_id</td><td>6ES7142-5AF00-0BA0</td></tr> </table>	PropertyName	PropertyValue	1 item_id	6ES7142-5AF00-0BA0	Properties (2)	<table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 object_name</td><td>BMECAT Object 0</td></tr> <tr> <td>2 object_desc</td><td>ET 200AL, DQ 8X24VDC/2A, 8XM12</td></tr> </table>	PropertyName	PropertyValue	1 object_name	BMECAT Object 0	2 object_desc	ET 200AL, DQ 8X24VDC/2A, 8XM12	ClassDefinition	0173-1#01-ADN579#006	UnitSystem	1	Properties (43)	
ID	6ES7142-5AF00-0BA0/A																																
ObjectType	PR																																
HierarchicalPosition	27-24-26-04																																
ClassifiedObject	<table> <tr> <td>ObjectType</td><td>Item</td></tr> <tr> <td>ClassifyRevision</td><td>true</td></tr> <tr> <td>ID (1)</td><td> <table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 item_id</td><td>6ES7142-5AF00-0BA0</td></tr> </table> </td></tr> <tr> <td>Properties (2)</td><td> <table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 object_name</td><td>BMECAT Object 0</td></tr> <tr> <td>2 object_desc</td><td>ET 200AL, DQ 8X24VDC/2A, 8XM12</td></tr> </table> </td></tr> </table>	ObjectType	Item	ClassifyRevision	true	ID (1)	<table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 item_id</td><td>6ES7142-5AF00-0BA0</td></tr> </table>	PropertyName	PropertyValue	1 item_id	6ES7142-5AF00-0BA0	Properties (2)	<table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 object_name</td><td>BMECAT Object 0</td></tr> <tr> <td>2 object_desc</td><td>ET 200AL, DQ 8X24VDC/2A, 8XM12</td></tr> </table>	PropertyName	PropertyValue	1 object_name	BMECAT Object 0	2 object_desc	ET 200AL, DQ 8X24VDC/2A, 8XM12														
ObjectType	Item																																
ClassifyRevision	true																																
ID (1)	<table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 item_id</td><td>6ES7142-5AF00-0BA0</td></tr> </table>	PropertyName	PropertyValue	1 item_id	6ES7142-5AF00-0BA0																												
PropertyName	PropertyValue																																
1 item_id	6ES7142-5AF00-0BA0																																
Properties (2)	<table> <tr> <th>PropertyName</th><th>PropertyValue</th></tr> <tr> <td>1 object_name</td><td>BMECAT Object 0</td></tr> <tr> <td>2 object_desc</td><td>ET 200AL, DQ 8X24VDC/2A, 8XM12</td></tr> </table>	PropertyName	PropertyValue	1 object_name	BMECAT Object 0	2 object_desc	ET 200AL, DQ 8X24VDC/2A, 8XM12																										
PropertyName	PropertyValue																																
1 object_name	BMECAT Object 0																																
2 object_desc	ET 200AL, DQ 8X24VDC/2A, 8XM12																																
ClassDefinition	0173-1#01-ADN579#006																																
UnitSystem	1																																
Properties (43)																																	

The **REFERENCE_FEATURE_GROUP** ⁴ is used to create the hierarchical position. The first two digits, in this case, **27**, indicate the segment being converted.

The utility creates a section in the JSON file for a classified object ⁵ and uses the **MANUFACTURER_PID** to identify the item ID. During import, the utility checks whether there is an item revision in the database with that ID. If so, it is automatically classified. If not, the item revision is created and automatically classified. The value of **DESCRIPTION_SHORT** is used as the classification object description ³.

Procedure

1. Obtain BMEcat XML product data and extract the contents into a directory, *<extraction-directory>*.

The extraction generally results in a directory structure with many sub-directories, each containing BMEcat files and reference folders.

2. Copy the following file to the *<extraction-directory>* location and unzip it.

<TC_DATA>/classification/eclass2json/eclass2jsonHashMaps.zip

A new *hashmaps* directory is created.

3. Open a Teamcenter command window and call the **eclass2json** Perl script:

```
%TC_PERL% %TC_BIN%/eclass2json/eclass2json.pl -DIR "<extraction-
directory>"
-inputFile: "<BMEcat-file-path-relative-to-extraction-location>"
-objectType:data
```

The script creates a **e2j_output** output directory in the directory where the input files are stored. The output directory contains the newly created JSON files.

4. Import the JSON files into the database as described in [Importing JSON files using the clsutility command](#).

Convert JSON files to BMEcat to share classified items with suppliers

Teamcenter exports classified product data, also referred to as classification objects (ICOs), in JSON format. To share these ICOs with an external system that uses BMEcat XML format, the JSON files must be converted.

Procedure

1. Export the desired ICOs using **clsutility** as follows:

clsutility -find -classification_objects request=<JSON-input-file>

<JSON-input-file> lists the ICO object IDs to be exported. The object IDs must be listed with the revision, for example, 123456/A. The JSON file must follow the format specified by the **Classification_FindRequest_advanced.schema.json** schema file found in the **TD\classification\json\schema\advanced** directory.

The **clsutility** creates a JSON file with the ICO information.

Caution

The content of the JSON file created by the **clsutility** may vary slightly in format to that required by the **eclass2json** utility as input. In this case, the JSON file must be manually modified.

2. Open a command window and navigate to the directory containing the BMEcat source files.
3. Run the **eclass2json** utility with the **-objectType:json2BMEcat** argument. For example:

```
%TC_PERL% %TC_BIN%\eclass2json\eclass2json.pl -objectType:json2BMEcat
```

Results

The BMEcat files are stored in the **e2j_output** directory found in the same directory as the input file. If there are attachments, these are also found there.

About the `eclass2json` properties file

The **eclass2json.properties** file allows you to configure the behavior of the conversions between the following formats:

- ontoML XML to JSON
- BMEcat XML to JSON
- JSON to BMEcat XML

Caution

Always create a backup copy of the **eclass2json.properties** file before you modify it in case you must revert to the default values.

Tip

Each property can be overridden in the command line arguments. See the **eclass2json.properties** file for the correct syntax.

Property	Description
targetReleaseVersion	Target release for which the JSON file is intended. This value helps determine which schema is used to create the JSON file.
eclassLevel	Can be either ADVANCED or BASIC .
status	Specifies the status the data will have in the database after conversion. It can be either Develop or Released . You can classify in released classes only.
skipDeprecatedDefinition	Does not convert deprecated definitions in the source XML. Siemens Digital Industries Software recommends keeping the default value of false .
skipDeprecatedReferences	Does not convert the deprecated references. Siemens Digital Industries Software recommends keeping the default value of true .
skipExistingIRDI	Adds the skipExistingIRDI option to the JSON file so that existing administrative objects are skipped during import.
addAsNewRelease	Adds the addAsNewRelease option to the JSON file so that the administrative objects are added as a new release. This begins a new revision chain in the database allowing the classification hierarchy to contain multiple releases . Additionally, you can specify that only objects of a specific type are added as a new release.
objectCountPerFile	Specifies the maximum number of objects described in a file.
topLevelNode	Specifies the IRDI of the root node in the classification hierarchy that contains the segment data that you import. The default value contains a variable for the revision level of the ECLASS release that you are importing. This results in a unique top level node for each new ECLASS release that you install.
topLevelNodeName	Specifies the name of the root node in the classification hierarchy in that contains the segment data that you import. The default value contains a variable for the revision level of the ECLASS release that you are importing. This results in a unique top level node for each new ECLASS release that you install.
WSO.revision_separator	Specifies the separator between item ID and item revision that is used at import when creating a classified item revision in Teamcenter. This property is used for converting BMEcat data only.
WSO.revision_id	Specifies the revision ID which is used to create a revision of this ID (or to classify a revision of this ID) when importing the product data into Teamcenter. This property is used for converting BMEcat data only.
WSO.revision_type	Specifies the workspace object type created when importing the converted JSON files into Teamcenter.

Property	Description
	This property is used for converting BMEcat data only.
WSO.classifyRevision	Specifies that the item revision is classified during import. If this option is set to false , the item is classified.
	This property is used for converting BMEcat data only.

Import ECLASS units

When viewing classification data, you can convert between units that exist in the database. The ECLASS standard data packages include a large number of unit definitions. These are contained in an XML file in the ECLASS data package that you downloaded. To be able to convert between units when using ECLASS data, you must first import the units into the database.

Units with conversion

Units without conversion

Net weight:

kg

kg

t

lb

Net weight:

kg

Procedure

- 1. In a command line, navigate to the directory containing the unit definition file and convert it to JSON with the **eclass2json** utility:

```
%TC_PERL% %TC_BIN%\eclass2json\eclass2json.pl -objectType:units
-inputFile:eClass11_1_UnitsML_EN_DE.xml
```

- 2. Import the newly created JSON file found in the **jsonOutputDirectory** folder as follows:

```
clsutility -create -unit_definitions -request=<JSON file>
```

Results

The ECLASS units are now available for use in the user interface. When converting between measures, the value is stored in the object's unit system (the one chosen in the **Unit System** radio buttons at the top of the **Properties** section). For example, if a property's base unit is **mm** and if the unit system of an object is metric, even if you set the value of the **Net weight** property to **lb** or **kg**, internally, the value is converted to **mm** before being stored.

Importing data (datasets) associated with BMEcat property records

When importing ECLASS product data originally in BMEcat file format, there are generally additional files associated with each property record. These must be imported into the database and associated to each imported classification object as a dataset using a specific named reference, depending on the file type. By default, metadata defining the datasets is included in the BMEcat file. For example:

```
"AttributeIDsForDocumentation": {
  "0173-1#AAQ680": {
    "PropertyIDsForDocumentation": {
      "0173-1#AAC311": "FileName",
      "0173-1#AAS193": "DocumentName",
      "0173-1#AAM660": "DocumentDescription"
    }
  }
}
```

When importing, the **clsutility** uses a configuration file to determine which property IDs relate to the datasets. Additionally, the configuration file specifies the named reference type used to import each file type. This configuration file, **ClsPropertyRecordDatasetConfiguration.json** is found in the `\<TC_DATA>\classification` directory. The default settings in the file correspond to the common property IDs and dataset types found in BMEcat data, but you can modify this file to suit your needs. The file also defines the types of relations with which the attachments are imported.

```
{
  "AttributeIDsForDocumentation": {
    "0173-1#AAQ680": {
      "PropertyIDsForDocumentation": {
        "0173-1#AAC311": "FileName",
        "0173-1#AAS193": "DocumentName",
        "0173-1#AAM660": "DocumentDescription"
      }
    }
  },
  "ClassifiedObjectAttachment": {
    "DocumentRevision": {
      "JPG": {
        "DatasetType": "JPEG_Thumbnail",
        "NamedReference": "JPEG_Reference",
        "RelationType": "IMAN_manifestation"
      },
      "PNG": {
        "DatasetType": "PNG_Thumbnail",
        "NamedReference": "PNG_Reference",
        "RelationType": "IMAN_manifestation"
      },
      "DXF": {
        "DatasetType": "UGPART",
        "NamedReference": "Autocad-files",
        "RelationType": "IMAN_Rendering"
      }
    }
  }
}
```

```

    "STP": {
      "DatasetType": "UGPART",
      "NamedReference": "STEP-files",
      "RelationType": "IMAN_Rendering"
    },
    "PDF": {
      "DatasetType": "PDF",
      "NamedReference": "PDF_Reference",
      "RelationType": "IMAN_reference"
    },
    "DEFAULT_EXT": {
      "DatasetType": "MISC",
      "NamedReference": "MISC_BINARY",
      "RelationType": "IMAN_reference"
    }
  },
  "ItemRevision": {
    "JPG": {
      "DatasetType": "JPEG_Thumbnail",
      "NamedReference": "JPEG_Reference",
      "RelationType": "IMAN_manifestation"
    },
    "PNG": {
      "DatasetType": "PNG_Thumbnail",
      "NamedReference": "PNG_Reference",
      "RelationType": "IMAN_manifestation"
    },
    "DXF": {
      "DatasetType": "UGPART",
      "NamedReference": "Autocad-files",
      "RelationType": "IMAN_Rendering"
    },
    "STP": {
      "DatasetType": "UGPART",
      "NamedReference": "STEP-files",
      "RelationType": "IMAN_Rendering"
    },
    "PDF": {
      "DatasetType": "PDF",
      "NamedReference": "PDF_Reference",
      "RelationType": "IMAN_reference"
    },
    "DEFAULT_EXT": {
      "DatasetType": "MISC",
      "NamedReference": "MISC_BINARY",
      "RelationType": "IMAN_reference"
    }
  }
}

```


Setting up classification with artificial intelligence

Overview of configuring Classification Artificial Intelligence

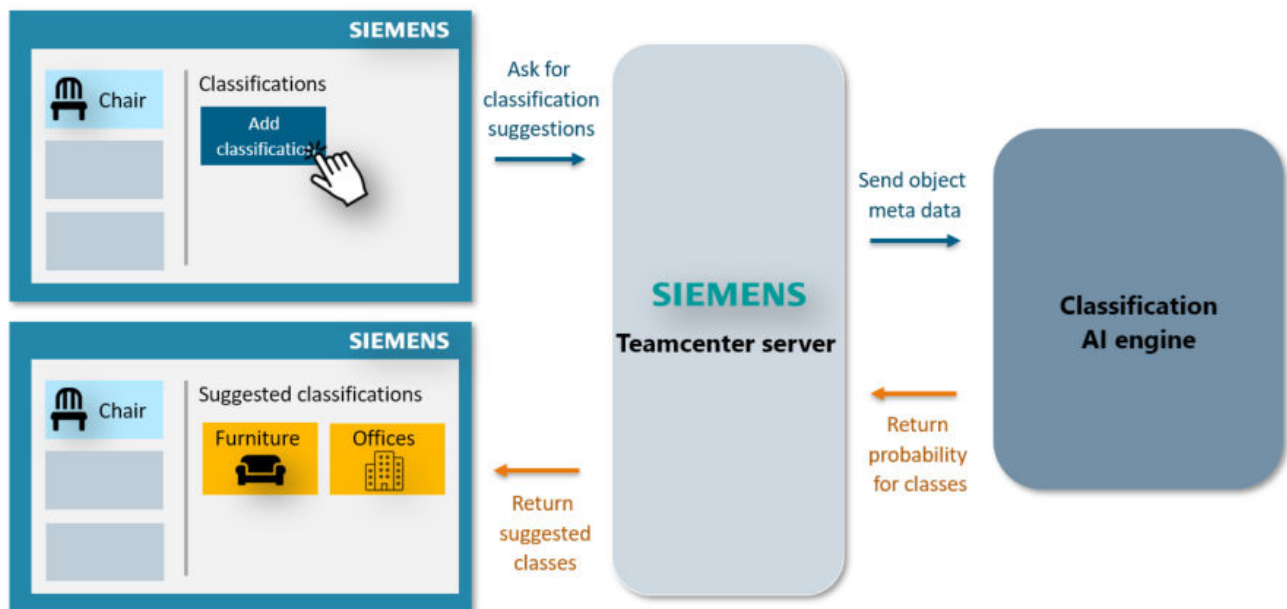
Note

Ensure that you have the appropriate license to use this feature. For more information, contact customer support.

Using artificial intelligence (AI), Active Workspace can guide you to the correct class in which to classify new objects. This is beneficial if your hierarchy is very large so you do not have to manually navigate to the desired class or remember the required class name. This saves time, especially for workflows where you often classify into similar classes.

The Classification AI engine is based on TensorFlow, an open source machine-learning library. It can, optionally, use Geolus, the Teamcenter shape search engine, to recommend a class based on the shape of the object being classified. After being trained on a database, the engine receives object metadata from the Teamcenter server and returns the probabilities for potential classes. You can specify which classes are displayed in the user interface based on these probabilities. For example, only classes with a probability of higher than 50% are displayed as potential Classification classes.

To configure Classification AI, the AI engine must first be trained on the data in your database. The training outputs a data model (files) that is used to deliver suggested classes.



Training the AI engine is carried out by running a utility. This utility need only be run when necessary. If your data changes often, new training is required. Training can be time-consuming depending on how much data you have.

The general process to install and configure Classification AI is as follows:

1. (Optional) To use the Geolus Shape Search with Classification AI, the Geolus Shape Search engine must be installed and configured on the Teamcenter server.
For details refer to the Geolus documentation.
2. **Install the feature.**
3. **Modify the required preferences.**
4. **Train the AI engine.**
5. View the suggestions in the Active Workspace client.

Note

If you use both traditional and Classification Standard Taxonomy (CST) data, the training is performed on CST data only.

Install Classification AI components

Procedure

Ensure that you have **installed Classification Active Workspace application**.

The installation creates a new folder in the Teamcenter root directory named **classification**. This folder contains all the files for Classification AI.

Deploying the classificationaiserving Docker image on Linux

Prior to deploying the image, you must perform the steps in **Install Classification AI components**.

1. Deploy the image to the appropriate stack by navigating to the machine and directory where microservices are installed. The `<Microservices-home-directory>/container` directory contains the Docker image and corresponding YML file.

Run the following command:

```
docker stack deploy --compose-file classificationaiserving.yml <your-stack>
```

The service is created and added to the stack.

2. Test that the service is running by typing the following in the address field of a browser:

```
<machine-or-IP-address-running-microservices>:9090/cais
```

For more information about general Docker administration commands, see their website:

<https://docs.docker.com/engine/reference/commandline/stack/>

When to run Classification AI training

Classification AI training creates a model using the classification data and metadata currently available in the database. Run training whenever changes can affect classification recommendations.

Training the Classification AI engine is not a one-time activity. The generated model reflects the state of classification structures, classified objects, and metadata at the time the training script is executed. When these inputs change, retraining is required to maintain accurate recommendations.

Recommended scenarios for training

Run training in the following scenarios:

- Initial deployment
Run training only after at least 100 objects are classified across multiple classes. Classification AI relies on existing classified objects. Training with little or no classified data does not produce useful results.
- Changes to classification hierarchies
Retraining is recommended when new classification hierarchies are introduced and classified objects are added under those hierarchies. Declassification or reclassification can also affect the AI model and should trigger retraining.
- Changes to training metadata
Retrain the AI engine when the metadata used for training changes. Recommendation quality depends on the relevance and consistency of extracted metadata. Updates to the [CLS_AI_Object_Properties](#) preference or to object metadata can improve results after retraining.
- Improved data availability
Retraining is recommended if initial training used limited or low quality data and a larger or more representative set of classified objects later becomes available.

Each training run creates a new AI model. The model does not retain information from previous training runs. Periodic retraining may be required as classification data evolves.

Train the Classification AI engine

Training the artificial intelligence engine on your data involves running a script that extracts data from the database in a format that the engine can subsequently use to create the training model. The properties that are extracted and trained on are specified in the [CLS_AI_Object_Properties](#) preference.

Procedure

1. Set the required preferences (see additional preferences [here](#)):

CLS_AI_Enable_AI_Engine

Set this preference to **true** to enable class suggestions.

To enable Geolus with Classification AI, additionally set this preference:

CLS_AI_Enable_Geolus

Enables the additional comparison of shape parameters using the Geolus search engine with Classification AI.

The URL of the microservice that supports Classification AI is specified in the environment variable named **TC_Microservices_Base_URL** located in `<TC_DATA>\tc_profilevars`.

2. In a Teamcenter command prompt window, navigate to the `<TC_ROOT>\classification\ai\bin` directory and run the following script:

`runClsAITraining`

This script trains the AI engine on the data in your database.

- The script creates two CSV files, **meta_data_train.csv** and **meta_data_val.csv**. The **meta_data_train.csv** contains the data used to train the model. The **meta_data_val.csv** file contains a subset of this data that is used to validate the training model. When complete, the model named **cait-artifacts-TrainOutput_<timestamp>**, is uploaded to the file repository and the two files used to generate it are subsequently deleted from your local directory.

Tip

If there are problems with the configuration, you can [run a utility to generate these files and retain a local copy](#).

- To obtain assistance for the **runClsAITraining** script, run it including the **-h** argument.

Each time you run the script, you create a new training model that has no memory of the last run.

Tip

As the training is run using a batch script, you can schedule it to run at regular intervals.

3. (Optional) [Set remaining Classification AI preferences](#).

Using Classification AI with traditional and Classification Standard Taxonomy data

If you use both traditional and next generation classification data, internally, different training engines are used to prepare the class predictions. To ensure that AI is correctly set up for using Classification Standard Taxonomy data, do the following:

1. Set the **CLS_is_presentation_hierarchy_active** preference to **true**.

2. Run the training. Pay attention to the message displayed when you do this. The message should read **Searching database for Next Generation Classification Objects**, and not **Searching database for Traditional Classification Objects** as is displayed when training traditional data.

A model specific to next generation or Classification Standard Taxonomy data is trained and uploaded to the file repository. If AI is set up correctly, the serving component immediately picks up the new model.

Only users that have Presentation Layer (Next Generation Classification) enabled should use the model trained with the **CLS_is_presentation_hierarchy_active** preference set to **true**. Training may not give accurate suggestion probabilities for traditional Classification users who are getting predictions from a Next Generation AI model, and vice versa.

Classifying objects in batch mode

A classification administrator can run a utility that classifies multiple objects simultaneously based on definable properties. This classification process is carried out using artificial intelligence. The process sends the objects to a workflow. Specified users receive a notification and can then decide whether to accept the suggested classification or, if it is not appropriate, delete it.

Note

Batch classification is not available for traditional basic classification.

1. Set the following preferences:

CLS_AI_Object_Properties

ICS_classifiable_types

CLS_AI_Engine_Probability_Cutoff

2. Run the **cls_ai_auto_classify** utility in dry run mode (without specifying the **-classifyObjects** argument), and examine the results found in the **AutoClassifyReport.csv** file residing in the **-outputPath** location that you specify.

The CSV file lists the objects that are found based on the preference settings, the probability that they match the suggested class, and whether they will be classified by the utility.

3. If you are satisfied with the results from the dry run, re-run the **cls_ai_auto_classify** utility and specify **-classifyObjects**.

The objects found by the utility are sent to the **Classification AI Review Classifications** workflow.

4. Select the classification experts responsible for approving the classifications.

The reviewers receive a notification and can approve the classification for the object, thereby classifying it, or remove the suggested classification.

Note

You cannot delegate the review task to another user.

Troubleshooting Classification AI

- **How do I know whether the Classification AI server is running?**

To ascertain whether the Classification AI server is running, open a Web browser and type the URL where microservices is installed:

<IP of machine on which microservices are installed>:port/<cais/>

A common port used is **9090**.

If the server is running correctly, the following is displayed:

Connected to Classification AI Serving

- **How do I prevent the training files from being uploaded to the file repo if problems arise during the configuration of AI?**

The **runClsAITraining** script calls several utilities:

cls_AI_data_generation Generates the two CSV files used to create the model and stores them in the following directory:

<TC_ROOT>\classification\ai\working\cai-data

app.js Creates the model from the two CSV files, uploads it to the file repository, and deletes the two CSV files from the local directory.

If there are problems with configuring AI, run the **cls_AI_data_generation** utility manually to generate the two CSV files but prevent them from being deleted after uploading to the file repository.

The files can then be found in the *<TC_ROOT>\classification\ai\working\cai-data* directory. This directory contains time-stamped subdirectories. The two relevant training files can be found in the most recent of the subdirectories. You can delete these files after you finish troubleshooting.

- **How do I make sure the data on which the AI is trained is valid?**

Classification AI trains on the metadata from existing classified objects. To discover the contents of this metadata, you can run an ITK utility, **cls_AI_data_generation**, that lists the existing metadata in a CSV file found in the following directory:

<TC_ROOT>\classification\ai\working\cai-data\<time-stamped-folder>

Run the utility and examine the output to ensure that the data on which it was trained is both meaningful and diverse enough that a pattern can be extracted by the learning algorithm.

- **Where do I find log files?**

The syslog created during training is stored in your local `<%TEMP%>` directory. It has the following format:

classificationaiserving`<instance_number>@<node_id>-msf.log`

For example:

classificationaiserving1@22504-msf.log

- **Why am I not seeing class suggestions?**

Ensure that the **following preference is set**:

- **CLS_AI_Enable_AI_Engine**

If this preference is not set to **true**, Teamcenter will not call the Classification AI serving component for suggestions.

- **What preferences are set during the Geolus installation?**

If Geolus is enabled, the following preferences are set by the installation and can be checked for accuracy:

SS1_DASS_cert_name

Defines the full name of the Geolus server certification file.

SS1_DASS_cert_path

Defines the path to the Geolus server certification file.

SS1_DASS_ssl_verify

Defines whether CURL verifies the certificate authenticity of the Geolus server.

GeolusServer

Defines a URL that points to the Geolus server.

- **What do I need to know about upgrading from an earlier version of AI?**

When upgrading from a later release, the old training model is still functional and is migrated to the file repository. When you train again, the model is no longer stored locally in the shared directory but automatically uploaded to the file repository. At this point, you can remove the shared directory as well as the old **CLS_AI_Shared_Directory** preference.

Configure access for classification

Give access to users for classifying objects

The following points determine whether you can classify an object (that is, whether the **Classify** button is visible for a selected object):

- The workspace object's type must be listed in the **ICS_classifiable_types** preference. If your business use case requires classifying items instead of item revisions, you must remove the **ItemRevision** entry from this preference.

- You must have write access to the workspace object. If this object is already released, you must have write access to the classification object (**write_ico** access). The workspace object access is set in the **Access Manager** application in rich client. The classification entries in the **Has Status()** → **Vault** rule must be set to write access.

The screenshot shows the 'Access Manager' application with the 'Access Manager Rules' tab selected. The 'Has Status' rule is highlighted in the list of rules. The right pane displays the configuration for the 'Has Status' rule, including the 'Object Access Control List' table.

Condition	Value	ACL Name
Has Class	POM_object	System Objects
Is Current Group External	true	
Has Class	WorkspaceObject	
Has Class	SavedSearch	
In Job	true	
In Current Program	false	Not Current Program
In Inactive Program	true	Inactive Program
Is Program Member	false	Not Program Member
In Invisible Program	true	Invisible Program
Has Type	EngChange Revision	
In IC Context	true	
Has Status	TCM Released	TCM Released Rule
Has Status		Vault
Has Object ACL	true	
Has Class	POM_classification object	Access/Freeze

The 'Object Access Control List' table in the 'Has Status' rule configuration is as follows:

Accessor Type	Accessor	Access
Owning Group		
Groups with Security	External	
World		✓

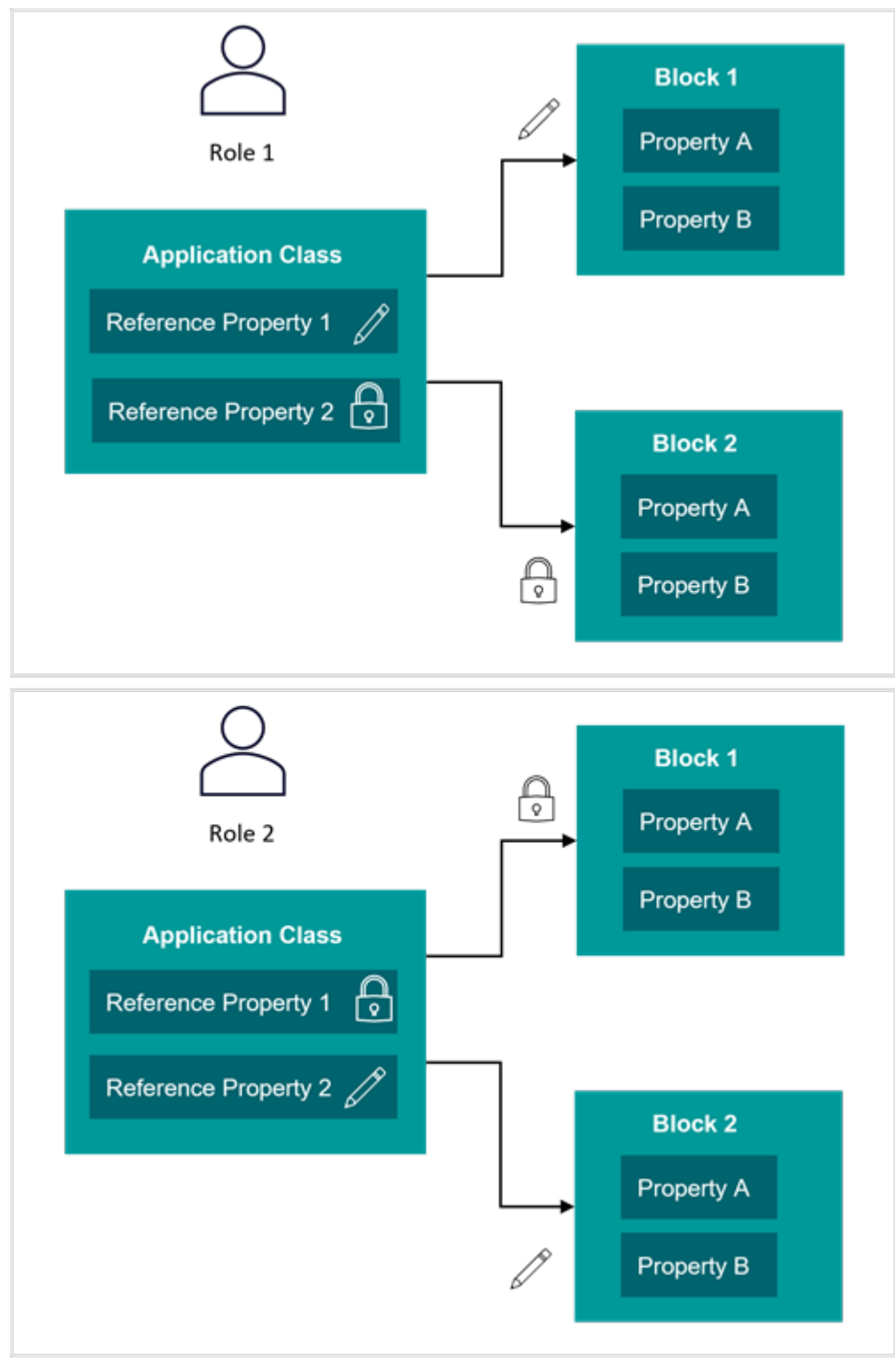
- You must **set additional classification preferences** depending on the classification features you use.

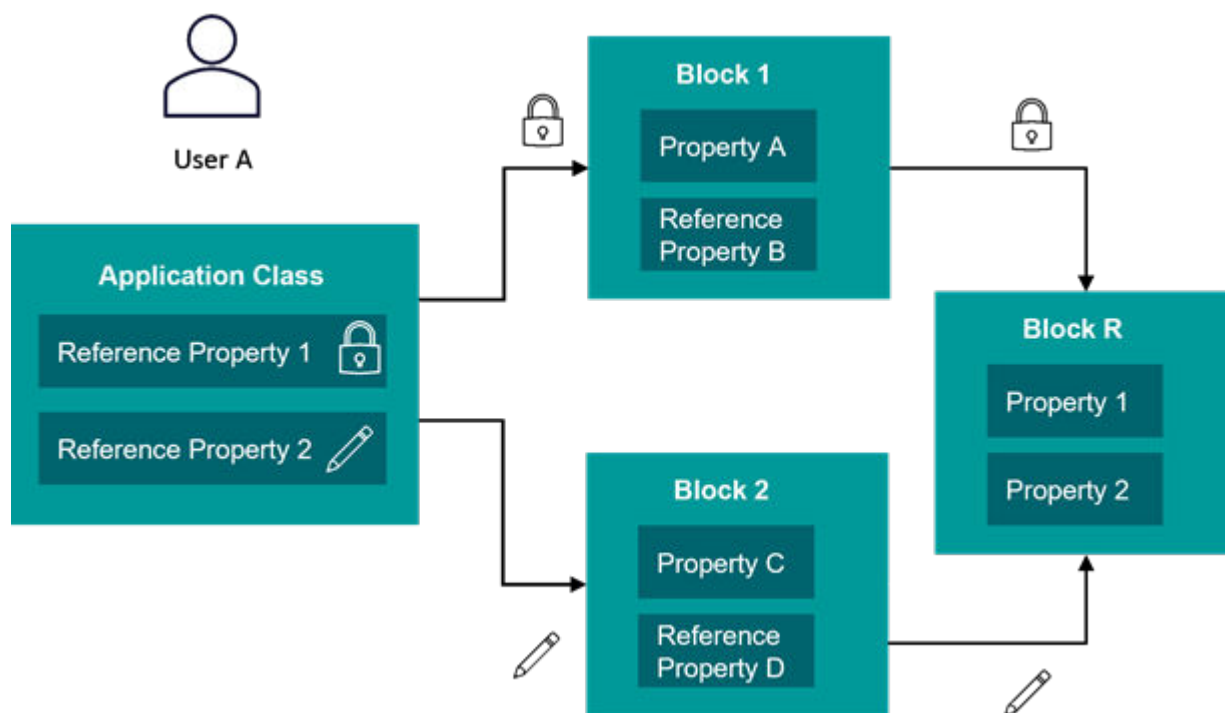
Controlling access to property blocks or groups

Write access on blocks is defined by setting access on the reference properties of blocks.

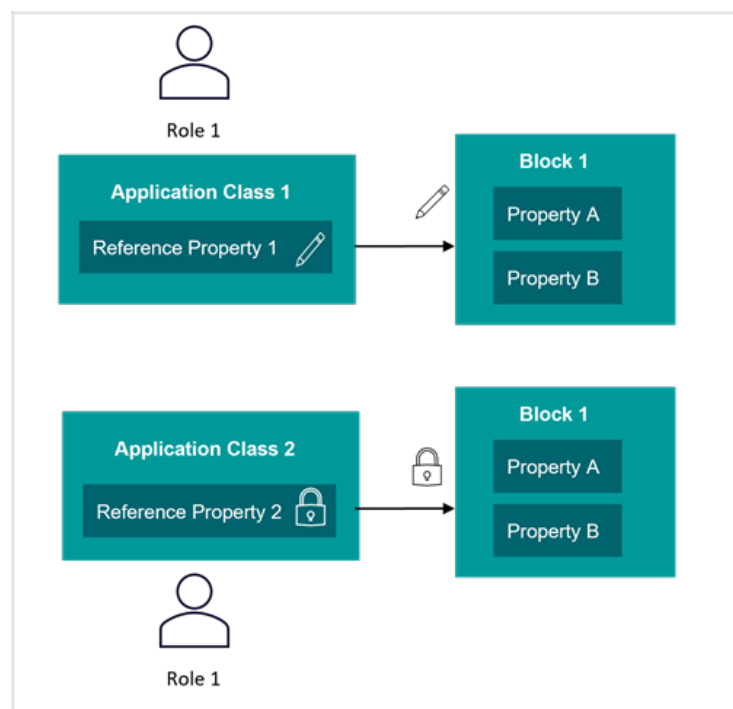
Block access controls allow you to:

- Use the Access Controls command to set access to blocks.
- Assign permissions at the group, role, or user level.
- Restrict write access as needed while preserving read access for all users.

Example

Example

When blocks are nested, denying access to the parent block denies access to the child block. Similarly giving access to the parent block gives access to the child block.

Example

Different classes can refer to the same block using different reference properties. In this case, the first application class receives access to the block. However, the second application class is denied access.

Control property block and group access

Block access control lets you manage write permissions for specific blocks through their reference properties.

Procedure

1. Open the **Active Admin** workspace.
2. From **Launcher**, search for **Classification Manager**.
If this option is unavailable, search for it. You can pin it from the search results for future access.
3. Select a class, and then select the class attribute that is a reference property.

The screenshot shows the 'Class Attributes' tab for a '1-Phase AC motor (parts)' block. The table below represents the data shown in the interface:

Name	Attribute Index	Type	Data Type	Reference	Metric Unit	Non Metric Unit
Add on Documentation	1	Aspect		0173-1#01-ADN464#006		
Commercial	2	Aspect		0173-1#01-ADR667#004		
Identification	3	Aspect		0173-1#01-ADN228#006		
Information	4	Aspect		0173-1#01-ADN329#002		
Mechanical and electrical const...	5	Property	Reference	0173-1#02-AAR080#006		
description of product type	6	Property	String	0173-1#02-AAO847#003		
Use for product	7	Property	String	0173-1#02-BAF281#003		

4. Click **Access Control** .
5. Select the type of access: **Grant** or **Deny**.
Grant gives a user write access. If you deny access to someone, only the specified groups, roles, or users lose access. All others get access.
6. Specify who you want to give access to from the **Group**, **Role**, or **User** lists.

Note

These lists operate independently. Your selection from one list, such as selecting a group, does not restrict choices in another list, such as selecting a role that is not linked to that group.

7. Click **Save**.
8. Log out of Teamcenter, and then log in again.

Related Topics

[id-3e416d555475ae41f4698277923e107a](#)

Set block access using clsutility

You can set block access by running the **clsutility** with a JSON file as the input.

Procedure

1. Create a JSON input file.

The following is a sample JSON file called *blaccess.json*:

```
{
  "SchemaVersion": "1.5.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "Update": "TD-DL#01-000911#001",
      "Values": {
        "ClassAttributeOptions": [
          {
            "AttributeIRDI": "TD-DL#02-000912#001",
            "PropertyAccess": {
              "Type": "DENY",
              "Group": [
                "4G Tester"
              ],
              "Role": [
                "4G Designer"
              ],
              "User": [
                "ed"
              ]
            }
          }
        ]
      }
    }
  ]
}
```

```
]
}
```

2. Run the **clsutility**.

```
clsutility -u=user -p=password -g=group -update -class_definitions
-request="blaccess.json"
```

Granting access to non-dba users for Classification Manager tasks

While configuring Access Manager rule conditions for a classification user, any user can be a classification admin user. It is not necessary for the user to be defined as a dba user for performing classification administrator tasks.

You must create an ACL to grant the **Read**, **Write**, **Delete**, **Change**, and **Copy** access for any non-dba user to access the Classification Manager application and perform classification administrator tasks.

Troubleshooting the installation and configuration

- **All classification options are installed and indexing is configured, but new objects are still not included in search results**

To use traditional basic classification in the Active Workspace client, it is not necessary to install the Presentation Layer (**Next Generation Classification**). If, however, you inadvertently do install this option, you *must* also extend the data using **clsutility** to make use of full functionality.

- **Filter categories are localized but facet values are not**

If you mark any Classification property as localizable, you must run the **bmide_modeltool** utility.

- **Search is incorrectly configured**

The following situations may arise if the search is not correctly configured:

- You cannot search by Classification properties.
- The Classification search works, but facets are not visible (or there are unassigned facets).
- You cannot search the library elements by Classification property.

Correctly configure the search.

- **The incorrect filters are displayed for library elements**

1. Set the dynamic prioritization preferences to display all the configured filters. This allows you to investigate further.
 - Set **AWC_dynamicPriority_hideSingleValueFacetCategories** to **true**.
 - Set **AWC_dynamicPriority_hideUnassignedValueFacetCategories** to **true**.

2. Make properties searchable in the global search.

- The Classification search works, but there only a limited number of properties available in the search filters.

Increase the number of search facets displayed using the **AWC_classification_facets_threshold** preference. The default value is **200** and the facets are sorted with the most common facets at the top.

Caution

Setting this value to a high number can cause a decrease in performance or even cause problems with Solr.

- Performing a bulk import of Classification data

Ensure that the **CLS_auto_sync_node_hierarchy** preference is set to **false** before performing a PLM XML import of hierarchy data.

- What business constants are involved in Classification installation?

The following business object constants are set in Business Modeler IDE:

- The **Awp0SearchClassifySearchEnabled** business object type constant is set to **true** at the workspace object level. This turns on the Classification search.
- The **Awp0SearchIsClassifyDataIndexed** business object type constant is set to **true** at the item and item revision level.

This turns on Classification indexing. The object type on which this constant is set is indexed for searching. All subtypes inherit the setting.

- The **Awp0SearchIsIndexed** business object type constant is set to **true** at the item and item revision level.

The object type on which it is set is indexed for searching. All subtypes inherit the setting.

Note

Do not set this constant at the workspace level since this prevents your database from being indexable.

Instead, set it true for each object that you want to index. For example, item and item revision.

Verify that these constants are correctly set using the **gettypeconstantvalue** utility, for example:

```
gettypeconstantvalue -u=username -p=password -g=dba
-type=WorkspaceObject -constantname=Awp0SearchClassifySearchEnabled
```

Enforce classification on specific object types

If your business process requires that all objects of a specific type, for example **Part** or **PartRevision**, must be classified, you can enforce classification.

- To enforce classification on a specific object type, list the internal type name in the **CLS_enforce_classification_on_types** preference.

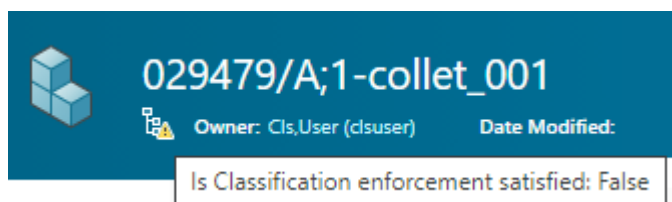
Example

Part or **PartRevision**.

Results

When you create an object of this type, it will be marked with a warning sign indicating that it has not yet been classified.

A tooltip will notify you that the enforcement requirement is not fulfilled.

**What to do next**

When you add the object to the preference, you can ensure that the object is always classified by submitting the object to a workflow as follows:

1. Submit the object to a workflow that uses the **Review Classifications** template.
2. Assign the workflow to the responsible subject-matter expert.
The object to be classified will be displayed as a target of this workflow.
3. The subject-matter expert can open the object from within the workflow and complete the workflow affirming that the object is correctly classified.

8. Administering classification

Viewing your hierarchy definitions

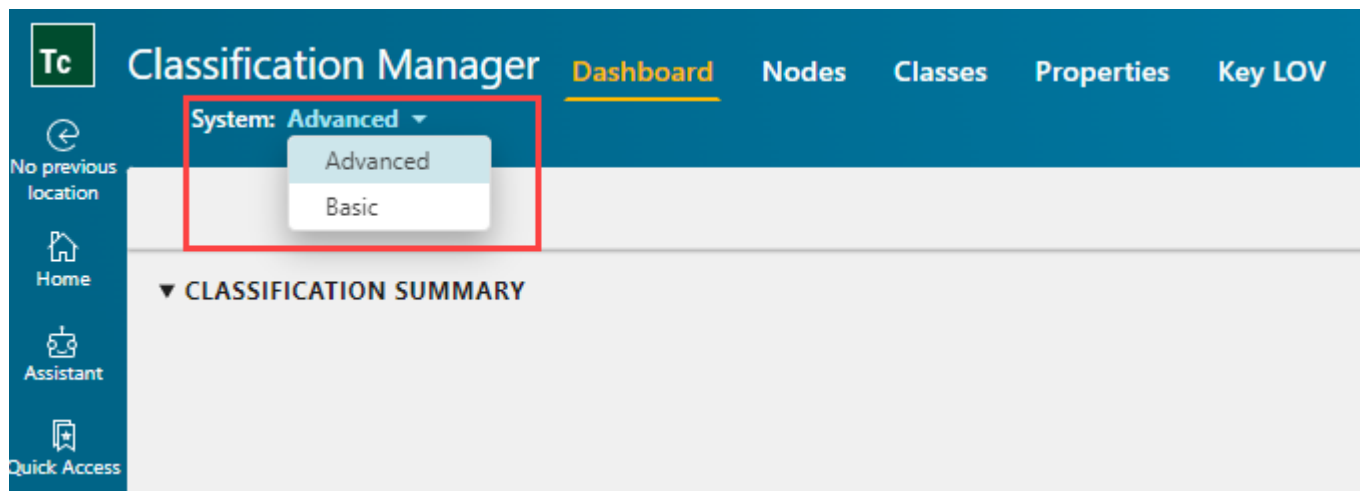
The basic and advanced classification hierarchy is created of nodes, classes, properties, and key-LOV administrative objects. These definitions are displayed in **Classification Manager**. The **Classification Manager** provides an overview of every classification definition in the database on the **Dashboard** tab, as well as details about each object on the **Nodes**, **Classes**, **Properties**, and **Key LOV** tabs.

CLASSIFICATION MANAGER is found in the **Active Admin** workspace.



By default, the **Active Admin** workspace is mapped to the **dba** group and role. You can add other groups and roles as needed. Alternatively, you can create your own workspace that includes the **CLASSIFICATION MANAGER**.

You can view the classification definitions for basic or advanced classification by selecting either the **Basic** or the **Advanced** system in **Classification Manager**.



Use the **Import** button on any of the tabs in **Classification Manager** to **import administrative objects** (key-LOV definitions, property definitions, class definitions and node definitions). The objects must always be **imported in the correct order**. For advanced classification you can import JSON, OntoML, or BMEcat file format.

Authoring hierarchy definitions

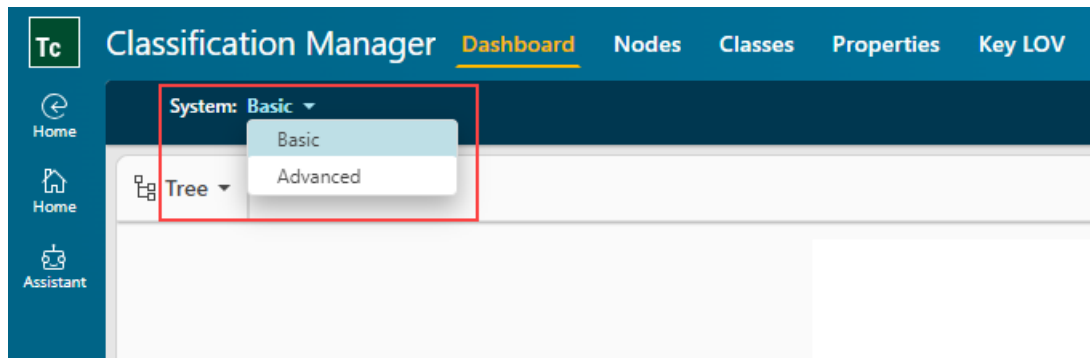
Creating classification hierarchy definitions

You use Classification Manager to define the nodes, classes, properties, and Key LOVs that form the classification hierarchy for basic and advanced classification. You also define and format the attributes that, when associated with a class, determine the type of information that is stored. The order when creating a classification hierarchy is to first create the Key LOV definitions, then the property definitions, then the class definitions, and finally the node definition so that the node is visible in the classification hierarchy for end users.

Use Classification Manager to do the following basic tasks:

- [Create a Key LOV definition](#)
- [Create a property](#)
- [Create a class](#)
- [Create a node](#)

You can create the classification definitions for basic and advanced classification. Select either **Basic** or **Advanced** from the **System** list available on the **Classification Manager** dashboard before creating the classification definitions.



Creating Key LOVs

Overview of Key LOVs

Key LOVs are preset lists of values used in Classification to keep data consistent. Using Key LOVs, users can choose standard options, such as countries or equipment types, and make it easier to reuse and maintain the same values across different parts of the system.

What you can do with Key LOVs

- Define entries

Create the values that belong to a Key LOV. In Classification, these values are called Entries.

- Create submenus (cascading values)

Add child values under an entry to form cascades.

Example

For a Key LOV that has an entry for the country, you can create submenus for the city and state.

▼ Entries

 Edit

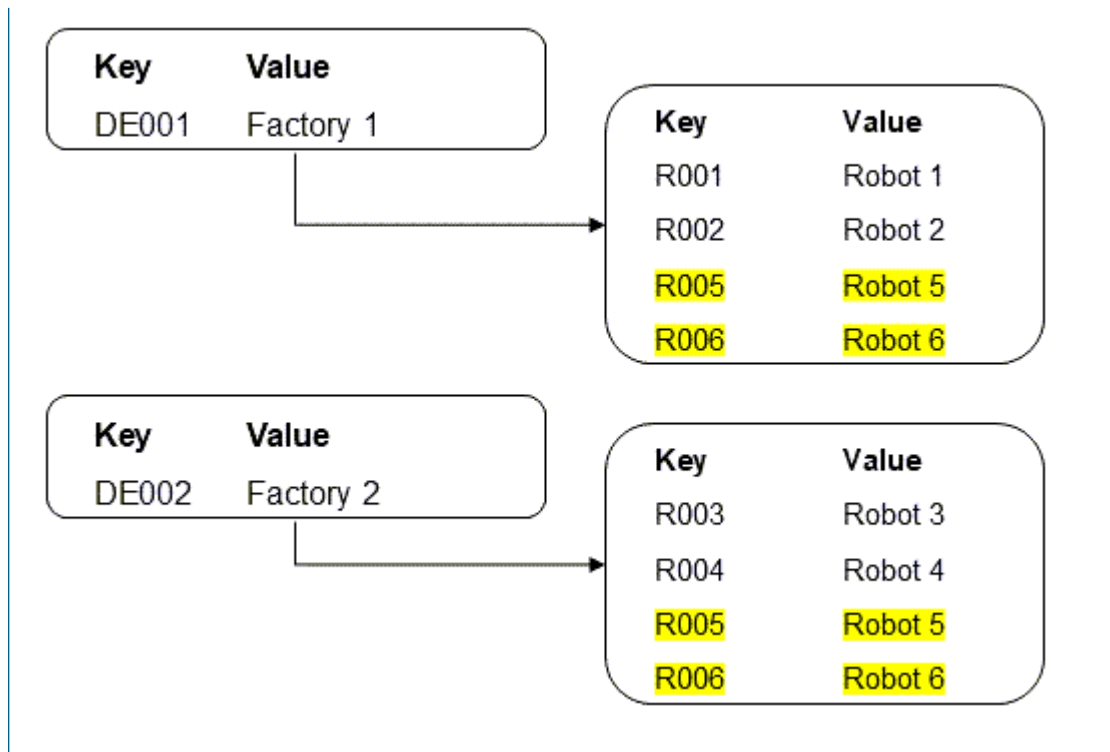
String Value	Is Submenu	Display Value
▼ Germany	▼	Germany
▼ Bavaria	▼	Bavaria
Munich		Munich
Passau		Passau

- Reuse entries

Use the same entry across different classes. When classifying data, you may want to assign the same values to different objects. For example, if the same type of robot is used across multiple manufacturing sites, reusable key-value pairs help you define the robot data only once and then assign the robots to different factories.

Example

In the following example, Robot 5 and Robot 6 are reused in Factory 1 and Factory 2.



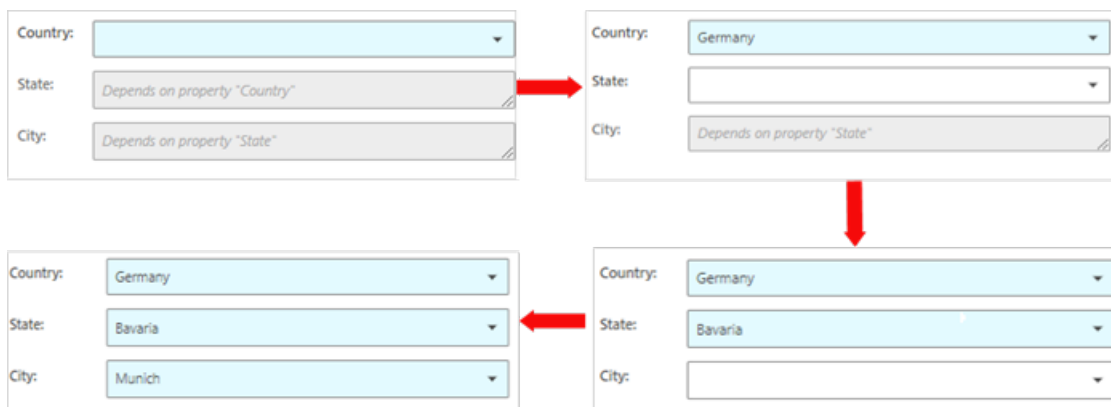
Reusable key-value pairs help you:

- Define a Key LOV once and use it multiple times across different classes or contexts.
- Update key-value pairs once and have the changes reflected everywhere the entry is used.
- Configure dependencies

Make one Key LOV dependent on another. Selecting a value in the first Key LOV filters or enables the next Key LOV.

Example

In a dependent Key LOV, if you select the country, the dependent Key LOV is available for selection. Additionally, when you select the state, the dependent Key LOV is available for selection.



- Use dynamic LOVs

Show LOV values at runtime by querying the database.

In Business Modeler IDE (BMIDE), you can create Dynamic LOVs. Dynamic LOVs are lists of values where the values are shown at runtime by querying the database, such as a list of users or a list of parts belonging to a particular group.

You can use these dynamic LOVs in Classification by associating the dynamic LOV with a Key LOV.

Create a Key LOV definition

You use Key LOVs to define one or more values that can be set for specific classification attributes. Once created, these lists are stored in the data dictionary and can be used and reused as required.

Procedure

1. On the **Classification Manager** dashboard, click the **Key LOV** tab.
2. Click **New** .
3. In the **Add Key LOV** panel, specify the following details:
 - **Namespace**
 - **ID**
 - **Revision**
 - **Name**
 - **LOV Items**
4. Click **Add**.

The Key LOV definition is created.
5. After creating a Key LOV, you can add the entry for the Key LOV definition.
6. Open the Key LOV definition. Under **Entries**, click **Edit**  in order to add entries to the Key LOV or to edit previously added entries.

☐

☒

☐

Metal Disc Material

Type: KeyLOV Definition ID: -5860 Owner: tcadmin [More...](#)

▼ Properties

ID:

-5860

Name:

Metal Disc Material

Is Hide Keys:

false

Create Date:

2024-04-27 22:34:05

Create User:



Create Group:

dba

Modify Date:

2024-04-27 22:34:05

Modify User:



▼ Entries Edit

String Value	Is Submenu	Display Value
0		42 ALLOY
1		BRASS
2		STAINLESS

▼ Multi-site

Shared Sites:

Owning Site:

IMC--1695479901

7. Click **Add** .

▼ Entries

 Add

 Cancel

 Save

8. Specify the details as required and click **Add**
You can add more entries to the Key LOV as required.
9. After all the entries are added, click **Save**.

▼ Entries

String Value	Is Submenu	Display Value	Is Deprecated
0		42 ALLOY	false
1		BRASS	false
2		STAINLESS	false
			false

10. To edit Key LOV definition, click **Edit**

The editable cells are activated.

☐ ☒ ☐ **Metal Disc Material**

☐ Type: KeyLOV Definition ID: -5860 Owner: tcadmin More...

▼ Properties

ID: -5860
 Name: Metal Disc Material
 Is Hide Keys: false
 Create Date: 2024-04-27 22:34:05
 Create User: tcadmin
 Create Group: dba
 Modify Date: 2024-04-27 22:34:05
 Modify User: tcadmin

11. Enter or update the Key LOV definition as required.
12. Save your changes for the Key LOV definition, by clicking **Save**. To cancel your edits, click **Cancel**.

Add submenus to a Key LOV

You can add submenu entries to the Key LOV entries, or you can add submenu entries to other submenus, creating cascading Key LOVs.

These steps show how to create state and city submenus for the country Key LOV entry.

Procedure

1. On the **Classification Manager** dashboard, click **Key LOV**, and open or create a Key LOV.
2. Create an entry for the Key LOV and in the **Add Entry** panel, select **Is Submenu** and click **Add**.

Example

Create an entry for the country, such as Germany.

Add Entry

Pin Panel

Close

* String Value:

Germany

Display Value:

Germany

Value Meaning:

Block Reference:

☐ Is Deprecated

☒ Is Submenu

Submenu Title:

3. To add a submenu entry to a Key LOV entry, select the Key LOV entry and click **Add**.

▼ Entries

+

 Add

✕

 Delete

↶

 Cancel

💾

 Save

String Value	Is Submenu	Display Value
Germany	▼	Germany

4. In the **Add Entry** panel, add details about the submenu entry and click **Add**.

Example

Add the values for the state, such as Bavaria.

The submenu entry is added under the Key LOV entry.

▼ Entries

 Add  Cancel  Save

String Value	Is Submenu	Display Value
▼ Germany	▼	Germany
Bavaria		Bavaria

- 5. You can also add submenus to existing submenus by selecting the **Is Submenu** option in the **Add Entry** panel.
- 6. In the **Entries** section, click **Save**.

Results

▼ Entries

 Edit

String Value	Is Submenu	Display Value
▼ Germany	▼	Germany
▼ Bavaria	▼	Bavaria
Munich		Munich
Passau		Passau

Reuse entries in a Key LOV

When classifying data, you may want to assign the same data to different objects. For example, if the same type of robot is used across multiple manufacturing sites, reusable key-value pairs help you define the robot data only once and then assign the same robots to different factories.

Procedure

- 1. On the **Classification Manager** dashboard, click **Key LOV**.
- 2. Open a Key LOV and click **Edit**.
[Add submenus to the Key LOV.](#)

▼ Entries

 Edit

String Value	Is Submenu	Display Value
▼ Factory 1	▼≡	Factory 1
Robot 1		Robot 1
Robot 2		Robot 2
▼ Factory 2	▼≡	Factory 2
Robot 3		Robot 3
Robot 4		Robot 4

3. Click **Edit** in the **Reusable Entries** section, and then click **Add** to add reusable Key LOV entries.
4. Add the reusable Key LOV entries as needed.

▼ Reusable Entries

 Edit

String Value	Display Value
Robot 5	Robot 5
Robot 6	Robot 6

5. To reuse the entries, add the reusable entries in the **Entries** section as follows:
 - a. Click **Edit** in the **Entries** section.
 - b. Select the entry where you want to add the reusable Key LOV values and click **Assign Reusable Entries** .
 - c. In the **Assign Reusable Entries** panel, select the reusable entries you want and click **Assign**.

Factories

▼ Properties

IRDI:

0173-5#09-AAD645#005

Object Type:

09

Name:

Factories

Namespace:

0173-5

ID:

AAD645

Revision:

005

Status:

Develop

Is Deprecated:

false

Is Dynamic:

false

Is Hide Keys:

false

Data Type:

String

▼ Entries

+

Add

Assign Reusab...

✕

Delete

↺

Cancel

💾

Save

String Value	Is Submenu	Display Value	Value Meaning
▶ Factory 1	▼≡	Factory 1	
▶ Factory 2	▼≡	Factory 2	

▼ Reusable Entries

String Value	Display Value	Value Meaning	Block Referen
Robot 5	Robot 5		
Robot 6	Robot 6		

Assign Reusable Entries

Robot 5

Robot 5

Robot 6

Robot 6

Assign

Results

The reusable Key LOV entries are added in the **Entries** section.

▼ Entries

 Edit

String Value	Is Submenu	Display Value
▼ Factory 1	▼≡	Factory 1
Robot 1		Robot 1
Robot 2		Robot 2
Robot 5		Robot 5
Robot 6		Robot 6
▼ Factory 2	▼≡	Factory 2
Robot 3		Robot 3
Robot 4		Robot 4
Robot 5		Robot 5
Robot 6		Robot 6

Create a dependent Key LOV

In a dependent Key LOV, setting the value of one Key LOV makes the next dependent value available for selection.

The process for creating an dependent Key LOV is as follows:

- Create a Key LOV and add the entries and sub menu entries such as entries for country, state, and city.
- Assign the Key LOV to the respective properties. For example, assign the Key LOV country to the country property, the state Key LOV to the state property and so on.
- Assign the properties to the respective class.
- Update the class attributes with information about the level of the dependency and the dependent attribute.

Procedure

1. On the **Classification Manager** dashboard, click **Key LOV**.
2. Create entries and submenus for the Key LOV.

Example

Create a Key LOV with entries for Country, State, and City.

String Value	Is Submenu	Display Value
DE	▼	Germany
DE01	▼	Bavaria
DE0101		Munich
DE0102		Passau
DE02	▼	NRW
DE03	▼	Berlin
US	▼	United States
UK	▼	United Kingdom
IN	▼	India

3. Assign the Key LOV you created to the respective properties.

For example, create properties for Country, State, and City, and assign the Key LOV to these properties.

Country

Type: String

Metric: SIM02#09-pbTr9w#

Entries:

- DE:Germany
 - DE01:Bavaria
 - DE0101:Munich
 - DE0102:Passau
 - DE02:NRW
 - DE03:Berlin
 - US:United States
 - UK:United Kingdom
 - IN:India

State

Type: String

Metric: SIM02#09-pbTr9w#

Entries:

- DE:Germany
 - DE01:Bavaria
 - DE0101:Munich
 - DE0102:Passau
 - DE02:NRW
 - DE03:Berlin
 - US:United States
 - UK:United Kingdom
 - IN:India

City

Type: String

Metric: SIM02#09-pbTr9w#001

Entries:

- DE:Germany
 - DE01:Bavaria
 - DE0101:Munich
 - DE0102:Passau
 - DE02:NRW
 - DE03:Berlin
 - US:United States
 - UK:United Kingdom
 - IN:India

4. Assign the properties to an application class.

Overview Class Attributes

Attribute Properties + Add ✕ Remove Attribute

Name	Attribute Index	Type	Reference	
Country	1	Property	SIM02#02-J9HKAA#001	
State	2	Property	SIM02#02-kPHKAA#001	
City	3	Property	SIM02#02-kfHKAA#001	

5. To assign the hierarchy level of the Key LOV, select a class attribute and click **Attribute Properties** .

Overview Class Attributes

Attribute Properties + Add ✕ Remove Attribute

Name	Attribute Index	Type	Reference	
Country	1	Property	SIM02#02-J9HKAA#001	

6. In the **Attribute Properties** panel, click **Edit** and select the hierarchy level from the **Dependency configuration level** list.

Example

Because Country is at the top of the hierarchy, select **Use KeyLOV Level(1)** for Country, select **Use KeyLOV Level(2)** for State, and select **Use KeyLOV Level(3)** for City.

Attribute Properties [Close]

Cancel Save Expand

Revision: 001
 Status: Released
 Name: Country
 SML Object ID: 13300111
 SML Sync Date: 2024-05-19T01:58:51+05:30
 Type: Property
 Annotation:
 [Text Area]

User Data:
 Metric Min Value:
 Non Metric Min Value:
 Metric Max Value:
 Non Metric Max Value:
 Metric Default Value:
 Non Metric Default Value:
 Metric Constant Value:
 Non Metric Constant Value:

Is Required:
 Dependency configuration Level:
 [Dropdown Menu]
 Use KeyLOV Level(1)
 Use KeyLOV Level(2)
 Use KeyLOV Level(3)

Is Protected:
 Is Auditable:

7. To specify the dependency between two attributes, select a class attribute and click **Attribute Properties** .
8. In the **Attribute Properties** panel, click **Edit**, and then select the dependent attribute from the **Dependent Attribute** list.

Example

State is dependent on Country. Therefore, for State, select the dependent attribute as Country. For City, select the dependent attribute as State.

Attribute Properties Close

Cancel Save Expand

Revision: 001

Status: Released

Name: State

SML Object ID: 13300112

SML Sync Date: 2024-05-19T01:58:51+05:30

Type: Property

Annotation:

User Data:

Dependency configuration Level:

useKeyLOVLevel(2)

Metric Min Value:

Non Metric Min Value:

Metric Max Value:

Non Metric Max Value:

Metric Default Value:

Non Metric Default Value:

Metric Constant Value:

Non Metric Constant Value:

Is Required:

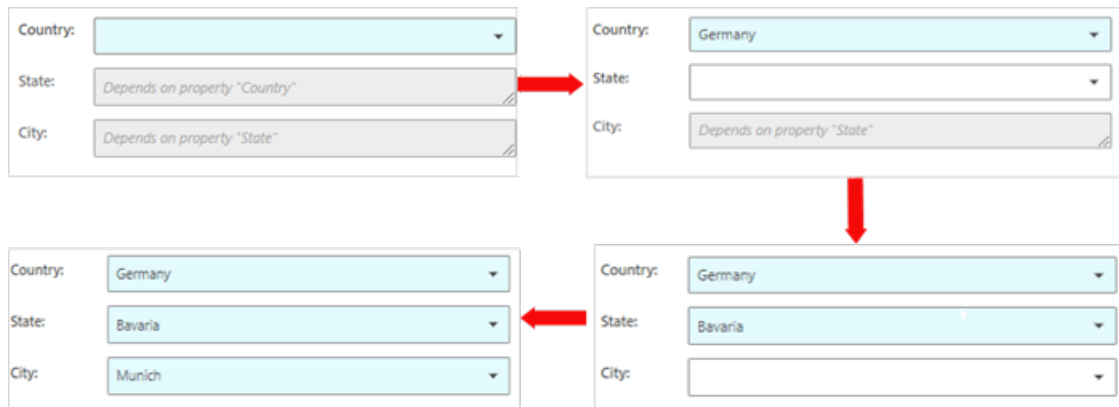
Dependent Attribute:

Country (SIM02#02-j9HKAA#001)

City (SIM02#02-kfHKAA#001)

Results

You can use the interdependent Key LOV in classification after releasing the class. If you select Country, the dependent Key LOV is available for selection. Also, when you select State, the dependent Key LOV is available for selection.



Using dynamic LOVs created in BMIDE

Dynamic LOVs are lists of values where the values are shown at runtime by querying the database. For example, a list of users or a list of parts belonging to a particular group. You can use these dynamic LOVs in Classification by associating the dynamic LOV with a Key LOV.

You can create Dynamic LOVs in Business Modeler IDE (BMIDE).

Procedure

1. On the **Classification Manager** dashboard, click **Key LOV** tab.
2. Click **New** .
3. In the **Add Key LOV** panel, specify the following details:
 - **Namespace**
 - **ID**
 - **Revision**
 - **Name**
 - **LOV Items**

String, integer, and double data types are supported.
4. From the **Dynamic LOV Name** list, select a dynamic LOV.

Add Key LOV Close

Object Type: KeyLOVDefinition

* Namespace:
0173-1

* ID:
AAD691

* Revision:
002

* Name:
Company Location

* LOV Items:
String

☒ Is Dynamic

* Dynamic LOV Name:
Fn0LocationCode : This is for Company Location

5. Click **Add**.

Create a property

A *property* is used to describe the attributes of a class. Attributes can be of any type, including string, integer, double, boolean, and reference.

Procedure

1. On the **Classification Manager** dashboard, click the **Properties** tab.
2. Click **New** .
3. In the **Add Property** panel, specify the following details:
 - **Namespace**
 - **ID**
 - **Name**
 - **Revision**

4. Select the **Data Type** from the list and then specify the required values for the selected data type, if available.
5. For certain data types, you can also specify the Key LOV by pasting the Key LOV ID in the **Key LOV** box.
6. Select the **Unit** applicable to the property from the list.

Add Property ✕ Close

★ ID: 124434

★ Name: 2d documentation

★ Data Type: String

★ Max Length: 5

Unit: len

- ▶ Length
- ▶ Force Per Unit Length
- ▶ Length Per Revolution

Add

You can expand the arrow to select a specific unit type. For example, for **Length**, you can select from the different units of length such as, **Meters**, **Centimeters**, **Nanometers**, and so on.

7. Click **Add**.
8. To edit any property, click **Edit** . The editable cells are activated.
9. Enter or update the property values as required.

▼ Data Type

Type:
String

Metric	Non-Metric
Max Length: 5	Max Length:
Unit: Nanometers	Unit:
Default Value: 	Default Value:
Options: 	Options:

10. Save your changes by clicking **Save**. To cancel your edits, click **Cancel**.

Create a class

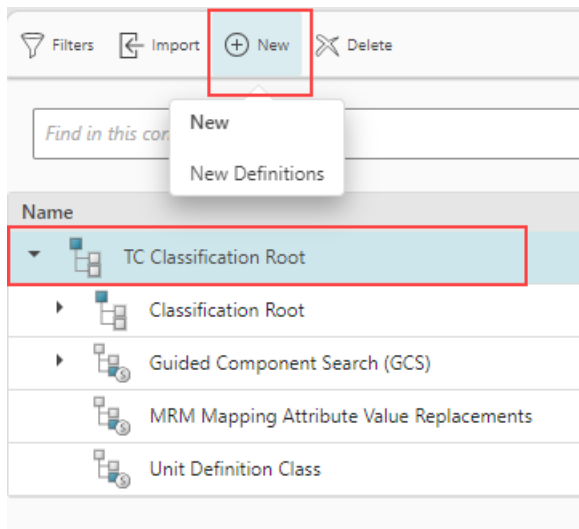
Create a class

Classes define the attributes that are stored for classified objects and determine which attributes are inherited by other classes. They are the primary building blocks of the Classification system. Workspace objects are classified by choosing a class from the hierarchy and providing values for the attributes of the class.

Procedure

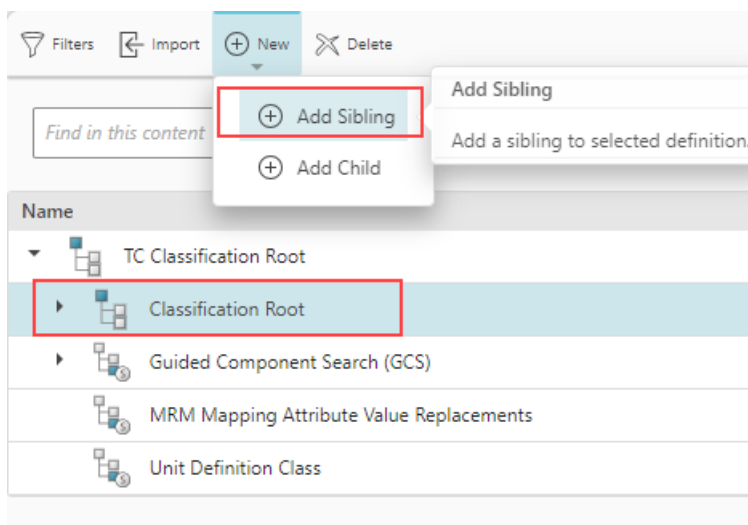
1. On the **Classification Manager** dashboard, click the **Classes** tab.
2. Select a hierarchy under which you want to add a new class. You can do any of the following:
 - Add a class definition directly under the top level root class.

To do this, select the top root class and click **New** ⊕.



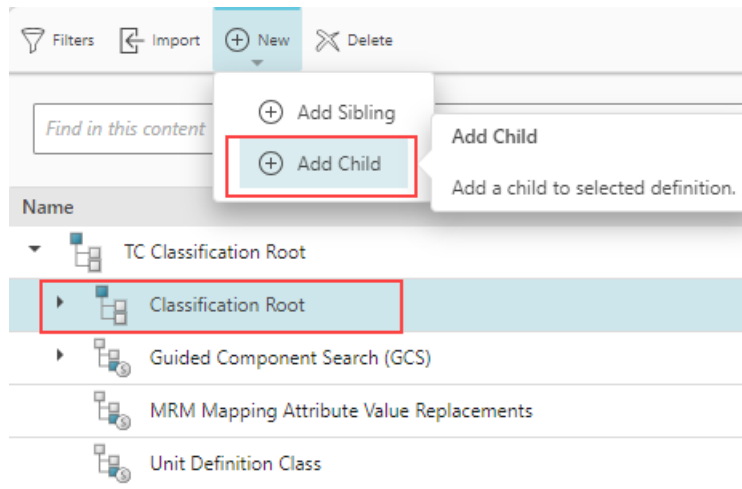
- Add a class definition as a sibling class at the same level as an existing class definition.

To do this, select an existing class definition, click **New** ⊕ > **Add Sibling** ⊕.



- Add a class definition as a child class under an existing class definition.

To do this, select an existing class definition, click **New** ⊕ > **Add Child** ⊕.



3. Specify the **Namespace**, **ID**, **Revision**, and **Name** for the class.
4. Select **Class Type** from the list.
 - **Application Class**
 - **Aspect**
 - **Block**
5. Select **Metric**, **Non-Metric**, or **both** from the **Unit System** list to specify the system of measure to be applied to the attributes of the class.
6. Click **Add**.

A new class is added within the selected hierarchy. You can click **Show All** in the **Overview** tab to view all the properties associated with the class.

After creating a class, you must [add attributes to the definition of the class](#). Attributes added to the class become inherited attributes of any child classes.

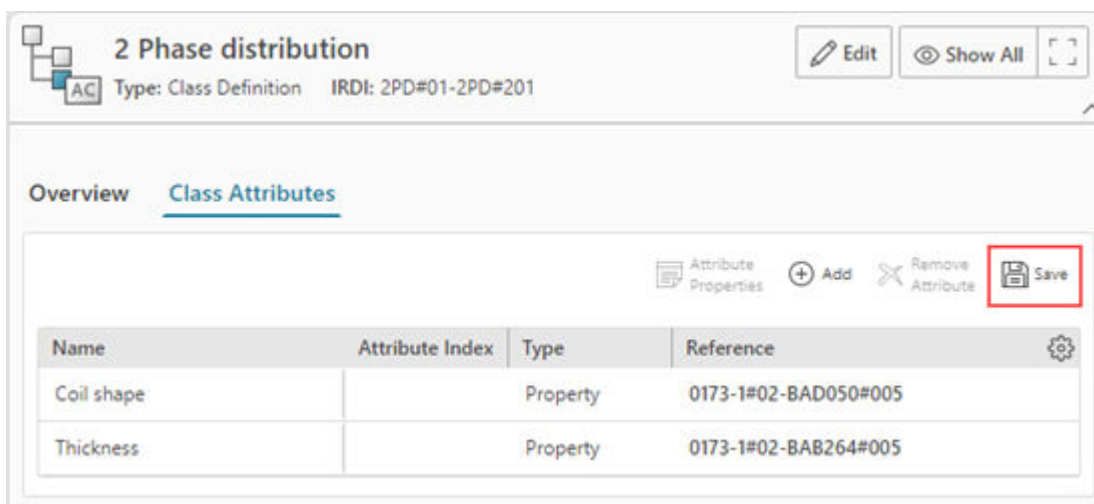
Create a class attribute

Attributes are either inherited or directly associated with the class. The **Inherited Attributes** list displays all the attributes that are inherited from any and all parent classes. Inherited attributes cannot be deleted or modified. You can, however, add attributes to the definition of the class. Attributes added to the class become inherited attributes of any child classes. A class can contain a maximum of 200 attributes, both inherited and local.

Procedure

1. Open the class definition to which you want to add the class attribute. Click **Class Attributes**.
2. Click **Add** (+).
3. In the **Add Class Attribute** panel, search for the attributes that you want to add to the class. Available search filter options are:

- **Name**
 - **ID**
 - **Revision**
 - **Namespace**
 - **Status**
4. Specify the **Value** of the selected attribute.
 5. Click **Search**.
 6. Select the class attribute that you want to add from the search results and click **Add**. You can also select multiple class attributes to add at a time.
 7. After all the attributes are added, click **Save**.



The new class attributes are displayed automatically in the **Class Attributes** table.

8. To view additional attribute properties for each class attribute, select the class attribute and click **Attribute Properties** .

If you select multiple class attributes and click **Attribute Properties** , you can compare attribute properties for all the selected class attributes.

9. To remove a class attribute, select the attribute and click **Remove Attribute** .
10. To edit any class attribute, click **Edit** .

The editable cells are activated.
11. Enter or update the class attributes as required.
12. To save your changes, click **Save** . To cancel your edits, click **Cancel** .

Capture additional information for class attributes

With User Data entries, you can attach additional information to a class attribute, even when the class is released.

Procedure

- 1. In **Classification Manager**, select a class attribute.
- 2. Select **User Data** .
- 3. To add a new entry, in the **User Data** panel, click **Add** .

Details ▾

 AC-DC supply

Overview

Class Attributes

Views

Name	Attribute Index	Type	Data Type	Referen
Design of electrical connection	13	Property	String	0173-1#
Direct assembly/mounting pos...	14	Property	Boolean	0173-1#
max. 1. output voltage with AC	15	Property	Double	0173-1#
max. 1. output voltage with DC	16	Property	Double	0173-1#
max. 2. output voltage with AC	17	Property	Double	0173-1#

User Data

 Pin Panel  Close

 Add

▼ User Data

User Data Entry

- 4. Add the **New User Data Name** and **User Data Value** and click **Save**.

User Data

 Pin Panel  Close

* New User Data Name:

Distance

* User Data Value:

500

Save

Discard

8-24
Software Version 2606

Advanced Classification — Deployment and Administration
Unpublished work. © 2026 Siemens

The screenshot shows a 'User Data' panel with a dark blue header. In the top right corner of the header are 'Pin Panel' and 'Close' buttons. Below the header, there is an 'Add' button with a plus icon. The main content area is titled 'User Data' with a dropdown arrow. It contains a 'User Data Entry' section with a dropdown menu showing 'Distance' and a text input field containing the value '500'.

5. To add an additional value to the existing user data entry:
 - a. Select the class attribute that has an existing user data entry.
 - b. Select **User Data** .
 - c. Select the user data entry.
 - d. Select **Add**  > **Add New User Data Entry** .
 - e. Add the additional value and click **Save**.

The screenshot shows the 'User Data' panel after adding a new entry. The header and 'Add' button are the same. The main content area now shows a new entry titled 'User Data Value' with a blue asterisk icon. Below the title is a large text input field containing the value '600'. At the bottom of the panel are two buttons: 'Save' and 'Discard'.

The screenshot shows the 'User Data' panel with a blue header bar containing 'Pin Panel' and 'Close' icons. Below the header is a '+ Add' button. The main content area is titled 'User Data' and contains a table with the following structure:

User Data Entry	
▼ Distance	
	500
	600

- To add additional user data entries in the same class attribute, in the **User Data** panel, click **Add** (+) and add the user data entry.

The screenshot shows the 'User Data' panel with a blue header bar containing 'Pin Panel' and 'Close' icons. Below the header is a '+ Add' button. The main content area is titled 'User Data' and contains a table with the following structure:

User Data Entry	
▼ Distance	
	500
	600
▼ Output notes	
	Warning when voltage crosses 100 volts

- To edit a user data entry value, select the value, click **Edit** (pencil icon), and update the value.
- To delete the user data entry or value, select the user data entry value and click **Delete User Data Entry** (X icon).

Create a node

You can create a node with details such as node properties, application class, parent, and the details about the class and its properties. You can use the property groups navigator to get the property details within each property group.

However, if the nodes are not created automatically, you can do so for the basic data as can be done for the advanced classification as follows.

Procedure

1. On the **Classification Manager** dashboard, click the **Nodes** tab.
2. Click **New** ⊕ > **Add Sibling** ⊕.
3. In the **Add Node** panel, select the **Node Type** from the list.
4. Specify the details in the remaining fields based on the selected node type.
5. Click **Add**.

A node is created, and the information about the node type and the application class is displayed under the **Overview** section.

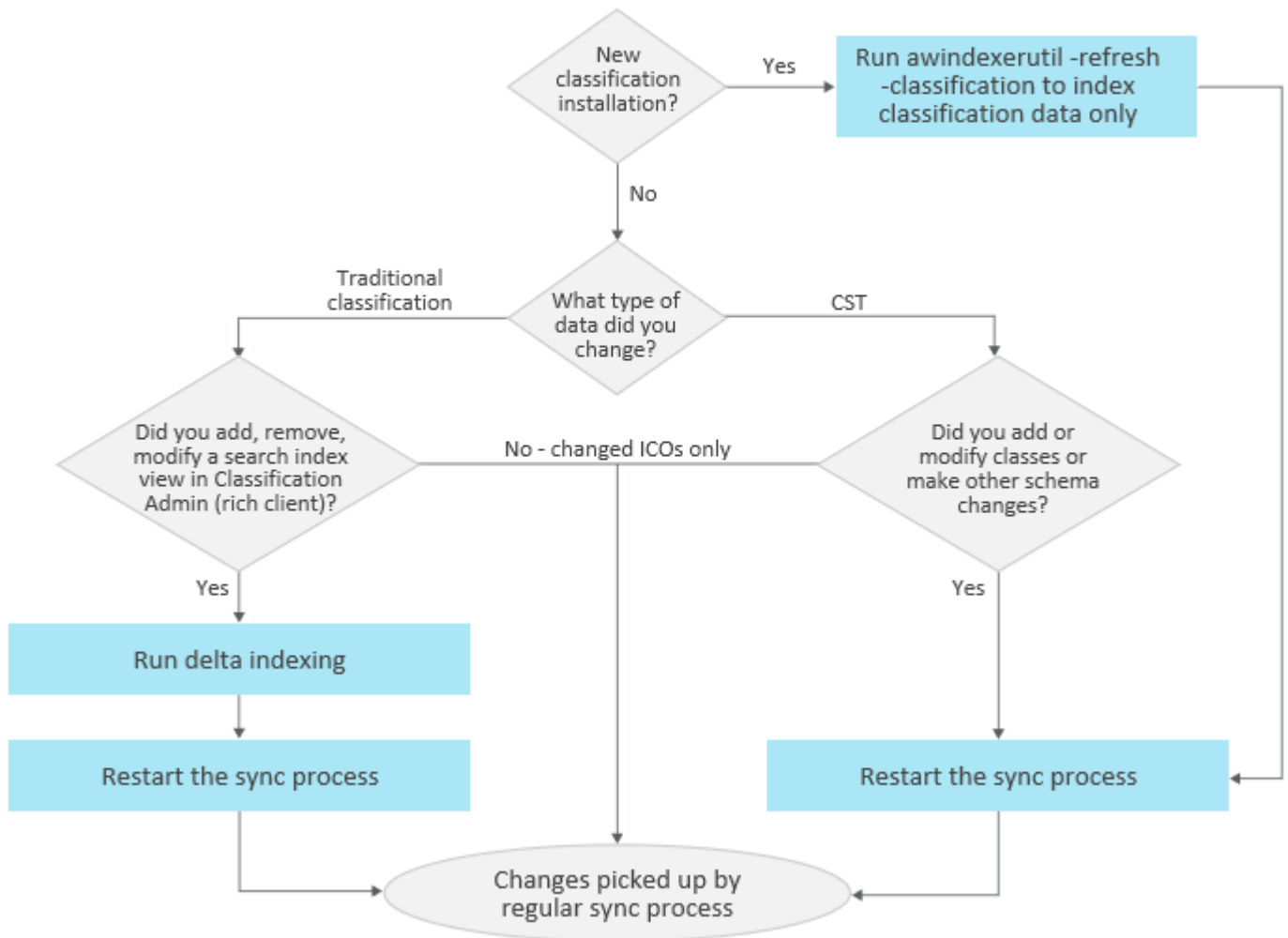
After you create or update the classification definitions for advanced classification in Classification Manager, you must [release these definitions](#) for the classification hierarchy to be available in the user interface for classifying the data.

Indexing classification data

About indexing classification data

Many changes to classification classes and data require indexing. There are several classification indexing options available to you. Classification indexing is based on the general Active Workspace indexing mechanism. As a prerequisite, classification assumes that general Active Workspace indexing is fully functional and synchronizing on a regular basis. The following workflow assists you with your indexing decisions:

Prerequisite: Set up Active Workspace indexing with the sync process running



Index classification data

If indexing is set up and running, use classification indexing to discover only the objects whose classification data has changed. This process marks these objects for indexing, and they are picked up by the next scheduled round of regular indexing.

Procedure

1. Ensure that regular indexing is running by entering the following in a command line:

```
runTcFTSIndexer -task=objdata:test
```

2. Do one of the following:

- Mark all classification data for indexing (or re-indexing):

Run the following command:

```
awindexerutil -u=user -p=password -g=group -classification -refresh
```

Use this command if you have performed an initial installation of a classification feature or made substantial changes to your classification data and want to refresh all data. Running this command marks the following objects for indexing:

- Workspace objects classified in either traditional basic or advanced CST hierarchies
- Standalone classification objects (**clsObject** and **clsCstObject**)
- Library elements

If the release you are upgrading to include new classification features, it generally also includes schema changes. During this upgrade procedure, all classification data is marked for re-indexing and included during your next indexing synchronization. This can cause the synchronization to take longer than usual.

- Index only changes made to the search index views in traditional basic classification:

The search index views in traditional basic classification define which classes and properties are searchable. If you make any changes to these views, you can index only the changes made in the search index views. Changes in the search index view that require indexing are:

- Adding or removing a search index view from a class
- Adding or removing properties from a search index view

a. **Make properties searchable in the global search.**

b. (Optional) Run the **awindexerutil** utility:

```
awindexerutil -u=user -p=password -g=group -delta -classification -dryrun
```

Running this utility using the **dryrun** option displays the objects that will be marked for indexing in the next step. Scanning this output can assist you in troubleshooting the indexing of your data.

c. Run the **awindexerutil** utility:

```
awindexerutil -u=user -p=password -g=group -delta -classification
```

This step marks changed data for indexing. Once marked, these objects are picked up in the next regular indexing synchronization flow.

Starting and stopping the indexing synchronization process

Before you begin classification indexing activities, you must stop the index synchronization process. The classification indexing mechanisms then mark objects for indexing. These objects are indexed the next time the synchronization process occurs.

After performing any classification indexing activities, it is important to restart the synchronization process, so the marked objects are picked up and indexed. Do this by scheduling regular syncing of the database with the **runTcFTSIndexer** with the **-task=objdata:sync** argument.

Classifying objects using clsutility

Understanding object classification with clsutility

In addition to classifying objects in the user interface, you can also perform this activity using **clsutility**.

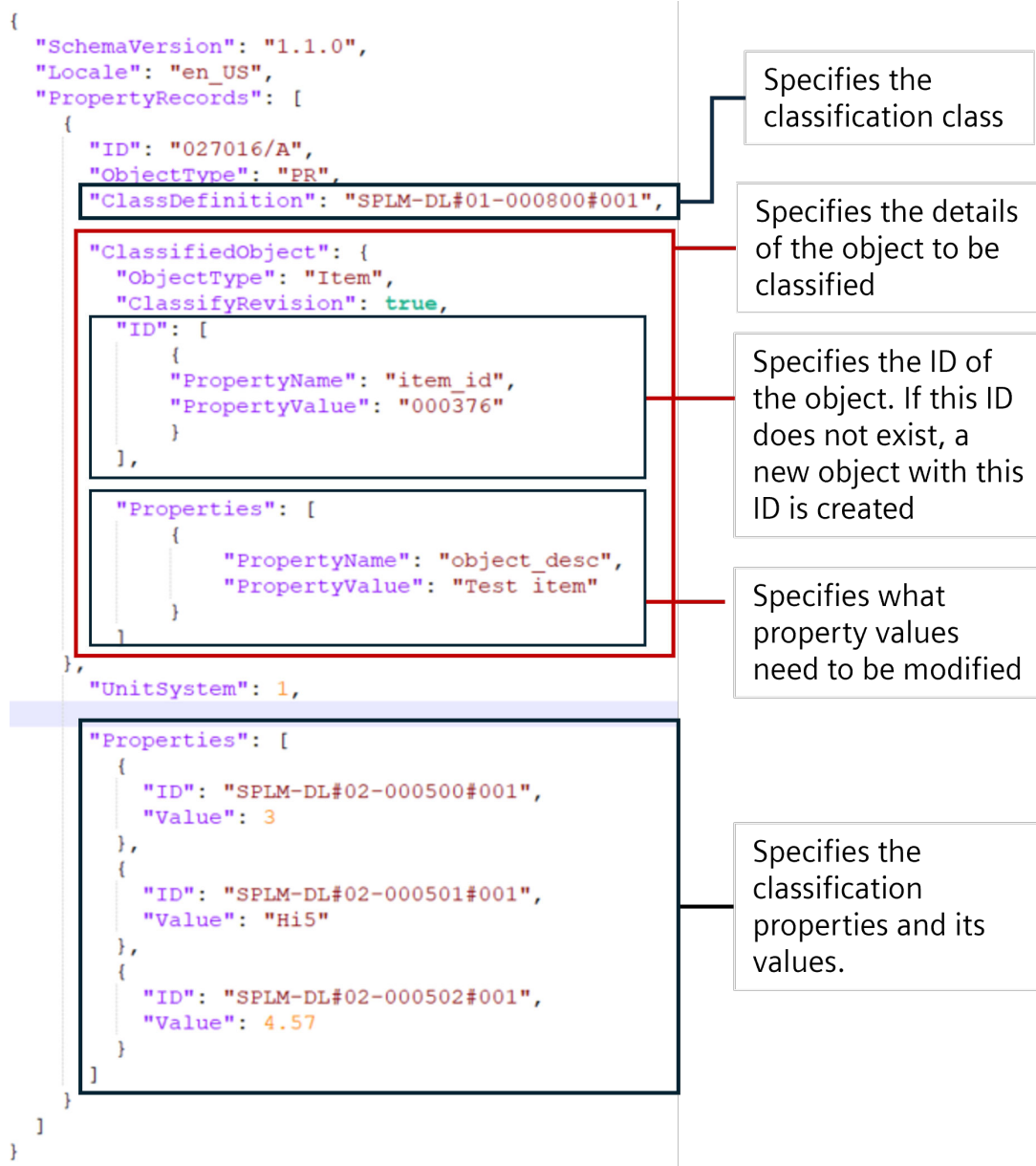
The utility requires a JSON file as input. The schema file used to create this request can be found in:

```
...\\<TC_DATA>\\classification
\\json\\schema\\advanced\\Classification_Save_PropertyRecords_Request_advanced.schema.json
```

The following sample request, called **classify.json**, is based on this schema:

```
{
  "SchemaVersion": "1.1.0",
  "Locale": "en_US",
  "PropertyRecords": [
    {
      "ID": "027016/A",
      "ObjectType": "PR",
      "ClassDefinition": "SPLM-DL#01-000800#001",
      "ClassifiedObject": {
        "ObjectType": "Item",
        "ClassifyRevision": true,
        "ID": [
          {
            "PropertyName": "item_id",
            "PropertyValue": "000376"
          }
        ],
        "Properties": [
          {
            "PropertyName": "object_desc",
            "PropertyValue": "Test item"
          }
        ]
      }
    }
  ],
}
```

```
"UnitSystem": 1,  
"Properties": [  
  {  
    "ID": "SPLM-DL#02-000500#001",  
    "Value": 3  
  },  
  {  
    "ID": "SPLM-DL#02-000501#001",  
    "Value": "Hi5"  
  },  
  {  
    "ID": "SPLM-DL#02-000502#001",  
    "Value": 4.57  
  }  
]  
]  
}
```



When updating the JSON file, specify the following details:

- For **ClassDefinition**, specify the IRDI of the classification class.
- In the **ClassifiedObject** section, specify the details for the object to be classified.

Note

For the **ClassifiedObject** section, you can either specify the details for the object to be classified, or you can include only the UID of the object.

Specify details of the object	Specify only the UID of the object
<pre> "ID": "000370", "ObjectType": "PR", "ClassDefinition": "TDOC#01-LLC#009", "ClassifiedObject": { "ObjectType": "Item", "ClassifyRevision": true, "ID": [{ "PropertyName": "item_id", "PropertyValue": "000376" }] }, </pre>	<pre> "ID": "000370", "ObjectType": "PR", "ClassDefinition": "TDOC#01-LLC#009", "ClassifiedObject": "AondLlu2JfCsbC" "UnitSystem": 1, </pre>

- In the **Properties** section, specify the classification properties and values.

Classify objects using clsutility

Classifying an object using **clsutility** requires a JSON file as input.

Note

The schema file used to create this request can be found in:

```

...ITC_DATA\classification
\json\schema\advanced\Classification_Save_PropertyRecords_Request_advanced.schema.json

```

Procedure

1. Create a JSON input file.

The following sample request, called *classify.json*, is based on this schema:

```

{
  "SchemaVersion": "1.1.0",
  "Locale": "en_US",
  "PropertyRecords": [
    {
      "ID": "027016/A",
      "ObjectType": "PR",
      "ClassDefinition": "SPLM-DL#01-000800#001",
      "ClassifiedObject": {
        "ObjectType": "Item",
        "ClassifyRevision": true,
        "ID": [
          {
            "PropertyName": "item_id",
            "PropertyValue": "000376"
          }
        ]
      }
    }
  ]
}

```

```

    },
    "Properties": [
      {
        "PropertyName": "object_desc",
        "PropertyValue": "Test item"
      }
    ]
  },
  "UnitSystem": 1,
  "Properties": [
    {
      "ID": "SPLM-DL#02-000500#001",
      "Value": 3
    },
    {
      "ID": "SPLM-DL#02-000501#001",
      "Value": "Hi5"
    },
    {
      "ID": "SPLM-DL#02-000502#001",
      "Value": 4.57
    }
  ]
}
]
}
}

```

2. Run the **clsutility**.

To classify the specified object, **027016/A**, in the **SPLM-DL#01-000800#001** class, enter the following command:

```
clsutility -u=user -p=password -g=group -create -classification_objects
-request="classify.json"
```

Classify objects in multiple classes using clsutility

By default, objects cannot be classified in the same class multiple times. However, after an object is classified once, you can classify it in other classes, using a process called **multiple classification**.

You can perform multiple classification using the same **clsutility** command and schema file that you used to complete a single object's classification, but the request must include the **"IsMultipleClassification":true** parameter.

Procedure

1. Create a separate JSON file with information about additional classification, and ensure that you include the **"IsMultipleClassification":true** parameter.

```
{
  "SchemaVersion": "1.1.0",
  "Locale": "en_US",
  "PropertyRecords": [
    {
      "ID": "027016/A",
      "ObjectType": "PR",
      "ClassDefinition": "SPLM-DL#01-000800#001",
      "IsMultipleClassification": true,
      "UnitSystem": 1,
      "Properties": [
        {
          "ID": "SPLM-DL#02-000500#001",
          "Value": 3
        },
        {
          "ID": "SPLM-DL#02-000501#001",
          "Value": "Hi5"
        },
        {
          "ID": "SPLM-DL#02-000502#001",
          "Value": 4.57
        }
      ]
    }
  ]
}
```

2. Run the **clsutility**.

To classify the specified object, **027016/A**, in the **SPLM-DL#01-000800#001** class, enter the following command:

```
clsutility -u=user -p=password -g=group -create -classification_objects
-request="classify.json"
```

Modify a classification using clsutility

To modify a classification using **clsutility**, the request must include the UID of the property record (ICO) that you want to modify, which is specified as the **PropertyRecordUID**.

Procedure

1. Create a JSON input file with information about the classification to modify, and specify the UID of the property record (ICO).

```
{
  "SchemaVersion": "1.1.0",
  "Locale": "en_US",
  "PropertyRecords": [
    {
      "ID": "027016/A",
      "ObjectType": "PR",
      "ClassDefinition": "SPLM-DL#01-000800#001",
      "PropertyRecordUID": "g_S588PW5c0mcA",
      "UnitSystem": 1,
      "Properties": [
        {
          "ID": "SPLM-DL#02-000500#001",
          "Value": 4
        },
        {
          "ID": "SPLM-DL#02-000501#001",
          "Value": "Hi5_new"
        },
        {
          "ID": "SPLM-DL#02-000502#001",
          "Value": 5.57
        }
      ]
    }
  ]
}
```

2. Run the **clsutility**.

To classify the specified object, **027016/A**, in the **SPLM-DL#01-000800#001** class, enter the following command:

```
clsutility -u=user -p=password -g=group -create -classification_objects
-request="classify.json"
```

Find the UID of a classified object

To find the UID of a classified object, use the `...\\TC_DATA>\\classification\\json\\schema\\advancedClassification_FindRequest_advanced.schema.json` schema to create a JSON request. This request finds the UID of the classifying object by using the ID of the classified object.

Procedure

1. Create a JSON input file.

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ObjectIDList": [
    {
      "ID": "027016/A"
    }
  ]
}
```

2. Run the **clsutility**.

```
clsutility -u=user -p=password -g=group -find -classification_objects
-request=find_request.json -output=<your-output-file.txt>
```

Results

The output of this command includes the value for **PropertyRecordUID**, which is the UID of the ICO that classifies the workspace object included in the request, in this case, **027016/A**.

Can I delete classifications using clsutility?

You can only delete classifications from the Teamcenter Classification user interface.

Updating the status of classification objects

Change the status of Classification objects

You can only use Classification objects that are in the released state.

Classification Standard Taxonomy supports three statuses:

- **Develop**
- **Released**
- **Retired**

Every class, property, and key-LOV is created with a status.

- When you create a new object and do not specify a release status, it is automatically assigned the **Develop** status. You can also specify the status explicitly:



This means the object is in a draft state. You can still modify it, for example, by adding or removing attributes in a class, but you cannot yet use it to classify data.

- Once the editing work is complete, you change the status of the object to **Released** using the **clsutility** command line utility or through the user interface.

- A **Released** class is available in the user interface to classify your data. You can [make minor changes in a released class without a new revision being created](#).
- If you create a new revision of a class, the status of the previously existing revision is automatically changed to **Retired**. All classified objects in the **Retired** class are automatically moved to the **Released** class, with the exception of those that are already released.
- When you create a new revision and change the status of the class, logoff and logon to Teamcenter to see the change in status.
- All revisions are imported with three digits, for example, **001**. If revisions are less than three digits, zeros are prefixed up to three digits so that **01** becomes **001**. If you then search for revision **01**, it is not found.

Note

Nodes do not have a status and cannot be used to change the status of the application classes they reference.

After releasing a class, it is good practice to [create a base view definition of the class](#). This improves the performance of the system and allows you to modify the order of presentation of the class properties.

Change the status of a class to Released using the clsutility command

You can change the status of the class to **Released** using the **clsutility** command line utility.

Procedure

To change the status of the class run **clsutility** as follows:

```
clsutility -u=<user> -p=<password> -g=<group> -update -status
-request=<JSON_file>
```

The JSON file must have a format similar to the following:

```
{
  "SchemaVersion": "1.0.0",
  "Options": ["recursive"],
  "Status": "Released",
  "ObjectIRDIList":
    [
      "TDOC#01-CLS001#001"
    ]
}
```

Running the utility using the **recursive** option changes the status of the class, as well as all the objects it references (properties, Key-LOV definitions, blocks, aspects) from **Develop** to **Released**.

Note

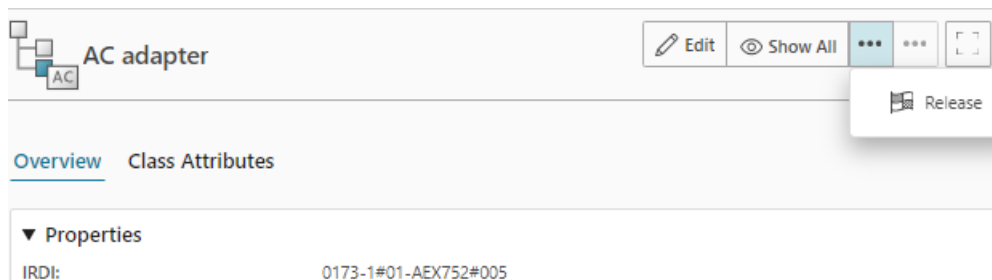
Nodes do not have a status and cannot be used to change the status of the application classes they reference.

Change the status of a class to Released from the user interface

You can change the status of the application class to **Released** from the user interface.

Procedure

To change the status of the class choose **More Commands*** > Release**.



Results

The status of the class is changed to **Released** and the status of all the properties and Key-LOVs associated with the class are also updated to released.

When to revise classification definitions

You can update classification definitions using revise or without revise. The decision depends on whether the change affects existing classified objects.

Use revise when the change can affect existing classification object data. Use update without revise when the change is small, safe, and does not change existing classification object data.

When to use revise

Use revise for changes that change, remove, convert, or reinterpret existing classification data.

Examples:

- Changing the data type of a property.
- Deleting an attribute that may already contain values.

- Moving an attribute in a way that changes existing classified object data.
- Changing the meaning of a property or list value.
- Making a change that is not supported by the update schema.
- Making a change when released classification definitions must remain unchanged for audit or compliance reasons.

When to update without revise

Use update without revise for changes that do not require existing classified objects to be modified.

Examples:

- Correcting a name or spelling mistake.
- Updating a description, note, remark, or translation.
- Adding a new value to a Key LOV.
- Adding a new attribute, if the business accepts that old classified objects may show the new attribute as empty.

For more information on updating without revise, see [Updating advanced classification objects without revisioning](#).

Planning for dependencies

Before you revise a Key LOV, property, class, or block, check where it is used. A revised object is not automatically adopted by all objects that reference it. You must decide which properties or classes should use the new revision.

If you are unsure whether a change requires revising, check the following schema file:

```
<TC_DATA>\classification\json\schema\advanced\SaveClassificationDefinitions_advanced.schema.json
```

In this schema, sections with names ending in **UpdateDefinition**, such as **PropertyUpdateDefinition**, usually define the changes that can be made without revising.

	{
	"\$schema": "http://json-schema.org/draft-04/schema#",
	"description": "Object defining the request that requir
	included in this file will be imported in the following
	Aspects, then Application Classes), ViewDefinitions, an
	"type": "object",
	"properties": {
	"required": [
	"additionalProperties": false,
	"definitions": {
	"PropertyDefinitions": {
	"PropertyDefinition": {
	"PropertyUpdateDefinition": {
	"KeyLOVDefinitions": {
	"KeyLOVDefinition": {
	"KeyLOVUpdateDefinition": {
	"ClassDefinitions": {
	"ClassDefinition": {
	"ClassUpdateDefinition": {
	"ClassAttribute": {
	"ClassAttrOverlayDefinition": {
	"ClassAttributeUpdate": {
	"ClassAttrOverlayUpdateDefinition": {

Updating advanced classification objects without revisioning

In general, major changes in definition objects should result in a new revision of that object. However, many companies have workflows that involve making smaller tweaks, delta updates, and corrections to classes, properties, or key-LOVs in their hierarchies. Revising the object each time a minor change is made may, in this case, result in a cluttered database with many meaningless revisions. To avoid this, you can make the following types of changes to an object definition without the object being revised:

- Updating or fixing spelling errors, for example, **Name** or **Description**
- Adding translations
- Adding key-LOV entries
- Deprecating key-LOV entries
- Adding class attributes
- Deprecating class attributes

Note the following:

- Partial update is available beginning with schema 1.5.0 using the **SaveClassificationDefinitions_advanced.schema.json** schema file, found in `...\\<TC_DATA>\\classification\\json\\schema\\advanced\\`. Using this schema, you can both create definition objects and update others in the same JSON file.
- Only modifications that do not change the existing classification objects and their properties are allowed.
- Semantic changes are not permitted.
- You cannot update nodes as nodes are just references to classes.

Example

You have a key-LOV, **SPLM-0#09-KEY002#001**, with the following entries:

ENTRIES		
StringValue	DisplayValue	ValueMeaning
Rear	Rear Seat	SPLM-0#07-ABC1#001
Front	Front Seat	SPLM-0#07-ABC2#001
3rd Seat Row	3	SPLM-0#07-ABC3#001

You want to change the display name of the third key-LOV value from **3** to **Fold Away Seat**.

Using schema 1.5.0, you create the following JSON file:

```
{
  "SchemaVersion": "1.5.0",
  "Locale": "en_US",
  "KeyLOVDefinitions": [{
    "Update": "SPLM-0#09-KEY002#001",
    "Values": {
      "LOVItems": [{
        "Update": "3rd Seat Row",
        "Values": {
          "DisplayValue": "Fold Away Seat"
        }
      }
    ]
  }
}]
}
```

You can [import this file](#) and then run the following command using **clsutility**:

```
clsutility -u=<user> -p=<password> -g=<group> -save -classification_definitions -request="<path-to-JSON-file>" -verbose -format
```

When complete, the key-LOV is displayed as follows:

▼ ENTRIES		
StringValue	DisplayValue	ValueMeaning
Rear	Rear Seat	SPLM-0#07-ABC1#001
Front	Front Seat	SPLM-0#07-ABC2#001
3rd Seat Row	Fold Away Seat	SPLM-0#07-ABC3#001

The revision of the key-LOV remains the same.

You can also perform these minor delta updates for class, class attributes, and properties from the user interface by using the **Edit** command.

Note that you cannot make these delta updates to key-LOV from the user interface.

Revise a class

Revising a class allows you to modify class definitions that are released.

Procedure

Select a release class and click **More Commands > Revise**.

Administering classification

Deleting CST structures and data

Deleting objects is performed using **clsutility** with the **-delete** arguments. You may delete any of the following using the utility:

- Nodes
- Classification objects

- Classes
- Properties
- Key-LOVs

The utility requires a JSON file as input. The format for the JSON is found in the [schema directory](#).

The following example contains the request used to test (using the **dryrun** option) whether the **TDOC#01-CLS001#001** class and all the objects it references (including other block classes, key-LOVs, and properties) can be deleted:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "Options": [ "recursive", "dryrun"],
  "ObjectIRDIList": [
    "TDOC#01-CLS001#001"
  ]
}
```

To actually delete the class and all the objects it references, remove the **dryrun** option in the JSON request file and run the **clsutility -delete** command again.

Note the following:

- To delete a branch in the classification hierarchy, delete the topmost class in the branch using the **recursive** argument. This deletes everything from that class downwards and prevents you from having to list all objects requiring deletion individually.
- You cannot delete a class in which objects are classified. You must first delete the classifications and then the classes.

Enable notifications for the objects you follow that have classification criteria

To receive notifications when an event occurs on the objects you follow, activate notifications.

Procedure

1. Setup Teamcenter Subscription.
For more information about subscriptions, see Subscription Management.
2. Update the value of the **CLS_enable_subscription** preference to show classification in Subscriptions.
You must also define classification criteria on the objects you follow.

What to do next

After enabling notifications, you can check if you receive notifications by defining classification criteria on the objects you follow.

About cardinality on property blocks

Cardinality on property blocks allows a user to specify the number of times that a property block class is used in an application class. For example, when designing a power supply for mobile devices, there is a class called **Power supply** containing two properties, **Designation** and **Type**. The number of times you can enter a designation and type depends on the value of a third property, **Number of slots**.

In the user interface, you first specify the number of slots , and then, for each slot, specify a designation and type.

PROPERTIES

keywords

Unit System: ☒ Metric ☐ Non-Metric

▼ SLOT

Number of slots:

2  **1**

▼ SLOT 1

Designation:

Type of slot:

▼ SLOT 2

Designation:

Type of slot:

PROPERTIES

keywords

Unit System: ☒ Metric ☐ Non-Metric

▼ SLOT

Number of slots:

3  **1**

▼ SLOT 1

Designation:

Type of slot:

▼ SLOT 2

Designation:

Type of slot:

▼ SLOT 3

Designation:

Type of slot:

The **Number of slots** property is referred to as the *cardinality controller*.

About polymorphism

Polymorphism refers to the idea of displaying different sets of properties depending on the value of another property. For example, when designing a power supply for mobile devices, two types of power supplies are produced with the following connector types:

Regular	Deluxe
Micro USB	Micro USB
USB	USB
Lightning	Lightning
	USB-Type C
	Thunderbolt

The set (property block class) of connector types (properties) displayed depends on a property called **Type of power supply** (key-LOV). This property is referred to as the *polymorphism controller*. On the user interface, this is displayed as follows. A user can choose between a **Regular** or a **Deluxe** power supply and the properties displayed vary.

ASSIGNED CLASSIFICATIONS  

Automotive Engineering > Assemblies > Electronics > Mobile power supplies

▼ **AVAILABLE CLASSES**



Automotive Engineering

Assemblies

Electronics

Mobile power supplies

▼ **PROPERTY GROUPS**

 Power supply

PROPERTIES

Unit System: ☒ Metric ☐ Non-Metric

▼ **POWER SUPPLY**

Type of power supply:

☐ Micro USB

☐ USB

☐ Lightning

▼ **POWER SUPPLY**

Type of power supply:

☐ Micro USB

☐ USB

☐ Lightning

☐ Thunderbolt

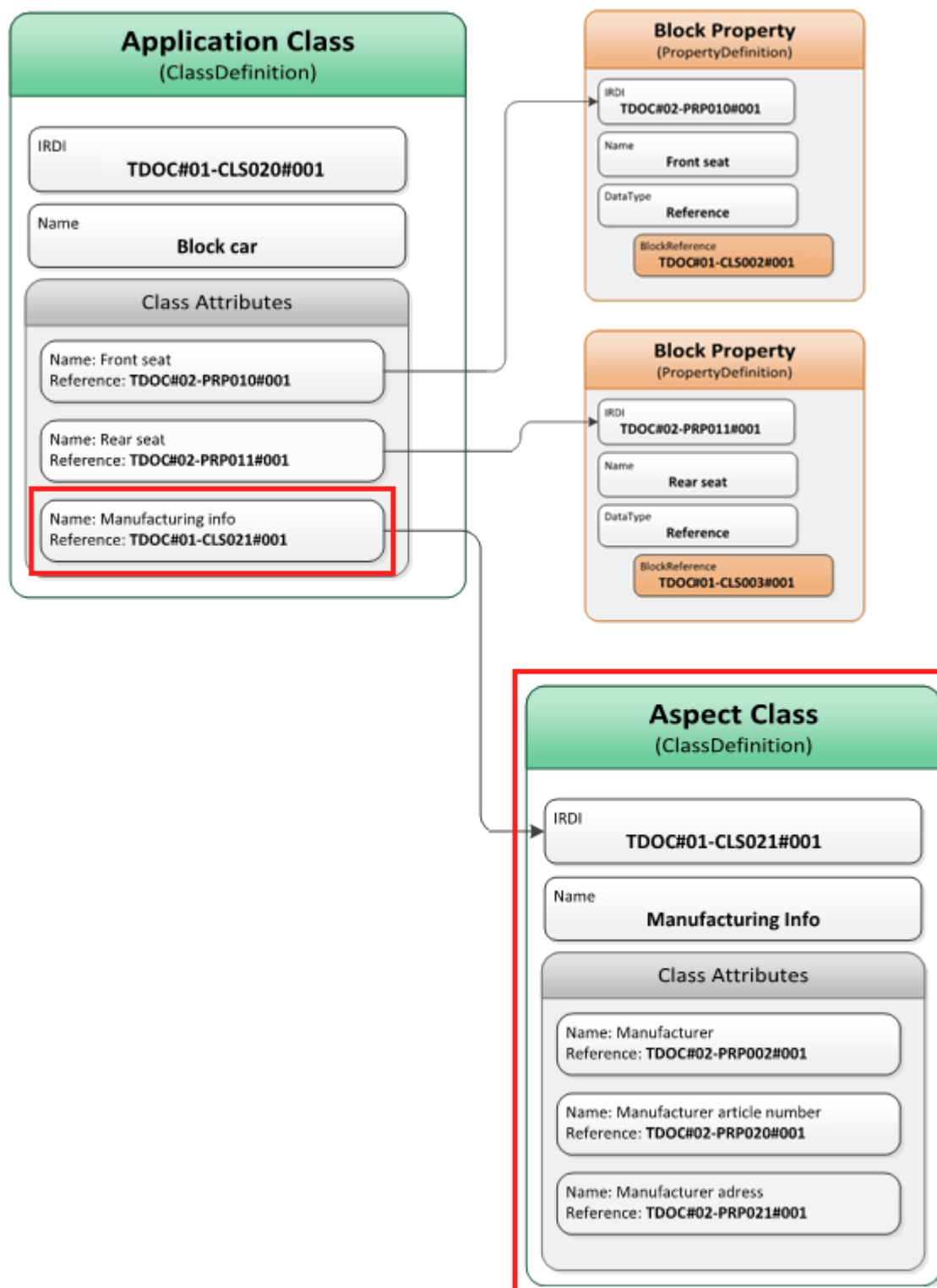
☐ USB Type C

About aspects

An aspect is an **ECLASS object** that represents a group of properties. In Classification Standard Taxonomy terms, an aspect is a kind of block class that can be referenced directly by another application class without going through a block property first.

Note that blocks, aspects, cardinality, and polymorphism are also supported in Advanced Classification without ECLASS taxonomy.

Example:



The two class attributes, **Front seat** and **Rear seat**, in the application class reference block properties that, in turn, reference property block classes. The third class attribute, **Manufacturer Info**, references the aspect class directly. The aspect class groups the properties pertaining to the manufacturer in one class.

The aspect class is represented in the JSON file as follows:

```

1  {
2    "SchemaVersion": "1.0.0",
3    "Locale": "en_US",
4    "ClassDefinitions": [
5      {
6        "ObjectType": "01",
7        "Namespace": "TDOC",
8        "ID": "CLS021",
9        "Revision": "001",
10       "Name": "Front seat",
11       "Status": "Develop",
12       "IsAbstract": "true",
13       "UnitSystem": 2,
14       "ClassType": "Aspect",
15       "ClassAttributes": [
16         {
17           "Type": "Property",
18           "Reference": "TDOC#02-PRP002#001"
19         },
20         {
21           "Type": "Property",
22           "Reference": "TDOC#02-PRP020#001"
23         },
24         {
25           "Type": "Property",
26           "Reference": "TDOC#02-PRP021#001"
27         }
28       ]
29     }
30   ]
31 }

```

The application class that references the aspect class is described as follows:

```

1  {
2    "SchemaVersion": "1.0.0",
3    "Locale": "en_US",
4    "ClassDefinitions": [
5      {
6        "ObjectType": "01",
7        "Namespace": "TDOC",
8        "ID": "CLS020",
9        "Revision": "001",
10       "Name": "Car",
11       "Status": "Develop",
12       "IsAbstract": "False",
13       "UnitSystem": 3,
14       "ClassType": "Application Class",
15       "ClassAttributes": [
16         {
17           "Type": "Property",
18           "Reference": "TDOC#02-PRP010#001"
19         },
20         {
21           "Type": "Property",
22           "Reference": "TDOC#02-PRP011#001"
23         },
24         {
25           "Type": "Block",
26           "Reference": "TDOC#01-CLS021#001"
27         }
28       ]
29     }
30   ]
31 }

```

Caution

- When importing classes with aspects, you must first import the JSON file containing the aspect classes. Next, you import the application class that references the aspect. You cannot import these two types of classes using the same JSON file.
- You cannot reference an aspect class in a block class.
- Aspects cannot reference other aspects.

Add classification properties to a table

To display specific classification properties in a table such as your search results table, you can add the desired properties to the table columns. This allows you to view the classification information that is most relevant to your needs.

To add classification properties in your search results table, you must:

- Choose the classification properties you want to add to the search results table and find the **Dynamic Compound Property (DCP)** value of the classification property using the **clsutility**.
- Add the DCP value to the table configuration.

Procedure

Find the DCP value of the classification property

- Run the **clsutility** command:

```
clsutility -list -hierarchy -nodeId=0173-1#AFX043#002 -showType
-showDCP -recursive > C:\output.txt
```

From the command replace the value of the **nodeId** with the **Node ID** of the node.

- From the output of the **clsutility**, copy the relevant DCP value .

```
143 : [cst00173-1#02-AAM549#001] (String)1 "Supplier item type"
    DCP :
    GRM(Cls0ClassifiedBy,Cst00173-1#AFR745#002).REF(cst00173-1#01-ADN228#006,Cst001
    73-1#01-ADN228#006).REF(cst00173-1#02-AAQ376#005,Cst00173-1#01-ADN199#005).cst0
    0173-1#02-AAM549#001
144 : [cst00173-1#02-AAO735#002] (String)1 "Supplier name"
    DCP :
    GRM(Cls0ClassifiedBy,Cst00173-1#AFR745#002).REF(cst00173-1#01-ADN228#006,Cst001
    73-1#01-ADN228#006).REF(cst00173-1#02-AAQ376#005,Cst00173-1#01-ADN199#005).cst0
    0173-1#02-AAO735#002
145 : [cst00173-1#02-AAM551#001] (String)1 "Supplier product designation"
    DCP :
    GRM(Cls0ClassifiedBy,Cst00173-1#AFR745#002).REF(cst00173-1#01-ADN228#006,Cst001
    73-1#01-ADN228#006).REF(cst00173-1#02-AAQ376#005,Cst00173-1#01-ADN199#005).cst0
    0173-1#02-AAM551#001
```

Note

Select only simple classification properties as complex properties are not supported.

Add the DCP value to the table configuration

- From the search results page, from the **Table Settings**  menu, choose **Configure Table** .

The screenshot shows the 'Table Configurator' interface. At the top, there are tabs for 'Shape', 'Saved', and 'Advanced'. Below these, a search bar contains 'Any Category' and a filter dropdown set to '3RT2018-1FB47/A;1'. A classification label 'eCI@ass ADVANCED 47...' is visible. The main toolbar includes icons for 'Table', 'Filters', 'Search Similar...', 'Save Search', 'Create Report', 'Selection Mode', 'Select All', 'Highlighting & Color...', 'Search Settings', and 'Edit'. A search input field 'Find in this content' is present. The table below has three columns: 'Name', 'Description', and 'Release Status'. It contains five rows of data. A context menu is open over the first row, displaying options: 'Arrange', 'Wrap Text', 'Clear all filters', and 'Configure Table'.

Name	Description	Release Status
3RT2018-1FB47	CONTACTOR,AC3:7,5KW 1NO DC24V M. DIODE	
3RH2344-1AF00	CONT.RELAY,4NO+4NC,AC110V 50/60HZ	
3RW5225-1AC05	3RW52 600V 63A 24V screw analog	
3RT2617-1AB03	CONT., AC6B:12.5KVAR 1NO+1NC24VAC50/60HZ	
6ES7144-5KD00-0BA0	ET 200AL, AI 4xU/I/RTD, 4xM12	

The **Table Configurator** opens with the table's column configuration selected.

6. In the **Table Configurator** page, click **Add Property** (+).
7. In the **Add Property** panel, select **Advanced** and specify the object type and the DCP value.

Add Property

Reset
 Pin Panel
 Close

☐ Basic
 ☒ Advanced

▼ Type

Workspace object

▼ Properties

* Dynamic Compound Property:

GRM(Cls0ClassifiedBy,Cst00173-1#AFR745#002).REF(cst00173-1#01-ADN228#006,Cst00173-1#01-ADN228#006).REF(cst00173-1#02-AAQ376#005,Cst00173-1#01-ADN199#005).cst00173-1#02-AAO735#002

Add

8. Click **Add**.
9. You can update the behavior for the classification property you added in the table, such as making the property visible by default, by editing the table configuration.

Results

The classification property is now available in the search results table. Users can now arrange the search results table to show or hide the classification properties.

Creating and managing views for class in Active Workspace

About classification views

You can use views in Teamcenter to customize the display of class attributes based on user roles and interests, ensuring relevant information is accessible to different user groups.

By creating specific views, you can configure the display of class attributes for different user groups, such as sales teams or contractors, ensuring that the most relevant information is available. You can also set attribute protections and hide class attributes on a user, group, role, or project basis to define how a class is displayed in the Classification application. You can create and manage following views in Classification Manager without modifying the classification data.

Note

Classification views are supported only in the released advanced classes.

- **User views:** Define how a class is displayed for a specific user. By default, user views take the highest precedence over role, project, group, and default views when the class is displayed.
- **Role views:** Define how a class is displayed for users with a specific role. By default, role views take precedence over project, group, and default views when the class is displayed.
- **Group views:** Define how a class is displayed for a specific group of users. By default, group views take precedence over default views but are subordinate to user, role and project views.
- **Base views:** Define how a class is displayed for users for whom no user or group view is defined. Base views take the lowest precedence when a class is displayed.

A Base View in Advanced Classification is used internally to cache data required to display the class, helping to improve performance in Advanced Classification. Additionally, you can use the Base View to modify the order in which the properties of a class are displayed in the user interface.

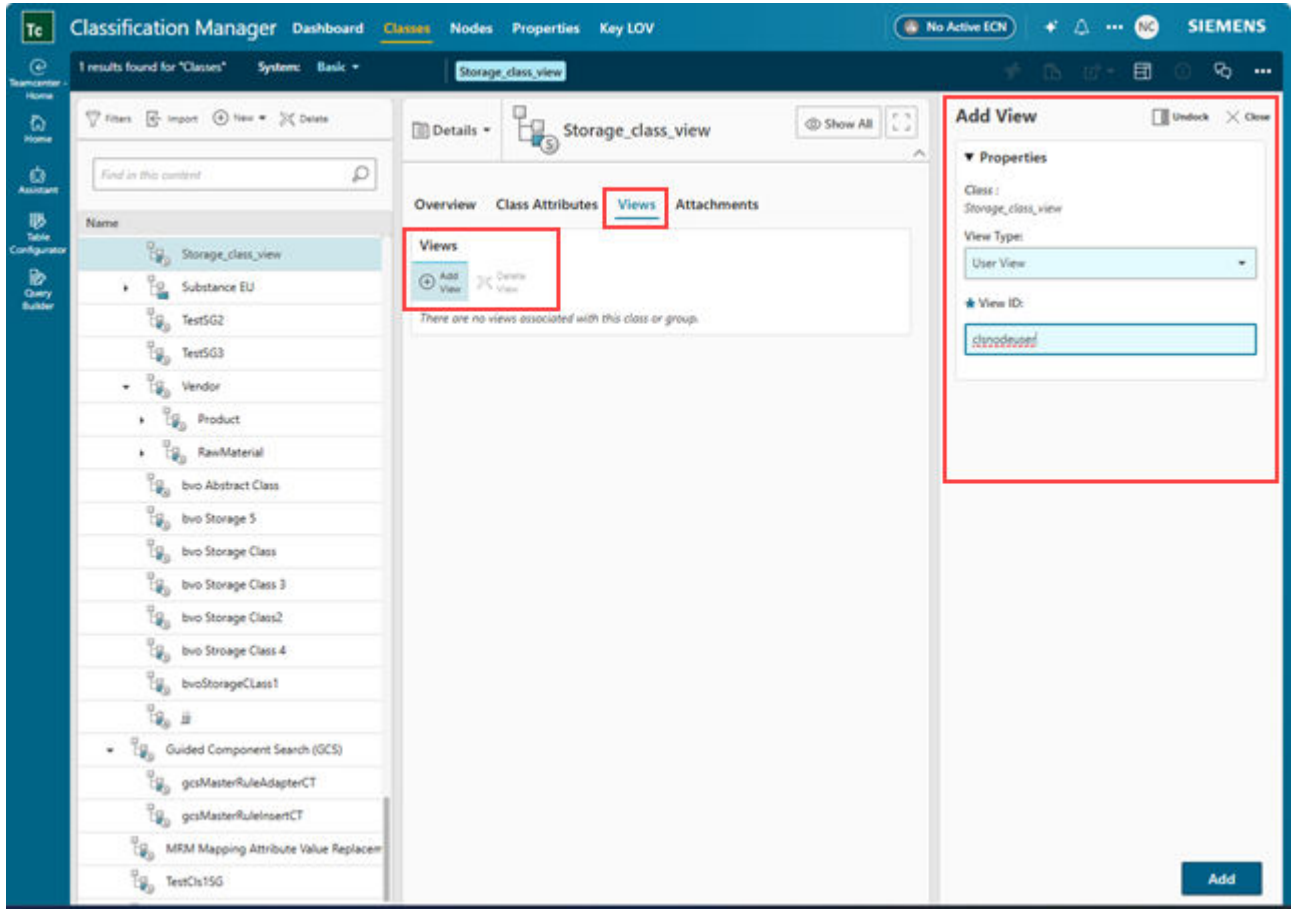
Create a view

You can set up classification views in Active Workspace that define which attributes are displayed and in what order, based on user roles or specific needs.

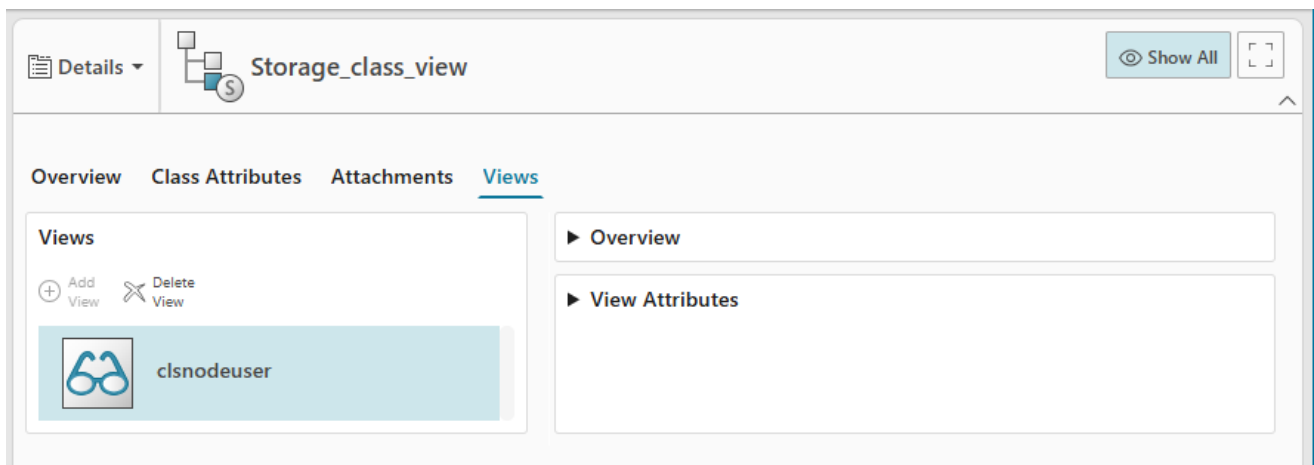
Procedure

1. From the hierarchy tree, select a storage or abstract class for which you want to create a view.
2. In the **Views** tab, click **Add View**.
3. In the **Add View** dialog box, choose **User View**, **Group View**, **Role View**, or **Base View** from the **View Type** list.
4. Specify **View ID** based on the view type selected.

- **User View:** The ID must exactly match the user's Teamcenter user ID.
- **Group View:** The ID must exactly match the Teamcenter group ID.
- **Role View:** The ID must exactly match the Teamcenter role ID.
- **Base View**



5. Click **Add** .
A view is created for the selected class.

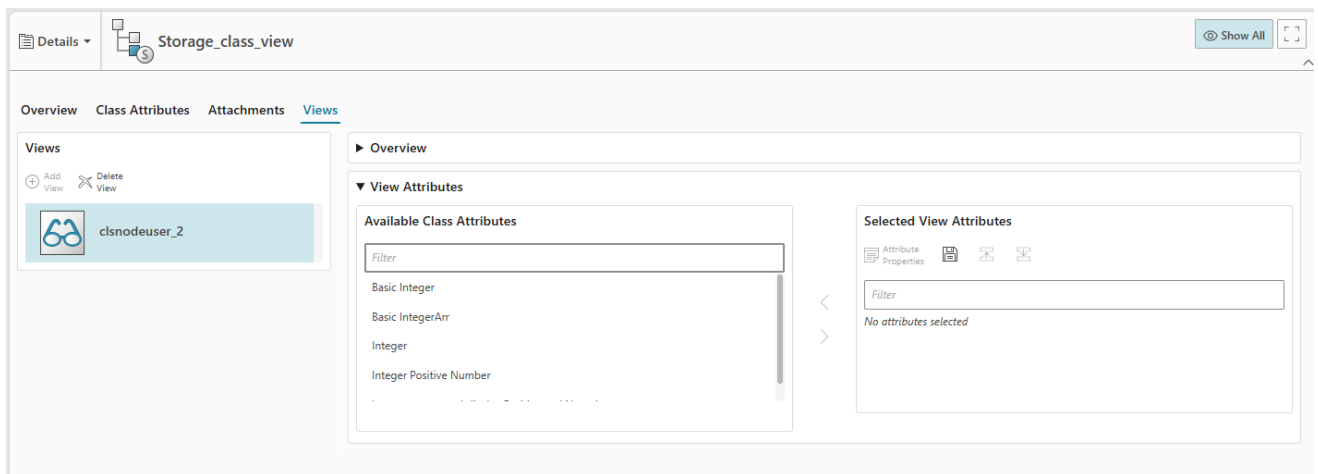


Assign attributes to a view

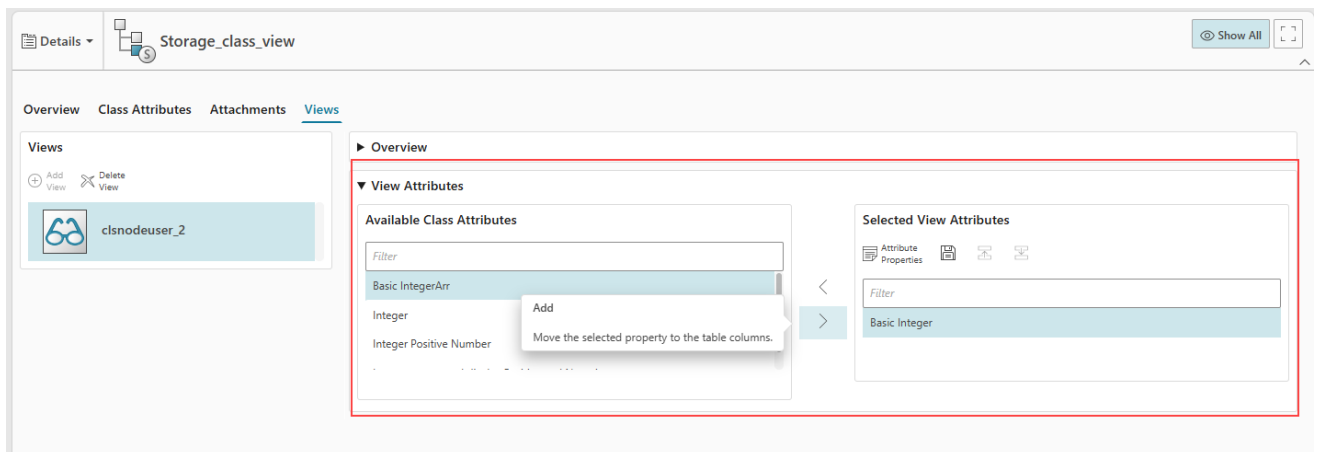
You can assign attributes to a view, ensuring that the most relevant information is displayed for end users while classifying objects. You can also rearrange attributes within a view to reflect changes in user requirements or organizational priorities.

Procedure

1. Open the class and click **Views**.
2. To choose the attributes to be displayed in the view, select the attribute in the **Available Class Attributes** list. You can also filter to select specific class attributes.



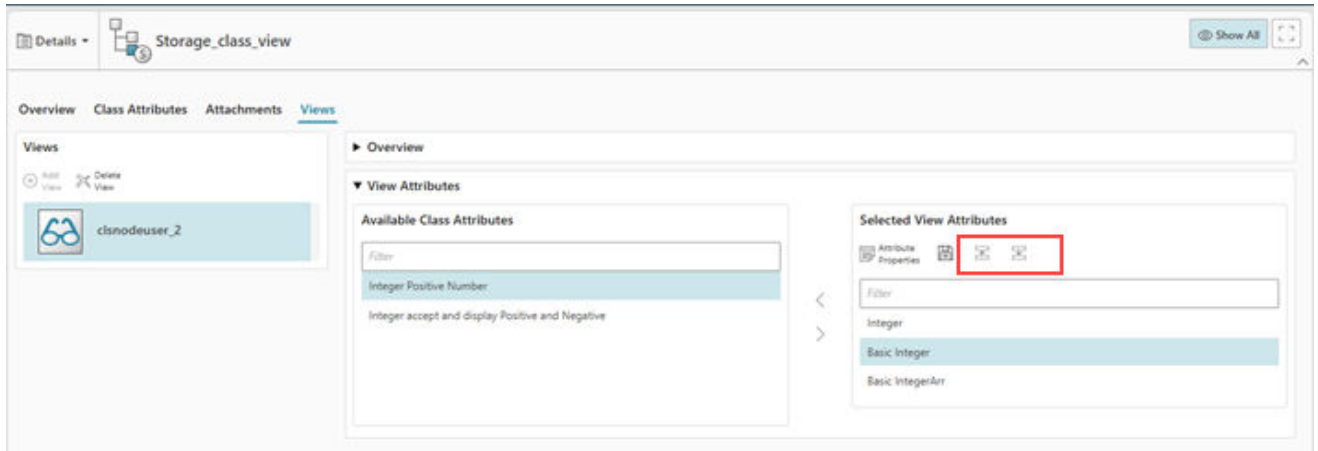
3. Click the right-arrow button to move the attribute to the **Selected View Attributes** list. Repeat the steps to select additional attributes.



Remove attributes from the **Selected View Attributes** list by choosing the attribute and clicking the left-arrow button.

Teamcenter displays the attributes in the **Selected View Attributes** list.

- To change the order in which the attributes are displayed on the Classification form, select the attribute and use the **Move up** and **Move down** buttons.



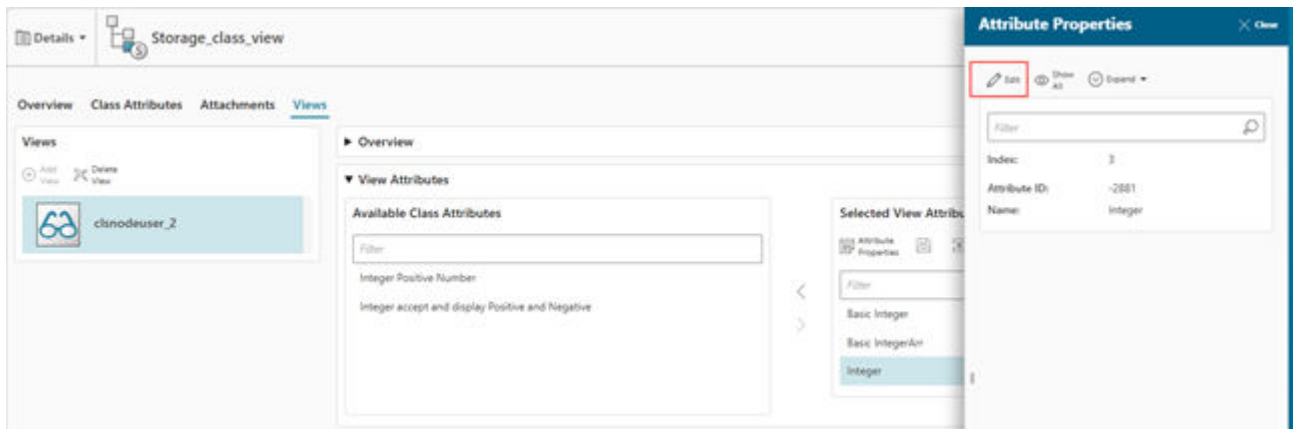
- Click **Save** .

Apply additional properties to the attributes in a view

You can apply additional properties to the attribute at the view level. The properties that were assigned to the attribute at the class level are displayed but cannot be modified.

Procedure

- Open the class and click **Views**.
- Select a **View**, and under **View Attributes**, select an attribute listed in the **Selected View Attributes** list.
- Click **Attribute Properties** .
- In the **Attribute Properties** dialog box, Click **Edit**  to apply additional properties to the selected attribute as required.



5. Click **Show All** to hide all the properties that do not contain any value.
6. Click **Expand** and then choose one of the following:
 - Expand All  to expand all attribute sections.
 - Collapse All  to collapse all attribute sections.
7. Click **Save** .

Delete a view

You can delete views that are no longer needed without affecting the associated class data, ensuring your Classification data remains intact.

Procedure

1. Open the class and click **Views**.
2. Under the **Views** pane, select the view that you want to delete, and click **Delete View**. Teamcenter displays the **Delete Confirmation** dialog box for you to confirm the delete action.
3. Click the **Delete** button to confirm the deletion. Teamcenter deletes the view from the hierarchy structure.

Using classification views to filter what information users can see

What are classification views?

Views are used to display all or a subset of the properties in a class in the user interface. There are two types of views used in classification classes:

Base view A base view is used internally to cache data required to display the class, helping to improve performance. Additionally, you can use the base view to modify the order in which the properties of a class are displayed in the user interface. For best results, create a base view for each application class in a **Released** state.

If no other view exists in the class, the user interface displays the properties contained in the base view.

User, group, or role view These types of views are used to override the order of properties in the base view and limit the number of properties shown to different types of users based on their needs. For example, an Engineering group may need to see one group of properties, whereas a Logistics group needs to see a different group of properties. You can create views for specific groups, roles, or individual users.

Note

You can create views only for application classes. Because of this, you can influence the order in which properties, blocks, and aspects are displayed in a class, but you cannot influence the order of properties within blocks or aspects.

Create base views in bulk using clsutility

You can create base views in bulk using the **clsutility**.

Procedure

You can create base views for released applications as follows:

- To create base views for every released application class, type:

```
clsutility -create -base_view_definitions
```

This command creates a base view definition for every application class in the **Released** state. If an application class already has a base view, it is skipped.

- To create a base view for a specific application class, type:

```
clsutility -create -base_view_definitions -class_definition=<IRDI of class>
```

This command creates a base view for the application class with the specified IRDI.

- To create a base view for all released application classes, regardless of whether they already have one:

```
clsutility -create -base_view_definitions -force
```

This command creates a new base view definition for every application class in the **Released** state and overwrites any existing base view.

Note

You cannot use the **-create -base_view_definitions** command to modify the order in which class properties are displayed in the user interface. To do this, you must use the **-create -view_definitions** command and specify the **ViewType** as **BaseView** in the JSON input file.

Create user, group, or role views using clsutility

You create user, group, or role views by importing a JSON file that defines the properties that are displayed for each group, role, or user in the user interface. The properties of the class are displayed in their order of appearance in the JSON file (from top to bottom). You can include only properties in the views that already exist in the base view of the class. Hide attributes by excluding them in the JSON file for the view.

Procedure

To create a group, role, or user view containing properties in a specific order run the utility as follows:

clsutility -create -view_definitions -request=<JSON file>

The schema file describing the JSON format is found here:

```
...\\<TC_DATA>\\classification
\\json\\schema\\advanced\\Classification_Save_ViewDefinitions_Request_advanced.schema.json
```

Refer also to the sample import file found here:

```
...\\<TC_DATA>\\classification
\\json\\schema\\advanced\\Classification_Save_ViewDefinition_Request_sample.json
```

The format of the input file must resemble the following:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ViewDefinitions": [
    {
      "ClassDefinition": "TDOC#01-APPC04#001",
      "ViewType": "GroupView",
      "ViewId": "Engineering",
      "ViewAttributes": [
        {
          "IRDI": "TDOC#02-PDF015#001"
        },
        {
          "IRDI": "TDOC#02-PDF016#001"
        }
      ]
    },
    {
      "ClassDefinition": "TDOC#01-APPC04#001",
      "ViewType": "RoleView",
      "ViewId": "Designer",
      "ViewAttributes": [
        {
          "IRDI": "TDOC#02-PDF011#001"
        },
        {
          "IRDI": "TDOC#02-PDF014#001",
          "IsProtected": true,
          "IsRequired": true
        }
      ]
    }
  ]
}
```

```

    "ClassDefinition": "TDOC#01-APPC04#001",
    "ViewType": "UserView",
    "ViewId": "JSmith",
    "ViewAttributes": [
      {
        "IRDI": "TDOC#02-PDF012#001"
      },
      {
        "IRDI": "TDOC#02-PDF013#001"
      }
    ]
  }
]
}

```

Notice that you can specify whether a property value must be entered for a property (**IsRequired**) or whether it is protected and cannot be modified in the user interface (**IsProtected**).

To create a base view to reorder its properties, the input must resemble the following:

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ViewDefinitions": [
    {
      "ClassDefinition": "TDOC#01-APPC04#001",
      "ViewType": "BaseView",
      "ViewId": "BaseView",
      "ViewAttributes": [
        {
          "IRDI": "TDOC#02-PDF013#001"
        },
        {
          "IRDI": "TDOC#02-PDF015#001"
        },
        {
          "IRDI": "TDOC#02-PDF011#001",
          "IsProtected": true
        },
        {
          "IRDI": "TDOC#02-PDF014#001",
          "IsRequired": true
        },
        {
          "IRDI": "TDOC#02-PDF012#001"
        },
        {
          "IRDI": "TDOC#02-PDF016#001"
        }
      ]
    }
  ]
}

```

Extract existing view definition using clsutility

You can extract an existing view definition from the database using the clsutility command.

Procedure

Run the following command:

```
clsutility -find -view_definitions -request=<JSON-schema-version>
```

The JSON schema file for the **-find** command is found in:

```
...\\<TC_DATA>\\classification  
\\json\\schema\\advanced\\Classification_Find_View_Request_advanced.schema.json
```

Refer also to the sample file found here:

```
...\\<TC_DATA>\\classification  
\\json\\samples\\advanced\\Classification_Find_ViewDefinition_Request_sample.json
```

The JSON file must resemble the following:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ViewDefinitions": [
    {
      "ClassDefinition": "TDOC#01-APPC10#001"
    },
    {
      "ClassDefinition": "TDOC#01-APPC12#001",
      "ViewType": "GroupView"
    },
    {
      "ClassDefinition": "TDOC#01-APPC09#001",
      "ViewId": "JSmith"
    },
    {
      "ClassDefinition": "TDOC#01-APPC08#001",
      "ViewType": "GroupView",
      "ViewId": "Engineering"
    },
    {
      "ViewType": "UserView",
      "ViewId": "JSmith"
    },
    {
      "ViewId": "MNancy"
    }
  ]
}
```



```
{
  "ClassDefinition": "TDOC#01-APPC11#001",
  "ViewType": "BaseView",
  "ViewId": "BaseView"
}
]
```

It is not necessary to uniquely identify each view that you want to display. It is sufficient to just request; for example, all the views for a particular class:

```
{
  "ClassDefinition": "TDOC#01-APPC10#001"
}
```

Alternatively, you could request all user views for a particular user:

```
{
  "ViewType": "UserView",
  "ViewId": "JSmith"
}
```

List the class descriptor for a view using clsutility

Whereas a class definition describes only the class, or a key-LOV definition only one key-LOV, the class descriptor compiles all the definitions required to describe an ICO. Each node of the hierarchy shares a class descriptor. The class descriptor provides a view on the data from the classification consumer perspective.

Procedure

To obtain the class descriptor for a particular view run the following command:

clsutility -get -class_descriptor -request=<JSON file name>

The schema file explaining the required JSON syntax is found in:

```
...\\<TC_DATA>\\classification
\\json\\schema\\advanced\\Classification_Get_ClassDescriptor_Request_advanced.schema.json
```

Delete view definitions using clsutility

You can delete a view definition using the clsutility.

Procedure

To delete a view definition, type:

```
clsutility -find -view_definitions -request=<JSON file name>
```

When deleting, the JSON request file uses the same schema as the JSON find request.

Note

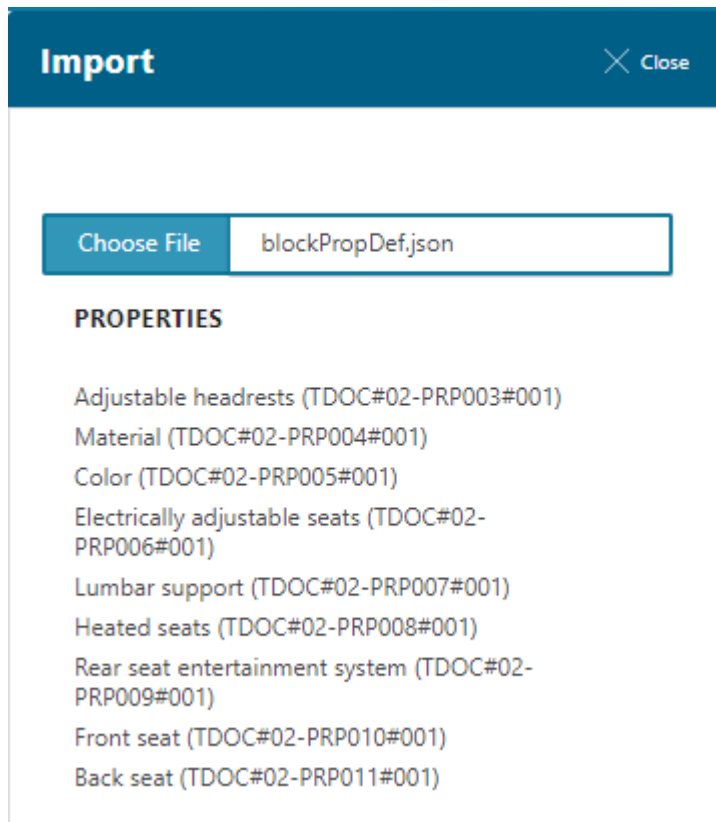
Be careful when deleting base views. For performance reasons, a released class should always contain a base view. It is advisable to overwrite an undesired base view with a corrected version instead of deleting it.

Deleting a class definition deletes all views belonging to the class.

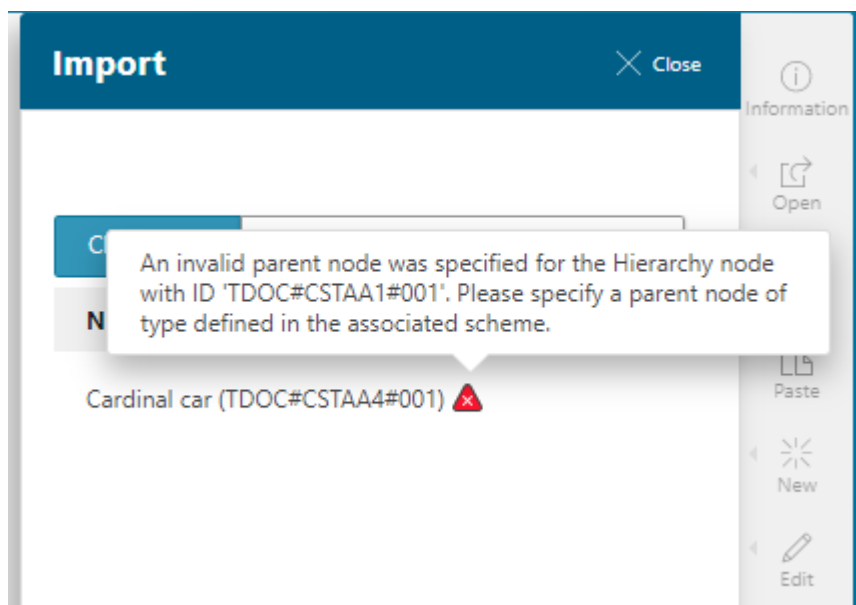
Importing and exporting classification data

Importing hierarchy definitions with the Classification Manager

For definition files based on schemas prior to version 1.5, you can import the definitions directly in the Teamcenter user interface. Use the **Import** button on the **Dashboard** tab or the **Import** command on any of the other tabs in **Classification Manager** to import key-LOV, property, class, and node definitions. They must always be **imported in the correct order**. After you upload the JSON file, you receive an overview of the objects you are about to import.



If there is an error in the files that you import, the error message provides assistance.



Tip

It does not matter on which tab you choose to import, you can import any type of hierarchy definition on any of the tabs.

Import classified data for advanced classification

You can import classified data in BMEcat XML or JSON format in the user interface. The XML files and supporting documents (for example, images) must be available in a local directory that you compress to a ZIP file. The import searches the compressed for the correct input file.

Procedure

1. Compress the directory structure of the data in a ZIP file.
If you download BMEcat files from the Siemens Industry Mall, these are already in the appropriate zipped format.
2. In your **Home** folder, choose **More Commands ***** > **Import/Export**  > **Import Classification Data**.
3. In the **Choose File** box, select the ZIP file containing the data.
Teamcenter searches inside the compressed file for all eligible import files and presents them to you in a list.
4. In the **Select file to import** box, select the XML file containing the data.
If you import BMEcat data, there may be several versions of the XML file in different languages. These are not localized values. If you import, for example, the English version, and then subsequently import the German version, the German version overwrites the English values. Each of these language files represent master locales.

Results

When the import is complete, you are notified in the **Alert** area where you can view the details of the import. As soon as the data is indexed, it is available for searching.

This feature requires that both the Subscription Manager and Dispatcher Client are running. If the Subscription Manager is not running, no notification is displayed in the **Alert** area, but the operation still occurs in the background.

If the imported data includes classification objects only, at import they are automatically attached to existing objects in the database if they have the same ID. This automatically classifies the existing objects or updates an existing classification on previously classified objects. If there is no existing object with the same ID in the database, a new object is created.

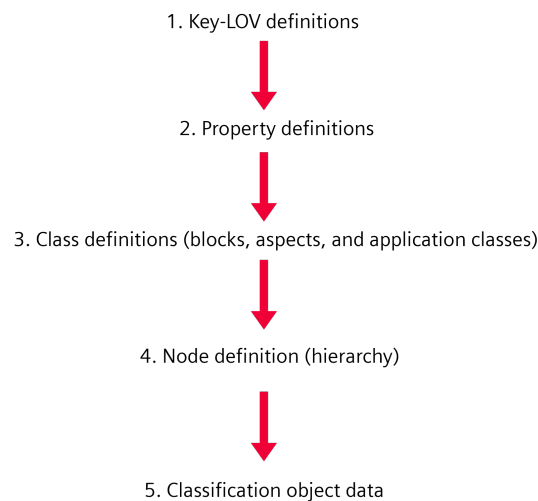
Importing JSON files using the clsutility command

You create a CST hierarchy by importing JSON files containing both hierarchy and data. The schema files defining the JSON format and example files are in the following directory:

```
...\\<TC_DATA>\\classification\\json\\schema\\advanced
```

The schema used to import classification definitions is evolving over time. Prior to version Teamcenter 14.1 (schema 1.5.0), key-LOVs, properties, class, and nodes had to be defined in separate JSON files and imported in the order listed. Using schema 1.5.0, you can import all types of class definitions in a single JSON file. This schema also supports importing older object definition JSON files with no need to rewrite them. You can define new objects or update existing ones in the same JSON file.

When importing, you can only reference objects that are already in the database. When importing classes that reference other classes (for example, when importing aspects), you must import the referenced classes first and then import the referencing classes. The definition file is read from top to bottom. If you import individual files, the order of import is as follows:



Data is imported from JSON files using the following methods:

- **Use the Classification Manager to import JSON files** in the user interface.
- Use **clsutility** to import a definition file based on any schema up to and including version 1.5.

```
clsutility -u=<user> -p=<password> -g=<group> -save -classification_definitions -request="<path-to-JSON-file>"
```

- Use **clsutility** to import object individual definition files.

The syntax of this utility is:

clsutility-u=<user> -p=<password> -g=<group> -<operation> <input/output file path>

If using schema 1.5, import the definition file using the following syntax:

clsutility -u=<user> -p=<password> -g=<group> -create -keylov_definitions -request="<path-to-JSON-file>"

For more information about the available commands and their exact syntax, type **clsutility -h** for the top-level help and **clsutility -<command-name> -h** for information about each available command.

The following table lists the supported objects and an example of the **clsutility** statements used to import these object types.

Object	Operation	clsutility command
Key-LOV	Import	clsutility -u=<user> -p=<password> -g=<group> -create -keylov_definitions -request="<path-to-JSON-file>"
Property definition	Import	clsutility -u=<user> -p=<password> -g=<group> -create -property_definitions -request="<path-to-JSON-file>"
Class definition	Import	clsutility -u=<user> -p=<password> -g=<group> -create -class_definitions -request="<path-to-JSON-file>"
Nodes	Import	clsutility -u=<user> -p=<password> -g=<group> -create -node_definitions -request="<path-to-JSON-file>"
Application data (ICOs)	Import	clsutility -u=<user> -p=<password> -g=<group> -create -classification_objects -request="<path-to-JSON-file>"

Note

There are pairs of very similar commands in the **clsutility**: **classification_objects** and **classification_object**, or **node_definitions** and **node_definition**. Be sure to use the plural forms of these commands (**objects** and **definitions**) for all CST imports. The singular refers to traditional classification objects.

Preparing JSON files for import (example)

Importing the hierarchy and classes of car seats

This example walks you through creating and importing the JSON files required to describe a class containing attributes that allow you to configure the seats in a car. The example consists of several levels of complexity:

- Classify a car in a class containing simple properties, for example, a key-LOV with the type of seat (front seat or back seat).
- Classify a car in a class containing reusable sets of properties (blocks).
- Specify the number of seats when classifying the car (cardinality).
- Specify the attributes shown depending on the type of seat selected (polymorphism).

- Specify the attributes shown for each seat when specifying multiple seats (cardinality and polymorphism).
- Import a property record into one of the newly created classes.

For each of these examples, the JSON files are explained. Once created, the JSON files can be imported using the Classification Manager or, alternatively, with **clsutility**. The **clsutility** commands required to import the JSON files are also listed [here](#).

To verify that the definitions in the following examples are correct or to create additional definitions, consult the following schema files found in ...\\TC_DATA>\\classification\\json\\schema\\advanced\\:

- **Classification_Save_KeyLOVDefinitions_Request_advanced.schema.json**
- **Classification_Save_PropertyDefinitions_Request_advanced.schema.json**
- **Classification_Save_ClassDefinitions_Request_advanced.schema.json**
- **Classification_Save_NodeDefinitions_Request_advanced.schema.json**
- **Classification_Save_PropertyRecords_Request_advanced.schema.json**

There are many online JSON validators to assist you in creating JSON files with the proper syntax. We recommend using any of these to verify the JSON files that you create prior to importing them.

Note the following points about these examples:

- The definitions are imported with a release status of **Develop**. This ensures that you can still modify them if required. When you are satisfied with the structure and classes, you must [change the status to Released](#). Objects can be classified only in **Released** classes.
- The **TDOC** name space is used to differentiate the structures imported in this example from other structures that you use. Creating your own name space helps when creating queries. When searching for your newly-imported objects, you can filter the tabs in Classification Manager to only show those with **Namespace = TDOC**.

Import a class with simple properties

The simplest use case is to import a class containing properties. This example shows how to import a **Car** class containing three properties.

- **Type of seat**
A key-LOV with two entries: **Front** and **Back**
- **Manufacturer**
A simple string property
- **Adjustable headrests**

A Boolean property

In the user interface, this class is displayed as follows:

Overview Content **Classification** Finishes Made From Where Used Changes Attachments History

ASSIGNED CLASSIFICATIONS

Automotive Engineering > Assemblies > Cars > Simple Car

AVAILABLE CLASSES

Filter

- Automotive Engineering
 - Assemblies
 - Cars
 - Simple Car**
 - Material Families
 - Miscellaneous

PROPERTIES

keywords

Unit System: * ☒ Metric ☐ Non-Metric

Type of seat:

Manufacturer:

☐ Adjustable headrests

Type of seat:

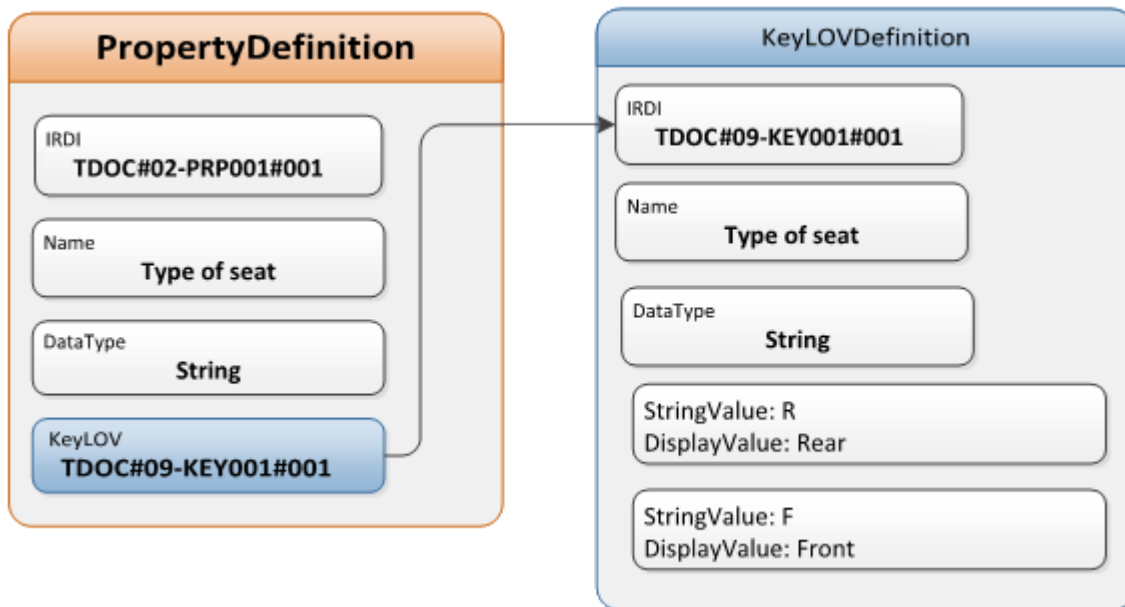
- F Front
- R Rear
- F Front

The order when importing this class is to first import the key-LOV definitions, then the property definitions, then the class definition, and finally the node definition so that the class is visible in the Classification hierarchy.

Procedure

1. Import the key-LOV definition.

In Classification Standard Taxonomy, a key-LOV definition is separate from its property definition.



In this example, there is one key-LOV whose JSON definition is stored in a file titled **simpleKeylovDef.json**:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "KeyLOVDefinitions": [
    {
      "ObjectType": "09",
      "Namespace": "TDOC",
      "ID": "KEY001",
      "Revision": "001",
      "Name": "Type of seat",
      "Status": "Develop",
      "LOVItems": {
        "DataType": "String",
        "LOVStringItems": [
          {
            "StringValue": "R",
            "DisplayValue": "Rear Seat"
          },
          {
            "StringValue": "F",
            "DisplayValue": "Front Seat"
          }
        ]
      }
    }
  ]
}
```

2. Import the property definitions.

PropertyDefinition

IRDI
TDOC#02-PRP001#001

Name
Type of seat

DataType
String

KeyLOV
TDOC#09-KEY001#001

PropertyDefinition

IRDI
TDOC#02-PRP002#001

Name
Manufacturer

DataType
String

PropertyDefinition

IRDI
TDOC#02-PRP003#001

Name
Adjustable headrests

DataType
Boolean

In this example, there are three properties to import, one of which (**Type of seat**) is a key-LOV that references the key-LOV definition imported in the previous step. The JSON definition for the properties is stored in a file titled **simplePropDef.json**:

```
{  
  "SchemaVersion": "1.0.0",
```

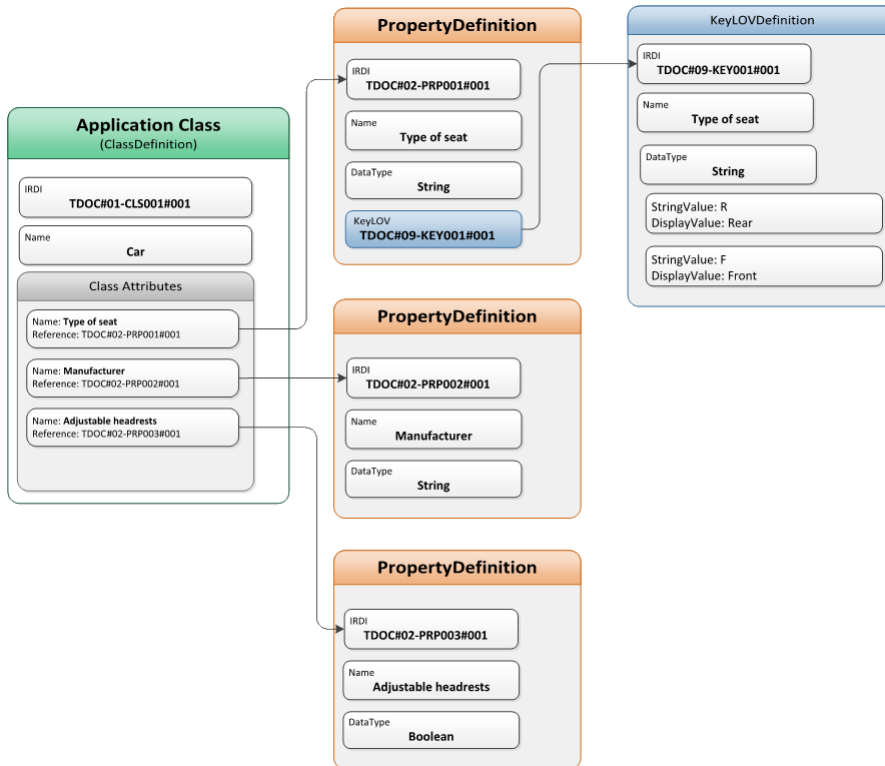
```

"Locale": "en_US",
"PropertyDefinitions": [
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP001",
    "Revision": "001",
    "Status": "Develop",
    "Name": "Type of seat",
    "DataType": {
      "Type": "String",
      "KeyLOV": "TDOC#09-KEY001#001"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP002",
    "Revision": "001",
    "Name": "Manufacturer",
    "Status": "Develop",
    "DataType": {
      "Type": "String"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP003",
    "Revision": "001",
    "Name": "Adjustable headrests",
    "Status": "Develop",
    "DataType": {
      "Type": "Boolean"
    }
  }
]
}

```

3. Import the class definition.

The **Car** class contains only three properties: **Type of seat**, **Manufacturer**, and **Adjustable headrests**:



The JSON file, named **simpleApplicationClassDef.json**, consists of the following:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "ObjectType": "01",
      "Namespace": "TDOC",
      "ID": "CLS001",
      "Revision": "001",
      "Name": "Car",
      "Status": "Develop",
      "IsAbstract": false,
      "UnitSystem": 3,
      "ClassType": "Application Class",
      "ClassAttributes": [
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP001#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP002#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP003#001"
        }
      ]
    }
  ]
}
```

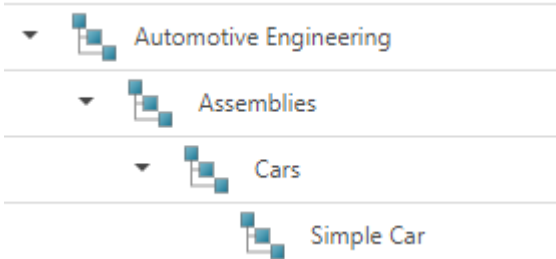
```

    ]
  }
}

```

4. Import the node definition that makes the class visible.

A nested hierarchy is required to organize the classes.



To import this hierarchy, create the following JSON file and name it **simpleNodeDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAE0",
      "Revision": "001",
      "Name": "Automotive Engineering"
    },
    {
      "Namespace": "TDOC",
      "ID": "CSTAA0",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAE0",
        "Revision": "001"
      },
      "Name": "Assemblies"
    },
    {
      "Namespace": "TDOC",
      "ID": "CSTAA1",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA0",
        "Revision": "001"
      },
      "Name": "Cars"
    },
    {
      "Namespace": "TDOC",

```

```

    "ID": "CSTAA2",
    "Revision": "001",
    "Parent": {
      "Namespace": "TDOC",
      "ID": "CSTAA1",
      "Revision": "001"
    },
    "Name": "Simple Car",
    "ApplicationClass": {
      "Namespace": "TDOC",
      "ID": "CLS001",
      "Revision": "001"
    }
  }
]
}

```

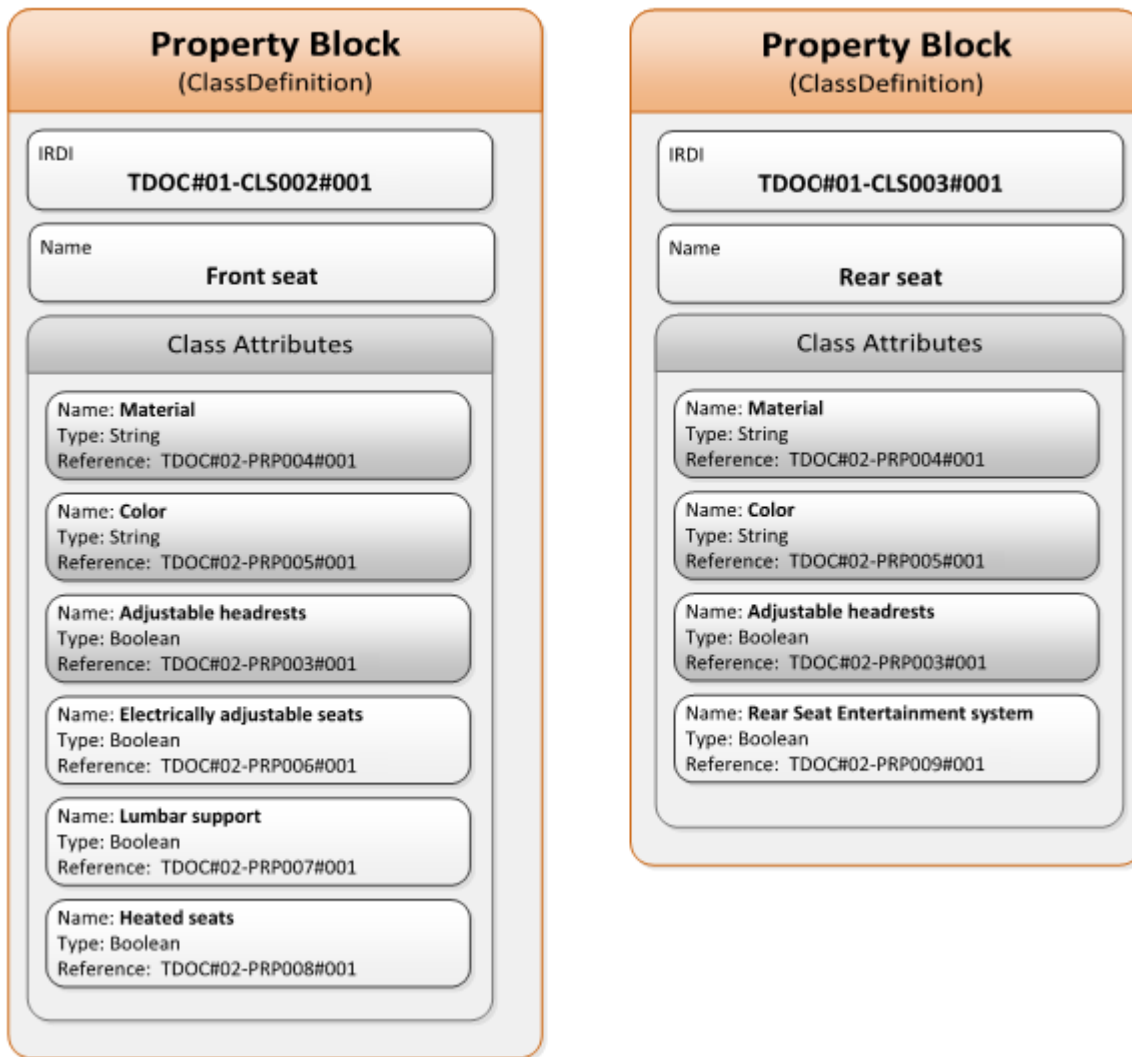
5. Import the definitions into the database using [Classification Manager](#) or with [clsutility commands](#). You must import the JSON files in the following order:
 - a. `simpleKeylovDef.json`
 - b. `simplePropDef.json`
 - c. `simpleApplicationClassDef.json`
 - d. `simpleNodeDef.json`

Import a class with a reusable set of properties (blocks)

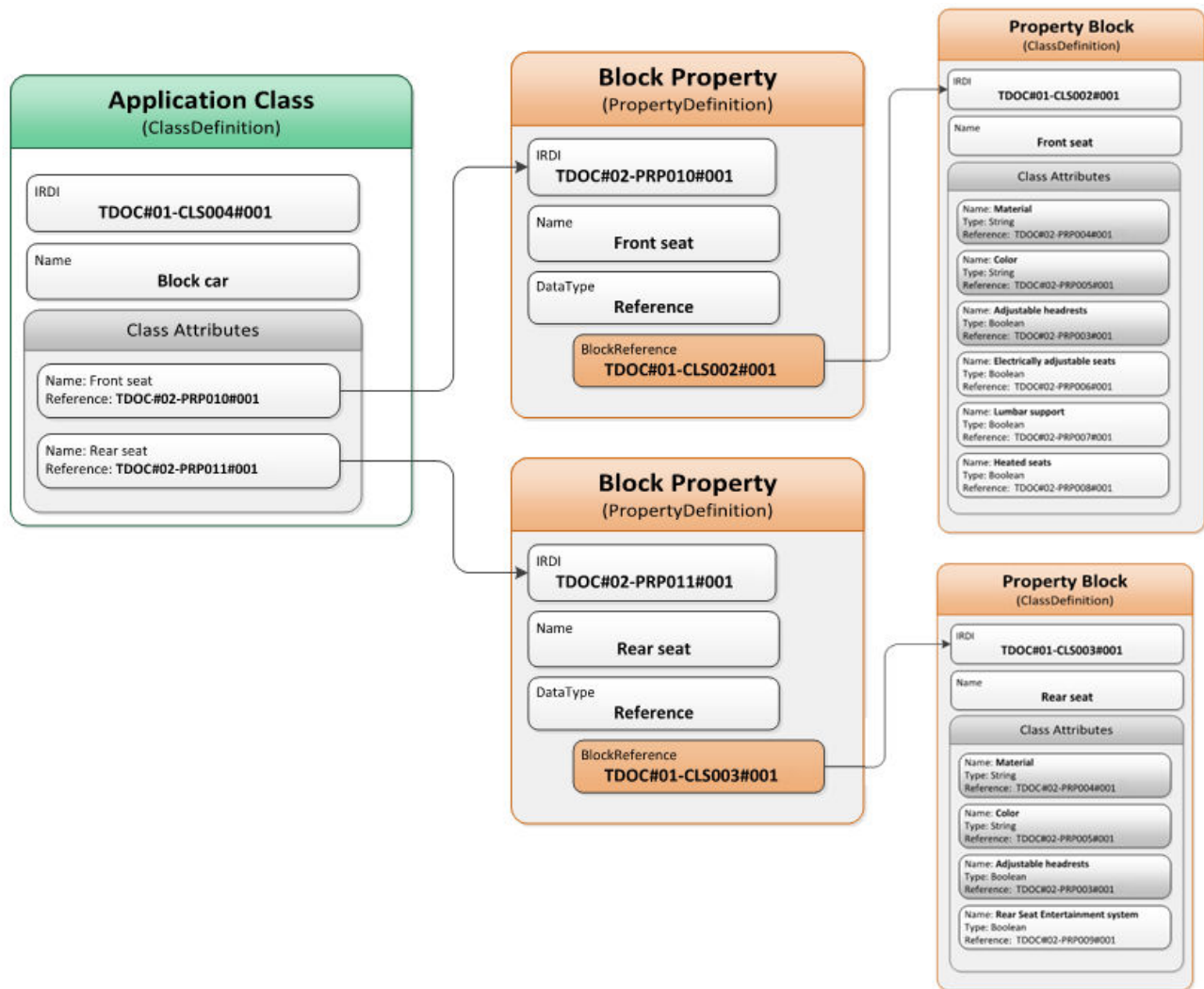
Frequently, there is a group of properties that is always used together. For example, the following set of properties describe a front seat and a rear seat:

Front seat	Rear seat
Material	Material
Color	Color
Adjustable headrests	Adjustable headrests
Electrically adjustable seats	Rear seat entertainment
Lumbar support	
Heated seats	

Each time you create a class, you could manually add each of these properties to the class to describe each of these seat types. However, it is easier to group them and select a single property called **Front seat** that contains all the required properties. This set of properties is called a property block. A property block is a special kind of class.



Property blocks are referenced by block properties which, in turn, are referenced in the application class containing the block properties:



To create the JSON files required to import these classes and properties:

Procedure

1. Create the individual property definitions.

Create the following JSON file and save it in a file named **blockPropDef.json** as follows:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyDefinitions": [
    {
      "ObjectType": "02",
      "Namespace": "TDOC",
      "ID": "PRP004",
      "Revision": "001",
      "Status": "Develop",
```



```

    "Name": "Material",
    "DataType": {
      "Type": "String"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP005",
    "Revision": "001",
    "Name": "Color",
    "Status": "Develop",
    "DataType": {
      "Type": "String"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP003",
    "Revision": "001",
    "Name": "Adjustable headrests",
    "Status": "Develop",
    "DataType": {
      "Type": "Boolean"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP006",
    "Revision": "001",
    "Name": "Electrically adjustable seats",
    "Status": "Develop",
    "DataType": {
      "Type": "Boolean"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP007",
    "Revision": "001",
    "Name": "Lumbar support",
    "Status": "Develop",
    "DataType": {
      "Type": "Boolean"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP008",
    "Revision": "001",
    "Name": "Heated seats",
    "Status": "Develop",

```

```

        "DataType": {
            "Type": "Boolean"
        }
    },
    {
        "ObjectType": "02",
        "Namespace": "TDOC",
        "ID": "PRP009",
        "Revision": "001",
        "Name": "Rear seat entertainment system",
        "Status": "Develop",
        "DataType": {
            "Type": "Boolean"
        }
    },
    {
        "ObjectType": "02",
        "Namespace": "TDOC",
        "ID": "PRP010",
        "Revision": "001",
        "Name": "Front seat",
        "Status": "Develop",
        "DataType": {
            "Type": "Reference",
            "BlockReference": "TDOC#01-CLS002#001"
        }
    },
    {
        "ObjectType": "02",
        "Namespace": "TDOC",
        "ID": "PRP011",
        "Revision": "001",
        "Name": "Back seat",
        "Status": "Develop",
        "DataType": {
            "Type": "Reference",
            "BlockReference": "TDOC#01-CLS003#001"
        }
    }
]
}

```

The last two properties, **PRP010** and **PRP011** reference the two property block classes that hold the sets of properties.

2. Create the class definitions, including the property block classes.

```

{
    "SchemaVersion": "1.0.0",
    "Locale": "en_US",
    "ClassDefinitions": [
        {
            "ObjectType": "01",
            "Namespace": "TDOC",

```

```

    "ID": "CLS002",
    "Revision": "001",
    "Name": "Front seat",
    "Status": "Develop",
    "IsAbstract": true,
    "UnitSystem": 3,
    "ClassType": "Block",
    "ClassAttributes": [
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP004#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP005#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP003#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP006#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP007#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP008#001"
      }
    ]
  },
  {
    "ObjectType": "01",
    "Namespace": "TDOC",
    "ID": "CLS003",
    "Revision": "001",
    "Name": "Back seat",
    "Status": "Develop",
    "IsAbstract": true,
    "UnitSystem": 3,
    "ClassType": "Block",
    "ClassAttributes": [
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP004#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP005#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP003#001"
      }
    ]
  }
}

```

```

    },
    {
      "Type": "Property",
      "Reference": "TDOC#02-PRP009#001"
    }
  ]
},
{
  "ObjectType": "01",
  "Namespace": "TDOC",
  "ID": "CLS004",
  "Revision": "001",
  "Name": "Car",
  "Status": "Develop",
  "IsAbstract": false,
  "UnitSystem": 3,
  "ClassType": "Application Class",
  "ClassAttributes": [
    {
      "Type": "Property",
      "Reference": "TDOC#02-PRP010#001"
    },
    {
      "Type": "Property",
      "Reference": "TDOC#02-PRP011#001"
    }
  ]
}
]
}

```

The first two classes, **CLS002** and **CLS003**, are the property block classes and the third class, **CLS004**, is the application class that references the two property blocks.

3. Import the node definition that makes the class visible in Active Workspace.

The **Block car** node must be displayed in the **Cars** node that you created in the first example. To do this, import the new node specifying the **Car** node, **CSTAA1**, as the parent. Save the following in a file named **blockNodeDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAA3",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA1",
        "Revision": "001"
      },
      "Name": "Block Car",
    }
  ]
}

```

```

"ApplicationClass": {
  "Namespace": "TDOC",
  "ID": "CLS004",
  "Revision": "001"
}
}
]
}

```

4. Import the definitions into the database using **Classification Manager** or with **clsutility commands**.

Results

The structure is displayed as follows:

ASSIGNED CLASSIFICATIONS



Automotive Engineering > Assemblies > Cars > Block Car

▼ AVAILABLE CLASSES

Filter
 

▶	Material Families
▶	Miscellaneous
▶	Resource Management
▼	Automotive Engineering
▼	Assemblies
▼	Cars
	Simple Car
	Block Car

▼ PROPERTY GROUPS

keywords

Front seat
Back seat

PROPERTIES

keywords

Unit System: *
 ☒ Metric
 ☐ Non-Metric

▼ Front seat

Material:

Color:

☐ Adjustable headrests

☐ Electrically adjustable seats

☐ Lumbar support

☐ Heated seats

▼ Back seat

Material:

Color:

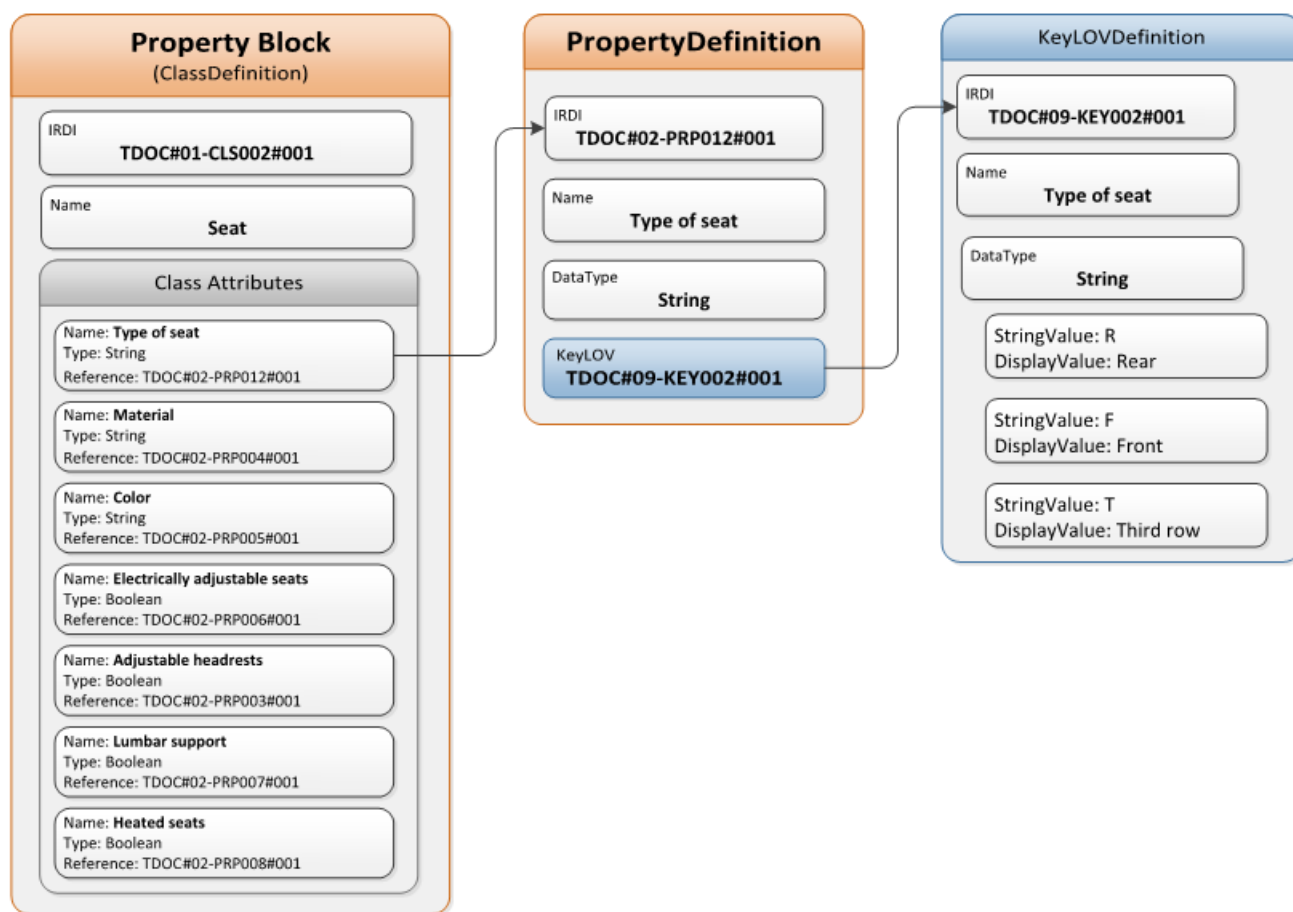
☐ Adjustable headrests

☐ Rear seat entertainment system

Import a class with cardinal properties

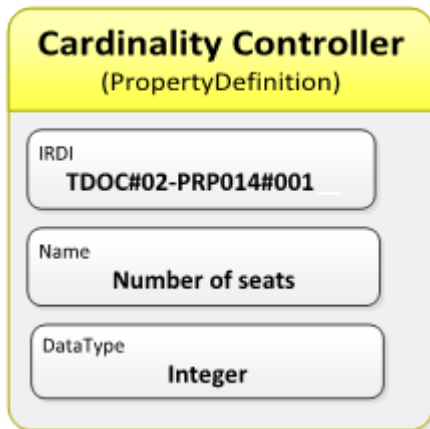
This example shows how to create a class where we can specify the number of seat rows in a car during classification. For example, when classifying a sports car, there is only one row of seats, so only one set of seat properties is necessary. When classifying a family van, there are three rows of seats that each need a set of properties to describe them.

We begin with a set of properties, a property block:

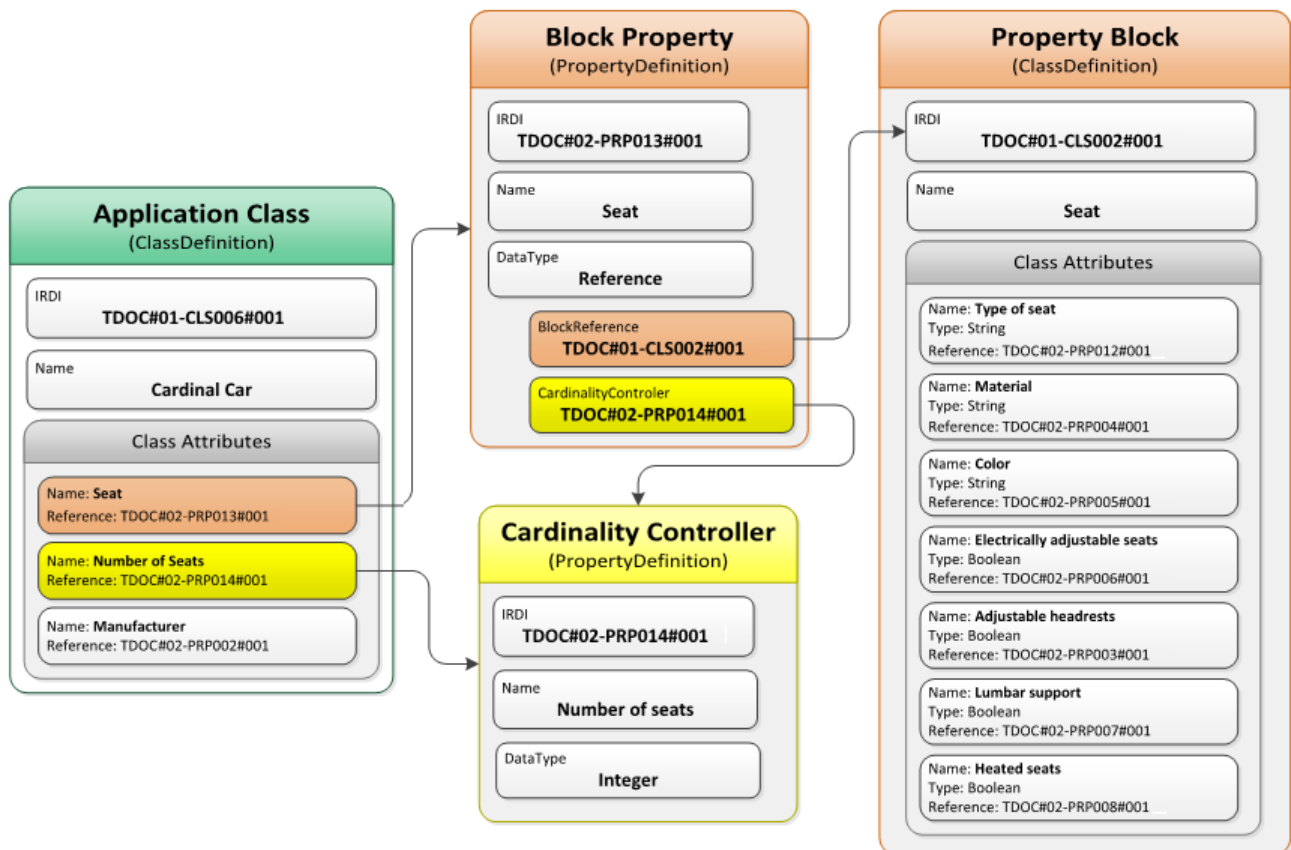


This property block class contains the same attributes as in the previous example, but additionally includes a new key-LOV property, **Type of seat**, where you can specify whether the seats are front, back, or third-row seats.

The number of times that this block of properties is displayed in the user interface is determined by a special property called a *cardinality controller*:



A cardinality controller is referenced by another property and is only of significance if set to **true**. If this is the case, then the user can specify the number of times that the set of properties is displayed.



To create the JSON files required to import these classes and properties:

Procedure

1. Create the key-LOV definitions.

This example introduces a new key-LOV, **Type of seat**, with three values, **Front**, **Rear**, and **Third**. Save the following in a file named **cardinalKeyLOVDef.json**

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "KeyLOVDefinitions": [
    {
      "ObjectType": "09",
      "Namespace": "TDOC",
      "ID": "KEY002",
      "Revision": "001",
      "Name": "Type of seat",
      "Status": "Develop",

      "LOVItems": {
        "DataType": "String",
        "LOVStringItems": [
          {
            "StringValue": "F",
            "DisplayValue": "Front"
          },
          {
            "StringValue": "R",
            "DisplayValue": "Rear"
          },
          {
            "StringValue": "T",
            "DisplayValue": "Third row"
          }
        ]
      }
    }
  ]
}
```

2. Create the individual property definitions.

There are three new properties in this example: **Type of seat** that references the new key-LOV, **Seat**, the property that specifies the property block class that is to be displayed and acts as an indicator that cardinality is involved, and the **Number of seats** that holds the count.

Save the following in a file named **cardinalPropDef.json**:

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyDefinitions": [
    {
      "ObjectType": "02",
      "Namespace": "TDOC",
      "ID": "PRP012",
      "Revision": "001",
      "Name": "Type of seat",
```



```

    "Status": "Develop",
    "DataType": {
      "Type": "String",
      "KeyLOV": "TDOC#09-KEY002#001"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP013",
    "Revision": "001",
    "Name": "Seat",
    "Status": "Develop",
    "DataType": {
      "Type": "Reference",
      "BlockReference": "TDOC#01-CLS005#001",
      "CardinalityController": "TDOC#02-PRP014#001"
    }
  },
  {
    "ObjectType": "02",
    "Namespace": "TDOC",
    "ID": "PRP014",
    "Revision": "001",
    "Name": "Number of seats",
    "Status": "Develop",
    "DataType": {
      "Type": "Integer"
    }
  }
]
}

```

3. Create the class definitions, including the property block classes.

This example introduces two new classes, **CLS005**, the property block class, and **CLS006**, the application class. Save the following in a file named **cardinalClassDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "ObjectType": "01",
      "Namespace": "TDOC",
      "ID": "CLS005",
      "Revision": "001",
      "Name": "Seat properties",
      "Status": "Develop",
      "IsAbstract": true,
      "UnitSystem": 3,
      "ClassType": "Block",
      "ClassAttributes": [
        {
          "Type": "Property",

```

```

        "Reference": "TDOC#02-PRP012#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP004#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP005#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP006#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP003#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP007#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP008#001"
      }
    ]
  },
  {
    "ObjectType": "01",
    "Namespace": "TDOC",
    "ID": "CLS006",
    "Revision": "001",
    "Name": "Cardinal car",
    "Status": "Develop",
    "IsAbstract": false,
    "UnitSystem": 3,
    "ClassType": "Application Class",
    "ClassAttributes": [
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP013#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP014#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP002#001"
      }
    ]
  }
]
}

```

4. Import the node definition that makes the class visible in.

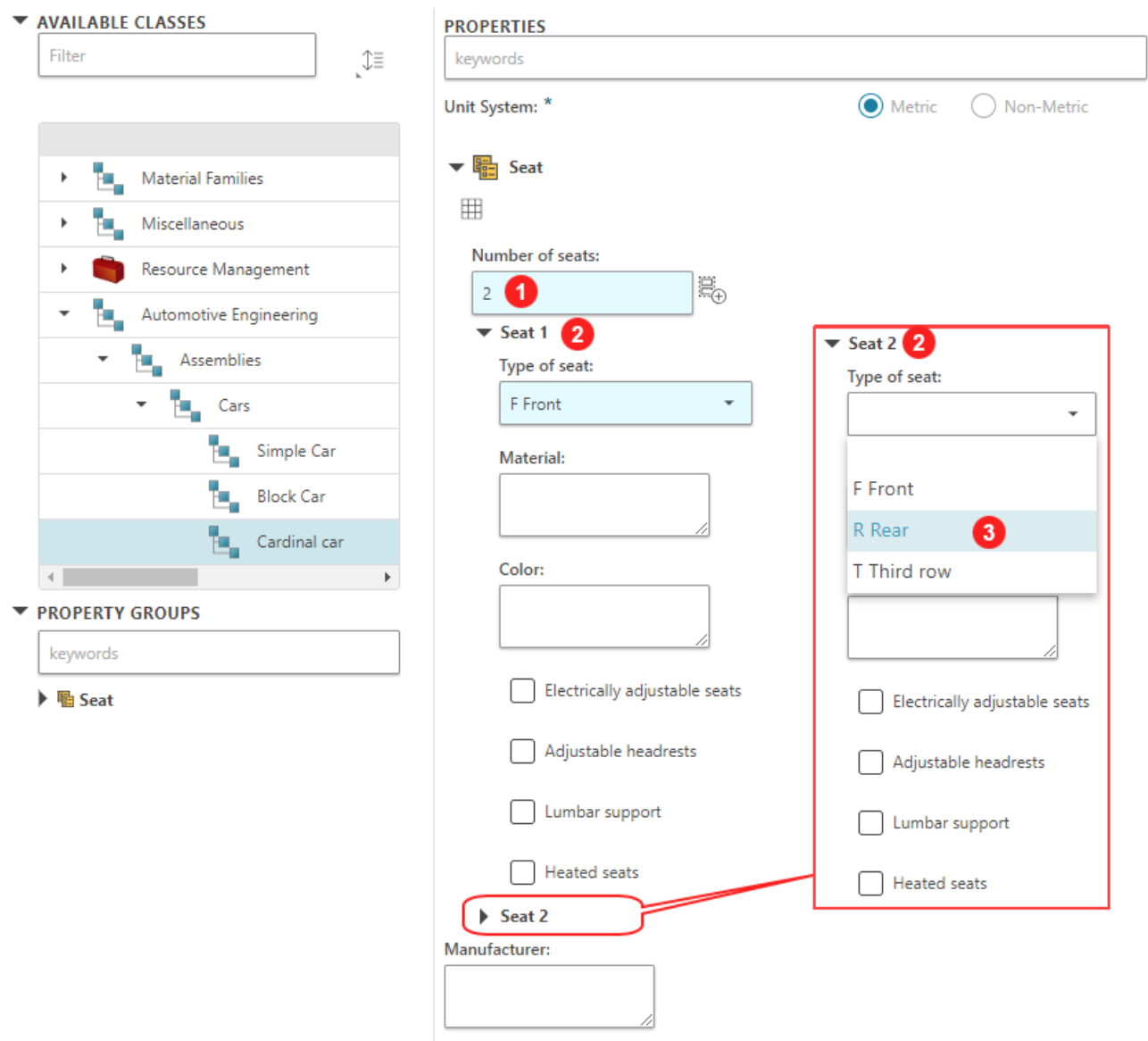
The **Cardinal car** node must be displayed in the **Cars** node that we created in the first example. To do this, import the new node specifying the **Car** node, **CSTAA1**, as the parent. Save the following in a file named **cardinalNodeDef.json**.

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAA4",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA1",
        "Revision": "001"
      },
      "Name": "Cardinal car",
      "ApplicationClass": {
        "Namespace": "TDOC",
        "ID": "CLS006",
        "Revision": "001"
      }
    }
  ]
}
```

5. Import the definitions into the database using [Classification Manager](#) or with [clsutility commands](#).

Results

The structure is displayed as follows:



After specifying the number of seats in 1, that number of **Seat** property blocks is displayed 2. You can assign different values for each seat row. With the new key-LOV 3, you can specify a separate type of seat for each row.

Import a class with polymorphic properties

This example shows how to create a class where we can specify the properties displayed depending on the type of seat row in a car that you select during classification. The following table displays the different properties available for each seat row type:

Front seat	Rear seat
Material	Material
Color	Color
Adjustable headrests	Adjustable headrests
Electrically adjustable seats	Rear seat entertainment
Lumbar support	
Heated seats	

These are the same properties used in the [second example](#). The difference is that in this example, you can specify the seat type during classification and, depending on this selection, display different properties. The concept of displaying different properties depending on the value of an additional property is referred to as *polymorphism*.

ASSIGNED CLASSIFICATIONS  

Automotive Engineering > Assemblies > Cars > Polymorphic car

▼ **AVAILABLE CLASSES**

Filter 

- Material Families
- Miscellaneous
- Resource Management
- Automotive Engineering
 - Assemblies
 - Cars
 - Simple Car
 - Block Car
 - Cardinal car
 - Polymorphic car**

▼ **PROPERTY GROUPS**

keywords

 **Type of seat**

PROPERTIES

keywords

Unit System: * ☒ Metric ☐ Non-Metric

▼  **Type of seat**

Type of seat: * Front  Rear

Material:

Color:

☐ Electrically adjustable seats

☐ Adjustable headrests

☐ Lumbar support

☐ Heated seats

Manufacturer:

Type of seat: *

Material:

Color:

☐ Adjustable headrests

☐ Rear seat entertainment system

This example contains two classes describing the front and rear seats:

Property Block (ClassDefinition)

IRDI

TDOC#01-CLS007#001

Name

Front Seat

Class Attributes

Name: **Type of Seat**
Type: String
Reference: TDOC#02-PRP0016#001

Name: **Material**
Type: String
Reference: TDOC#02-PRP004#001

Name: **Color**
Type: String
Reference: TDOC#02-PRP005#001

Name: **Adjustable headrests**
Type: Boolean
Reference: TDOC#02-PRP003#001

Name: **Electrically adjustable seats**
Type: Boolean
Reference: TDOC#02-PRP006#001

Name: **Lumbar support**
Type: Boolean
Reference: TDOC#02-PRP007#001

Name: **Heated seats**
Type: Boolean
Reference: TDOC#02-PRP008#001

Property Block (ClassDefinition)

IRDI

TDOC#01-CLS008#001

Name

Rear Seat

Class Attributes

Name: **Type of Seat**
Type: String
Reference: TDOC#02-PRP0016#001

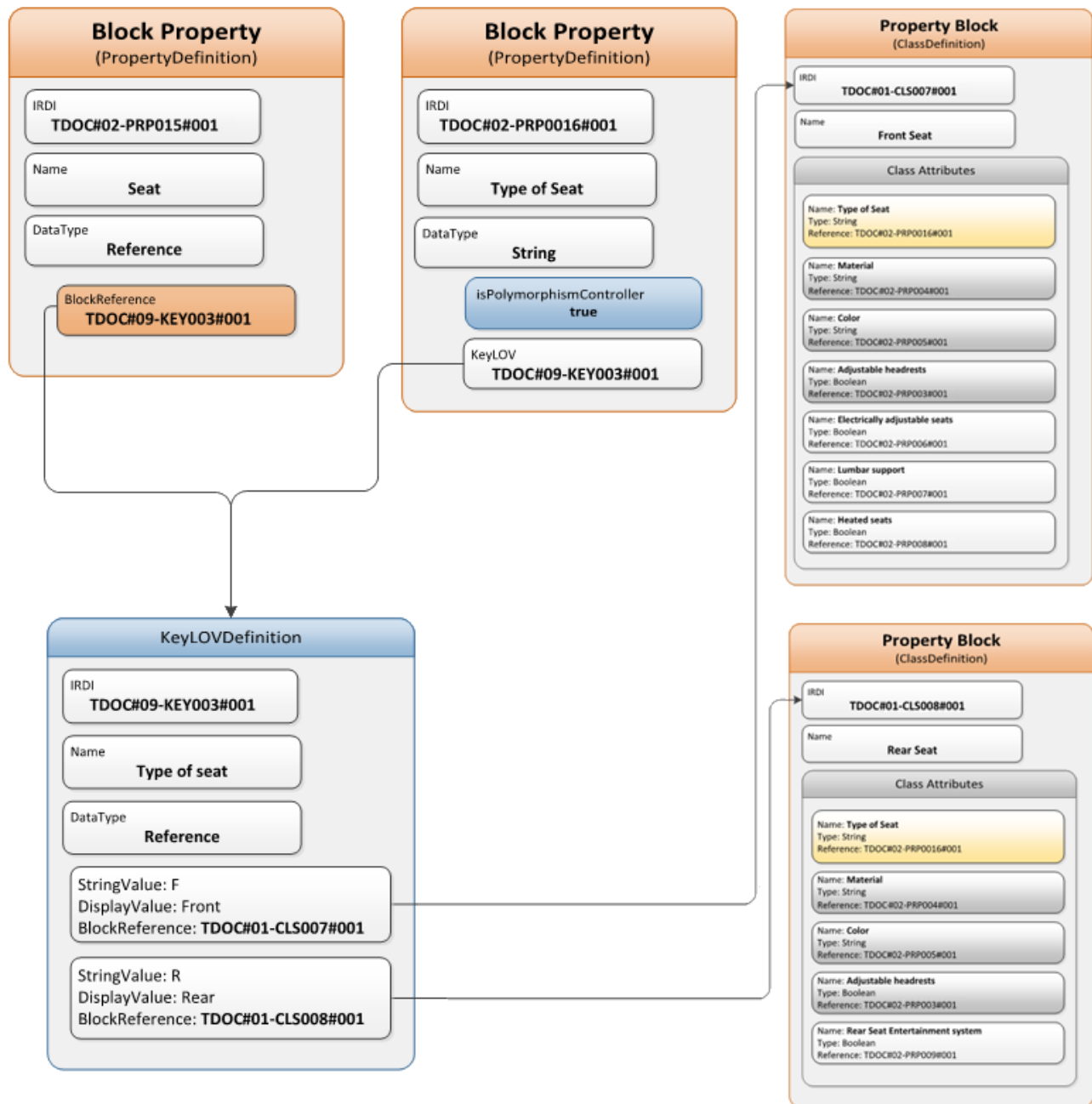
Name: **Material**
Type: String
Reference: TDOC#02-PRP004#001

Name: **Color**
Type: String
Reference: TDOC#02-PRP005#001

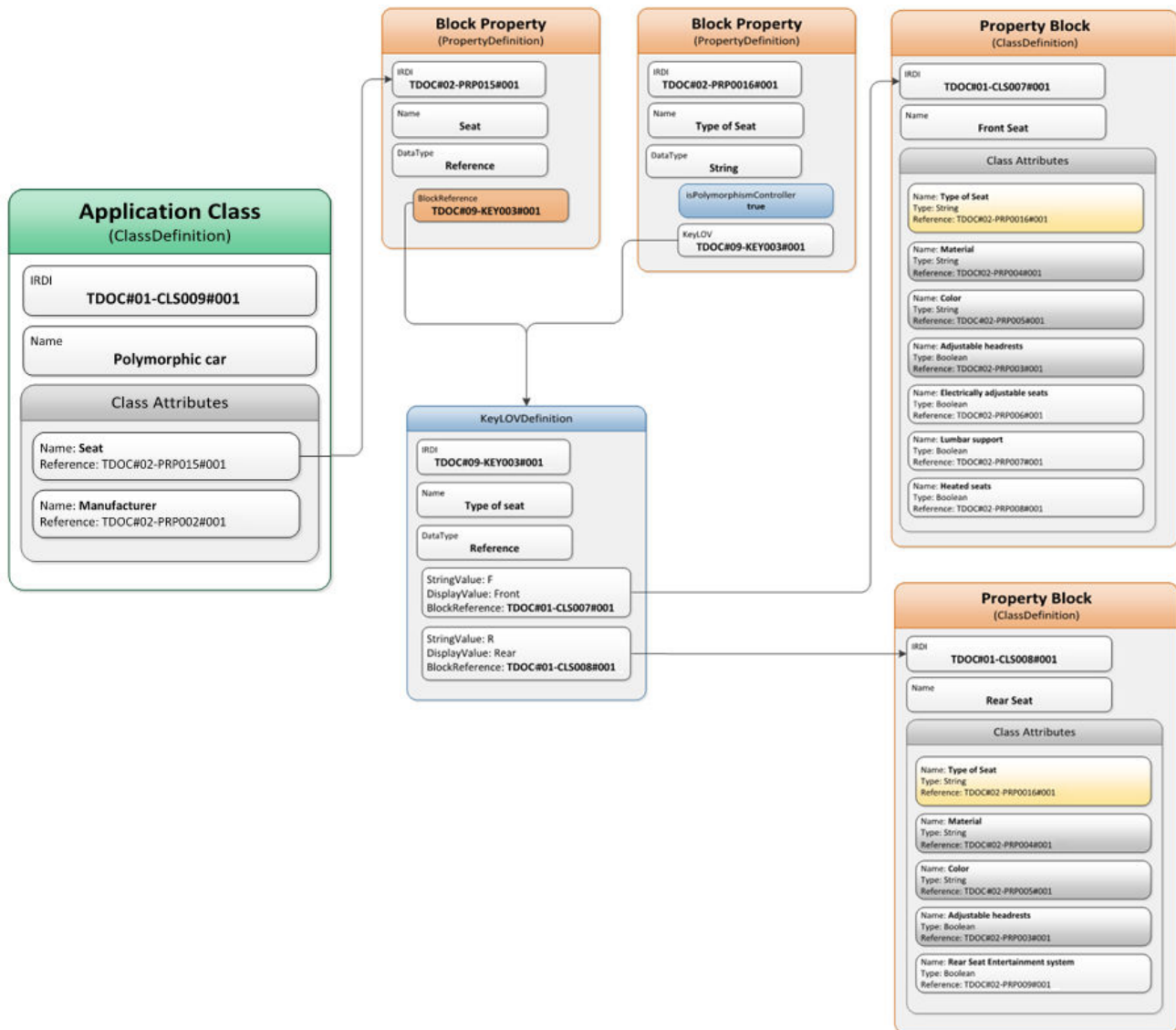
Name: **Adjustable headrests**
Type: Boolean
Reference: TDOC#02-PRP003#001

Name: **Rear Seat Entertainment system**
Type: Boolean
Reference: TDOC#02-PRP009#001

Each of the property block classes contains a special property that specifies the type of seat. This property references a key-LOV with the values **Front** or **Rear**.



The following application class contains the polymorphic **Seat** property that references the key-LOV.



Another property that references the key-LOV, **PRP016**, contains the **isPolymorphismController** attribute. When this property is used in a property block class (**CLS0007** or **CLS0008**), the **isPolymorphismController** attribute indicates that the property is polymorphic and that its value determines the list of class properties displayed in the user interface.

To create the JSON files for this example:

Procedure

1. Create the key-LOV definition.

This example introduces one new key-LOV that is saved in a file called **polymorphicKeylovDef.json**:

```
{
  "SchemaVersion": "1.0.0",
```



```

"Locale": "en_US",
"KeyLOVDefinitions": [
  {
    "ObjectType": "09",
    "Namespace": "TDOC",
    "ID": "KEY003",
    "Revision": "001",
    "Name": "Type of seat",
    "Status": "Develop",
    "LOVItems": {
      "DataType": "Reference",
      "LOVReferenceItems": [
        {
          "StringValue": "F",
          "DisplayValue": "Front",
          "BlockReference": "TDOC#01-CLS007#001"
        },
        {
          "StringValue": "R",
          "DisplayValue": "Rear",
          "BlockReference": "TDOC#01-CLS008#001"
        }
      ]
    }
  }
]
}

```

The values of the key-LOV, **Front** and **Rear**, reference their respective property block classes, **CLS007** and **CLS008**.

2. Create the individual property definitions.

This example uses many of the properties created in the earlier examples, as well as two new properties. Create the following JSON file and save it in a file named **polymorphicPropDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyDefinitions": [
    {
      "ObjectType": "02",
      "Namespace": "TDOC",
      "ID": "PRP015",
      "Revision": "001",
      "Name": "Seat",
      "Status": "Develop",
      "DataType": {
        "Type": "Reference",
        "BlockReference": "TDOC#09-KEY003#001"
      }
    },
    {
      "ObjectType": "02",
    }
  ]
}

```

```

    "Namespace": "TDOC",
    "ID": "PRP016",
    "Revision": "001",
    "Name": "Type of Seat",
    "Status": "Develop",
    "DataType": {
      "Type": "String",
      "KeyLOV": "TDOC#09-KEY003#001",
      "IsPolymorphismController": true
    }
  }
]
}

```

PRP016 contains the **IsPolymorphismController** attribute.

3. Create the class definitions, including the property block classes.

This example introduces two new property block classes very similar to those in the block class example, but additionally containing the polymorphic **PRP016** property. Save this in a file named **polymorphicClassDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "ObjectType": "01",
      "Namespace": "TDOC",
      "ID": "CLS007",
      "Revision": "001",
      "Name": "Front seat",
      "Status": "Develop",
      "IsAbstract": true,
      "UnitSystem": 3,
      "ClassType": "Block",
      "ClassAttributes": [
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP016#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP004#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP005#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP006#001"
        }
      ]
    }
  ]
}

```

```

        "Type": "Property",
        "Reference": "TDOC#02-PRP003#001"
    },
    {
        "Type": "Property",
        "Reference": "TDOC#02-PRP007#001"
    },
    {
        "Type": "Property",
        "Reference": "TDOC#02-PRP008#001"
    }
]
},
{
    "ObjectType": "01",
    "Namespace": "TDOC",
    "ID": "CLS008",
    "Revision": "001",
    "Name": "Back seat",
    "Status": "Develop",
    "IsAbstract": true,
    "UnitSystem": 3,
    "ClassType": "Block",
    "ClassAttributes": [
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP016#001"
        },
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP004#001"
        },
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP005#001"
        },
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP003#001"
        },
        {
            "Type": "Property",
            "Reference": "TDOC#02-PRP009#001"
        }
    ]
},
{
    "ObjectType": "01",
    "Namespace": "TDOC",
    "ID": "CLS009",
    "Revision": "001",
    "Name": "Polymorphic car",
    "Status": "Develop",
    "IsAbstract": false,
    "UnitSystem": 3,
    "ClassType": "Application Class",

```

```

    "ClassAttributes": [
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP015#001"
      },
      {
        "Type": "Property",
        "Reference": "TDOC#02-PRP002#001"
      }
    ]
  }
]
}

```

The application class **CLS009** references **PRP015**, which is associated with the polymorphic attribute **PRP016** through the key-LOV.

4. Import the node definition that makes the class visible.

The **Polymorphic car** node must be displayed in the **Cars** node that we created in the first example. To do this, import the new node specifying the **Car** node **CSTAA1** as the parent. Save the following in a file named **polymorphicNodeDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAA5",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA1",
        "Revision": "001"
      },
      "Name": "Polymorphic car",
      "ApplicationClass": {
        "Namespace": "TDOC",
        "ID": "CLS009",
        "Revision": "001"
      }
    }
  ]
}

```

5. Import the definitions into the database using **Classification Manager** or with **clsutility commands**.

Import a class with polymorphic and cardinal properties

This example shows how to create a class where you can first specify the number of seat rows a car has during classification. For each seat row, the properties displayed depend on the type of seat row that you select.

ASSIGNED CLASSIFICATIONS  

Automotive Engineering > Assemblies > Cars > Polymorphic cardinal car

▼ **AVAILABLE CLASSES**

Filter 

- Material Families
- Miscellaneous
- Resource Management
- Automotive Engineering
 - Assemblies
 - Cars
 - Simple Car
 - Block Car
 - Cardinal car
 - Polymorphic car
 - Polymorphic cardinal car**

▼ **PROPERTY GROUPS**

keywords

►  **Seat**

PROPERTIES

keywords

Unit System: * ☒ Metric ☐ Non-Metric

▼  **Seat**

Number of seats:  

▼  **Seat 1**

Type of seat: *

Material:

Color:

☐ Electrically adjustable seats

☐ Adjustable headrests

☐ Lumbar support

☐ Heated seats

►  **Seat 2**

▼  **Seat 2**

Type of seat: *

Material:

Color:

☐ Adjustable headrests

☐ Rear seat entertainment system

Manufacturer:

This example uses the same two property block classes as the [previous example](#), describing the front and rear seats:

Property Block (ClassDefinition)

IRDI

TDOC#01-CLS007#001

Name

Front Seat

Class Attributes

Name: **Type of Seat**
Type: String
Reference: TDOC#02-PRP0016#001

Name: **Material**
Type: String
Reference: TDOC#02-PRP004#001

Name: **Color**
Type: String
Reference: TDOC#02-PRP005#001

Name: **Adjustable headrests**
Type: Boolean
Reference: TDOC#02-PRP003#001

Name: **Electrically adjustable seats**
Type: Boolean
Reference: TDOC#02-PRP006#001

Name: **Lumbar support**
Type: Boolean
Reference: TDOC#02-PRP007#001

Name: **Heated seats**
Type: Boolean
Reference: TDOC#02-PRP008#001

Property Block (ClassDefinition)

IRDI

TDOC#01-CLS008#001

Name

Rear Seat

Class Attributes

Name: **Type of Seat**
Type: String
Reference: TDOC#02-PRP0016#001

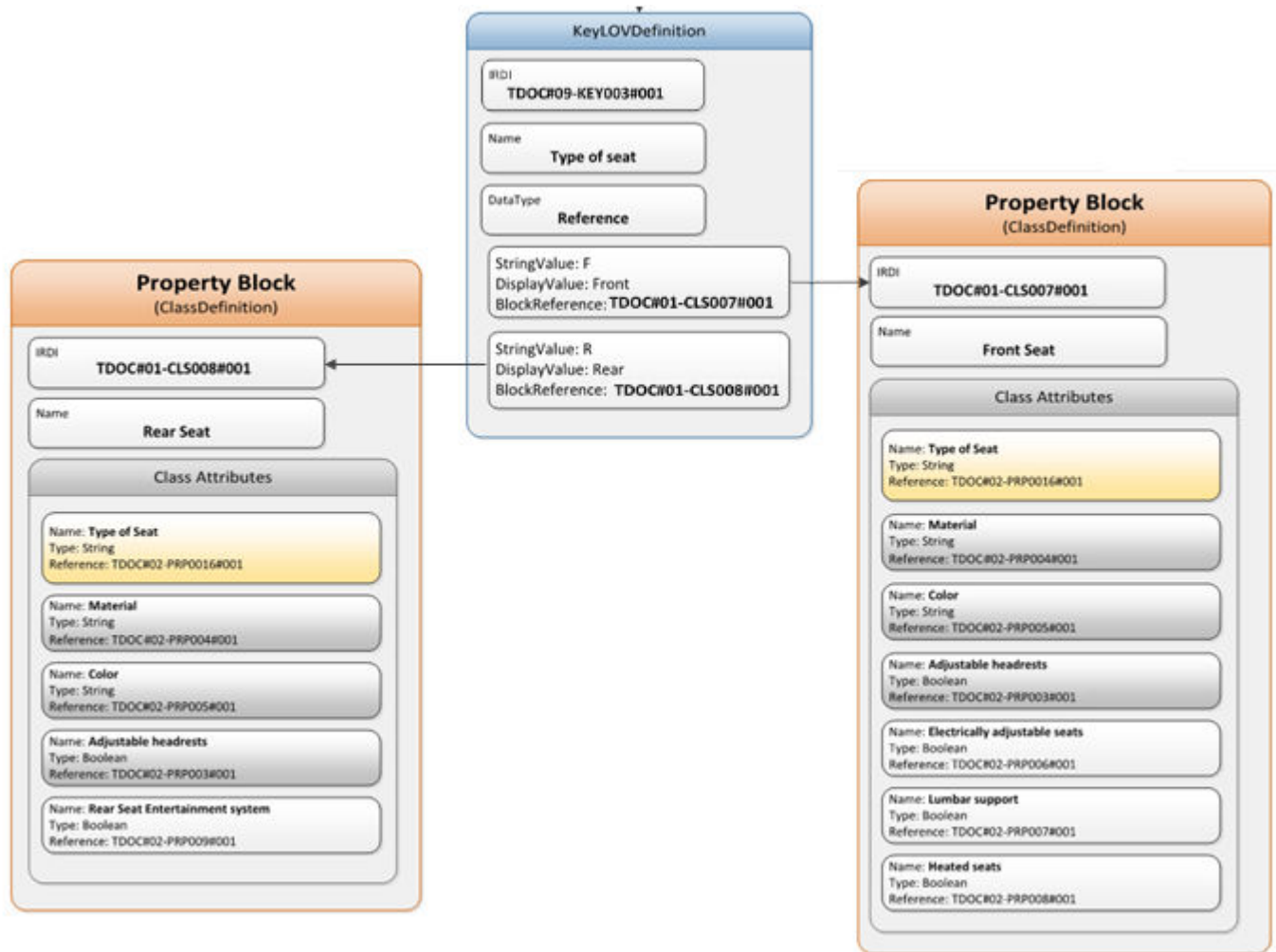
Name: **Material**
Type: String
Reference: TDOC#02-PRP004#001

Name: **Color**
Type: String
Reference: TDOC#02-PRP005#001

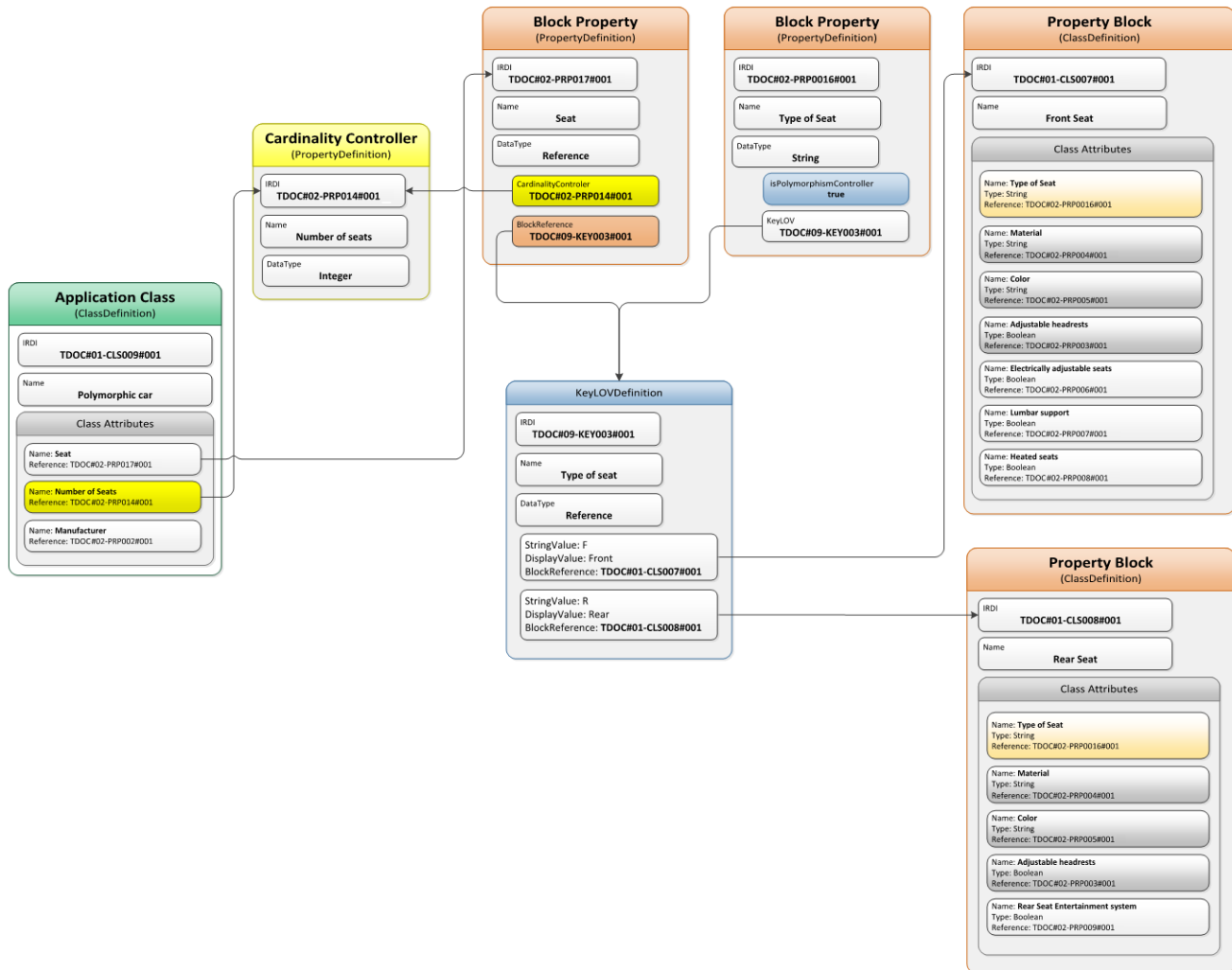
Name: **Adjustable headrests**
Type: Boolean
Reference: TDOC#02-PRP003#001

Name: **Rear Seat Entertainment system**
Type: Boolean
Reference: TDOC#02-PRP009#001

Again, these classes are referenced by the same key-LOV as in the previous example, which determines which of these classes is displayed:



The application class references the properties that determine the polymorphism and cardinality:



After completing the previous examples, there are only a few new objects required to import this example. To create the JSON files for this example:

Procedure

1. Create the individual property definitions.

This example uses **PRP014** created in the cardinality example and **PRP016** created in the polymorphism example. One new property is required that references the key-LOV created in the polymorphism example. Create the following JSON file and save it in a file named **poly_cardPropDef.json**.

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyDefinitions": [
    {
      "ObjectType": "02",
      "Namespace": "TDOC",
```



```

    "ID": "PRP017",
    "Revision": "001",
    "Name": "Seat",
    "Status": "Develop",
    "DataType": {
      "Type": "Reference",
      "BlockReference": "TDOC#09-KEY003#001",
      "CardinalityController": "TDOC#02-PRP014#001"
    }
  }
]
}

```

This property references the key-LOV definition, as well as the cardinality controller property. The polymorphism is achieved by the reference of **PRP016** to the key-LOV.

2. Create the class definitions.

This example introduces only one new application class. Save this in a file named **poly_cardClassDef.json**.

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "ObjectType": "01",
      "Namespace": "TDOC",
      "ID": "CLS010",
      "Revision": "001",
      "Name": "Polymorphic cardinal car",
      "Status": "Develop",
      "IsAbstract": false,
      "UnitSystem": 3,
      "ClassType": "Application Class",
      "ClassAttributes": [
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP014#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP017#001"
        },
        {
          "Type": "Property",
          "Reference": "TDOC#02-PRP002#001"
        }
      ]
    }
  ]
}

```

3. Import the node definition that makes the class visible.

The **Polymorphic cardinal car** node must be displayed in the **Cars** node that we created in the first example. To do this, import the new node specifying the **Car** node **CSTAA1** as the parent. Save the following in a file named **poly_cardNodeDef.json**.

```
{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "NodeDefinitions": [
    {
      "Namespace": "TDOC",
      "ID": "CSTAA6",
      "Revision": "001",
      "Parent": {
        "Namespace": "TDOC",
        "ID": "CSTAA1",
        "Revision": "001"
      },
      "Name": "Polymorphic cardinal car",
      "ApplicationClass": {
        "Namespace": "TDOC",
        "ID": "CLS010",
        "Revision": "001"
      }
    }
  ]
}
```

4. Import the definitions into the database using **Classification Manager** or with **clsutility** commands.

Importing classification objects to CST

Classification objects can be imported using the JSON format. The schema file for this is:

..\TD\classification\json\schema\advanced\Classification_Save_PropertyRecords_Request_advanced.schema.json

When creating the JSON file, you can specify whether only the item for the property record is created, or additionally specify a class into which the item is immediately classified during import. Importing the following example classifies an item into the **Polymorphic cardinal car** class that was created in the [import example](#).

Tip

Find the example as text that you can copy [here](#).

```

1  {
2      "SchemaVersion": "1.0.0",
3      "Locale": "en_US",
4      "PropertyRecords": [
5          {
6              "ID": "029159/A",
7              "ObjectType": "PR",
8              "ClassDefinition": "TDOC#01-CLS010#001",
9              "UnitSystem": 1,
10             "ClassifiedObject": {
11                 "ObjectType": "Item",
12                 "ClassifyRevision": true,
13                 "ID": [
14                     {
15                         "PropertyName": "item_id",
16                         "PropertyValue": "029159"
17                     }
18                 ],
19                 "Properties": [
20                     {
21                         "PropertyName": "object_name",
22                         "PropertyValue": "my_car"
23                     }
24                 ]
25             },
26             "Properties": [
27                 {
28                     "ID": "TDOC#02-PRP014#001",
29                     "Name": "Number of seats",
30                     "Value": 2
31                 },
32                 {
33                     "ID": "TDOC#02-PRP017#001",
34                     "Name": "Seat",
35                     "Value": [
36                         {
37                             "Index": 1,
38                             "ClassDefinition": "TDOC#01-CLS008#001",
39                             "Properties": [
40                                 {
41                                     "ID": "TDOC#02-PRP004#001",
42                                     "Name": "Material",
43                                     "Value": "Polyester"
44                                 },
45                                 {
46                                     "ID": "TDOC#02-PRP005#001",
47                                     "Name": "Color",
48                                     "Value": "Black"
49                                 },
50                                 {
51                                     "ID": "TDOC#02-PRP003#001",
52                                     "Name": "Adjustable headrests",
53                                     "Value": true
54                                 },
55                                 {
56                                     "ID": "TDOC#02-PRP009#001",
57                                     "Name": "Rear seat entertainment system",
58                                     "Value": true
59                                 }
60                             ]
61                         }
62                     ]
63                 }
64             ],
65             "ID": "TDOC#02-PRP002#001",
66             "Name": "Manufacturer",
67             "Value": "Fancy Car Company"
68         }
69     ]
70 }

```

In the JSON file:

- Lines 10-25 create the classified item with ID **029159** and name **my_car**.
- Line 30 creates 2 instances (a cardinality of 2) of **Number of seats** and these seats are described in lines 36-99.
- The index shown in line 37 and 73 introduces each of the new seat instances.
- Lines 39 and 75 begin the list of properties for each type of seat.
- The list of properties specific to each seat type is described in lines 40-71 and lines

The **Name** property is not mandatory and is added to this example for purposes of clarity.

To import this property record that is saved in a file called **poly_car_prop_rec.json**, you must use the **clsutility**:

```
clsutility -create -classification_objects -request="poly_car_prop_rec.json"
```

JSON classification object input example

```

{
  "SchemaVersion": "1.0.0",
  "Locale": "en_US",
  "PropertyRecords": [
    {
      "ID": "029159/A",
      "ObjectType": "PR",
      "ClassDefinition": "TDOC#01-CLS010#001",
      "UnitSystem": 1,
      "ClassifiedObject": {
        "ObjectType": "Item",
        "ClassifyRevision": true,
        "ID": [
          {
            "PropertyName": "item_id",
            "PropertyValue": "029159"
          }
        ],
        "Properties": [
          {
            "PropertyName": "object_name",
            "PropertyValue": "my_car"
          }
        ]
      },
      "Properties": [
        {
          "ID": "TDOC#02-PRP014#001",
          "Name": "Number of seats",
          "Value": 2
        },
        {
          "ID": "TDOC#02-PRP017#001",
          "Name": "Seat",
          "Value": [
            {
              "Index": 1,
              "ClassDefinition": "TDOC#01-CLS007#001",
              "Properties": [
                {
                  "ID": "TDOC#02-PRP004#001",
                  "Name": "Material",
                  "Value": "Leather"
                }
              ],
              {
                "ID": "TDOC#02-PRP005#001",
                "Name": "Color",
                "Value": "Red"
              }
            ],
            {
              "ID": "TDOC#02-PRP006#001",
              "Name": "Electrically adjustable seats",
              "Value": true
            }
          ]
        }
      ]
    }
  ]
}

```

```

    },
    {
      "ID": "TDOC#02-PRP003#001",
      "Name": "Adjustable headrests",
      "Value": true
    },
    {
      "ID": "TDOC#02-PRP007#001",
      "Name": "Lumbar support",
      "Value": true
    },
    {
      "ID": "TDOC#02-PRP008#001",
      "Name": "Heated seats",
      "Value": true
    }
  ]
},
{
  "Index": 2,
  "ClassDefinition": "TDOC#01-CLS008#001",
  "Properties": [
    {
      "ID": "TDOC#02-PRP004#001",
      "Name": "Material",
      "Value": "Polyester"
    },
    {
      "ID": "TDOC#02-PRP005#001",
      "Name": "Color",
      "Value": "Black"
    },
    {
      "ID": "TDOC#02-PRP003#001",
      "Name": "Adjustable headrests",
      "Value": true
    },
    {
      "ID": "TDOC#02-PRP009#001",
      "Name": "Rear seat entertainment",
      "Value": true
    }
  ]
}
]
},
{
  "ID": "TDOC#02-PRP002#001",
  "Name": "Manufacturer",
  "Value": "Fancy Car Company"
}
]
}
system",
}

```

```
} ]
```

clsutility commands required to import example json files

When importing files for [this example](#), if you choose to import using the **clsutility**, run each of the following commands in a Teamcenter command window:

Import a class with simple properties

```
clsutility -u=user -p=password -g=group -create -keylov_definitions  
-request="simpleKeylovDef.json"  
clsutility -u=user -p=password -g=group -create -property_definitions  
-request="simplePropDef.json"  
clsutility -u=user -p=password -g=group -create -class_definitions  
-request="simpleApplicationClassDef.json"  
clsutility -u=user -p=password -g=group -create -node_definitions  
-request="simpleNodeDef.json"
```

Import a class with a block properties

```
clsutility -u=user -p=password -g=group -create -property_definitions  
-request="blockPropDef.json"  
clsutility -u=user -p=password -g=group -create -class_definitions  
-request="blockClassDef.json"  
clsutility -u=user -p=password -g=group -create -node_definitions  
-request="blockNodeDef.json"
```

Import a class with cardinal properties

```
clsutility -u=user -p=password -g=group -create -keylov_definitions  
-request="cardinalKeyLOVDef.json"  
clsutility -u=user -p=password -g=group -create -property_definitions  
-request="cardinalPropDef.json"  
clsutility -u=user -p=password -g=group -create -class_definitions  
-request="cardinalClassDef.json"  
clsutility -u=user -p=password -g=group -create -node_definitions  
-request="cardinalNodeDef.json"
```

Import a class with polymorphic properties

```
clsutility -u=user -p=password -g=group -create -keylov_definitions  
-request="polymorphicKeylovDef.json"  
clsutility -u=user -p=password -g=group -create -property_definitions  
-request="polymorphicPropDef.json"  
clsutility -u=user -p=password -g=group -create -class_definitions  
-request="polymorphicClassDef.json"  
clsutility -u=user -p=password -g=group -create -node_definitions  
-request="polymorphicNodeDef.json"
```


Import a class with polymorphic and cardinal properties

```
clsutility -u=user -p=password -g=group -create -property_definitions
-request="poly_cardPropDef.json"
clsutility -u=user -p=password -g=group -create -class_definitions
-request="poly_cardClassDef.json"
clsutility -u=user -p=password -g=group -create -node_definitions
-request="poly_cardNodeDef.json"
```

Import a property record

```
clsutility -create -classification_objects -request="poly_car_prop_rec.json"
```

Export classification data in JSON format

You can export classification data in JSON format from the user interface.

Restrictions and limitations

This feature only exports the selected node or class. Any dependent data and groups are not included in the export.

Procedure

1. Open the **Active Admin** workspace.
2. From **Launcher**, search for **Classification Manager**.
If this option is unavailable, search for it. You can pin it from the search results for future access.
3. Select either a node, class, property, or key LOV, and click **Export** .
4. From the **Export** panel, select **Export Dependent Object** to include all dependent objects along with the selected object.

Tip

As a best practice, export Nodes without the dependent data.

5. Click **Export** to start the export.

Results

A notification appears in **Alerts** when the export completes successfully.

The file is available for download in the **Target Object** section of the alert report.

Export classified data for advanced classification BMEcat XML format

You can export classified data in the BMEcat XML from the user interface.

Procedure

1. Open the classified data that you want to export.
2. Choose **More Commands ***** > **Import/Export**  > **Export Classification Data**.
3. In the **File Name** box, give a name to the file that will be downloaded.
4. Click **Export**.

Results

When the export is complete, you are notified in the **Alert** area where you can view the details of the export.

The file is available in the **Target Object** section of the alert report. Download the file from that section.

This feature requires that Subscription Manager is running. If the Subscription Manager is not running, no notification is displayed in the **Alert** area, but the operation still occurs in the background.

Export classified objects using the clsutility command

To export the classification information (the *ICO*) for an object classified in a CST hierarchy, use the following **clsutility** command:

```
clsutility -u=<user> -p=<password> -g=<group> -find -classification_objects -request=<JSON_file> -output=<path-to-output-file>
```

This command requires a JSON file resembling the following (see the [Classification_FindRequest_advanced.schema.json](#) schema file):

```
{
  "SchemaVersion": "1.0.0",
  "Locale" : "en_US",
  "ObjectIDList": [
    {
      "ID": "026144/A"
    }
  ]
}
```

The input JSON file contains a list of object IDs for which the ICO information is exported to a file of your choice.

Share objects with classification data across sites with Briefcase files

You can share objects that have classification data with other sites using Briefcase files.

Select the object to share with the other site, export the object to a Briefcase file, provide that file to the other site, and upload the data to that site.

Procedure

1. Select the object that has classified data and choose **More commands *** > Import/Export**  > **Export Briefcase**  to display the **Export to Briefcase** panel.
2. Specify the export properties on the **Export Briefcase** panel.

Property	Description
Target	Select Teamcenter if you are sharing data with a site managing its data with Teamcenter.
Target Site	Enter the name of the site to which you are exporting data.
Transfer Option Set	Select TIEUnconfiguredLLBCZExportDefault .
Briefcase Name	Accept the default filename or update it as necessary.

3. Click **Export** to create the Briefcase file. You receive an alert when the export is complete.
4. Download and give the exported Briefcase (BCZ) file to the target site.
5. In the target site, select a folder and choose **More commands *** > Import/Export**  > **Import Briefcase**  to display the **Import Briefcase** panel.
6. Click **Choose File** and navigate to the BCZ file location.
7. In the **Transfer Option Set** choose **TIELLBCZImportOptionSetDefault**.
8. Click **Import**. You receive an alert when the import is complete.

Related Topics

[Using Briefcase files](#)

Enable auditing for classification events

Overview of auditing classification events

You can allow audit logging of classification events as follows:

- **Make the audit logs visible for users.**

You can specify who can view audit logs by updating the values of preferences.

- **Allow auditing for certain event types**

By default, only the audit logs of modify and delete events are saved. You can activate audit logging for other classification events in the **Audit Definition** editor in Business Modeler IDE.

- **Allow auditing for class attributes**

By default, the audit logs of class attributes are not saved. You can specify which class attributes can support auditing.

- **Test if you can view the audit logs**

Make audit logs visible for users

To make audit logs of Classification events visible for users, you must update certain preferences.

Procedure

1. Update the value of the **AWC_show_audit_logs** preference to **true**.
2. Update the value of the **AWC_show_classification_audit_log** preference to **true** for the group or role that needs access to the audit log.

Allow auditing for certain event types

By default, audit logs for modification and deletion events are saved. If you want to allow auditing for other event types, perform the following configurations.

Procedure

1. In Business Modeler IDE, open the **Audit Definition** editor.
2. Navigate to the classification event and select the **Is Active?** check box.

Allow auditing for class attributes using JSON file

To log updates for class attributes, you must permit auditing for each class attribute using the JSON file.

Example

For a class with IRDI TST02#01-CLS05#001, you want to allow auditing for the class attributes **Country** and **City**.

Overview Class Attributes



Name	Attribute I...	Type	Reference
Country	1	Property	TST02#02-PRP1#001
State	2	Property	TST02#02-PRP2#001
City	3	Property	TST02#02-PRP3#001

Restrictions and limitations

Complex data types are not supported.

Procedure

1. Create a JSON file as follows:

```
{
  "SchemaVersion": "1.5.0",
  "Locale": "en_US",
  "ClassDefinitions": [
    {
      "Update": "TST02#01-CLS05#001",
      "Values": {
        "ClassAttributeOptions": [
          {
            "AttributeIRDI": "TST02#02-PRP1#001"
            "IsAuditable": true
          },
          {
            "AttributeIRDI": "TST02#02-PRP2#001"
            "IsAuditable": false
          },
          {
            "AttributeIRDI": "TST02#02-PRP3#001"
            "IsAuditable": true
          }
        ]
      }
    }
  ]
}
```

Update the JSON file as follows:

- In the **ClassDefinitions** section, specify the IRDI of the class in the **Update** section.
- In the **ClassAttributeOptions** section, update the **AttributeIRDI** attribute with the IRDI of the class attribute that you want to allow audit logging on.
- In the **ClassAttributeOptions** section, update the **IsAuditable** attribute to **true**.

2. Upload the JSON file.

Allow auditing for class attributes using the user interface

To log updates for class attributes, you must allow auditing for each class attribute.

Example

For a class called **Location**, you want to allow auditing for the class attributes **Country** and **City**.

Overview Class Attributes



Name	Attribute I...	Type	Reference
Country	1	Property	TST02#02-PRP1#001
State	2	Property	TST02#02-PRP2#001
City	3	Property	TST02#02-PRP3#001

Restrictions and limitations

Complex data types are not supported.

Procedure

1. Open the class and choose **Class Attributes**.
2. Choose the class attribute that you want to allow auditing for and select **Attribute Properties**.
3. In the **Attribute Properties** panel, click **Edit**.
4. Select the **Is Auditable** check box.

Attribute Properties

Cancel Save Expand ▼

Overview **Class Attributes**

Name

Country

State

City

Metric Default Value:

Non Metric Default Value:

Metric Constant Value:

Non Metric Constant Value:

Is Required: ☐

Is Display Optimized: ☐

Is Protected: ☐

☒ **Is Auditable**

Dependent Attribute:

5. Click **Save**.

View audit logs for classification events

You can view audit logs for classification events.

This feature is available only with Advanced Classification.

Procedure

Open a component, and click **Audit Logs**.

The audit logs for classification events appear in the **Classification Logs** section.

Classification preferences and utilities

eclass2json.pl

Converts the various ECLASS formats to JSON or, alternatively, converts JSON files to BMEcat format.
Specifically:

- Converts ontoML XML ECLASS hierarchy data to JSON format
- Converts BMEcat XML property record data to JSON format
- Converts JSON property records to BMEcat XML format

The **eclass2json** utility is a platform-independent Perl script that can be found in the following directory:

`\\TR\\bin\\eclass2json`

Syntax

**eclass2json -inputFile: -objectType:
-h**

Arguments

-inputFile:

Contains the source file for the conversion. This can be an ontoML XML file, a BMEcat XML file or a JSON file, depending on the conversion taking place.

-objectType:

Must be one of the following:

unitsmap

Creates an internal unit conversion file necessary for converting from ontoML files to JSON. This conversion is required once per ECLASS release.

definitions

Used to convert the following administrative objects in the order given:

1. Key-LOVs
2. Properties
3. Aspects
4. Blocks
5. Application classes
6. Classification classes

data

Used to convert BMEcat XML property records to JSON format.

json2BMEcat

Used to convert JSON format property records to BMEcat XML format.

-h

Displays assistance in using the utility.

Environment

This utility must be run in the Teamcenter shell environment. It is configured in the **eclass2json.properties** file contained in the same directory as the utility. The properties file contains directions on how to set the properties. In this property file you configure such things as:

- The status of the object being converted, for example, **Develop** or **Released**.
- Whether existing objects are skipped for conversion.
- Whether the converted object is to be added as a new release.
- The name of the top level node.
- The ECLASS level.
- The number of objects output in each file.

runClsAITraining

Runs the engine that trains classification data to create a model that is used for automatic class suggestions when classifying classes. Using artificial intelligence, classes are suggested based on the probability of them being an appropriate match.

Syntax

```
runClsAITraining -u=<user-name> {-p=<password> | -pf=<password-file>} -g=<group-name>
```

-h

Arguments

-u

Specifies the user ID.

This is generally a user with administration privileges.

-p

Specifies the password.

This argument is mutually exclusive with the **-pf** argument.

-pf

Specifies the password file.

This argument is mutually exclusive with the **-p** argument.

-g

Specifies the group associated with the user.

If used without a value, the user's default group is assumed.

-h

Displays help for this utility.

The **-filename** argument that was available in earlier releases is no longer supported.

Environment

This utility must be run in the Teamcenter shell environment. It is found in the following directory:

<Teamcenter-root (TR)>\classification\ai

cls_ai_auto_classify

Classifies many objects at the same time using AI. Objects classified in this way are sent to a workflow where administrators can check the validity of the AI predictions and make changes to the classification or reject the suggested classification.

Syntax

```
cls_ai_auto_classify -u=<user-name> {-p=<password> | -pf=<password-file>} -g=<group-name>
```

```
-startDate -endDate -objectType -classifyObjects -outputPath -item-h]
```

Arguments

-u

Specifies the user ID.

This is generally a user with administration privileges.

-p

Specifies the password.

This argument is mutually exclusive with the **-pf** argument.

-pf

Specifies the password file.

This argument is mutually exclusive with the **-p** argument.

-g

Specifies the group associated with the user.

If used without a value, the user's default group is assumed.

-startDate

Restricts classification AI training data to objects created after the date specified.

-endDate

Restricts classification AI training data to objects created before the date specified.

-objectType

Restricts classification AI training data to objects with a specified object type.

-classifyObjects

Classifies objects that have a maximum probability above the percentage set in the **CLS_AI_Engine_Probability_Cutoff** preference.

Running the utility without this argument runs it in a dry run mode. A report containing the object, its classification, the probability, and an indication of whether the utility is capable of classifying the object, is stored in the specified location.

-outputPath

Specifies the file path where the auto-classify report is stored.

-item

Specifies a suffix for the training model output.

-h

Displays help for this utility.

Environment

This utility must be run in the Teamcenter shell environment.

clsutility

Performs administrative tasks when the Presentation Layer is installed. Using a **multitude of arguments**, you can create, update, delete, and migrate data. You can also use this utility to **display the node hierarchy** in various levels of detail.

Syntax

```
clsutility -u=<user-name> {-p=<password> | -pf=<password-file>} -g=<group-name>
```

```
[-create | -delete | -save | -find | -ask | -add | -import | -search | -set | -update | -get |
```

```
-remove | -describe | -migrate | -list | -h]
```

Arguments

The **clsutility** utility contains many arguments and subarguments whose descriptions and syntax you can find in the help contained within the utility.

- List the utility help in the customary fashion:

```
clsutility -h
```

- List the utility help for subarguments as follows:

```
clsutility -<subargument> -h
```

For example:

```
clsutility -create -h
```

Running this command lists the help for all the subarguments of the **-show** command, such as the following:

USAGE

```
clsutility
-u=<username>
{-p=<password> | -pf=<password_file> }
-g=<group>
-output=<output file name>
<command> <sub-command> <command options...>
-----
For the command "-create" the following sub-commands are available:
  -default_objects      [Creates default Classification Context,
Hierarchy and Scheme objects.]
  -node                 [Creates a Classification Hierarchy node
```

object.]	
-class_definitions	[Creates a Classification Standard
Taxonomy Class.]	
-property_definitions	[Creates a Classification Standard
Taxonomy Property.]	
-keylov_definitions	[Creates a Classification Standard
Taxonomy Keylov.]	
-classification_object	[Creates a new instance of Classification
object.]	
-classification_objects	[Creates a set of Classification objects.]
-node_hierarchy	[Creates a Classification node hierarchy
from Classification hierarchy.]	
-node_definitions	[Creates a Classification standard
taxonomy Master node.]	
-view_definitions	[Creates Classification Standard Taxonomy
View Definitions.]	

Environment

This utility must be run in the Teamcenter shell environment.

clsutility reference tables

All **clsutility** commands begin with the following validation:

clsutility -u=<user-name> {-p=<password> | -pf=<password-file>} -g=<group-name>

Add the following to the command to perform the required actions. Optional arguments are displayed in brackets ([]).

All hierarchy definitions

Use this command	To
-save -classification_definitions -request=<JSON-file-name>	Create or update key-LOV, property, class, or node definitions described in the given JSON file. This argument is available beginning with schema 1.5.0. and can be used to import all previous hierarchy definitions.

Key-LOV definitions

Use this command	To
-create -keylov_definitions -request=<JSON-file-name>	Create key-LOVs described in the given JSON file.
-delete -keylov_definitions -request=<JSON-file-name>	Delete key-LOVs described in the given JSON file.
-update -keylov_definitions -request=<JSON-file-name>	Update key-LOV definition. If it does not exist, the utility creates a new key-LOV based on the data in the JSON file.
-update -status -request=<JSON-file-name>	Update the key-LOV definition status.

Property definitions

Use this command	To
-create -property_definitions -request=<JSON-file-name>	Create properties described in the given JSON file.
-delete -property_definitions -request=<JSON-file-name>	Delete properties described in the given JSON file.
-find -property_definitions -request=<JSON-file-name>	Find properties described in the given JSON file.
-update -property_definitions -request=<JSON-file-name>	Update properties described in the given JSON file.
-update -status -request=<JSON-file-name>	Update the property definition status.

Class definitions

Use this command	To
-create -class_definitions -request=<JSON-file-name>	Create classes described in the given JSON file.
-delete -class_definitions -request=<JSON-file-name>	Delete classes described in the given JSON file.
-find -class_definitions -request=<JSON-file-name>	Find classes described in the given JSON file.
-update -class_definitions -request=<JSON-file-name>	Update classes described in the given JSON file.
-update -status -request=<JSON-file-name>	Update the class definition status.

Nodes

Use this command	To
<code>-create -node_definitions -request=<JSON-file-name></code>	Create the hierarchy nodes described in the given JSON file.
<code>-delete -node_definitions -request=<JSON-file-name></code>	If a hierarchy node already exists, the utility updates the node with the values provided.
<code>-update -node -id=<hierarchy-node-ID> -name=<new-name-of-hierarchy-node></code>	Delete the hierarchy nodes described in the given JSON file.
<code>[-descr=<new-description-for-given-node>]</code>	Update a hierarchy node with the values provided.
<code>-find -all_nodes</code>	Update a hierarchy node with a new description.
<code>-find -node -id=<hierarchy-node-ID-including-namespace></code>	Find and display all nodes in the system.
<code>-find -top_level_nodes</code>	Find and display a single node based on its ID.
<code>-get -classification_objects -id=<hierarchy-node-ID></code>	Display all top-level nodes in the system.
<code>-ask -node_class_props -id=<hierarchy-node-ID></code>	List the classification objects belonging to the given hierarchy node.
<code>-ask -node_class_props -id=<hierarchy-node-ID> -name=<name-of-property- including-prefix></code>	List all properties belonging to a hierarchy node object.
<code>-ask -node_parent -id=<hierarchy-node-ID></code>	List all properties belonging to a hierarchy node object with a given name.
<code>[-full_hierarchy]: lists all ancestor nodes of a hierarchy node</code>	List the parent of a given hierarchy node object.
<code>-ask -node_ancestors -id=<hierarchy-node-ID></code>	List all parents of a given hierarchy node object.
<code>-ask -node_children -id=<hierarchy-node-ID></code>	List all child nodes of a given type for the specified hierarchy node object and optionally, for the given type.
<code>[-type=<{>0<: group >1<: master >3<: all}>]</code>	
<code>-ask -is_top_level_node -id=<hierarchy-node-ID></code>	Check whether a hierarchy node is a top-level node.
<code>-ask -is_descendent_node -id=<hierarchy-node-ID> -ancestor_id=<ancestor-hierarchy-node-ID></code>	Check whether a hierarchy node is a descendent of another node.
<code>-ask -node_type -id=<hierarchy-node-ID></code>	Display the type of the given hierarchy node.
<code>-ask -node_instances -id=<hierarchy-node-ID></code>	List the classification objects belonging to the given hierarchy node, including all the objects belonging to child nodes.
<code>-ask -node_class_props -id=<hierarchy-node-ID></code>	List properties belonging to a hierarchy node.
<code>-ask -node_class_prop -id=<hierarchy-node-ID> -name=<name-of-property-being-sought-including-prefix></code>	List all properties belonging to a hierarchy node.
<code>-ask -node_attachments -id=<hierarchy-node-ID></code>	List all the attachments belonging to the given hierarchy node.
<code>-ask -node_characteristic -id=<hierarchy-node-ID> -characteristic=<{>0<: is-an-assembly >1<: has-multiple-instances >3<: is-a-leaf-node}></code>	List whether a given node is an assembly, has multiple instances, or is a leaf node.
<code>-import -image -id=<hierarchy-node-ID> -os_path=<path-to-image-file></code>	Import an image for the given hierarchy node. Images are displayed in the right pane of the user interface.
<code>[-new_file_name=<new-file-name-used-by-Active-Workspace>]</code>	Teamcenter supports the following file types for class images: GIF, JPG, JPEG, PNG, BMP, SVG, and PDF.

Use this command	To
<code>[-do_update]</code> : updates existing primary image	
<code>[-is_primary_image]</code> : sets the image to primary image	
<code>-import -icon -id=<hierarchy-node-ID> -os_path=<path-to-image-file></code>	Import an icon for the given hierarchy node. Icons are displayed in the classification hierarchy and help to visualize the class names.
<code>[-new_file_name=<new-file-name-used-by-Active-Workspace>]</code>	Teamcenter supports the following file types for node icons: GIF, JPG, JPEG, PNG, BMP, SVG, and PDF.
<code>[-do_update]</code> : updates existing icon	
<code>-remove -image -id=<hierarchy-node-ID></code>	Remove an image attached to a node.
<code>[-delete_dataset]</code> : removes image dataset in addition to image	
<code>-remove -icon -id=<hierarchy-node-ID></code>	Remove an icon attached to a node.
<code>[-delete_dataset]</code> : removes icon dataset in addition to icon	
<code>-get -image -id=<hierarchy-node-ID></code>	Identify the primary image used by the given hierarchy node.
<code>-get -images -id=<hierarchy-node-ID></code>	List all the images used by the given hierarchy node.
<code>-get -icon -id=<hierarchy-node-ID></code>	Identify the icon used by the given hierarchy node.

Classification objects (ICOs)

Use this command	To
<code>-create -classification_objects -node_id=<hierarchy-node-ID></code>	Create multiple classification instances.
<code>[-obj_count=<number-of-classification-objects>]</code>	
<code>[-prefixid=<prefix-assigned-to-new-objects>] </code>	
<code>[-request=<JSON-file-name>]</code>	
<code>-find -classification_objects -request=<JSON-file-name></code>	Export the classification properties of the classification objects listed in the JSON request file. In the request file, the ICO object ID must be listed with its revision, for example, 123456/A.

Classification views

Use this command	To
<code>-create -base_view_definitions</code>	Create base views for every released application class. If a base already exists for a class, the class is skipped.
<code>-create -base_view_definitions -class_definition <IRDI-of-class></code>	Create a base view for the given application class.
<code>-create -base_view_definitions -force</code>	Create or recreates a base view for every released application class.

Use this command	To
<code>-create -view_definitions -request=<JSON-file-name></code>	Create a user, group, or role view for a class, or modifies the order of or suppresses properties in a base view for a class.
<code>-get -class_descriptor -request=<JSON-file-name></code>	List the class descriptor for specified classes.

Examine the hierarchy using the clsutility -list -hierarchy utility

For most general use cases, Classification admin objects are listed in the **Classification Manager**. Additionally, you can use the **clsutility**, to list Classification Standard Taxonomy hierarchy objects to help you better understand the Classification data in the database. This method of viewing the hierarchy is based on the *class descriptor*. Whereas a class definition describes only the class, or a key-LOV definition only one key-LOV, the class descriptor compiles all the definitions required to describe a Classification object (ICO). Each node of the hierarchy shares a class descriptor. The class descriptor provides a view on the data from the Classification consumer perspective.

Note
When you list all the nodes in the database, the utility lists hierarchies that are not migrated to CST, but it cannot display ICOs belonging to these hierarchies.

You can direct the output to a text file that you can open in a text editor by using the `-output=<output-file-name.txt>`.

The following arguments are available when using **clsutility -list -hierarchy**:

Argument	Description
<code>-nodeId=</code>	Specifies the unique ID of the node for which you want to display the hierarchy. The utility displays all the parents of the given node, as well as the node itself. If a node ID is not provided, all top level nodes are displayed. To view the children of the given node, additionally use the -recursive argument. If you use the -nodeId argument, you cannot use the -classId and -classNamespace arguments.
<code>-classId=</code>	Specifies the class ID of the class definition referenced from the given node. If you specify a class ID here, you must also specify a namespace in the -classNamespace argument. If you use this argument, you cannot use the -nodeId argument.

Argument	Description
-classNameSpace=	Specifies the namespace of the class definition referenced from the node. If you specify a namespace here, you must also specify a class ID in the -classId argument. If you use this argument, you cannot use the -nodeId argument.
-recursive	Displays the entire hierarchy below the specified node. You cannot use this argument with the -showClass argument.
-showClass	Shows detailed information about the referenced classes of the given node. You cannot use this argument with the -recursive argument.
-showJSON	Displays the JSON response from CST_class_describe , which is used to build the report of a class.
-showType	Displays the class definition of the dynamic runtime type. This argument is used for Siemens Digital Industries Software internal troubleshooting purposes.
-showICOs	Displays the ICOs of the nodes.
-showICOProperties	Displays all ICO properties if you have also specified -showICOs .
-icoLimit=	Limits the number of ICOs of a single node that are to be displayed.
-icold=	Shows only the ICO with this ID.

- To view all the top nodes in the hierarchy, type:

clsutility -list -hierarchy

- To view all the nodes in the hierarchy, type:

clsutility -list -hierarchy -recursive

- To view all the nodes underneath a specified node, type:

clsutility -list -hierarchy -nodeId=<node ID> -recursive

- To view the details of a specific class, type:

clsutility -list -hierarchy -nodeId=<node ID> -showClass

For example, for an ECLASS class called **AC-DC supply**, the **clsutility -list -hierarchy -recursive** command can be used to find the node ID (in this example, **0173-1#AFX043**):

```
"27-04-06-90" [0173-1#BAB258 ==> 0173-1#01-ADP432#006] "[27-04-06-90] No-break power supply (complete, unspecified)"
}
"27-04-07-00" [0173-1#AFR649] "[27-04-07] Power supply device"{
  "27-04-07-01" [0173-1#AFX040 ==> 0173-1#01-AFR739#002] "[27-04-07-01] Continuous current supply"
  "27-04-07-02" [0173-1#AFX041 ==> 0173-1#01-AFR741#002] "[27-04-07-02] Voltage stabilizer"
  "27-04-07-03" [0173-1#AFX042 ==> 0173-1#01-AFR743#002] "[27-04-07-03] Alternating current supply"
  "27-04-07-04" [0173-1#AFX043 ==> 0173-1#01-AFR745#002] "[27-04-07-04] AC-DC supply"
  "27-04-07-05" [0173-1#AFX039 ==> 0173-1#01-AFR737#002] "[27-04-07-05] Stand-by unit"
  "27-04-07-90" [0173-1#AFX044 ==> 0173-1#01-AFR747#002] "[27-04-07-90] Power supply device (unspecified)"
}
"27-04-91-00" [0173-1#AAB638] "[27-04-91] Power supply (parts)" {
  "27-04-91-01" [0173-1#AFQ933 ==> 0173-1#01-AFR005#003] "[27-04-91-01] No-break power supply (complete, parts)"
  "27-04-91-90" [0173-1#AFZ646 ==> 0173-1#01-ADP554#006] "[27-04-91-90] Power supply (parts, unspecified)"
}
}
"27-04-92-00" [0173-1#AKP085] "[27-04-92] Power supply (accessories)" {
  "27-04-92-01" [0173-1#AKN587 ==> 0173-1#01-ADO360#006] "[27-04-92-01] UPS system (accessories)"
  "27-04-92-90" [0173-1#AEI674 ==> 0173-1#01-AEX753#004] "[27-04-92-90] Power supply (accessories, unspecified)"
}
}
```

You can then use the node ID as input for the **clsutility -list -hierarchy -nodeId=0173-1#AFX043 -showClass** command. The output resembles the following:

```
1 | "00-00-00-00" [0173-1#4711A1] "eClass ADVANCED 4711-A1"
2 | "27-00-00-00" [0173-1#AAB572] "[27] Electric engineering, automation, process control engineering"
3 | "27-04-00-00" [0173-1#AAB631] "[27-04] Power supply devices"
4 | "27-04-07-00" [0173-1#AFR649] "[27-04-07] Power supply device"
5 | "27-04-07-04" [0173-1#AFX043 ==> 0173-1#01-AFR745#002] "[27-04-07-04] AC-DC supply"
6 | { Class[0173-1#01-AFR745#002] "AC-DC supply"
7 |   Status: "Released"
8 |   01: [0173-1#01-ADN464#006] Aspect "Add on Documentation"
9 |     { Class[0173-1#01-ADN464#006] "Add on Documentation"
10 |       Status: "Released"
11 |       01:01 [0173-1#02-AAN469#002] Integer "number of documentation"
12 |       IsCardinalityController
13 |       02: [0173-1#02-AAQ680#006] Reference "Additional Information"
14 |       CardinalityController: 0173-1#02-AAN469#002
15 |       { Class[0173-1#01-ADN356#006] "Additional Information"
16 |         Status: "Released"
17 |         01:01 [0173-1#02-AAC312#001] Date "calendrical validity from"
18 |         02: [0173-1#02-AAN466#001] L10NString "Description"
19 |         03:01 [0173-1#02-AAN467#003] String "Country/region classification the document"
20 |         { KeyLOV[0173-1#09-AAD528#002] String (entries:246)
21 |           001: "AF" "Afghanistan" " " "0173-1#07-AAS015#001" ""
22 |           002: "AX" "Åland" " " "0173-1#07-AAS156#001" ""
23 |           003: "AL" "Albania" " " "0173-1#07-AAS157#001" ""
24 |           004: "DZ" "Algeria" " " "0173-1#07-AAS158#001" ""
25 |           005: "AS" "American Samoa" " " "0173-1#07-AAS159#001" ""
26 |           006: "AD" "Andorra" " " "0173-1#07-AAS161#001" ""
27 |           007: "AO" "Angola" " " "0173-1#07-AAS162#001" ""
28 |           008: "AI" "Anguilla" " " "0173-1#07-AAS163#001" ""
29 |           ...
30 |         }
```

This output displays all the parent nodes of **0173-1#AFX043**, as well as all the classes (application, block, aspects) referenced by it, the properties, and the key-LOVs included in the class. The properties are listed as follows:

```
15:04 [0173-1#02-AAB774#006] Double ("V") "max. 1. output voltage with AC"
```

1

2

3

4

5

1

2

3

Index:internal code

IRDI

Data type

4

Unit

5

Property name

If key-LOVs are used repeatedly in a class, their details are listed the first time they are used.

```
04:02 [0173-1#02-AAN350#003] String "Part relation to the main device"
{ KeyLOV[0173-1#09-AAD520#003] String (entries:7)
  1: "accessory" "accessory" "0173-1#07-AAR847#001" ""
  2: "counterpart" "counterpart" "0173-1#07-AAR849#001" ""
  3: "Label parts (inscription)" "Label parts (inscription)" "0173-1#07-AAR848#002" ""
  4: "Wartungsteil" "maintenance part" "0173-1#07-AAR846#001" ""
  5: "resolve part" "resolve part" "0173-1#07-AAR844#001" ""
  6: "spare" "spare" "0173-1#07-AAR845#002" ""
  7: "system component" "system component" "0173-1#07-AAR843#002" ""
}
```

All subsequent uses of the key-LOV are abbreviated with a reference to the first usage:

```
04:02 [0173-1#02-AAN350#003] String "Part relation to the main device"
{ KeyLOV[0173-1#09-AAD520#003] { ADDITIONAL USE: For details see output above }
```

- To find the node from which you want to begin displaying the hierarchy, assuming you know the class ID and the class namespace, type:

clsutility -list -hierarchy -classId= <class ID> -classNamespace=<class namespace>

If the class ID of the **AC-DC supply** class is **AFR745** and the namespace is **0173-1**, type:

clsutility -list -hierarchy -classId=AFR745 -classNamespace=0173-1

The output is as follows:

```
"00-00-00-00" [0173-1#4711A1] "eCl@ass ADVANCED 4711-A1"
"27-00-00-00" [0173-1#AAB572] "[27] Electric engineering, automation, process control engineering"
"27-04-00-00" [0173-1#AAB631] "[27-04] Power supply devices"
"27-04-07-00" [0173-1#AFR649] "[27-04-07] Power supply device"
"27-04-07-04" [0173-1#AFX043 => 0173-1#01-AFR745#002] "[27-04-07-04] AC-DC supply"
```

- To display the JSON file from which the output information is drawn, type:

clsutility -list -hierarchy -showClass -showJSON

This argument functions only with the **-showClass** argument.

The output includes the description of the class as well as the JSON that the user interface uses to display the class.

- To display all the ICOs of a specific node, type:

clsutility -list -hierarchy -nodeId= 0173-1#AFR842 -showICOs

All the ICOs are displayed in addition to the complete hierarchy, with the number of instances displayed for each class:

```
"27-40-06-29" [0173-1#AFR842 ==> 0173-1#01-AFR844#001] "J27-40-06-29] Labeling material (electronic installation)"
{ Instance Count: 4
  Type Name: Cst00173-1#01-AFR844#001
  [0] "8Bfx$prFZsQeXC"->"MqVx$prFZsQeXC" "PC-0816799/A" "Name of PC-0816799/A"
  [1] "8wUx$prFZsQeXC"->"cVax$prFZsQeXC" "PC-0816155/A" "Name of PC-0816155/A"
  [2] "JbTx$pv6ZsQeXC"->"Z$Zx$pv6ZsQeXC" "PC-0803254/A" "Name of PC-0803254/A"
  [3] "tVRx$prFZsQeXC"->"95Xx$prFZsQeXC" "PC-0816786/A" "Name of PC-0816786/A"
```

To display a specific ICO from the preceding list, type

```
clsutility -list -hierarchy -showICOs -icold=<ICO ID>
```

This is particularly useful when combined with **-showICOProperties**.

- To display no more than x number of ICOs within a class, type

```
clsutility -list -hierarchy -showICOs -recursive -icoLimit=<x>
```

View output in a text editor

The text files created by this utility can be extremely large. To assist you in manipulating these files, you can try enabling the collapsing of code in a text editor. This can be done, for example, by viewing the text file as C++ code, or a similar viewing feature depending on the text editor. This adds the ability to collapse at every curly bracket in the output of the hierarchy. Collapsing unwanted portions of the output makes it easier to find what you are looking for.

Setting Classification preferences

The following preferences are available to configure your Classification environment.

ICS_classifiable_types

Stores the business object types that can be classified. If your company works with custom business objects, you must add the item type to this preference to be able to classify it. If you want to classify, for example, items only, you must remove the **Item Revision** entry from the list of classifiable types.

AWC_classification_facets_threshold

Sets the number of search facets displayed using the preference. The default value is 200. The facets are sorted with the most common facets at the top. Exercise caution when setting the number of returned facets as a very large number can cause a decrease in performance.

CLS_is_presentation_hierarchy_active

Specifies whether traditional classes or the presentation hierarchy (nodes) are visible in the Classification hierarchy. If these preferences are set to **true**, the presentation node hierarchy is visible. If it is set to **false**, traditional classes are visible. The default value of the preference is **true**.

This user preference is not delivered by default and must be added manually after installing the Presentation Layer (**Next Generation Classification Server** installation option).

CLS_auto_sync_node_hierarchy

Allows the system to mirror operations performed on the Classification storage class hierarchy onto the node hierarchy in the Presentation Layer.

The default value of the preference is **true**.

CLS_search_similar_wso_props_enabled

Specifies the list of workspace object properties that are displayed in the **Filters** pane when using the **Search Similar** feature.

AWC_cls_object_filter_sorting

Defines how the Classification hierarchy is sorted in the **Filter** panel. Valid values are:

- **Ascending**
Sorts the hierarchy in alphabetical order, from A to Z.
- **Descending**
Sorts the hierarchy in reverse alphabetical order, from Z to A.
- **Count** (default)
Sorts the hierarchy based on the number of instances for the given filter value.

CST_supported_eclass_releases

The following preferences are required when migrating from traditional basic classes to advanced (Classification Standard Taxonomy, CST) classes using the **clsutility -migrate -classification2cst** utility:

CST_default_namespace

Specifies the namespace of all CST objects created by the migration utility.

CST_default_migration_status

Specifies the status of the CST objects created by the migration utility. Valid values are **Develop**, **Released**, or **Retired**. Objects set to **Released** can no longer be modified.

The following preferences are used to configure Classification Artificial Intelligence (AI):

CLS_AI_Enable_AI_Engine

Set this preference to **true** to enable class suggestions.

CLS_AI_Object_Properties

Specifies the properties that are extracted from the Teamcenter database and used to train the AI engine. These properties form the basis for the class suggestions offered when classifying objects.

By default, this property is set to:

object_name

object_desc

object_type

owning_user

owning_group

CLS_AI_Enable_Geolus

Set this preference to **true** to enable the additional comparison of shape parameters using the Geolus search engine with Classification AI.

CLS_AI_Geolus_Max_Num_Results

Limits the number of results returned by the Geolus search engine. By default, this preference is set to **1000**.

CLS_AI_Engine_Probability_Cutoff

Specifies the percentage probability above which classes are considered appropriate suggestions for Classification AI. If too many class suggestions are displayed, or you would like to increase the accuracy of the classes displayed, you can increase the value of this preference. By default, this preference is set to **10**.

CLS_AI_Engine_Suggestions_Cutoff

Specifies the maximum number of suggestions displayed to the user. The default value is **10**. This preference is not delivered with the software. You must add it manually.

CLS_AI_Query_Start_Date

Specifies the earliest modification date of the data to include in training. Only objects created or modified after this date are included in the training. If this date is not set, there is no restriction on objects used in training.

CLS_AI_Query_End_Date

Specifies the latest modification date of the data to include in training. Only objects created or modified before this date are included in the training. If this date is not set, there is no restriction on objects used in training.

CLS_AI_Query_Object_Type

Specifies the type of data to include in training. Only objects of the given type (and any child objects of this type) are included for training. If left blank, the value of this preference is **WorkspaceObject** by default.